

Ordonnabilité des espaces métriques

Le sujet comporte 12 pages, numérotées de 1 à 12.

Début de l'épreuve.

Dans ce sujet, on s'intéresse au problème d'ordonner les éléments d'un espace métrique de sorte que deux éléments successifs soient à distance bornée.

Ce sujet est constitué de quatre parties. La première partie est formée de préliminaires utiles dans le reste du sujet, avec des questions de programmation en C et OCaml. La deuxième partie propose une implémentation en C d'une structure de données utilisée notamment dans la troisième partie. Cette troisième partie considère les graphes, munis de la distance de plus court chemin, comme espace métrique, avec implémentations en C. La quatrième partie, enfin, indépendante des deux précédentes, considère l'ordonnabilité des langages réguliers pour une certaine distance d'édition ; cette partie contient de la programmation en OCaml.

Dans les questions de programmation en C ou OCaml, on n'utilisera pas de fonctions qui ne sont pas incluses dans la bibliothèque standard du langage. Pour les codes en C, on pourra supposer que les en-têtes `<stdbool.h>`, `<stdio.h>`, `<stdlib.h>` et `<assert.h>` ont été inclus.

Espaces métriques et d -ordres

Dans ce sujet, on s'intéresse à des collections de données discrètes que l'on cherche à explorer ou engendrer de proche en proche. On modélise ce cadre général à l'aide des notions d'*espace métrique*, de *d -suite* et de *d -ordre* :

Définition 1 (espace métrique). *Un espace métrique $\mathcal{M} = (X, \delta)$ est constitué d'un ensemble dénombrable X dont les éléments sont appelés points et d'une distance sur X , c'est-à-dire une application $\delta : X \times X \rightarrow \mathbb{N}$ satisfaisant :*

Symétrie. Pour tout $(x, y) \in X^2$, $\delta(x, y) = \delta(y, x)$.

Séparation. Pour tout $(x, y) \in X^2$, $\delta(x, y) = 0$ si et seulement si $x = y$.

Inégalité triangulaire. Pour tout $(x, y, z) \in X^3$, $\delta(x, y) \leq \delta(x, z) + \delta(z, y)$.

Pour un espace métrique $\mathcal{M} = (X, \delta)$ et $X' \subseteq X$, on note $\mathcal{M}[X']$ le couple $(X', \delta|_{X'})$, où $\delta|_{X'}$ dénote la restriction de δ au domaine $X' \times X'$. On observe que $\mathcal{M}[X']$ est encore un espace métrique, que l'on appellera sous-espace de $\mathcal{M}$.

Définition 2 (d -suite). *Pour $d \in \mathbb{N}^*$, une d -suite $\mathbf{s}$ dans un espace métrique $\mathcal{M} = (X, \delta)$ est une suite (finie ou infinie dénombrable) $\mathbf{s} = x_1, x_2, \dots$ de points de $\mathcal{M}$ telle que :*

- $\mathbf{s}$ ne contient pas de doublons, c'est-à-dire que pour tous x_i, x_j de la suite, si $i \neq j$ alors $x_i \neq x_j$;
- deux points consécutifs de la suite sont à distance au plus d ; formellement, pour tous x_i, x_{i+1} de la suite on a $\delta(x_i, x_{i+1}) \leq d$.

On dit que $\mathbf{s}$ commence en x_1 et, dans le cas où $\mathbf{s}$ est finie et a n éléments, que $\mathbf{s}$ termine en x_n .

Définition 3 (d -ordre, d -ordonnable). *Pour $d \in \mathbb{N}^*$, un d -ordre de $\mathcal{M}$ est une d -suite dans $\mathcal{M}$ contenant tous les points de $\mathcal{M}$. L'espace $\mathcal{M}$ est dit d -ordonnable lorsqu'il existe un d -ordre de $\mathcal{M}$. On dit que $\mathcal{M}$ est ordonnable lorsque $\mathcal{M}$ est d -ordonnable pour un certain d .*

Distances d'édition sur les mots

On fixe dans tout le sujet l'*alphabet* $\Sigma := \{a, b\}$ ayant pour seules lettres a et b . Un *mot* w est une suite finie de lettres $w = \alpha_1 \cdots \alpha_n$. La longueur de w , notée $|w|$, est n . On note ε le mot vide, de longueur nulle. On note Σ^* l'ensemble des mots sur Σ . Un *langage* est un sous-ensemble de Σ^* .

Un espace métrique d'intérêt, sur l'ensemble des mots d'un langage, est donné par deux distances d'édition sur les mots : la distance *push-pop* et la distance *push-pop-droite*, que l'on définit ci-après.

Définition 4 (distance push-pop). *La distance push-pop, dénotée δ_{pp} , est définie de la manière suivante : pour $w, w' \in \Sigma^*$, $\delta_{pp}(w, w')$ est le nombre minimal d'opérations nécessaires pour passer de w à w' , où les opérations autorisées sont :*

- pour un mot w , insérer la lettre $\alpha \in \Sigma$ à la fin, ce qui donne le mot $w\alpha$;
- pour un mot w , insérer la lettre $\alpha \in \Sigma$ au début, ce qui donne le mot αw ;
- pour un mot de la forme $w\alpha$ avec $\alpha \in \Sigma$, supprimer la dernière lettre, ce qui donne le mot w ;
- pour un mot de la forme αw avec $\alpha \in \Sigma$, supprimer la première lettre, ce qui donne le mot w .

Exemple 1. *Les mots à distance push-pop 1 du mot aab sont aa (on a supprimé la dernière lettre), $aaba$ (on a ajouté un a à la fin), $aabb$ (on a ajouté un b à la fin), ab (on a supprimé la première lettre), $aaab$ (on a ajouté un a au début) et $baab$ (on a ajouté un b au début).*

Définition 5 (distance push-pop-droite). *La distance push-pop-droite, dénotée δ_{ppr} , est définie de la même manière que δ_{pp} mais seules les insertions et suppressions à la fin du mot sont autorisées.*

Exemple 2. *Les mots qui sont à distance push-pop-droite 1 du mot aab sont aa (on a supprimé la dernière lettre), $aaba$ (on a ajouté un a à la fin) et $aabb$ (on a ajouté un b à la fin).*

Algorithmes d'énumération push-pop et push-pop-droite

Lorsqu'on travaillera sur les mots, on considèrera parfois des programmes produisant une suite (potentiellement infinie) de mots de Σ^* , en utilisant les instructions spéciales suivantes : `popL()`, `popR()` ; `pushL( $\alpha$ )` et `pushR( $\alpha$ )` pour $\alpha \in \Sigma$; et `output()`.

Le programme dispose, comme état interne, d'une liste L d'éléments de Σ , qui est interprétée comme un mot de Σ^* . La liste L est initialement vide et représente donc le mot vide. Voici ce qu'il se passe lorsque le programme utilise les instructions spéciales :

- `popL()` a pour effet de supprimer la première lettre de la liste (et ne peut être appelée que si la liste est non vide) ;
- `popR()` a pour effet de supprimer la dernière lettre de la liste (et ne peut être appelée que si la liste est non vide) ;
- `pushL( $\alpha$ )` a pour effet d'ajouter la lettre $\alpha \in \Sigma$ en début de liste ;
- `pushR( $\alpha$ )` a pour effet d'ajouter la lettre $\alpha \in \Sigma$ en fin de liste ;
- `output()` *produit* le mot étant actuellement représenté par la liste (par exemple, il l'affiche en sortie du programme).

On souligne que la liste L n'est accessible par le programme que via ces instructions spéciales. De plus, on fait l'hypothèse que chacune des instructions `popL`, `popR`, `pushL` et `pushR` a une complexité en $O(1)$.

Un tel programme *produit* alors la suite de mots $w_1, w_2, \dots$, où w_1 est le premier mot produit par le programme lors de son exécution, w_2 le second et ainsi de suite.

On appellera un tel programme un *programme push-pop*. De manière similaire, un programme *push-pop-droite* est un programme push-pop qui n'utilise pas les instructions `popL()` ni `pushL( $\alpha$ )` pour $\alpha \in \Sigma$.

Exemple 3. On suppose que les fonctions `C` `pushL`, `pushR`, `popL`, `popR`, `output` ont été prédéfinies et manipulent toutes une même variable représentant la liste. Le programme `C` suivant est un programme *push-pop-droite* qui produit la suite de mots $w_1, w_2, \dots$ où w_i est $(aa)^{i-1}b$:

```
int main(void)
{
    pushR('b'); output();
    while (true) {
        popR(); pushR('a'); pushR('a'); pushR('b'); output();
    }
}
```

On remarque que ce programme ne termine jamais.

En OCaml, on suppose qu'on dispose d'un type `lettre` défini par :

```
type lettre = A | B
```

On représente alors les mots de Σ^* par des listes OCaml `lettre list`, dont la tête correspond à la première lettre du mot. Par exemple, le mot *aab* est représenté par la liste `[A;A;B]`. On suppose avoir accès en OCaml à des fonctions de type `pushR : lettre -> unit`, `pushL : lettre -> unit`, `popR : unit -> unit`, `popL : unit -> unit`, ainsi que `output : unit -> unit`, qui représentent les instructions spéciales.

Exemple 4. Le programme OCaml *push-pop* suivant produit la même suite que le programme `C` *push-pop-droite* de l'exemple 3 :

```
let main () =
    pushR B; output ();
    while true do
        pushL A; pushL A; output ();
    done
```


Partie I : Préliminaires

Question I.1. Soit $\mathcal{M} = (X, \delta)$ un espace métrique tel que X est un ensemble fini. Montrer que $\mathcal{M}$ est ordonnable.

Question I.2. Écrire une fonction OCaml

`delta_ppr : lettre list -> lettre list -> int`

prenant en entrée deux listes `l1`, `l2` représentant des mots w_1, w_2 et renvoyant $\delta_{\text{ppr}}(w_1, w_2)$, la distance push-pop-droite entre w_1 et w_2 . On attend de cette fonction que sa complexité soit linéaire en $|w_1| + |w_2|$.

Question I.3. On considère la suite $\mathbf{s} := w_1, w_2, \dots$, où w_i est a^{i^2} . Écrire un programme push-pop-droite en C qui produit la suite $\mathbf{s}$.

Question I.4. Montrer que $\mathcal{M}_{\text{pp}} := (\Sigma^*, \delta_{\text{pp}})$ et $\mathcal{M}_{\text{ppr}} := (\Sigma^*, \delta_{\text{ppr}})$ sont des espaces métriques.

On s'intéresse maintenant aux sous-espaces de $\mathcal{M}_{\text{pp}}$ et $\mathcal{M}_{\text{ppr}}$, c'est-à-dire (comme indiqué dans la Définition 1) aux espaces métriques de la forme $\mathcal{M}_{\text{pp}}[L] = (L, \delta_{\text{pp}|L})$ ou $\mathcal{M}_{\text{ppr}}[L] = (L, \delta_{\text{ppr}|L})$, pour L un langage. Par exemple, la suite produite par les programmes push-pop des exemples 3 et 4 est un 4-ordre pour $\mathcal{M}_{\text{ppr}}[L]$ et un 2-ordre pour $\mathcal{M}_{\text{pp}}[L]$, où $L := (aa)^*b$.

Question I.5. Écrire un programme push-pop en C qui produit un d -ordre pour $\mathcal{M}_{\text{pp}}[L]$, où $L := a^*b^* \mid b^*a^*$ et un certain $d \in \mathbb{N}$. Expliquer comment fonctionne ce programme.

Question I.6. Écrire un programme push-pop en C qui produit un 1-ordre pour $\mathcal{M}_{\text{pp}}[L]$, où $L := a^*b^*$. [*Indice* : on peut visualiser le langage L comme la grille $\mathbb{N} \times \mathbb{N}$, où à la position (i, j) correspond le mot $a^i b^j$.] Ne pas hésiter à faire un dessin pour se faire comprendre.

Question I.7. Soit $L := b^* \mid ab^*$.

- Écrire un programme push-pop en C qui produit un 1-ordre pour $\mathcal{M}_{\text{pp}}[L]$.
- Prouver que $\mathcal{M}_{\text{ppr}}[L]$ n'est pas ordonnable.

Question I.8. Un *chemin hamiltonien* dans un graphe non-orienté est un chemin (aussi appelé *chaîne*) qui visite tous les sommets du graphe exactement une fois. Un *graphe grille* est un graphe de la forme $G = (V, E_V)$ avec V une partie finie de $\mathbb{N} \times \mathbb{N}$ et $E_V = \{(i, j), (i', j')\} \mid (i, j), (i', j') \in V \text{ et } |i - i'| + |j - j'| = 1\}$, autrement dit, deux sommets sont connectés par une arête s'ils sont à distance 1 dans la grille $\mathbb{N} \times \mathbb{N}$.

On définit le problème de décision **HamGrille** de la manière suivante : en entrée on a un graphe grille G et la sortie est OUI si G possède un chemin hamiltonien et NON sinon. On admettra que ce problème est NP-complet.

Pour t un entier naturel non nul, on définit le problème de décision $\mathcal{P}_t$, de la façon suivante : $\mathcal{P}_t$ prend en entrée un ensemble fini de mots L et la sortie est OUI si $\mathcal{M}_{\text{pp}}[L]$ est t -ordonnable, NON sinon. On fixe maintenant un entier naturel $t \geq 1$ arbitraire. Montrer que pour cet entier t , le problème $\mathcal{P}_t$ est NP-complet.

Partie II : Implémentation en C des programmes push-pop

Dans cette partie, on souhaite réaliser une implémentation en langage C des opérations des programmes push-pop sur une structure de données `liste` codant une liste doublement chaînée. Le type de la structure de données `liste` est défini comme suit :

```
typedef struct chainon_s chainon;
```

```
struct chainon_s
{
    int val;
    chainon *prec;
    chainon *suiv;
};
```

```
struct liste_s
{
    chainon *premier;
    chainon *dernier;
};
```

```
typedef struct liste_s liste;
```

En particulier, les lettres sont donc représentées par des entiers (on rappelle qu'en C, une variable de type `char` peut être implicitement interprétée comme variable de type `int` sans perte d'informations); cela permettra de réutiliser cette structure `liste` dans d'autres contextes.

Dans l'implémentation de cette structure de données, on veillera à ce qu'un pointeur dont la valeur n'est pas `NULL` pointe toujours vers un espace mémoire valide et pas, par exemple, vers une donnée libérée.

Question II.1. Définir et initialiser une variable globale `lg` représentant la liste chaînée globale manipulée par les programmes push-pop; on souhaite que cette liste soit initialement vide.

Question II.2. Programmer une fonction `est_vide` de prototype `bool est_vide(void)` renvoyant `true` si `lg` représente une liste vide, `false` sinon.

Question II.3. Programmer des fonctions `pushL` et `pushR` de prototypes `void pushL(int)` et `void pushR(int)` implémentant les opérations `pushL` et `pushR` sur la liste représentée par `lg`. On utilisera une assertion pour vérifier que l'allocation dynamique de mémoire est bien réalisée. Quelle est la complexité en temps de ces deux fonctions ?

Question II.4. Programmer des fonctions `popL` et `popR` de prototypes `int popL(void)` et `int popR(void)` implémentant les opérations `popL` et `popR` sur la liste représentée par `lg` (chacune renvoyant également la valeur extraite de la liste). On utilisera une assertion pour s'assurer que la liste n'est pas vide et on veillera à libérer la mémoire devenue inutile. Quelle est la complexité en temps de ces deux fonctions ?

Question II.5. Programmer des fonctions `output` et `vide_liste` de prototypes respectifs `void output(void)` et `void vide_liste(void)`; `output` affiche le mot codé par la liste de caractères sur la sortie standard (suivi d'un retour à la ligne), tandis que `vide_liste` vide la liste (retire tous ses éléments) et libère la mémoire dynamique associée. Quelle est la complexité en temps de ces deux fonctions ?

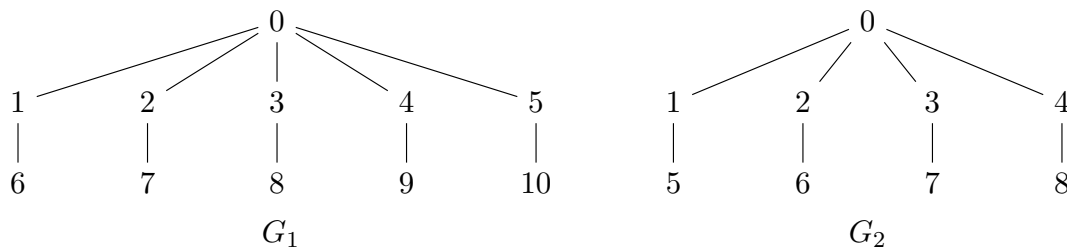
Partie III : Ordonnabilité des graphes

Dans cette partie on s'intéresse à des espaces métriques définis à partir de graphes non-orientés connexes. Pour un graphe non-orienté connexe $G = (V, E)$ (où V est l'ensemble fini non-vide de nœuds du graphe et E son ensemble d'arêtes), on note $\delta_G : V \times V \rightarrow \mathbb{N}$ la fonction qui à un couple de nœuds (u, v) de V associe la longueur d'un plus court chemin de u à v dans G .

L'objectif est de montrer que les espaces de la forme $\mathcal{M}_G := (V, \delta_G)$ sont 3-ordonnables et d'étudier des algorithmes qui calculent de tels ordres. On travaillera en langage C.

Question III.1. Soit $G = (V, E)$ un graphe non-orienté connexe. Montrer que le couple $\mathcal{M}_G := (V, \delta_G)$ est effectivement un espace métrique.

Question III.2. On considère les graphes G_1 et G_2 représentés ci-dessous.



- a. Donner un 3-ordre de $\mathcal{M}_{G_1}$ (aucune justification n'est attendue).
- b. Donner un 2-ordre de $\mathcal{M}_{G_2}$ (aucune justification n'est attendue).

On souhaite calculer un arbre couvrant d'un graphe non-orienté connexe. Un *arbre* est un graphe orienté *acyclique* $T = (V_T, E_T)$ avec V_T un ensemble fini non vide de nœuds et $E_T \subseteq V_T \times V_T$ un ensemble d'arcs tel que :

- il existe un unique nœud $r \in V_T$, appelé la *racine* de T tel que pour tout $v \in V_T$, $(v, r) \notin E_T$;
- pour tout nœud $u \in V_T$ qui n'est pas la racine, il existe un unique nœud $v \in V_T$ tel que $(v, u) \in E_T$ et on dit que u est un *fil* de v .

On définit maintenant un *arbre couvrant* d'un graphe non-orienté connexe $G = (V, E)$ comme un arbre $T = (V', E')$ tel que :

- $V' = V$;
- pour tout $(u, v) \in E'$, $\{u, v\} \in E$.

On va travailler avec la représentation en matrice d'adjacence des graphes : on suppose que le nombre de nœuds N est prédéfini en C comme une constante globale `N`, que les nœuds de G sont $\{0, \dots, N-1\}$ et que G est représenté par un tableau bidimensionnel `bool graphe[N][N]` où `g[u][v]` et `g[v][u]` valent tous les deux `true` si $\{u, v\}$ est une arête de G , et `false` sinon.

L'arbre couvrant T sera également représenté par un tableau `bool arbre[N][N]`, où cette fois `arbre[u][v]` vaut `true` si v est un fils de u et `false` sinon.

Question III.3. Écrire une fonction

```
void calcule_arbre_couvrant(bool graphe[N][N], bool arbre[N][N])
```

qui remplit la structure `arbre` pour que celle-ci représente un arbre couvrant quelconque du graphe G représenté par `graphe`, avec le nœud 0 comme racine. On supposera G connexe et on ne vérifiera pas ce point dans le code. On pourra utiliser les fonctions définies dans la partie II pour gérer une file (ou une pile) de nœuds à traiter. Justifier brièvement la correction de votre algorithme.

On supposera par la suite que `arbre` représente effectivement un arbre couvrant comme indiqué plus haut et que le nœud 0 est la racine de celui-ci.

Question III.4. On considère la fonction C suivante :

```
void visite(bool arbre[N][N], int noeud, bool mystere) {
    if(!mystere)
        pushR(noeud);
    for (int fils = 0; fils < N; ++fils) {
        if (arbre[noeud][fils])
            visite(arbre, fils, !mystere);
    }
    if(mystere)
        pushR(noeud);
}
```

On suppose que la liste `lg` utilisée par la fonctions `pushR` est initialement vide et qu'on appelle `visite(arbre, 0, false)` sur un arbre `arbre` de racine 0.

- Que vaut le troisième argument `mystere` lors d'un appel récursif à `visite` avec comme deuxième argument un nœud n de l'arbre, en fonction de la position de n dans l'arbre ?
- Que contient la liste `lg` à la fin de l'exécution de ce programme sur le graphe G_2 de la question III.2, vu comme un arbre enraciné en 0 ?

Question III.5. Montrer que l'appel `visite(a, 0, false)`, lorsque a est la représentation d'un arbre couvrant d'un graphe non-orienté connexe G de racine 0, calcule (dans la liste `lg`) un 3-ordre pour $\mathcal{M}_G$.

Question III.6. Quelle est la complexité en temps de l'appel à `visite(a, 0, false)`, en fonction du nombre N de nœuds ? Peut-on améliorer cette complexité (par exemple en changeant notre méthode de représentation des graphes) ?

Question III.7. On observe que la fonction `visite` est une fonction récursive.

- Quel(s) problème(s) cela pourrait-il poser ?
- Proposer une méthode pour rendre ce calcul non récursif. On ne s'attend pas ici à du code, mais à une explication qui puisse être facilement transformée en du code.

Partie IV : Ordonnabilité des langages réguliers pour la distance push-pop-droite

Dans cette partie on souhaite caractériser les langages réguliers ordonnables pour la distance push-pop-droite. On travaillera en OCaml.

Automates et langages réguliers. Le terme *automate* désignera systématiquement un automate fini déterministe. Formellement, un automate $A = (Q, q_{\text{init}}, F, \text{trans})$ consiste en un ensemble fini Q d'états, un état initial q_{init} , un ensemble $F \subseteq Q$ d'états finaux, ainsi qu'une fonction de transition $\text{trans} : Q \times \Sigma \rightarrow Q \cup \{\perp\}$, où $\perp$ signifie que la transition n'est pas définie. Un chemin dans A d'un état $q \in Q$ à un état $q' \in Q$ est une suite finie de la forme $q_1, \alpha_1, q_2, \alpha_2, \dots, \alpha_{n-1}, q_n$ où les q_i sont des états de Q et les α_i des lettres de Σ , telle que $q = q_1$, $q' = q_n$ et telle que pour tout $i \in \{1, \dots, n-1\}$ on a $q_{i+1} = \text{trans}(q_i, \alpha_i)$; l'étiquette d'un tel chemin est alors le mot $\alpha_1 \dots \alpha_{n-1}$. En particulier, il y a toujours un chemin de longueur nulle, avec étiquette ε , entre n'importe quel état et lui-même (la suite comprenant ce seul état). Le langage accepté par A , noté $L(A)$, est l'ensemble des mots qui étiquettent un chemin depuis q_{init} jusqu'à un état final. Un langage est *régulier* s'il est accepté par un automate ou, de manière équivalente, s'il est représenté par une expression régulière.

On rappelle qu'un automate $A = (Q, q_{\text{init}}, F, \text{trans})$ est *émondé* si tout état q est à la fois accessible depuis l'état initial (c'est-à-dire qu'il y a un chemin depuis q_{init} à q) et co-accessible depuis un état final (c'est-à-dire qu'il y a un chemin depuis q vers un état final). Pour tout automate A , on peut calculer en temps linéaire un automate A' émondé tel que $L(A) = L(A')$.

Automates poêle. D'après la question I.1, si L est fini alors $\mathcal{M}_{\text{ppr}}[L]$ est ordonnable. **On supposera L infini dans cette partie.** On définit dans ce qui suit la notion d'*automate poêle-à-frîre* (ou *automate poêle*), puis on montre que $\mathcal{M}_{\text{ppr}}[L]$ est ordonnable si et seulement si n'importe quel automate émondé acceptant L est un automate poêle.

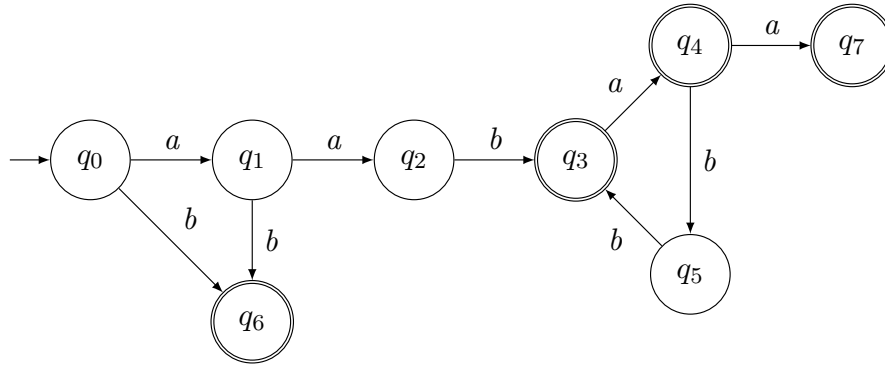
Soit $A = (Q, q_{\text{init}}, F, \text{trans})$ un automate. Un *cycle* dans A est un chemin de longueur non nulle dans A depuis un état q vers lui-même sans répétition d'état (à part q). On observe que ce chemin est possiblement de longueur 1, dans le cas où $\text{trans}(q, \alpha) = q$ pour $\alpha \in \Sigma$. On note que si $q_1, \alpha_1, q_2, \alpha_2, \dots, \alpha_{n-1}, q_1$ est un cycle, alors $q_2, \alpha_2, \dots, \alpha_{n-1}, q_1, \alpha_1, q_2$ et plus généralement $q_i, \alpha_i, \dots, \alpha_{n-1}, q_1, \alpha_1, \dots, \alpha_{i-1}, q_i$ pour tout $2 \leq i \leq n-2$ sont également des cycles : on dit que ce sont les mêmes cycles à *permutation près*.

Un *chemin strict vers un cycle* dans A est un chemin sans répétition d'état depuis l'état initial jusqu'à un état faisant partie d'un cycle tel que tous les états sauf le dernier ne font pas partie d'un cycle dans A ; formellement, c'est un chemin $q_1, \alpha_1, \dots, \alpha_{n-1}, q_n$ pour $n \in \mathbb{N}$ avec $q_1 = q_{\text{init}}$ tel que q_n fait partie d'un cycle et, pour $1 \leq i < n$, q_i ne fait partie d'aucun cycle. En particulier, si q_{init} fait partie d'un cycle alors le seul tel chemin est de longueur nulle.

On dit de A qu'il est *pseudo-acyclique* s'il a au plus un cycle à permutation près. On note qu'un automate tel que $\text{trans}(q, a) = \text{trans}(q, b) = q$ n'est jamais pseudo-acyclique, car q, a, q et q, b, q sont deux cycles différents.

Un automate A est alors un *automate poêle* s'il est pseudo-acyclique et a un unique chemin strict vers un cycle.

Exemple 5. *Considérons l'automate ci-dessous :*



Son ensemble d'états est $\{q_0, \dots, q_7\}$, son état initial q_0 , ses états finaux q_3, q_4, q_6 et q_7 ; les transitions non spécifiées par des flèches sont non définies. Cet automate est clairement déterministe et il est aisé de vérifier qu'il est émondé.

L'automate est également pseudo-acyclique : il comporte en effet un unique cycle, le cycle $q_3, a, q_4, b, q_5, b, q_3$ (qu'on peut également écrire $q_4, b, q_5, b, q_3, a, q_4$ ou encore $q_5, b, q_3, a, q_4, b, q_5$). Et comme il a un unique chemin strict vers un cycle (le chemin $q_0, a, q_1, a, q_2, b, q_3$), c'est un automate poêle.

Si une transition de q_3 à q_4 étiquetée par b était ajoutée, ce ne serait plus un automate pseudo-acyclique car $q_3, a, q_4, b, q_5, b, q_3$ et $q_3, b, q_4, b, q_5, b, q_3$ seraient deux cycles différents. De même, si une transition étiquetée par a de q_2 à l'un des états du cycle était ajoutée, l'automate aurait deux chemins stricts distincts vers un cycle et ne serait donc plus un automate poêle.

Question IV.1. Proposer une méthode permettant de déterminer si un automate émondé est un automate poêle, en supposant n'importe quelle représentation raisonnable des automates (on supposera l'automate émondé). On ne demande pas du code, mais on attend une explication qui puisse être facilement transformée en du code.

Lorsque A est un automate poêle, on appelle *état d'entrée* l'état qui se trouve à la fin de l'unique chemin strict vers l'unique cycle de A (cet état peut être l'état initial si celui-ci fait partie du cycle). Dans l'exemple 5, l'état q_3 est l'état d'entrée.

Question IV.2. Si A est un automate poêle, montrer que $L(A)$ peut s'écrire de la forme

$$F \mid uv^*F'$$

où F et F' sont des ensembles finis de mots et u, v sont des mots avec $v \neq \varepsilon$.

Question IV.3. On suppose que A est un automate poêle. Écrire un programme push-pop-droite en OCaml qui calcule un d -ordre pour $\mathcal{M}_{\text{ppr}}[L(A)]$, pour un certain $d \in \mathbb{N}$ que l'on ne cherchera pas à calculer, étant donnés les mots u, v et ensembles F, F' de la question précédente représentés en OCaml par des variables :

```

u : lettre list
v : lettre list
f : (lettre list) list
f' : (lettre list) list

```

On a ainsi établi que lorsqu'un langage L est accepté par un automate poêle alors il est ordonnable pour la distance push-pop-droite. On montre dans la suite de cette partie que c'est en fait une condition nécessaire, dans le sens où si $\mathcal{M}_{\text{ppr}}[L]$ est ordonnable, alors n'importe quel automate émondé pour L est un automate poêle.

Pour ce faire, on considère l'arbre infini T_Σ de Σ^* défini ainsi : les nœuds de T_Σ sont les mots de Σ^* , le mot vide est la racine de T_Σ et si $w \in \Sigma^*$ alors wa et wb sont les fils de w dans T_Σ . Une *branche infinie* B dans T_Σ est une suite infinie de la forme $w_1, w_2, \dots$, où $w_1 = \varepsilon$ et $w_{i+1} = w_i \alpha_i$ avec $\alpha_i \in \Sigma$ pour tout $i \in \mathbb{N}$. Pour $w \in \Sigma^*$, on note T_w le sous-arbre infini de T_Σ enraciné en w .

Pour un langage L , une *branche lourde* est une branche infinie $B = w_1, w_2, \dots$ dans T_Σ telle que pour tout $i \in \mathbb{N}$, T_{w_i} contient une infinité de mots de L .

Question IV.4. Montrer que, pour n'importe quel langage L , si L est infini alors L a (au moins) une branche lourde.

Question IV.5. Montrer que, pour n'importe quel langage L , si $\mathcal{M}_{\text{ppr}}[L]$ est ordonnable alors L a au plus une branche lourde.

Question IV.6. Soit A un automate émondé qui a au moins deux chemins stricts (distincts) vers des cycles (potentiellement identiques) et soient u et u' des mots étiquetant deux tels chemins.

- Montrer que u n'est pas un préfixe de u' et que u' n'est pas un préfixe de u .
- Montrer que $L(A)$ a au moins deux branches lourdes.

Question IV.7. Soit A un automate émondé qui a un seul chemin strict vers un cycle. Montrer que si A n'est pas pseudo-acyclique alors $L(A)$ a au moins deux branches lourdes. En déduire que pour un langage régulier L infini, $\mathcal{M}_{\text{ppr}}[L]$ est ordonnable si et seulement si n'importe quel automate émondé A acceptant L est un automate poêle.

Fin du sujet.

**ECOLE POLYTECHNIQUE
ECOLES NORMALES SUPERIEURES**

CONCOURS D'ADMISSION 2023

**MARDI 18 AVRIL 2023
14h00 - 18h00**

**FILIERE MP-MPI - Epreuve n° 4
INFORMATIQUE A (XULSR)**

Durée : 4 heures

***L'utilisation des calculatrices n'est pas autorisée pour
cette épreuve***

*Cette composition ne concerne qu'une partie des candidats de la
filière MP, les autres candidats effectuant simultanément la
composition de Physique et Sciences de l'Ingénieur.
Pour la filière MP, il y a donc deux enveloppes de Sujets pour
cette séance.*

Compression entropique

Le sujet comporte 11 pages, numérotées de 1 à 11.

Vue d'ensemble du sujet.

Ce sujet s'intéresse à la compression et à la décompression de mots sur un alphabet fini $\mathcal{S}$ de cardinal $|\mathcal{S}| \geq 2$. Plus précisément, on se donne une fonction q de $\mathcal{S}$ dans $\mathbb{N}$ et on s'intéresse aux mots de $\mathcal{S}^*$ tels que chaque lettre $\sigma \in \mathcal{S}$ a exactement $q(\sigma)$ occurrences dans ces mots. On note $\mathcal{S}^q$ l'ensemble de ces mots. Remarque : les mots de $\mathcal{S}^q$ ont pour longueur $N = \sum_{\sigma \in \mathcal{S}} q(\sigma)$.

Une lettre de $\mathcal{S}$ peut être représentée avec $\lceil \log_2 |\mathcal{S}| \rceil$ bits, où $\lceil x \rceil$ désigne la partie entière supérieure de x , c'est-à-dire l'entier tel que $\lceil x \rceil - 1 < x \leq \lceil x \rceil$. Si l'on concatène les bits représentant chacune des lettres d'un mot de $\mathcal{S}^q$, on peut donc représenter ce mot de façon non ambiguë à l'aide d'une séquence de bits de taille

$$\sum_{\sigma \in \mathcal{S}} q(\sigma) \cdot \lceil \log_2 |\mathcal{S}| \rceil = \lceil \log_2 |\mathcal{S}| \rceil \cdot N.$$

Il s'agit d'une représentation non compressée.

La théorie de l'information affirme qu'il faut au moins $\sum_{\sigma \in \mathcal{S}} q(\sigma) \cdot \log_2(N/q(\sigma))$ bits pour représenter tous les mots de $\mathcal{S}^q$ de façon non ambiguë. Ce sujet explore quelques façons de compresser les mots pour s'approcher de cette borne théorique.

La partie I s'intéresse à la fonction q , c'est-à-dire au comptage des lettres d'un mot. La partie II étudie les arbres binaires dont les feuilles sont étiquetées par des lettres de $\mathcal{S}$. La partie III utilise ces arbres pour (dé)compresser des mots de $\mathcal{S}^*$. La partie IV s'intéresse aux arbres qui donnent les meilleurs taux de compression pour les mots de $\mathcal{S}^q$. Certains de ces arbres ont une représentation compacte ; c'est l'objet de la partie V. D'autres arbres sont encore plus compacts, mais au prix d'un moindre taux de compression, comme le montre la partie VI. Finalement, la partie VII attaque le problème d'une façon complètement différente afin de s'approcher un peu plus du taux de compression théorique optimal.

Les différentes parties sont indépendantes : il n'est pas nécessaire d'avoir répondu aux questions d'une partie pour répondre aux questions d'une autre partie ; cependant les notions abordées dans une partie sont souvent utiles aux parties suivantes.

Rappels d'OCaml

On rappelle ici quelques opérations de base sur les tableaux :

- `Array.length t` renvoie la longueur du tableau `t`.
- `Array.make n v` crée un tableau de `n` cases qui sont toutes initialisées avec `v`.
- `Array.of_list l` renvoie un tableau initialisé avec les valeurs de la liste `l`, tandis que `Array.to_list t` réalise l'opération inverse.
- La case numéro `i` du tableau `t` peut être accédée avec `t.(i)`. Les cases sont numérotées à partir de zéro.

Partie I. Comptage d'occurrences

Un texte non compressé est un mot de $\mathcal{S}^*$. Dans la suite, à chaque fois qu'il s'agira de définir une fonction en OCaml, les lettres de $\mathcal{S}$ seront représentées par des entiers et un mot de $\mathcal{S}^*$ sera représenté par un tableau d'entiers (`int array`) ou une liste d'entiers (`int list`) en fonction des questions. La première étape consiste à calculer le nombre d'occurrences $q(\sigma)$ de chaque lettre σ dans un mot.

Question I.1. Supposons $\mathcal{S} = [0, 255]$. Définissez une fonction OCaml `occurrences : int list -> int array` qui reçoit en argument un mot représenté par une liste d'entiers et renvoie le nombre d'occurrences $q(\sigma)$ de chaque lettre $\sigma \in \mathcal{S}$ sous forme d'un tableau $\llbracket q(0); q(1); \dots; q(255) \rrbracket$.

Il peut être intéressant de ne considérer que le sous-ensemble des lettres qui ont au moins une occurrence. Dans la question suivante, on ne représentera pas q par $\llbracket q(0); q(1); \dots; q(|\mathcal{S}| - 1) \rrbracket$ mais par un tableau de paires $\llbracket (\sigma_1, q(\sigma_1)); (\sigma_2, q(\sigma_2)); \dots \rrbracket$ avec $\{\sigma_1, \sigma_2, \dots\} = \{\sigma \in \mathcal{S} \mid q(\sigma) \neq 0\}$ et $\sigma_1 < \sigma_2 < \dots$.

Question I.2. Définissez une fonction OCaml `nonzero_occurrences : int array -> (int * int) array` qui passe de la représentation de q de la question I.1 (tableau $\llbracket q(0); q(1); \dots; q(|\mathcal{S}| - 1) \rrbracket$) à la représentation sous forme d'un tableau de paires. Cette fonction devra avoir une complexité temporelle en $O(|\mathcal{S}|)$.

Partie II. Arbres binaires

Soit $\mathcal{T}_{\mathcal{S}}$ l'ensemble des arbres binaires dont les feuilles sont en bijection avec un ensemble fini $\mathcal{S}$. Autrement dit, un arbre de $\mathcal{T}_{\mathcal{S}}$ a exactement $|\mathcal{S}|$ feuilles ; chacune de ses feuilles est étiquetée par un élément de $\mathcal{S}$; toutes les feuilles sont étiquetées par des éléments différents de $\mathcal{S}$. Ces arbres peuvent être construits comme suit :

- Une feuille étiquetée par x est notée $F(x)$. C'est un arbre de $\mathcal{T}_{\{x\}}$.
- Si g et d sont des arbres appartenant respectivement à $\mathcal{T}_{\mathcal{S}_1}$ et $\mathcal{T}_{\mathcal{S}_2}$, alors l'arbre dont le sous-arbre gauche est g et le sous-arbre droit est d , noté $N(g, d)$, est un arbre de $\mathcal{T}_{\mathcal{S}_1 \cup \mathcal{S}_2}$ à condition que $\mathcal{S}_1 \cap \mathcal{S}_2 = \emptyset$.

Remarque : un tel arbre binaire possède exactement $|\mathcal{S}| - 1$ nœuds internes. Par ailleurs, on étiquettera implicitement les arêtes par 0 ou 1 suivant qu'elles mènent à un sous-arbre gauche (0) ou droit (1).

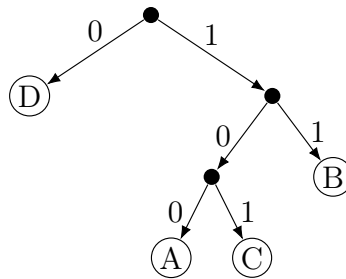


FIGURE 1 – Exemple d'arbre de $\mathcal{T}_{\{A,B,C,D\}}$.

Question II.1. Montrez que l'ensemble $\mathcal{T}_{\{A,B,C,D\}}$ contient 120 arbres.

Le type OCaml utilisé pour représenter les arbres est le suivant :

```
type tree = F of int | N of tree * tree
```

Étant donné un arbre t de $\mathcal{T}_{\mathcal{S}}$ et un élément σ de $\mathcal{S}$, on note $\ell_t(\sigma)$ la profondeur de la feuille étiquetée par σ , c'est-à-dire le nombre d'arêtes entre la racine de t et cette feuille. Par exemple, avec l'arbre de la figure 1, on a $\ell_t(A) = 3$.

Comme précédemment, soit q une fonction de $\mathcal{S}$ dans $\mathbb{N}$. Pour tout arbre $t \in \mathcal{T}_{\mathcal{S}'}$ avec $\mathcal{S}' \subseteq \mathcal{S}$, on note $c_q(t)$ la somme pondérée suivante :

$$c_q(t) = \sum_{\sigma \in \mathcal{S}'} q(\sigma) \ell_t(\sigma).$$

Question II.2. Supposons $\mathcal{S} = [0, n-1]$. Définissez une fonction OCaml `cq : tree -> int array -> int` qui reçoit deux arguments, un arbre $t \in \mathcal{T}_{\mathcal{S}}$ et un tableau $\llbracket q(0); q(1); \dots; q(n-1) \rrbracket$ représentant q , et qui renvoie la valeur de $c_q(t)$. On cherchera à écrire une fonction efficace. Donnez et justifiez sa complexité temporelle.

Étant donné un arbre t de $\mathcal{T}_{\mathcal{S}}$, on représente une lettre $\sigma \in \mathcal{S}$ par la séquence des bits obtenus en parcourant t de la racine vers la feuille $F(\sigma)$. Par exemple, dans l'arbre de $\mathcal{T}_{\{A,B,C,D\}}$ de la figure 1, D est représenté par 0 tandis que A est représenté par 100.

Question II.3. Définissez une fonction OCaml `get_path : int -> tree -> int list` qui reçoit en argument un entier $\sigma \in \mathcal{S}$ et un arbre $t \in \mathcal{T}_{\mathcal{S}}$ et qui renvoie la liste de 0 et 1 représentant σ dans t . Donnez et justifiez la complexité temporelle de cette fonction.

Considérons les séquences de bits définies de la façon suivante. Elles commencent par $k \geq 0$ bits valant 1, suivis d'un bit à 0, suivis de k bits arbitraires, ce que l'on notera $1^k 0(0 \mid 1)^k$. Les dix premières séquences par ordre lexicographique sont donc 0, 100, 101, 11000, 11001, 11010, 11011, 1110000, 1110001, 1110010. L'arbre dont les branches constituent ces séquences et dont les feuilles sont étiquetées de gauche à droite par des entiers croissants commence comme illustré sur la figure 2.

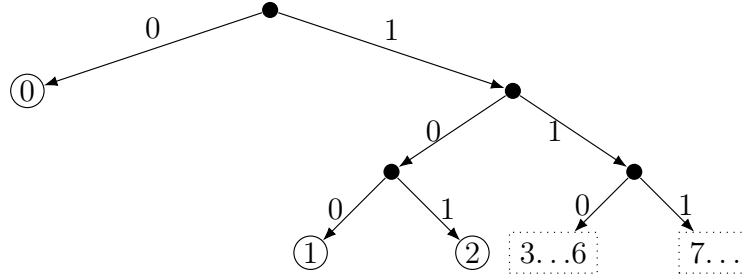


FIGURE 2 – Arbre des séquences $1^k 0(0 \mid 1)^k$.

Remarque : l'étiquette de chaque feuille correspond à l'indice de la séquence quand elles sont triées par ordre lexicographique. Les séquences croissant indéfiniment, l'arbre est *a priori* infini. C'est pourquoi la question suivante se limite aux séquences de bits $1^k 0(0 \mid 1)^k$ pour lesquelles k n'excède pas une certaine borne ℓ , afin d'obtenir un arbre de profondeur bornée $2\ell + 1$.

Question II.4. Définissez une fonction OCaml `integers : int -> tree` qui prend un entier $\ell \geq 0$ en argument et renvoie l'arbre dont les branches sont les séquences de la forme $1^k 0(0 \mid 1)^k$ avec $k \leq \ell$, ainsi que la séquence $1^{\ell+1}$. Les feuilles seront étiquetées de gauche à droite par les entiers 0, 1, 2, etc.

Partie III. Codes préfixes

Étant donnés un alphabet $\mathcal{S}$ et un arbre t de $\mathcal{T}_{\mathcal{S}}$, un mot de $\mathcal{S}^*$ peut être représenté par la concaténation des séquences de bits représentant chacune de ses lettres dans t , de la gauche vers la droite. Supposons que l'arbre t est l'exemple de la figure 1. Le mot « ADBDCD » est alors représenté par la séquence de bits 100|0|11|0|101|0. Remarque : les barres verticales ne servent qu'à rendre l'exemple plus lisible ; elles ne font pas partie de la séquence de bits et ne sont pas nécessaires pour retrouver le mot initial.

Question III.1. Soit t un arbre de $\mathcal{T}_{\mathcal{S}}$. Étant donnée une séquence de bits, montrez qu'il existe au plus un mot de $\mathcal{S}^*$ qui est représenté par cette séquence.

Question III.2. Supposons $\mathcal{S} \subseteq \mathbb{N}$. Définissez une fonction OCaml `decomp1 : int list -> tree -> int list` qui reçoit en argument une liste d'entiers 0 ou 1 et un arbre de $\mathcal{T}_{\mathcal{S}}$ et qui renvoie la liste des éléments de $\mathcal{S}$ représentée par cette liste de bits. On sera attentif au cas où la liste en argument ne représente aucun mot de $\mathcal{S}^*$. Donnez et justifiez la complexité temporelle de cette fonction.

Cette fonction est dite de *décompression*. Supposons que $\mathcal{S}$ est $\{A, B, C, D\}$ et que la fonction q donne le nombre d'occurrences de chaque lettre dans le mot « ADBDCD », par exemple $q(D) = 3$. Dans ce cas, $c_q(t)$ vaut 11 pour l'arbre exemple de la Figure 1. Il s'agit de la longueur de la séquence 10001101010 représentant le mot « ADBDCD ». Plus généralement, $c_q(t)$ est la longueur de n'importe quelle séquence représentant un mot de $\mathcal{S}^q$ compressé avec l'arbre t . Il s'avère qu'il n'existe aucun arbre $t' \in \mathcal{T}_{\mathcal{S}}$ tel que $c_q(t') < 11$; t est donc optimal.

Étant donnée une fonction q , l'objectif de la partie suivante sera de construire un des arbres de $\mathcal{T}_{\mathcal{S}}$ offrant la meilleure *compression* des mots de $\mathcal{S}^q$, c'est-à-dire un arbre qui minimise c_q .

Partie IV. Arbres optimaux

Soit q une fonction dont le domaine inclut $\mathcal{S}$. Un arbre de $\mathcal{T}_{\mathcal{S}}$ est dit *optimal* pour $(q, \mathcal{S})$ s'il minimise c_q parmi tous les arbres de $\mathcal{T}_{\mathcal{S}}$. Autrement dit, un arbre optimal $t \in \mathcal{T}_{\mathcal{S}}$ vérifie $c_q(t) = \min_{t' \in \mathcal{T}_{\mathcal{S}}} c_q(t')$. De tels arbres optimaux existent et, dès lors que $|\mathcal{S}| \geq 2$, ils ne sont pas uniques.

Question IV.1. Soit $t = N(g, d) \in \mathcal{T}_{\mathcal{S}}$ un arbre optimal pour $(q, \mathcal{S})$. Soit $\mathcal{S}_g$ l'ensemble des lettres qui étiquettent les feuilles du sous-arbre g . Montrez que g est un arbre de $\mathcal{T}_{\mathcal{S}_g}$ optimal pour $(q, \mathcal{S}_g)$.

Malheureusement, cette propriété ne permet pas d'en déduire un algorithme de type « diviser pour régner » pour trouver l'arbre optimal. En effet, il faudrait que l'algorithme puisse efficacement deviner comment partitionner $\mathcal{S}$ entre les feuilles du sous-arbre gauche et celles du sous-arbre droit.

Question IV.2. Supposons qu'il existe une lettre $\sigma_0 \in \mathcal{S}$ telle que $q(\sigma_0) > \sum_{\sigma \in \mathcal{S} \setminus \{\sigma_0\}} q(\sigma)$. Montrez que, pour tout arbre de $\mathcal{T}_{\mathcal{S}}$ optimal pour $(q, \mathcal{S})$, la feuille $F(\sigma_0)$ est à profondeur 1, c'est-à-dire directement attachée à la racine.

Question IV.3. Soient σ_1 et σ_2 deux éléments différents de $\mathcal{S}$ tels que, pour tout $\sigma \in \mathcal{S} \setminus \{\sigma_1, \sigma_2\}$, on a $q(\sigma) \geq q(\sigma_1)$ et $q(\sigma) \geq q(\sigma_2)$. Soit $\mathcal{S}' = \mathcal{S} \setminus \{\sigma_1, \sigma_2\} \cup \{\sigma_3\}$ avec σ_3 un tout nouvel élément. On définit q' de telle sorte que $q'(\sigma_3) = q(\sigma_1) + q(\sigma_2)$ et $\forall \sigma \in \mathcal{S} \setminus \{\sigma_1, \sigma_2\}$, $q'(\sigma) = q(\sigma)$.

1. Montrez qu'il existe un arbre t de $\mathcal{T}_{\mathcal{S}}$ optimal pour $(q, \mathcal{S})$ ayant un nœud $N(F(\sigma_1), F(\sigma_2))$.
2. Soit un arbre t' de $\mathcal{T}_{\mathcal{S}'}$ optimal pour $(q', \mathcal{S}')$. Montrez que l'arbre $t \in \mathcal{T}_{\mathcal{S}}$ obtenu en remplaçant dans t' la feuille $F(\sigma_3)$ par $N(F(\sigma_1), F(\sigma_2))$ est optimal pour $(q, \mathcal{S})$.

On définit la fonction $\bar{q}$ en étendant la fonction q aux arbres de la façon suivante. Pour une feuille, $\bar{q}(F(\sigma))$ vaut $q(\sigma)$. Pour un nœud interne, $\bar{q}(N(g, d))$ vaut récursivement $\bar{q}(g) + \bar{q}(d)$. Remarque : en général, $c_q(t) \neq \bar{q}(t)$.

La question précédente montre qu'il est possible de construire un arbre optimal pour $(q, \mathcal{S})$ à l'aide de l'algorithme suivant. On initialise un ensemble d'arbres E avec toutes les feuilles $F(\sigma)$ avec $\sigma \in \mathcal{S}$. À chaque étape, on retire de E deux arbres t_1 et t_2 qui minimisent $\bar{q}$; puis on ajoute à E l'arbre $N(t_1, t_2)$. On répète cette procédure jusqu'à ce qu'il ne reste qu'un seul arbre dans E . Il s'agit alors d'un arbre optimal pour $(q, \mathcal{S})$.

Question IV.4. Implantez l'algorithme décrit ci-dessus en définissant les deux fonctions OCaml suivantes : `insert` et `optimal`.

1. La fonction `insert : (int * tree) -> (int * tree) list -> (int * tree) list` insère dans une liste son premier argument. La liste en argument est supposée triée par rapport à la première composante de chacune de ses paires. La liste renvoyée doit l'être aussi.
2. La fonction `optimal : (int * int) list -> tree` renvoie un arbre optimal. La liste passée en argument est de taille $|\mathcal{S}|$; elle contient toutes les paires $(\sigma, q(\sigma))$ pour $\sigma \in \mathcal{S}$; ces paires sont triées par valeur croissante de $q(\sigma)$.
3. Donnez et justifiez la complexité temporelle de la fonction `optimal`.

Les deux questions suivantes montrent que, quand la liste initiale des $(\sigma, q(\sigma))$ est triée par valeur croissante de $q(\sigma)$, de simples listes et/ou tableaux suffisent pour écrire une fonction `optimal` dont la complexité temporelle est en $O(|\mathcal{S}|)$.

Question IV.5. Montrez que, lors de l'exécution de l'algorithme décrit ci-dessus, les arbres $t = N(t_1, t_2)$ sont ajoutés à l'ensemble E par valeur croissante de $\bar{q}(t)$.

Question IV.6. Expliquez comment écrire la fonction `optimal` pour que sa complexité temporelle soit linéaire. Le code OCaml n'est pas demandé.

Cette partie a permis de calculer un arbre qui minimise la longueur c_q de la séquence de bits représentant un mot de $\mathcal{S}^q$. Mais pour décompresser ce mot, il faut disposer de l'arbre et donc l'avoir stocké quelque part. Parmi tous les arbres optimaux, il est donc important d'en choisir un qui nécessitera le moins de bits pour être représenté; c'est l'objet de la partie suivante.

Partie V. Arbres canoniques

On suppose maintenant qu'il existe un ordre alphabétique noté $<$ sur les éléments de $\mathcal{S}$. Pour $\mathcal{S} \subseteq \mathbb{N}$, il s'agira de l'ordre usuel sur $\mathbb{N}$. Soit t un arbre de $\mathcal{T}_{\mathcal{S}}$. On définit la relation $\prec_t$ entre éléments de $\mathcal{S}$ de la façon suivante : pour toute paire $(\sigma_1, \sigma_2) \in \mathcal{S}^2$,

$$\sigma_1 \prec_t \sigma_2 \iff \ell_t(\sigma_1) < \ell_t(\sigma_2) \text{ ou } (\ell_t(\sigma_1) = \ell_t(\sigma_2) \text{ et } \sigma_1 < \sigma_2).$$

L'arbre t est dit *canonique* si, en parcourant les feuilles de la gauche vers la droite, leurs étiquettes respectent la relation $\prec_t$. Autrement dit, plus une feuille est à gauche, plus elle est proche de la racine ; et les étiquettes des feuilles à une même profondeur sont triées de gauche à droite par ordre alphabétique.

L'arbre de la figure 1 vérifie bien la deuxième partie de la propriété : les feuilles à une même profondeur sont triées de gauche à droite par ordre alphabétique. Mais il ne respecte pas la première partie : la feuille étiquetée par A est plus profonde que celle étiquetée par B alors qu'elle est plus à gauche. Cet arbre n'est donc pas canonique. L'arbre suivant est par contre canonique :

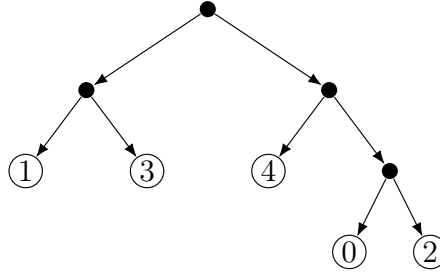


FIGURE 3 – Exemple d'arbre canonique.

Un arbre canonique peut être représenté par deux tableaux d'entiers. Le premier tableau associe à chaque indice i le nombre de lettres $\sigma \in \mathcal{S}$ telles que $\ell_t(\sigma) = i$. Le deuxième tableau contient les éléments de $\mathcal{S}$ obtenus en parcourant les feuilles de l'arbre de gauche à droite. Ainsi, l'arbre de $\mathcal{T}_{[0,4]}$ de la figure 3 est représenté par les deux tableaux $\llbracket 0; 0; 3; 2 \rrbracket$ et $\llbracket 1; 3; 4; 0; 2 \rrbracket$.

Question V.1. Définissez une fonction OCaml `canonical : int array -> int array -> tree` qui, étant donnés deux tels tableaux, renvoie l'arbre canonique qu'ils représentent, s'il existe. Donnez et justifiez la complexité temporelle de cette fonction.

Partie VI. Arbres alphabétiques

L'arbre de la figure 1 était optimal pour le mot « ADBDCD ». Un autre arbre optimal pour ce mot est le suivant :

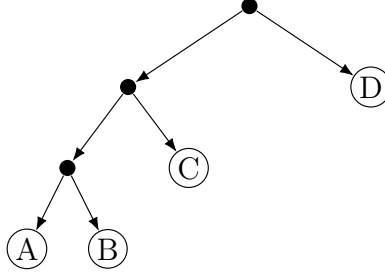


FIGURE 4 – Arbre à la fois optimal pour le mot « ADBDCD » et alphabétique.

Il n'est pas canonique mais il a une propriété très intéressante : ses feuilles sont dans l'ordre alphabétique. Cela signifie que lors du stockage de l'arbre, il n'est pas nécessaire de stocker les étiquettes des feuilles, il suffit de stocker la forme de l'arbre. Un tel arbre est dit *alphabétique*.

Plus généralement, un arbre de $\mathcal{T}_S$ est *alphabétique* si, en parcourant ses feuilles de gauche à droite, les étiquettes apparaissent dans l'ordre alphabétique. On note $\mathcal{A}_S$ l'ensemble de ces arbres. On dit d'un arbre qu'il est *alphabétique-optimal* pour (q, S) s'il minimise c_q parmi tous les arbres de $\mathcal{A}_S$. Remarque : un arbre alphabétique-optimal pour (q, S) n'est pas nécessairement optimal pour (q, S) .

Soit $n \in \mathbb{N}$ et $S = [0, n-1]$. On se donne une fonction $q : S \rightarrow \mathbb{N}$. Étant donnés deux entiers i et j tels que $0 \leq i \leq j < n$, on note $m_{i,j}$ la valeur de $c_q(t)$ pour n'importe quel arbre t de $\mathcal{A}_{[i,j]}$ alphabétique-optimal pour $(q, [i, j])$. En particulier, si t est un arbre de $\mathcal{A}_S$ alphabétique-optimal pour (q, S) , il vérifie $c_q(t) = m_{0,n-1}$.

Question VI.1. Exprimez la valeur de $m_{i,j}$ en fonction des valeurs de $m_{i',j'}$ avec $[i', j'] \subsetneq [i, j]$. Déduisez en une fonction OCaml `alpha_optimal : int array -> tree` qui calcule un arbre de $\mathcal{A}_{[0,n-1]}$ alphabétique-optimal pour $(q, [0, n-1])$. La fonction q est donnée par le tableau de taille n passé en argument. La complexité temporelle de la fonction devra être en $O(n^3)$.

Un arbre alphabétique de $\mathcal{A}_{[0,n-1]}$ peut être représenté par la séquence de $2n-1$ bits obtenue en effectuant un parcours en profondeur préfixe, c'est-à-dire que la racine d'un sous-arbre est visité avant ses feuilles et qu'un sous-arbre gauche est parcouru avant un sous-arbre droit. Pour chaque nœud, les chiffres 0 et 1 indiquent respectivement s'il s'agit d'un nœud interne ou d'une feuille. Par exemple, l'arbre de la figure 4 est représenté par la séquence 0001111.

Question VI.2. Définissez une fonction OCaml `alpha : int array -> tree` qui reçoit en argument un tableau de 0 et 1 et renvoie l'arbre alphabétique correspondant. On sera attentif au cas où le tableau en argument ne représente aucun arbre. Cette fonction devra avoir une complexité temporelle en $O(n)$ avec n la taille du tableau.

Partie VII. Codes arithmétiques

Soient un alphabet $\mathcal{S}$ et une fonction $q : \mathcal{S} \rightarrow \mathbb{N}$. Comme précédemment, $\mathcal{S}^q$ est le sous-ensemble des mots de $\mathcal{S}^*$ qui contiennent exactement $q(\sigma)$ occurrences de σ pour chaque $\sigma \in \mathcal{S}$.

Question VII.1. Supposons que $\mathcal{S}$ est $\{A, B, C\}$ et que q satisfait $q(A) = 15$, $q(B) = 4$, $q(C) = 1$.

1. Combien de bits faut-il pour représenter un mot de $\mathcal{S}^q$ en utilisant un arbre optimal pour $(q, \mathcal{S})$?
2. Combien y a-t-il de mots dans $\mathcal{S}^q$? On pourra exprimer ce nombre sous forme d'un produit de nombres premiers.
3. Montrez que 17 bits suffisent pour représenter chaque entier entre 0 et $|\mathcal{S}^q| - 1$ (et donc n'importe quel mot de $\mathcal{S}^q$). Remarque : $15 \cdot 17 = 2^8 - 1$.

L'exemple de la question précédente montre que, dans certains cas, par exemple quand une lettre est bien plus fréquente que les autres, l'utilisation d'un code préfixe n'est pas l'approche optimale. Il faut alors se tourner vers d'autres mécanismes de compression.

On se restreint maintenant au cas où $\mathcal{S} = [0, n - 1]$. Soit $N = \sum_{\sigma \in \mathcal{S}} q(\sigma)$. La fonction $E_q : \mathbb{N} \times \mathcal{S} \rightarrow \mathbb{N}$ est définie de la façon suivante pour $q(\sigma) \neq 0$:

$$E_q(x, \sigma) = \lfloor x/q(\sigma) \rfloor \cdot N + (x \bmod q(\sigma)) + \sum_{0 \leq k < \sigma} q(k),$$

où $\lfloor a \rfloor$ désigne la partie entière de a .

La fonction E_q peut être étendue en une fonction $C_q : \mathbb{N} \times \mathcal{S}^* \rightarrow \mathbb{N}$ qui travaille sur les mots de la façon suivante :

$$C_q(x, \sigma_0 \sigma_1 \dots \sigma_k) = E_q(\dots E_q(E_q(x, \sigma_0), \sigma_1), \dots \sigma_k).$$

Ainsi, après avoir choisi x arbitrairement, un mot $\sigma_0 \dots \sigma_{N-1} \in \mathcal{S}^q$ peut être compressé en un entier $y = C_q(x, \sigma_0 \dots \sigma_{N-1})$. Il est alors possible de retrouver le mot original à partir de y en calculant progressivement σ_{N-1} , σ_{N-2} , etc, jusqu'à σ_0 .

Considérons par exemple le mot « 0120000 » ($N = 7$, $q = \llbracket 5, 1, 1 \rrbracket$). Si l'on part de $x = 0$, sa version compressée sera l'entier 151 :

$$0 \xrightarrow{0} 0 \xrightarrow{1} 5 \xrightarrow{2} 41 \xrightarrow{0} 57 \xrightarrow{0} 79 \xrightarrow{0} 109 \xrightarrow{0} 151$$

où $x \xrightarrow{\sigma} y$ signifie $E_q(x, \sigma) = y$.

Question VII.2. Définissez une fonction OCaml `decomp2 : int array -> int -> int -> int * int array` qui reçoit en argument un tableau représentant q et deux entiers y et k , et renvoie un entier x et un tableau $[\sigma_0, \sigma_1, \dots, \sigma_{k-1}]$ tels que $E_q(x, \sigma_0 \sigma_1 \dots \sigma_{k-1}) = y$.

Le nombre de bits de l'entier $C_q(x, \sigma_0 \dots \sigma_{N-1})$ est proche de l'optimum théorique $\sum_{\sigma \in \mathcal{S}} q(\sigma) \cdot \log_2(N/q(\sigma))$ mais un problème se pose en pratique. Sauf pour de tout petits mots sur de tout petits alphabets, le calcul de $C_q(x, \sigma_0 \dots \sigma_{N-1})$ va nécessiter de manipuler des entiers très grands. Pour éviter ce problème, une solution consiste, avant chaque appel à E_q , à se débarrasser d'un certain nombre k_i de bits de poids faible. Plus précisément, on construit les suites (x_n) et (y_n) suivantes :

$$\begin{cases} y_i &= \lfloor x_i / 2^{k_i} \rfloor \\ x_{i+1} &= E_q(y_i, \sigma_i) \end{cases}$$

Comme précédemment, la valeur de x_0 est fixée arbitrairement. Le mot $\sigma_0 \dots \sigma_{N-1}$ peut alors être représenté par d'une part x_N et d'autre part la concaténation des séquences de bits suivantes, les bits de poids forts apparaissant en premier :

$$\underbrace{x_{N-1} \bmod 2^{k_{N-1}}}_{k_{N-1} \text{ bits}}; \quad \dots; \quad \underbrace{x_2 \bmod 2^{k_2}}_{k_2 \text{ bits}}; \quad \underbrace{x_1 \bmod 2^{k_1}}_{k_1 \text{ bits}}.$$

Pour chaque i en ordre décroissant, le décompresseur déduit de x_{i+1} les valeurs de y_i et σ_i (par exemple avec `decomp2`, question **VII.2**, avec $k = 1$), puis il lit k_i bits dans le mot compressé et les concatène à y_i pour obtenir x_i , et ainsi de suite jusqu'à avoir décompressé tout le mot.

Dans la suite, on supposera que N est une puissance de deux : $N = 2^K$. On supposera par ailleurs que le processeur n'est efficace que pour des entiers ne dépassant pas 2^B pour un certain B bien plus grand que K . En particulier, $x_{i+1} = E_q(y_i, \sigma_i)$ ne doit jamais atteindre cette borne 2^B lors de la compression.

Pour que le taux de compression se rapproche de l'optimum théorique, il faut que chaque k_i soit le plus petit possible (idéalement 0) et donc que chaque y_i soit le plus grand possible. Par ailleurs, les différents k_i ne font pas partie du mot compressé. Autrement dit, en ne connaissant que y_i , le décompresseur doit pouvoir deviner le nombre k_i de bits qui doivent être lus dans le mot compressé pour reconstruire x_i . Une solution correcte mais mauvaise serait, par exemple, de fixer tous les k_i à la constante K ; le décompresseur pourrait alors deviner trivialement les k_i mais le mot résultant ne serait absolument pas compressé.

Question VII.3. Proposez une façon pour le compresseur de choisir k_i en fonction de x_i et σ_i . Expliquez comment le décompresseur choisit k_i en fonction de y_i et prouvez que ce choix correspond bien à celui du compresseur. Si cela a une importance, expliquez comment le compresseur doit choisir x_0 .

Remarque : la contrainte imposant que N soit une puissance de deux ne pose aucune difficulté en pratique. En effet, en matière de compression entropique, la seule chose qui importe est que $q(\sigma)/N$ soit proche de la proportion de la lettre σ dans le mot à compresser.

Fin du sujet.

ECOLES NORMALES SUPERIEURES

CONCOURS D'ADMISSION 2023

**VENDREDI 21 AVRIL 2023
14h00 - 18h00**

FILIERES MP et MPI

Epreuve n° 10

INFO-FONDAMENTALE (ULSR)

Durée : 4 heures

***L'utilisation des calculatrices n'est pas
autorisée pour cette épreuve***

Épreuve d'informatique fondamentale

Concision et ambiguïté

Le sujet porte sur les automates finis, dont la définition est rappelée dans le préambule. Le sujet s'intéresse à la propriété d'ambiguïté des automates finis, propriété également définie dans le préambule. La première partie lie les notions de déterminisme et d'ambiguïté d'un automate fini en utilisant la notion de miroir d'un automate. La deuxième partie amène à définir un algorithme qui teste si un automate est ambigu et vous demande de déterminer sa complexité asymptotique. La troisième et la quatrième partie étudient la concision des automates non ambigus et ambigus.

Les parties 1 et 2 sont indépendantes ; il est conseillé de les traiter en premier car les parties 3 et 4 en dépendent. Il est permis d'admettre les réponses à certaines questions pour répondre aux suivantes.

Préambule

On note Σ un *alphabet* fini, c'est à dire un ensemble fini de symboles appelés *lettres*. Un *mot* sur Σ est une suite finie de lettres. On note Σ^n l'ensemble des mots de longueur n sur Σ et Σ^* l'ensemble de tous les mots sur Σ . Soient u, v deux mots sur Σ , on note $u \cdot v$ la *concaténation* de u et v . Un *langage* sur Σ est un sous-ensemble de Σ^* .

Un automate sur Σ est un tuple $\mathcal{A} = (Q, T, I, F)$ où :

- Q est un ensemble fini de symboles appelés *états* ;
- $T \subseteq Q \times \Sigma \times Q$ est appelé ensemble des *transitions* ;
- $I \subseteq Q$ est l'ensemble des états *initiaux* ;
- $F \subseteq Q$ est l'ensemble des états *finaux*.

La Figure 1 représente graphiquement trois automates. Les symboles dans Q sont encerclés, avec deux cercles pour les symboles dans F . Une transition $(q, a, q') \in T$ est représentée par une flèche étiquetée par $a \in \Sigma$, allant de l'état source q à l'état destination q' . Les états initiaux sont indiqués par une flèche sans état source.

Un calcul de $\mathcal{A}$ sur un mot $w = a_0 \dots a_{n-1} \in \Sigma^*$ est une suite finie d'états $q_0 \dots q_n \in Q^*$ telle que $q_0 \in I$ et pour tout $i < n$, $(q_i, a_i, q_{i+1}) \in T$. Un tel calcul est dit *acceptant* si $q_n \in F$. On dit alors que $\mathcal{A}$ accepte w . Le langage de $\mathcal{A}$, noté $L(\mathcal{A})$, est l'ensemble des mots acceptés par $\mathcal{A}$.

Un automate $\mathcal{A}$ est dit *déterministe* si $|I| \leq 1$ et pour tout $(q, a) \in Q \times \Sigma$, $|\{q' \mid (q, a, q') \in T\}| \leq 1$.

Un automate $\mathcal{A}$ est dit *complet* si $|I| \geq 1$ et pour tout $(q, a) \in Q \times \Sigma$, $|\{q' \mid (q, a, q') \in T\}| \geq 1$.

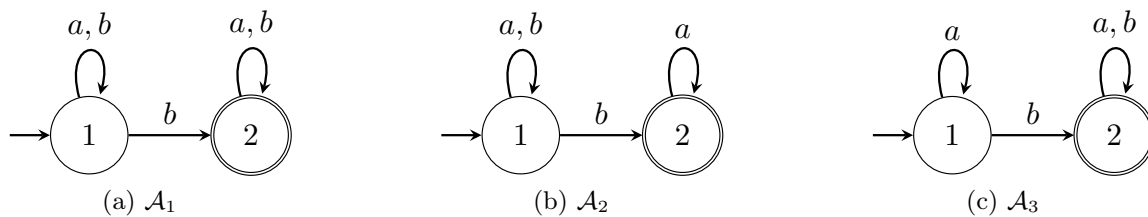


FIGURE 1 – Trois automates reconnaissant le même langage

Soient $w \in \Sigma^*$ et $\mathcal{A}$ un automate. Le mot w est dit *ambigu* pour $\mathcal{A}$ s'il existe deux calculs acceptants ρ et ρ' de $\mathcal{A}$ sur w avec $\rho \neq \rho'$. On définit $d_{\mathcal{A}}(w)$, le *degré d'ambiguïté* de w dans $\mathcal{A}$, comme étant le nombre de calculs acceptants différents de $\mathcal{A}$ sur w . Ainsi, w est ambigu pour $\mathcal{A}$ si et seulement si $d_{\mathcal{A}}(w) > 1$. On note $\text{Amb}(\mathcal{A})$ l'ensemble des mots ambigus pour $\mathcal{A}$. L'automate $\mathcal{A}$ est dit *ambigu* si $\text{Amb}(\mathcal{A}) \neq \emptyset$.

Ce sujet s'intéresse tout particulièrement aux automates *non ambigus*.

1 Déterminisme et ambiguïté

Question 1.1 Les trois automates de la Figure 1 acceptent le même langage. Donnez-en, sans justification, une description intuitive.

Question 1.2 Pour chacun des automates de la Figure 1, dites s'il est déterministe ou non déterministe. Justifiez vos affirmations.

Question 1.3 Pour l'automate $\mathcal{A}_1$ de la Figure 1, calculez $\text{Amb}(\mathcal{A}_1)$ et déduisez en si $\mathcal{A}_1$ est ambigu ou non. Faites de même pour les automates $\mathcal{A}_2$ et $\mathcal{A}_3$.

Soit $\mathcal{A}$ un automate. On note $\tilde{\mathcal{A}} = (\tilde{Q}, \tilde{T}, \tilde{I}, \tilde{F})$ l'*automate miroir* de $\mathcal{A}$, défini par :

- $\tilde{Q} = Q$;
- $\tilde{I} = F$;
- $\tilde{F} = I$;
- $\tilde{T} = \{(t, a, s) \mid (s, a, t) \in T\}$.

Un automate est dit *co-déterministe* si son automate miroir est déterministe.

Question 1.4 Soit $\mathcal{A}$ un automate, $w \in L(\mathcal{A})$ un mot accepté par $\mathcal{A}$ et $q_0 \dots q_n$ un calcul acceptant de $\mathcal{A}$ sur w . Montrez qu'il existe un mot $\tilde{w}$ tel que $q_n \dots q_0$ soit un calcul acceptant de $\tilde{\mathcal{A}}$ sur $\tilde{w}$.

Question 1.5 Montrez qu'un automate $\mathcal{A}$ est ambigu si et seulement si $\tilde{\mathcal{A}}$ est ambigu.

Question 1.6 Montrez que si un automate $\mathcal{A}$ est déterministe, alors $\mathcal{A}$ n'est pas ambigu.

Question 1.7 Montrez que si un automate $\mathcal{A}$ est co-déterministe, alors $\mathcal{A}$ n'est pas ambigu.

Question 1.8 Pour chacune des questions suivantes, donnez un automate $\mathcal{A}$ ayant au plus 4 états et respectant les propriétés demandées. Justifiez vos réponses.

- (i) $\mathcal{A}$ est non ambigu mais ni déterministe, ni co-déterministe ;
- (ii) $L(\mathcal{A})$ est infini et $\text{Amb}(\mathcal{A}) = L(\mathcal{A})$;
- (iii) $\text{Amb}(\mathcal{A})$ est infini, et $\text{Amb}(\mathcal{A}) \neq L(\mathcal{A})$.

2 Test d'ambiguïté

Le but de cette partie est d'obtenir un algorithme qui teste si un automate est ambigu et de déterminer sa complexité asymptotique.

Dans cette partie, $\mathcal{A}$ est un automate (Q, T, I, F) tel que $L(\mathcal{A}) \neq \emptyset$.

2.1 Une construction utile

En utilisant $\mathcal{A}$, on définit l'automate $\hat{\mathcal{A}} = (\hat{Q}, \hat{T}, \hat{I}, \hat{F})$ comme suit :

- $\hat{Q} = Q \times Q \times \{0, 1\}$;
- $\hat{I} = \{(i, i, 0) \mid i \in I\} \cup \{(i, i', 1) \mid i \in I, i' \in I, i \neq i'\}$;
- $\hat{F} = \{(f, f', 1) \mid f \in F, f' \in F\}$;
- $\hat{T} = T_1 \cup T_2 \cup T_3$ avec :
 - $T_1 = \{((s, s, 0), a, (t, t, 0)) \mid (s, a, t) \in T\}$;
 - $T_2 = \{((s, s, 0), a, (t, t', 1)) \mid (s, a, t) \in T, (s, a, t') \in T, t \neq t'\}$;
 - $T_3 = \{((s, s', 1), a, (t, t', 1)) \mid (s, a, t) \in T, (s', a, t') \in T\}$.

Question 2.1 Soit $\mathcal{A}_1$ le premier automate de la figure Figure 1. Construisez l'automate $\hat{\mathcal{A}}_1$. Il est inutile de faire figurer les états qui ne sont pas accessibles à partir d'un état initial. Donnez, sans justification, le langage $L(\hat{\mathcal{A}}_1)$.

Question 2.2 Soient $w \in \Sigma^*$ un mot et $\rho = (q_0, q'_0, b_0) \dots (q_n, q'_n, b_n)$ un calcul de $\hat{\mathcal{A}}$ sur w . On pose $\mu = q_0 \dots q_n$ et $\mu' = q'_0 \dots q'_n$.

- (i) Montrez que μ et μ' sont des calculs de $\mathcal{A}$ sur w .
- (ii) Montrez que $b_n = 0$ si et seulement si $\mu = \mu'$.
- (iii) Montrez que, si ρ est acceptant, alors μ et μ' sont acceptants.
- (iv) Montrez qu'il existe un calcul ρ tel que μ et μ' sont acceptants mais ρ ne l'est pas.

Question 2.3 Montrez que $L(\hat{\mathcal{A}}) = \text{Amb}(\mathcal{A})$.

2.2 L'algorithme

Pour deux fonctions $f, g : \mathbb{N} \rightarrow \mathbb{N}$, on dit que g est une borne asymptotique de f , et on le note par $f \in \mathcal{O}(g)$, s'il existe deux constantes strictement positives $n_0 \in \mathbb{N}$ et $c \in \mathbb{N}$ telles que pour tout $n \geq n_0$, $f(n) \leq c \times g(n)$. Cette définition se généralise naturellement à des fonctions f et g avec plusieurs paramètres.

Question 2.4 Pour chaque ensemble $\widehat{Q}$, $\widehat{T}$, $\widehat{I}$ et $\widehat{F}$, donnez une borne asymptotique au nombre d'éléments qu'il contient en fonction des tailles de Q , T , I et F .

Question 2.5 Donnez un algorithme qui a comme entrée $\mathcal{A}$ et comme sortie $\widehat{\mathcal{A}}$ en précisant :

- (i) quelle structure de données classique (matrice d'adjacence ou liste d'adjacence) est utilisée pour représenter $\mathcal{A}$ et $\widehat{\mathcal{A}}$,
- (ii) une borne asymptotique de la complexité en temps d'exécution de cet algorithme en fonction de la somme des tailles des ensembles Q , T , I et F .

Question 2.6 Décrivez une méthode permettant de tester si $\mathcal{A}$ est ambigu en utilisant l'automate $\widehat{\mathcal{A}}$. Donnez une borne asymptotique de la complexité en temps de cette méthode en fonction de la somme des tailles des ensembles Q , T , I et F .

2.3 Généralisation

Soit k un entier strictement positif.

Question 2.7 Soit $w \in L(\mathcal{A})$ un mot de longueur k .

- (i) Donnez, en fonction de k et $|Q|$, une borne supérieure sur le degré d'ambiguïté de w dans $\mathcal{A}$.
- (ii) Donnez un automate $\mathcal{A}$ et un mot de longueur k pour lesquels la borne supérieure indiquée ci-dessus est atteinte.

Question 2.8 On pose $\text{Amb}_{\geq k}(\mathcal{A}) = \{w \in L(\mathcal{A}) \mid d_{\mathcal{A}}(w) \geq k\}$. Montrez que $\text{Amb}_{\geq k}(\mathcal{A})$ est régulier.

Question 2.9 On pose $\text{Amb}_k(\mathcal{A}) = \{w \in L(\mathcal{A}) \mid d_{\mathcal{A}}(w) = k\}$. Montrez que $\text{Amb}_k(\mathcal{A})$ est régulier.

3 Concision des automates non ambigus

Le but de cette partie est de prouver que les automates non ambigus peuvent être exponentiellement plus concis que leurs équivalents déterministes et complets.

Dans toute cette partie, on fixe l'alphabet $\Sigma = \{a, b\}$. Pour $n \geq 1$ un entier, on pose $L_n = \{w_1 \cdot b \cdot w_2 \mid w_1, w_2 \in \Sigma^*, |w_2| = n - 1\}$, le langage des mots dont la n -ième lettre en partant de la fin est un b .

Question 3.1 *Donnez un automate non ambigu acceptant L_3 , puis donnez un automate déterministe et complet acceptant L_3 .*

Question 3.2 *Montrez que pour tout $n \geq 1$ entier, il existe un automate non ambigu $\mathcal{A}$ avec $n + 1$ états tel que $L(\mathcal{A}) = L_n$.*

Question 3.3 *Montrez que pour tout $n \geq 1$ entier, il existe un automate déterministe et complet $\mathcal{B}$ avec 2^n états tel que $L(\mathcal{B}) = L_n$.*

On veut à présent prouver que tout automate déterministe et complet acceptant L_n a au moins 2^n états.

Question 3.4 *Soit $\mathcal{B}$ un automate déterministe et complet. Soit $w \in \Sigma^*$. Montrez qu'il existe un unique calcul, non nécessairement acceptant, de $\mathcal{B}$ sur w .*

Soit $\mathcal{B}$ un automate déterministe et complet et $w \in \Sigma^*$. On note q_w l'état atteint par l'unique calcul de $\mathcal{B}$ sur w , c'est-à-dire l'état q_m tel que $q_0 \dots q_m$ est un calcul de $\mathcal{B}$ sur w .

Question 3.5 *Soit $n \in \mathbb{N}$ et $\mathcal{B}$ un automate déterministe et complet reconnaissant L_n . Soient w et w' deux mots de Σ^* de longueur n . Montrez que $q_w = q_{w'}$ si et seulement si $w = w'$.*

Question 3.6 *Soit $n \in \mathbb{N}$. Montrez que tout automate déterministe et complet reconnaissant L_n a au moins 2^n états.*

4 Concision des automates ambigus

Le but de cette partie est de prouver que les automates ambigus peuvent être exponentiellement plus concis que leurs équivalents non ambigus.

Pour $n \in \mathbb{N}$, on pose Σ_n un alphabet à n lettres et K_n le langage des mots sur Σ_n^* dont au moins une lettre apparaît au moins deux fois, c'est-à-dire :

$$K_n = \{w_1 \cdot x \cdot w_2 \cdot x \cdot w_3 \mid w_1, w_2, w_3 \in \Sigma_n^*, x \in \Sigma_n\}$$

Question 4.1 Pour $\Sigma_3 = \{a, b, c\}$,

- (i) donnez un automate acceptant K_3 et ayant au plus 5 états,
- (ii) donnez un automate non ambigu acceptant K_3 et ayant au plus 9 états.

Question 4.2 Montrez que pour tout $n \in \mathbb{N}$, il existe un automate avec au plus $n + 2$ états acceptant K_n .

Question 4.3 Montrez que pour tout $n \in \mathbb{N}$, il existe un automate non ambigu avec au plus $2^n + 1$ états acceptant K_n .

On veut à présent prouver que tout automate non ambigu acceptant K_n a au moins $2^n + 1$ états. Jusqu'à la fin de cette partie, on fixe un entier $n \geq 2$ et un automate non ambigu $\mathcal{A} = (Q, T, I, F)$ acceptant K_n .

Soient u et v deux mots de Σ_n^* . Lorsque $u \cdot v \in K_n$, on note $q_{u,v}$ l'état de $\mathcal{A}$ atteint après avoir lu u lors de l'unique calcul acceptant de $\mathcal{A}$ sur $u \cdot v$. Autrement dit, soit $\ell = |u|$ et $q_0 \dots q_m$ l'unique calcul acceptant de $\mathcal{A}$ sur $u \cdot v$, alors $q_{u,v} = q_\ell$.

Question 4.4 Soient u, u' et v, v' quatre mots de Σ_n^* tels que $u \cdot v \in K_n$ et $u' \cdot v' \in K_n$. On suppose que $q_{u,v} = q_{u',v'}$.

- (i) Montrez que $u \cdot v' \in K_n$ et $u' \cdot v \in K_n$.
- (ii) Montrez que $q_{u,v} = q_{u,v'} = q_{u',v} = q_{u',v'}$.

L'objectif à présent est de prouver que ces contraintes garantissent que $\mathcal{A}$ a au moins 2^n états.

Soit $s = (s_i)$ une suite de mots et α une lettre. On note $s \cdot \alpha$ la suite $(s_i \cdot \alpha)$, c'est-à-dire la suite obtenue en ajoutant α à la fin de chaque mot de s .

On fixe un ordre total $<$ sur Σ_n , et on note $\Sigma_n = \{\alpha_1, \dots, \alpha_n\}$ avec pour tout $1 \leq i < j \leq n$, $\alpha_i < \alpha_j$. On construit une famille $s^0, \dots, s^n$ de suites de mots de Σ_n comme suit :

- s^0 est la suite contenant uniquement ε ;

— Pour $0 < i \leq n$, $s^i = s^{i-1} \cdot s^{i-1} \cdot \alpha_i$, la suite constituée de s^{i-1} , suivie de la suite $s^{i-1} \cdot \alpha_i$.

Finalement, on pose $s = s^n$. Par exemple, pour $n = 2$, $\Sigma_2 = \{a, b\}$ et $a < b$, la suite s contient 4 éléments : $s_0 = \varepsilon$, $s_1 = a$, $s_2 = b$, $s_3 = ab$.

Finalement, on remarque que s contient 2^n éléments et on pose M_n la matrice de dimension $2^n \times 2^n$ à coefficients en $\mathbb{Z}$ définie par :

$$M_n[i, j] = \begin{cases} 1 & \text{si } s_i \text{ et } s_j \text{ ont une lettre en commun} \\ 0 & \text{sinon} \end{cases}$$

Par exemple, M_2 est donnée ci-dessous :

$$M_2 = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 \end{pmatrix}$$

Question 4.5 Montrez que M_n peut s'écrire sous la forme :

$$M_n = \begin{pmatrix} M_{n-1} & M_{n-1} \\ M_{n-1} & \mathbf{1}_{n-1} \end{pmatrix}$$

où, pour $n \geq 1$, $\mathbf{1}_n$ est la matrice carrée de dimension $2^n \times 2^n$ dont tous les coefficients valent 1.

Question 4.6 Soient $i < 2^n$ et $j < 2^n$. Montrez que $M_n[i, j] = 1$ si et seulement si $s_i \cdot s_j \in K_n$.

À chaque état q de $\mathcal{A}$, on associe le vecteur colonne v_q de dimension 2^n défini par :

$$v_q[i] = \begin{cases} 1 & \text{s'il existe } j < 2^n \text{ tel que } q = q_{s_i, s_j} \\ 0 & \text{sinon} \end{cases}$$

Question 4.7 Montrez que l'ensemble de vecteurs $(v_q)_{q \in Q}$ est une famille génératrice du sous-espace vectoriel de $\mathbb{Z}^n$ engendré par les vecteurs colonnes de M_n .

Indication : Soit u_j la j -ème colonne de M_n . On pourra montrer que u_j est une combinaison linéaire de vecteurs de $(v_q)_{q \in Q}$ en prouvant que :

$$u_j = \sum_{\substack{q \in Q \\ \exists i, q = q_{s_i, s_j}}} v_q$$

Question 4.8 *Montrez que M_n est de rang $2^n - 1$.*

Question 4.9 *En déduire que $\mathcal{A}$ contient au moins $2^n - 1$ états q tels que $v_q \neq 0$.*

Question 4.10 *Montrez que si $v_q \neq 0$, alors $q \notin I$ et $q \notin F$.*

Question 4.11 *Conclure que $\mathcal{A}$ a au moins $2^n + 1$ états.*

A2023 – INFO MPI I

**ÉCOLE DES PONTS PARISTECH,
ISAE-SUPAERO, ENSTA PARIS,
TÉLÉCOM PARIS, MINES PARIS,
MINES SAINT-ÉTIENNE, MINES NANCY,
IMT ATLANTIQUE, ENSAE PARIS,
CHIMIE PARISTECH - PSL.**

Concours Mines-Télécom

CONCOURS 2023

PREMIÈRE ÉPREUVE D'INFORMATIQUE

Durée de l'épreuve : 3 heures

L'usage de la calculatrice et de tout dispositif électronique est interdit.

*Les candidats sont priés de mentionner de façon apparente
sur la première page de la copie :*

INFORMATIQUE I - MPI

L'énoncé de cette épreuve comporte 9 pages de texte.

Cette épreuve concerne uniquement les candidats de la filière MPI.

*Si, au cours de l'épreuve, un candidat repère ce qui lui semble être une erreur d'énoncé,
il le signale sur sa copie et poursuit sa composition en expliquant les raisons
des initiatives qu'il est amené à prendre.*

Les sujets sont la propriété du GIP CCMP. Ils sont publiés sous les termes de la licence
Creative Commons Attribution - Pas d'Utilisation Commerciale - Pas de Modification 3.0 France.
Tout autre usage est soumis à une autorisation préalable du Concours commun Mines Ponts.



Préliminaires

Présentation du sujet

L'épreuve est composée d'un problème unique comportant 32 questions divisées en trois sections. L'objectif du problème est de construire des *listes à accès direct* : une liste à accès direct est un type de donnée abstrait qui permet, d'une part, d'empiler et de dépiler efficacement un élément en tête de liste et, d'autre part, d'accéder efficacement au k^{e} élément de la liste, pour n'importe quel indice k valide.

Dans la première section (page 1), nous étudions un système de numération : la représentation binaire gauche des entiers naturels. Dans la deuxième section (page 4), nous étudions les arbres binaires parfaits. Dans la troisième section (page 6), nous implémentons le type liste à accès direct par la structure de données concrète *liste gauche*, que nous construisons en exploitant les résultats obtenus dans les sections précédentes.

Dans tout l'énoncé, un même identificateur écrit dans deux polices de caractère différentes désignera la même entité, mais du point de vue mathématique avec la police en italique (par exemple n) et du point de vue informatique avec celle en romain avec espacement fixe (par exemple `n`).

Travail attendu

Pour répondre à une question, il est permis de réutiliser le résultat d'une question antérieure, même sans avoir réussi à établir ce résultat. Des rappels de programmation sont faits en annexe et peuvent être utilisés directement.

Selon les questions, il faudra coder des fonctions à l'aide du langage de programmation C exclusivement, en reprenant le prototype de fonction fourni par le sujet, ou en pseudo-code (c-à-d. dans une syntaxe souple mais conforme aux possibilités offertes par le langage C). Il est inutile de rappeler que les entêtes `<assert.h>`, `<stdbool.h>`, etc. doivent être inclus.

Quand l'énoncé demande de coder une fonction, sauf indication explicite de l'énoncé, il n'est pas nécessaire de justifier que celle-ci est correcte ou de tester que des préconditions sont satisfaites. On suppose que le type `int` n'est jamais sujet à des débordements.

Le barème tient compte de la clarté des programmes : nous recommandons de choisir des noms de variables intelligibles ou encore de structurer de longs codes par des blocs ou par des fonctions auxiliaires dont on décrit le rôle. Lorsqu'une réponse en pseudo-code est permise, seule la logique de programmation est évaluée, même dans le cas où un code en C a été fourni en guise de réponse.

1. Représentation binaire gauche des entiers naturels

1.1. Mise en place

Soient m un entier naturel et N un entier naturel non nul. Il est classique d'appeler *représentation binaire standard de l'entier m sur N chiffres*, ou plus simplement *représentation standard*, toute suite finie $b = (b_n)_{0 \leq n < N}$ de longueur N telle que, pour tout indice n compris entre 0 et $N - 1$, le chiffre b_n appartient à $\{0, 1\}$ et l'égalité suivante est vérifiée

$$m = \sum_{n=0}^{N-1} b_n 2^n.$$

Première épreuve d'informatique MPI 2023

Définition : Nous appelons *représentation binaire gauche de l'entier m sur N chiffres* toute suite finie $g = (g_n)_{0 \leq n < N}$ de longueur N telle que,

- (i) pour tout indice n compris entre 0 et $N - 1$, le chiffre g_n appartient à $\{0, 1, 2\}$,
- (ii) l'égalité suivante est satisfaite

$$m = \sum_{n=0}^{N-1} g_n(2^{n+1} - 1),$$

- (iii) le chiffre « 2 » n'apparaît qu'au plus une fois parmi les chiffres $(g_n)_{0 \leq n < N}$,
- (iv) s'il existe une position p tel que le chiffre g_p est 2, alors, pour tout indice n compris entre 0 et $p - 1$, le chiffre g_n est nul.

De manière plus courte, nous parlons simplement de *représentation gauche*.

La figure 1 ci-dessous donne une représentation standard et une représentation gauche sur quatre chiffres des seize premiers entiers. Conformément à l'usage habituel, nous écrivons toute représentation, qu'elle soit standard ou gauche, sous la forme d'un mot $b_{N-1} \cdots b_0$ ou $g_{N-1} \cdots g_0$ dans lequel les chiffres de poids faibles sont écrits à droite.

Entier	Repr. standard	Repr. gauche	Entier	Repr. standard	Repr. gauche
0	0000	0000	8	1000	0101
1	0001	0001	9	1001	0102
2	0010	0002	10	1010	0110
3	0011	0010	11	1011	0111
4	0100	0011	12	1100	0112
5	0101	0012	13	1101	0120
6	0110	0020	14	1110	0200
7	0111	0100	15	1111	1000

FIGURE 1 – Représentations standard et gauche des seize premiers entiers

□ 1 – Soit c un entier. Donner la représentation standard de l'entier dont une représentation gauche est $10 \cdots 0$ (avec c chiffres nuls) en justifiant sommairement. Faire de même avec l'entier dont une représentation gauche est $20 \cdots 0$ (avec c chiffres nuls).

□ 2 – Déterminer, en justifiant, le plus grand entier naturel M_N qui admet une représentation gauche sur N chiffres. Préciser la représentation gauche de cet entier.

Définition : Soit n_0 un indice compris entre 0 et $N - 1$. On dit que l'indice n_0 est la *position du chiffre de plus fort poids* d'une représentation $g_{N-1} \cdots g_0$ si l'indice n_0 est le plus petit entier tel que, pour tout indice $n > n_0$, le chiffre g_n est nul. On appelle le chiffre g_{n_0} le *chiffre de plus fort poids*.

□ 3 – Soient N un entier naturel non nul, $g = g_{N-1} \cdots g_0$ et $h = h_{N-1} \cdots h_0$ deux représentations gauches d'un même entier m . Démontrer que les chiffres de plus fort poids de g et de h sont de même valeur et à la même position.

□ 4 – Démontrer que tout entier appartenant à l'intervalle $\llbracket 0, M_N \rrbracket$, où l'entier M_N a été introduit à la question 2, ne possède au plus qu'une seule représentation gauche sur N chiffres.

Première épreuve d'informatique MPI 2023

Indication C : L'entier N est déclaré comme constante globale. Nous utilisons la structure C déclarée comme suit pour écrire la représentation gauche sur N chiffres d'un entier $g_{N-1} \cdots g_0$:

```

1.  const int N = 8;
2.
3.  struct RepGauche {
4.      int position;
5.      bool chiffres[N];
6.  };
7.  typedef struct RepGauche rg;

```

Le champ `position` repère la position éventuelle du chiffre 2; il vaut -1 au cas où le chiffre 2 n'apparaît pas. Pour tout indice n compris entre 0 et $N - 1$, la n^e case du champ `chiffres` vaut `true` si le chiffre g_n est non-nul et vaut `false` sinon.

Par exemple, les entiers 15 et 21 ont pour représentation gauche les variables `entier_15` et `entier_21` suivantes :

```

8.  rg entier_15 = { .position = -1,
9.                  .chiffres = { 0, 0, 0, 1, 0, 0, 0, 0 } };
10. rg entier_21 = { .position = 1,
11.                 .chiffres = { 0, 1, 0, 1, 0, 0, 0, 0 } };

```

□ 5 – Formaliser sous la forme d'un ou de plusieurs invariants le fait qu'une valeur C de type **rg** est la représentation gauche d'un entier.

□ 6 – Écrire une fonction C **int** `rg_to_int(rg g)`, qui renvoie l'entier dont g est la représentation gauche. On supposera que l'invariant de la question 5 est satisfait.

1.2. Incrémentation et décrémentation

Nous proposons l'algorithme suivant :

ALGORITHME MYSTÈRE :

Entrée : Représentation gauche $g = g_{N-1} \cdots g_0$ d'un certain entier m .

Effet :

- Si aucun des chiffres $(g_n)_{0 \leq n < N}$ ne vaut 2, changer le chiffre g_0 en $g_0 + 1$.
- Sinon, en notant p la position du chiffre 2, changer le chiffre g_p en 0 et le chiffre g_{p+1} en $g_{p+1} + 1$.

Nous notons m' l'entier dont la représentation gauche est g après exécution de l'algorithme.

FIGURE 2 – Un algorithme

□ 7 – Vérifier que l'invariant de la question 5 n'est pas rompu par l'algorithme mystère (cf. figure 2). Avec les notations m et m' de la figure 2, caractériser, en fonction de l'entier m , la valeur de l'entier m' .

Première épreuve d'informatique MPI 2023

- 8 – Écrire une fonction C `bool rg_incr(rg *s)` dont la spécification suit :
- Précondition* : La variable s est un pointeur vers la représentation gauche d'un entier m .
- Effet* : La valeur pointée par s est modifiée afin de représenter l'entier $m + 1$.
- Valeur de retour* : Booléen `true` si l'incrément de m peut avoir lieu et `false` si un débordement se produit car $m + 1$ n'est pas représentable sur le même nombre de chiffres.
- 9 – Calculer la complexité en temps dans le pire des cas de la fonction `rg_incr` en fonction de N . Comparer avec la complexité de la même opération sur la représentation standard.
- 10 – Écrire une fonction C `bool rg_decr(rg *s)` dont la spécification suit :
- Précondition* : Le pointeur s désigne la représentation gauche d'un entier m .
- Effet* : La valeur pointée par s est modifiée afin de représenter l'entier $m - 1$.
- Valeur de retour* : Booléen `true` si la décrémentation de m peut avoir lieu et `false` si un débordement se produit car m est nul.
- Il est recommandé d'expliquer son intention avant de donner son code.

2. Arbres binaires parfaits

2.1. Opérations sur les arbres binaires

Indication C : Nous représentons des arbres binaires à valeurs entières au moyen du type C `arb` suivant, qui est un pointeur vers une structure contenant la valeur entière dans le champ `valeur` et les deux fils dans les champs `fils_g` et `fils_d`. L'arbre vide se représente par le pointeur `NULL`.

```

12. typedef struct Noeud *arb;
13. struct Noeud {
14.     int valeur;
15.     arb fils_g;
16.     arb fils_d;
17. };

```

L'arbre vide est, par convention, de hauteur -1 .

- 11 – Écrire une fonction C `int hauteur(arb a)` qui calcule la hauteur de l'arbre a .
- 12 – Écrire une fonction C `arb noeud(int v, arb ag, arb ad)` qui construit un arbre dont la racine a pour étiquette v , dont le fils gauche est a_g et le fils droit est a_d . Dans cette question, il est demandé de se défendre, par le truchement d'une assertion, de toute erreur liée à un échec d'allocation dynamique de mémoire.

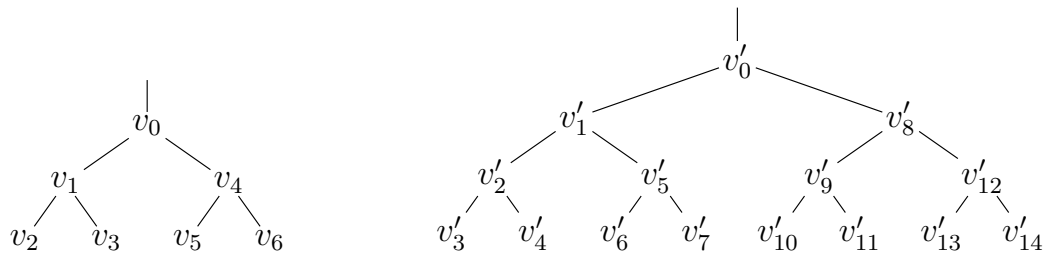


FIGURE 3 – Arbres binaires parfaits de hauteur 2 et de hauteur 3.

2.2. Arbres parfaits

Définition : Un *arbre binaire parfait*, ou simplement *arbre parfait*, est un arbre binaire dont tous les nœuds internes ont exactement deux fils, dont toutes les feuilles sont à la même profondeur et dont les nœuds sont étiquetés par des valeurs entières.

Des exemples d'arbres binaires parfaits sont donnés figure 3.

- 13 – Démontrer que tout arbre binaire parfait de hauteur n possède $2^{n+1} - 1$ nœuds.
- 14 – Écrire une fonction C `bool est_parfait(arb a, int n)` dont la spécification suit :
Précondition : Le pointeur a désigne la racine d'un arbre binaire (dont les nœuds sont bien tous distincts).
Valeur de retour : Booléen `true` si l'arbre pointé est parfait de hauteur n et `false` sinon.
- 15 – Calculer la complexité en temps dans le pire des cas de l'exécution de `est_parfait(a, n)` en fonction de l'entier n .

2.3. Opérations sur les arbres parfaits

Définition : Soient S une structure de données qui permet de stocker une collection d'entiers et t le cardinal de S . Nous disons que la structure de données S est à *accès direct* s'il existe une manière systématique de numéroter chaque élément de S entre 0 et $t - 1$ et s'il existe deux primitives, `acces` et `modif`, qui permettent respectivement de consulter et de modifier n'importe quel élément de S en temps logarithmique en t à partir seulement de son numéro.

Nous souhaitons vérifier qu'un arbre parfait est une structure de données à accès direct. Nous numérotons les éléments d'un arbre parfait en utilisant l'ordre préfixe de gauche à droite de l'arbre (comme illustré dans la figure 3).

- 16 – Écrire une fonction C `arb arb_trouve(arb a, int n, int k)` ainsi spécifiée :
Précondition : Le pointeur a désigne la racine d'un arbre binaire parfait de hauteur n . L'entier k satisfait les inégalités $0 \leq k < 2^{n+1} - 1$.
Valeur de retour : Pointeur vers le k^e nœud de l'arbre dans l'ordre préfixe.

Il est rappelé que la racine d'un arbre est à profondeur 0. Nous donnons la formule de sommation, valable pour tout réel $a \neq 1$ et tout entier t ,

$$\sum_{k=0}^t k \cdot a^k = \frac{((a-1)t-1)a^{t+1} + a}{(a-1)^2}.$$

Première épreuve d'informatique MPI 2023

- 17 – Nous appelons P la profondeur d'un nœud choisi aléatoirement et uniformément parmi les $2^{n+1} - 1$ nœuds d'un arbre binaire parfait de hauteur n . Calculer l'espérance $\mathbb{E}(P)$ de la variable aléatoire P .
- 18 – Déterminer la complexité moyenne en temps de l'instruction `arb_trouve(a, n, k)` lorsque la hauteur n et l'arbre a sont fixés et le numéro k est choisi aléatoirement et uniformément dans l'intervalle $\llbracket 0, 2^{n+1} - 2 \rrbracket$.
- 19 – Écrire une fonction C `int arb_acces(arb a, int n, int k)` qui renvoie la k^{e} valeur de l'arbre a de hauteur n ainsi qu'une fonction C `void arb_modif(arb a, int n, int k, int v)` qui remplace la k^{e} valeur de l'arbre a de hauteur n par la valeur v . Dire finalement si la structure de données arbre binaire parfait est à accès direct.

3. Listes gauches

Les arbres binaires parfaits seuls ne permettent de représenter que des collections d'entiers dont le cardinal est de la forme $2^{n+1} - 1$ où n est entier naturel. Afin de représenter des collections de cardinal quelconque, nous introduisons des suites d'arbres parfaits, aux hauteurs strictement croissantes et savamment choisies.

Définition : Nous appelons *liste binaire gauche de cardinal m sur N arbres* la structure de données constituée de $N + 1$ arbres binaires parfaits $((a_n)_{0 \leq n < N}, e)$ comme suit. Nous décomposons l'entier m selon sa représentation binaire gauche sur N chiffres : $g_{N-1} \dots g_0$. Pour tout indice n compris entre 0 et $N - 1$, si le chiffre g_n n'est pas nul, l'arbre binaire parfait a_n est un arbre de hauteur n , sinon il s'agit de l'arbre vide. Si le chiffre 2 apparaît parmi les chiffres de la représentation gauche de m et si l'indice p est sa position, alors l'arbre e est un arbre binaire parfait de hauteur p , sinon l'arbre e est l'arbre vide. De manière plus courte, nous parlons simplement de *liste gauche*.

Une liste binaire gauche de cardinal m sur N arbres permet de stocker une collection de m éléments entiers. Il apparaît qu'avec N arbres, une liste binaire gauche peut stocker jusqu'à M_N entiers (où l'entier M_N a été introduit à la question 2) et que m est inférieur à M_N .

Indication C : Nous adoptons le type C suivant

```
18. struct ListeGauche {
19.     int hauteur_e;
20.     arb extra;
21.     int nb_arbres;
22.     arb *arbres;
23. };
24. typedef struct ListeGauche lg;
```

Le champ `hauteur_e` contient la hauteur p de l'arbre exceptionnel e . Le champ `extra` désigne la racine de l'arbre exceptionnel e . Le champ `nb_arbres` contient l'entier N . Enfin, le champ `arbres` désigne un tableau de N pointeurs vers les arbres $(a_n)_{0 \leq n < N}$ qui a été alloué dynamiquement.

3.1. Opérations simples sur les listes gauches

□ 20 – Écrire une fonction C `lg lg_init(int N)` qui renvoie une liste gauche de cardinal nul sur N arbres.

□ 21 – Écrire une fonction C `int lg_card(lg l)` qui calcule le cardinal m de la liste gauche ℓ .

Définition : Afin de numéroter l'ensemble des éléments d'une liste gauche $\ell = ((a_n)_{0 \leq n < N}, e)$, nous parcourons d'abord les éléments de l'arbre exceptionnel e dans l'ordre préfixe, puis nous parcourons les éléments des arbres $a_0, a_1, \dots, a_{N-1}$ dans l'ordre préfixe. Les numéros sont attribués aux valeurs rencontrées par ordre de première rencontre.

□ 22 – Écrire une fonction C `arb lg_trouve(lg l, int k)` qui renvoie le k^{e} nœud de la liste gauche ℓ . On supposera que k est un indice valide ($0 \leq k < m$).

□ 23 – Calculer la complexité en temps dans le pire des cas de `lg_trouve l` en fonction de la capacité maximale M_N de la liste gauche ℓ .

□ 24 – Écrire une fonction C `int lg_acces(lg l, int k)` qui renvoie la k^{e} valeur de la liste gauche ℓ ainsi qu'une fonction C `void lg_modif(lg l, int k, int v)` qui remplace la k^{e} valeur de la liste gauche ℓ par la valeur v .

3.2. Ajout et suppression en tête de liste gauche

□ 25 – Soient v une valeur entière et $\ell = ((a_n)_{0 \leq n < N}, e)$ une liste gauche de cardinal m , avec $m < M_N$, qui contient les éléments $v_1, \dots, v_m$ dans cet ordre. Décrire, en fonction de la liste gauche ℓ , la liste gauche $\ell' = ((a'_n)_{0 \leq n < N}, e')$ de cardinal $m + 1$ dont les éléments sont $v, v_1, \dots, v_m$ dans cet ordre. En déduire le principe d'une fonction C `bool lg_empile(int v, lg l)` réalisant l'insertion de la valeur v en tête de la liste gauche ℓ où le booléen résultat vaut `true` si l'ajout a eu lieu et vaut `false` si un débordement de capacité de la liste se produit. On ne demande pas le code complet de cette fonction.

□ 26 – Déterminer la complexité en temps dans le pire des cas de la fonction `lg_empile`.

□ 27 – Donner le principe d'une fonction C `bool lg_depile(int *w, lg l)` réalisant le retrait de l'élément de tête de la liste gauche ℓ et son affectation à l'adresse w . Le booléen résultat vaut `true` si le retrait a eu lieu et vaut `false` si l'opération a échoué en raison d'une liste vide. On ne demande pas le code complet.

□ 28 – Donner la complexité en temps dans le pire des cas de la fonction `lg_depile`.

□ 29 – Discuter la possibilité d'obtenir une complexité plus faible à la question 28, quitte à modifier légèrement la définition du type `lg`.

□ 30 – Soit N un entier et $\ell = ((a_n)_{0 \leq n < N}, e)$ une liste gauche. Discuter la possibilité de modifier en place et avec une faible complexité en temps la liste gauche ℓ de sorte que le nombre d'arbres N devienne $N + 1$ et que les mêmes éléments demeurent dans la liste.

3.3. Utilisation concurrente des listes gauches

Dans cette sous-section, on raisonne sur les algorithmes en supposant que les fils d'exécution peuvent s'entrelacer mais que les instructions d'un même fil s'exécutent dans l'ordre du programme. L'entête `#include <pthread.h>` a été déclarée ; la syntaxe de certaines fonctions s'y rattachant est rappelée en annexe.

□ 31 – Deux fils d'exécution distincts exécutent la fonction `lg_empile` sur la même liste gauche. Montrer qu'une course critique advient de leur exécution concurrente et que la cohérence de ladite liste gauche n'est pas garantie, autrement dit que certains invariants qui caractérisent la bonne formation d'une liste gauche peuvent être violés à l'issue des exécutions.

Nous nous intéressons finalement au *problème des producteurs et des consommateurs* qui s'échangent des entiers au travers d'un tampon, ici constitué par une unique liste gauche ℓ à N arbres, l'entier N étant fixé à l'avance. Les producteurs écrivent dans la liste gauche ℓ en empilant un entier à la fois en tête, à condition que la liste gauche ℓ ne soit pas pleine ; les consommateurs vident la liste gauche ℓ en dépilant l'entier en tête de la liste, à condition que la liste gauche ℓ ne soit pas vide.

Un seul agent peut accéder au tampon à la fois. Lorsqu'un consommateur souhaite supprimer une donnée alors que le tampon est vide, il est mis en attente ; lorsqu'un producteur souhaite écrire une donnée alors que le tampon est plein, il est mis en attente.

□ 32 – Décrire une solution au problème des producteurs et des consommateurs en s'appuyant sur un verrou et sur deux sémaphores. Détailler sous la forme de code C ou bien de pseudo-code ce que font les producteurs et les consommateurs.

* *

*

Les listes binaires gauches, ou *skew binary lists*, ont été inventées par Eugene Myers en 1983.

A. Rappels de programmation en C

L'expression $1 \ll n$ représente le décalage de la valeur 1 sur n bits : elle a pour valeur l'entier 2^n .

Le type **pthread_t** désigne des fils d'exécution.

L'instruction `pthread_create(pthread_t *th_id, NULL, &ma_fonction, void *args)` crée un nouveau fil d'exécution qui appelle la fonction `ma_fonction` sur le ou les arguments désignés par `args` et qui s'exécute simultanément avec le fil d'exécution appelant.

L'instruction `pthread_join(th_id, NULL)` suspend l'exécution du fil d'exécution appelant jusqu'à ce que le fil d'exécution identifié par `th_id` achève son exécution.

Le type **pthread_mutex_t** désigne des verrous.

L'instruction `pthread_mutex_lock(&v)` verrouille le verrou `v`.

L'instruction `pthread_mutex_unlock(&v)` déverrouille le verrou `v`.

Le type **sem_t** désigne des sémaphores.

L'instruction `sem_init(&s, 0, v)` initialise le sémaphore `s` à la valeur v (avec $v \geq 0$).

L'instruction `sem_wait(&s)` décrémente le compteur du sémaphore `s` : si le compteur est toujours positif, l'appel se termine ; sinon le fil d'exécution appelant est bloqué.

L'instruction `sem_post(&s)` incrémente le compteur du sémaphore `s` et, si le compteur redevient strictement positif, réveille un fil d'exécution bloqué sur `s`.

FIN DE L'ÉPREUVE

A2023 – INFO MPI II

**ÉCOLE DES PONTS PARISTECH,
ISAE-SUPAERO, ENSTA PARIS,
TÉLÉCOM PARIS, MINES PARIS,
MINES SAINT-ÉTIENNE, MINES NANCY,
IMT ATLANTIQUE, ENSAE PARIS,
CHIMIE PARISTECH - PSL.**

Concours Mines-Télécom

CONCOURS 2023

DEUXIÈME ÉPREUVE D'INFORMATIQUE

Durée de l'épreuve : 4 heures

L'usage de la calculatrice et de tout dispositif électronique est interdit.

*Les candidats sont priés de mentionner de façon apparente
sur la première page de la copie :*

INFORMATIQUE II - MPI

L'énoncé de cette épreuve comporte 12 pages de texte.

Cette épreuve concerne uniquement les candidats de la filière MPI.

*Si, au cours de l'épreuve, un candidat repère ce qui lui semble être une erreur d'énoncé,
il le signale sur sa copie et poursuit sa composition en expliquant les raisons
des initiatives qu'il est amené à prendre.*

Les sujets sont la propriété du GIP CCMP. Ils sont publiés sous les termes de la licence
Creative Commons Attribution - Pas d'Utilisation Commerciale - Pas de Modification 3.0 France.
Tout autre usage est soumis à une autorisation préalable du Concours commun Mines Ponts.



Préliminaires

L'épreuve est composée d'un problème unique, comportant 38 questions. Le problème est divisé en quatre sections qui peuvent être traitées séparément, à condition de lire toutes les définitions de la section 1. Dans la première section (page 1), nous introduisons la *complexité de Kolmogoroff* et étudions des propriétés de calculabilité. Dans la deuxième section (page 3), nous estimons la complexité de Kolmogoroff à l'aide du codage de Huffman. La troisième section (page 4) contient des prolégomènes pour la section suivante. Dans la quatrième section (page 5), nous nous appuyons sur un modèle de calcul épuré, introduit à travers plusieurs langages formels, afin toujours d'estimer la complexité de Kolmogoroff.

Dans tout l'énoncé, un même identificateur écrit dans deux polices de caractère différentes désigne la même entité, mais du point de vue mathématique pour la police en italique (par exemple n , $\mathcal{D}$ ou π) et du point de vue informatique pour celle en romain avec espacement fixe (par exemple `n`, `d` ou `pi`).

Travail attendu

Pour répondre à une question, il est permis de réutiliser le résultat d'une question antérieure, même sans avoir réussi à établir ce résultat.

Il faudra coder des fonctions à l'aide du langage de programmation OCaml exclusivement, en reprenant l'en-tête de fonctions fourni par le sujet, sans s'obliger à recopier la déclaration des types. Il est permis d'utiliser la totalité du langage OCaml mais il est recommandé de s'en tenir aux fonctions les plus courantes afin de rester compréhensible. Des rappels ponctuels de documentation du langage OCaml peuvent être proposés à titre d'aide. Quand l'énoncé demande de coder une fonction, sauf demande explicite de l'énoncé, il n'est pas nécessaire de justifier que celle-ci est correcte ou de tester que des préconditions sont satisfaites.

Le barème tient compte de la clarté des programmes : nous recommandons de choisir des noms de variables intelligibles ou encore de structurer de longs codes par des blocs ou par des fonctions auxiliaires dont on décrit le rôle.

1 Complexité de Kolmogoroff

Nous notons Σ l'ensemble ordonné des 256 caractères ASCII étendus usuels et Σ^* l'ensemble des chaînes de caractères. Pour toute chaîne de caractères $x \in \Sigma^*$, la longueur de x , notée $|x|$, est le nombre de caractères qui la composent. Par exemple, la longueur de la chaîne "abac" est 4. Le nombre d'occurrences d'un symbole $\sigma \in \Sigma$ dans une chaîne de caractères $x \in \Sigma^*$ est noté $|x|_\sigma$. Par exemple, $|abac|_a = 2$.

Dans l'ensemble du sujet, nous nous appuyons sur une *machine universelle*, c'est-à-dire une fonction OCaml `eval`, de type `string -> string`, telle que :

- si la chaîne x , de type `string`, est le code source d'une expression OCaml y de type `string` et si l'exécution du code x se termine sans erreur, alors `eval x` se termine et a pour valeur de retour la valeur de y ;
- sinon, `eval x` ne se termine pas.

Les exécutions de `eval` ont lieu sur une machine idéale dont la mémoire est infinie et qui est capable de gérer des types natifs de taille quelconque.

Seconde épreuve d'informatique MPI 2023

Définition : Pour toute chaîne de caractères $y \in \Sigma^*$, nous disons que la chaîne de caractères $x \in \Sigma^*$ est une *description* de la chaîne y si le calcul `eval x` se termine et renvoie la chaîne y . Nous appelons *complexité de Kolmogoroff* de y et notons $K(y)$ la longueur de la plus courte chaîne de caractères $x \in \Sigma^*$ qui décrit y .

L'objet de ce sujet est d'étudier des propriétés et diverses majorations de la complexité de Kolmogoroff. Dans toutes nos illustrations, nous nous concentrons sur la description de la chaîne de caractères

$$y_0 = "1000000\dots" \in \Sigma^*,$$

qui correspond à l'entier $10^{(10^{10})}$ écrit en base 10 et que nous fixons une fois pour toutes.

1.1 Un premier exemple

□ 1 – Proposer une première majoration de la complexité de Kolmogoroff $K(y_0)$, qui s'appuie sur la chaîne de caractères $x_0 = y_0 = "1000000\dots"$ comme description de y_0 .

Indication OCaml : Il est rappelé que la fonction `string_of_int` convertit un entier en une chaîne de caractères.

□ 2 – Soient n un entier naturel et n' la partie entière de $\frac{n}{2}$. Exprimer la quantité 10^n en fonction de $10^{n'}$. Compléter le code OCaml suivant en utilisant une stratégie « diviser pour régner » :

```
let exp10 n = (* Calcul de 10^n a ecrire *)
in string_of_int (exp10 (exp10 10))
```

afin d'en faire une description de la chaîne de caractères $y_0 = "1000000\dots"$. En déduire une nouvelle borne grossière (à 10^2 près) de la complexité de Kolmogoroff $K(y_0)$, significativement meilleure que celle de la question 1.

□ 3 – Décrire quelle ou quelles difficultés adviendraient si l'on exécutait le code de la question 2 sur une machine réelle.

1.2 Quelques propriétés

Indication OCaml : L'expression `String.make (n : int) (sigma : char) : string` désigne la chaîne de caractères répétant n fois le caractère $\sigma \in \Sigma$. Pour tout entier n compris entre 0 et 255, `Char.chr (n : int) : char` désigne le n^{e} caractère dans la numérotation ASCII.

□ 4 – Présenter une bijection $\varphi : \mathbb{N} \rightarrow \Sigma^*$ entre l'ensemble des entiers naturels et l'ensemble de chaînes de caractères. En écrire le code sous la forme d'une fonction OCaml `phi (n : int) : string`.

Seconde épreuve d'informatique MPI 2023

Nous prétendons avoir écrit une fonction OCaml `kolmogoroff` (`y : string`) : `int` qui calcule la complexité de Kolmogoroff $K(y)$.

□ 5 – Écrire une fonction OCaml `psi` (`m : int`) : `int` dont la valeur de retour est l'entier

$$\psi(m) = \min \{n \in \mathbb{N}; K(\varphi(n)) \geq m\}$$

où $\varphi : \mathbb{N} \rightarrow \Sigma^*$ est la bijection définie à la question 4 et qui utilise la fonction `kolmogoroff`.

□ 6 – Établir d'une part que, pour tout entier naturel m , on a

$$K(\varphi(\psi(m))) \geq m$$

et d'autre part que l'on a

$$K(\varphi(\psi(m))) = O(\log m).$$

Discuter l'existence de la fonction OCaml `kolmogoroff`.

Définition : Nous appelons *décompresseur* toute fonction OCaml `d : string -> string`. Pour toute chaîne de caractères $y \in \Sigma^*$ et pour tout décompresseur $\mathcal{D}$ (noté informatiquement `d`), nous disons que la chaîne de caractères z est une *description* de la chaîne y par rapport à $\mathcal{D}$ si le calcul `eval (d z)` se termine sans erreur et a pour valeur de retour y . Nous appelons *complexité de Kolmogoroff par rapport à $\mathcal{D}$* de la chaîne y , et notons $K_{\mathcal{D}}(y)$, la longueur de la plus courte chaîne de caractères $z \in \Sigma^*$ qui décrit y par rapport à $\mathcal{D}$.

□ 7 – Dire comment se nomme en informatique un programme qui transforme un code source dans un certain langage de programmation en un code équivalent dans un second langage.

□ 8 – Montrer que pour tout décompresseur $\mathcal{D}$, il existe une constante entière $c_{\mathcal{D}}$ telle que, pour toute chaîne de caractères y , nous avons :

$$K(y) \leq K_{\mathcal{D}}(y) + c_{\mathcal{D}}.$$

2 Estimation de la complexité grâce au décompresseur de Huffman

Nous fixons dans cette section la chaîne de caractères $x_0 \in \Sigma^*$ suivante

```
"let rec t e=if e=0 then 1 else let n=e-1 in 10*t n in let n=t 10 in t n".
```

La chaîne x_0 contient 71 caractères, l'espace étant un caractère et les guillemets ne faisant pas partie de la chaîne.

□ 9 – Inférer le type OCaml de l'expression dénotée par la chaîne de caractères x_0 .

Seconde épreuve d'informatique MPI 2023

- 10 – Déterminer la valeur de l'évaluation de x_0 en tant que code source OCaml.
- 11 – Signaler une ou plusieurs caractéristiques du code source x_0 qui rend la lecture de ce code impénétrable par un humain.

Indication OCaml : Nous rappelons le détail de quelques fonctions du module OCaml `Hashtbl` permettant de manipuler des dictionnaires (ou tableaux associatifs) mutables.

- `Hashtbl.create (n : int) : ('a, 'b) Hashtbl.t` crée un dictionnaire vide de taille initiale n .
- `Hashtbl.add (d : ('a, 'b) Hashtbl.t) (k : 'a) (v : 'b) : unit` ajoute une association entre la clé k et la valeur v au dictionnaire d .
- `Hashtbl.find_opt (d : ('a, 'b) Hashtbl.t) (k : 'a) : 'b option` vaut `Some v` si le dictionnaire d possède une association entre la clé k et la valeur v et vaut `None` sinon.

- 12 – Écrire une fonction OCaml `count (x : string) : (char, int) Hashtbl.t` dont la valeur de retour est un dictionnaire qui associe chaque caractère $\sigma \in \Sigma$ présent dans la chaîne x à son nombre d'occurrences $|x|_\sigma$.

Nous exécutons `count x0` et obtenons le dictionnaire suivant :

'n'	't'	'i'	'l'	'1'	'c'	'f'	'h'	'r'	's'	'-'	'*'	'0'	'='	'e'	' '
8	8	4	4	4	1	1	1	1	1	1	1	3	4	10	19

Nous appelons z_0 la chaîne de bits correspondant à la compression de la chaîne x_0 par l'algorithme de Huffman.

- 13 – Dessiner un arbre de Huffman associé à la chaîne de caractères x_0 . Il est recommandé de placer les feuilles de gauche à droite comme dans le tableau ci-dessus.

Nous notons $\mathcal{H}$ le décompresseur qui utilise l'arbre de Huffman de la question 13 pour transformer une chaîne de bits $z \in \{0, 1\}^*$ en un mot $x \in \Sigma^*$ et renvoie la chaîne de caractères `"string_of_int (x)"`.

- 14 – Calculer la longueur $|z_0|$ de la chaîne obtenue après compression de x_0 . En déduire que la complexité de Kolmogoroff de $y_0 = 10000\dots$ par rapport au décompresseur $\mathcal{H}$ vérifie

$$K_{\mathcal{H}}(y_0) \leq 239.$$

3 Interlude

Nous souhaitons écrire une fonction `new_string : unit -> string` ainsi spécifiée : à chaque appel, une chaîne de caractères inédite est produite. Nous offrons trois propositions de code.

Seconde épreuve d'informatique MPI 2023

```

let inc s = s := "a" ^ (!s); !s    (* Commun aux 3 propositions *)

let new_string1 =                  (* Proposition 1 *)
  let seed1 = ref "#" in
  fun () -> inc seed1

let new_string2 () =              (* Proposition 2 *)
  let seed2 = ref "#" in
  inc seed2

let seed3 = ref "#"              (* Proposition 3 *)
let new_string3 () =
  inc seed3

```

- 15 – Analyser la portée de la variable `seed1`, respectivement `seed2` et `seed3`, dans la fonction `new_string1`, respectivement `new_string2` et `new_string3`.
- 16 – Dédurre de la question 15 laquelle des trois fonctions `new_string1`, `new_string2` et `new_string3` ne respecte pas la spécification. Écrire un test qui permet de discriminer la fonction erronée.
- 17 – Dédurre de la question 15 laquelle des deux fonctions correctes restantes est plus propice à des erreurs de manipulation. Expliquer.

4 Estimation de la complexité grâce au décompresseur de De Bruijn

Nous nous avisons que la complexité de Kolmogoroff est influencée par la verbosité et l'expressivité du langage de programmation. Dans cette section, nous tentons de réduire l'encombrement ou la facilité d'écriture dus à la syntaxe du langage OCaml en introduisant un modèle de calcul nouveau et épuré.

4.1 Construction syntaxique d'un langage

Nous présentons systématiquement les *grammaires* sous la forme d'un quadruplet (N, Γ, S, Π) où N désigne l'alphabet des symboles non terminaux, Γ l'alphabet des symboles terminaux, $S \in N$ le symbole initial et Π l'ensemble des règles de production, de la forme $X \rightarrow \gamma$ avec $X \in N$ et $\gamma \in (N \cup \Gamma)^*$.

Une *dérivation immédiate* se note $\alpha \Rightarrow \beta$, avec $(\alpha, \beta) \in ((N \cup \Gamma)^*)^2$, et indique qu'il existe une règle de production $X \rightarrow \gamma$ de Π et des décompositions $\alpha = \alpha_1 X \alpha_2$ et $\beta = \alpha_1 \gamma \alpha_2$, avec $(\alpha_1, \alpha_2) \in ((N \cup \Gamma)^*)^2$. Une *dérivation* se note $\Rightarrow^*$ et indique l'existence d'une suite finie, éventuellement vide, de dérivations immédiates. Enfin, le *langage engendré par une grammaire* $\mathcal{G}$ se note $L(\mathcal{G})$ et désigne l'ensemble des mots de Γ^* qui dérivent du symbole initial S .

Seconde épreuve d'informatique MPI 2023

Selon les questions, nous représentons les mots de Γ par le type `char list` ou le type `string`.

Soit $\mathcal{G}_0$ la grammaire $(\{V\}, \{a, b, \#\}, V, \Pi_0)$ dont les règles de production sont

$$\Pi_0 : \quad V \rightarrow aV \mid bV \mid \#.$$

□ 18 – Exhiber une expression régulière qui dénote le langage $L(\mathcal{G}_0)$.

□ 19 – Dessiner l'automate de Glushkov associé à l'expression régulière de la question 18.

□ 20 – Écrire une fonction OCaml `parseV (w : char list) : string * char list` dont la spécification suit :

Pré-condition : Il existe une décomposition du mot w en $w = vs$ avec $v \in L(\mathcal{G}_0)$ et $s \in \Sigma^*$.

Valeur de retour : Couple (v, s) où le préfixe v est représenté par le type `string` et s est le suffixe restant.

Effet : Une exception `SyntaxError` est levée si la pré-condition n'est pas satisfaite.

Soit $\mathcal{G}_1$ la grammaire $(\{T\}, \{(_, _)\}, T, \Pi_1)$ dont les règles de production sont

$$\Pi_1 : \quad T \rightarrow (TT) \mid _.$$

□ 21 – Montrer que le langage $L(\mathcal{G}_1)$ est sans préfixe, c'est-à-dire qu'il n'existe pas deux mots non vides w et w' dans $L(\mathcal{G}_1)$ tels que w soit un préfixe strict de w' . On pourra, par exemple, raisonner par induction structurale sur le nombre de parenthèses ouvrantes et fermantes qui se trouvent dans un préfixe d'un mot de $L(\mathcal{G}_1)$.

□ 22 – Montrer que la grammaire $\mathcal{G}_1$ n'est pas ambiguë, c'est-à-dire que, pour tout mot du langage $L(\mathcal{G}_1)$, il n'existe qu'un seul arbre d'analyse (parfois aussi appelé arbre de dérivation) associé.

Soit $\mathcal{G}_2$ la grammaire $(\{T\}, \{\text{var}, (_, _), ->\}, T, \Pi_2)$ dont les règles de production sont

$$\Pi_2 : \quad T \rightarrow \text{var} \mid (TT) \mid \text{var} \rightarrow T.$$

□ 23 – Montrer que la grammaire $\mathcal{G}_2$ n'est pas ambiguë.

Soit finalement $\mathcal{G}$ la grammaire $(\{T, V\}, \{a, b, \#, (_, _), ->\}, T, \Pi)$ dont les règles de production sont

$$\Pi : \quad \begin{cases} T \rightarrow V \mid (TT) \mid V \rightarrow T \\ V \rightarrow aV \mid bV \mid \# \end{cases}$$

Nous admettons que la grammaire $\mathcal{G}$ n'est pas ambiguë. Nous appelons *variable* les mots de $L(\mathcal{G}_0)$. Informellement, la grammaire $\mathcal{G}$ engendre un langage $L(\mathcal{G})$ qui permet de parler de *variables*, d'*applications* d'une expression à une autre et de *fonctions*. Il nous reste à en construire une sémantique, ce qui sera fait en section 4.3.

Seconde épreuve d'informatique MPI 2023

Afin de représenter l'*arbre d'analyse* d'un mot du langage $L(\mathcal{G})$, nous introduisons le type `ada` par la déclaration suivante.

```
type ada = V of string | A of (ada * ada) | F of (string * ada)
```

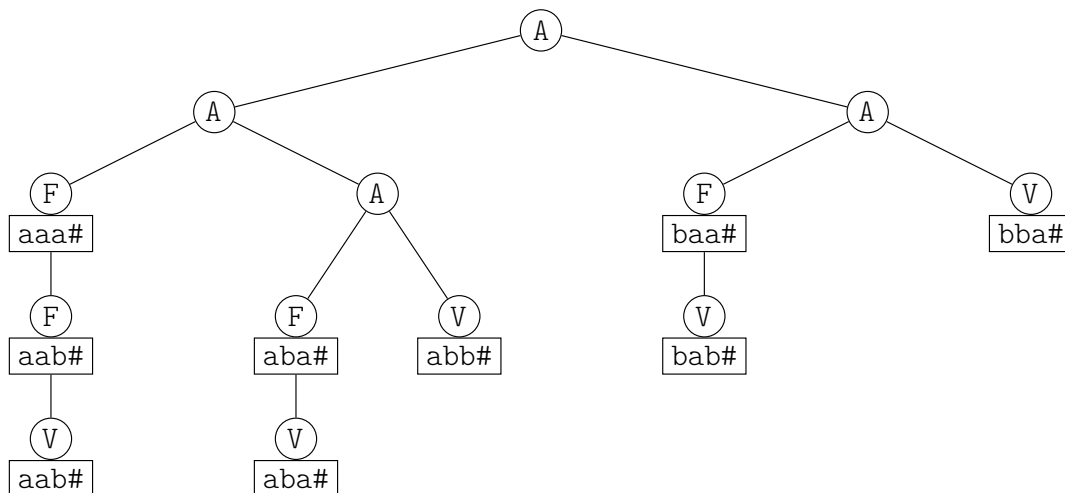
Nous notons $\mathcal{A}$ l'ensemble des arbres d'analyse.

Le constructeur OCaml `V` représente les dérivations immédiates de règle $T \rightarrow V$ et introduit les feuilles ; le constructeur OCaml `A` représente les dérivations immédiates de règle $T \rightarrow (TT)$ et introduit des nœuds internes d'arité 2 ; le constructeur OCaml `F` représente les dérivations immédiates de règle $V \rightarrow T$ et introduit des nœuds internes d'arité 1. Les dérivations à partir du symbole non terminal V sont directement représentées par une valeur OCaml de type `string`.

Par exemple, le mot

$$((aaa\# \rightarrow aab\# \rightarrow aab\#(aba\# \rightarrow aba\#abb\#))(baa\# \rightarrow bab\#bba\#)) \in L(\mathcal{G})$$

admet pour arbre d'analyse :



□ 24 – Écrire une fonction OCaml `parseT (w : char list) : ada * char list` dont la spécification suit :

Pré-condition : Il existe une décomposition du mot w en $w = ps$ avec $p \in L(\mathcal{G})$ et $s \in \Sigma^*$, dans laquelle le préfixe p est de longueur maximale.

Valeur de retour : Couple (a, s) où a est l'arbre d'analyse de p et s est le suffixe restant.

Effet : Une exception `SyntaxError` est levée si la pré-condition n'est pas satisfaite.

Indication OCaml : Nous supposons définie une fonction `charlist_of_string : string -> char list` qui transforme une chaîne de caractères en une liste de caractères.

□ 25 – Écrire une fonction OCaml `parse (w : string) : ada` dont la valeur de retour est l'unique arbre d'analyse de w quand $w \in L(\mathcal{G})$ et qui lève une exception `SyntaxError` sinon.

Seconde épreuve d'informatique MPI 2023

Définition : Pour tout entier naturel $n \in \mathbb{N}$, nous définissons le mot

$$[n] = b\# \rightarrow a\# \rightarrow (b\#(b\# \dots (b\#(b\#a\#)) \dots)) \in L(\mathcal{G})$$

où la variable $b\#$ apparaît n fois à droite des deux symboles $\rightarrow$.

□ 26 – Indiquer s'il existe un automate fini capable de reconnaître le langage $\{[n]; n \in \mathbb{N}\}$ et justifier la réponse.

Définition : Plus généralement, nous disons qu'un mot w de $L(\mathcal{G})$ *incarne* un entier naturel $n \in \mathbb{N}$ s'il existe deux variables distinctes v_1 et v_2 dans $L(\mathcal{G}_0)$ telles que w est de la forme

$$w = v_2 \rightarrow v_1 \rightarrow (v_2(v_2 \dots (v_2(v_2v_1)) \dots)) \in L(\mathcal{G})$$

où la variable v_2 apparaît n fois à droite des deux symboles $\rightarrow$.

□ 27 – Écrire une fonction OCaml `int_of_ada (a : ada) : int` dont la valeur de retour est l'entier naturel n incarné par a . Si un tel entier n'existe pas, une exception `SyntaxError` est levée.

4.2 Réécriture des variables et sérialisation

□ 28 – Nommer une structure de donnée concrète efficace qui réalise le type de donnée abstrait « ensemble » lorsqu'il existe une relation d'ordre entre les objets à ranger.

Indication OCaml : Nous supposons définis un module OCaml `StringSet` permettant de construire des ensembles de chaînes de caractères persistants, de type `StringSet.t`, et des fonctions :

- `StringSet.mem : string -> StringSet.t -> bool` qui teste l'appartenance d'un élément à un ensemble.
- `StringSet.remove : string -> StringSet.t -> StringSet.t` qui retourne un ensemble privé d'un élément.
- `StringSet.singleton : string -> StringSet.t` qui construit un singleton à partir d'un élément.
- `StringSet.union : StringSet.t -> StringSet.t -> StringSet.t` qui construit l'union de deux ensembles.

Définition : L'ensemble des *variables libres* $VL(a) \subseteq \Sigma^*$ d'un arbre d'analyse $a \in \mathcal{A}$ est défini par induction structurale avec les règles d'inférence suivantes :

- si l'arbre d'analyse a est de la forme $V \ v$, où $v \in L(\mathcal{G}_0)$, alors $VL(a)$ est le singleton $\{v\}$;
- si l'arbre d'analyse a est de la forme $A \ (a_1, a_2)$, où a_1 et a_2 sont deux arbres d'analyse, alors $VL(a)$ est la réunion $VL(a_1) \cup VL(a_2)$;
- si l'arbre d'analyse a est de la forme $F \ (v, a_1)$, où $v \in L(\mathcal{G}_0)$ et a_1 est un arbre d'analyse, alors $VL(a)$ est l'ensemble $VL(a_1) \setminus \{v\}$.

□ 29 – Écrire une fonction OCaml `free_vars (a : ada) : StringSet.t` dont la valeur de retour est l'ensemble des variables libres de l'arbre d'analyse a .

Seconde épreuve d'informatique MPI 2023

Définition : Un arbre d'analyse est dit *clos* s'il ne contient pas de variables libres ; de même, un mot de $L(\mathcal{G})$ est dit *clos* si son arbre d'analyse est clos.

Dans un arbre d'analyse clos $a \in \mathcal{A}$, pour toute chaîne de caractères $v \in L(\mathcal{G}_0)$, si la construction OCaml $V\ v$ apparaît comme feuille de l'arbre a , alors il existe un nœud interne de la forme $F\ (v, _)$ parmi les ascendants de $V\ v$ qui coïncide avec l'« introduction » de la variable v .

Afin de ne plus s'encombrer avec des variables nommées par une chaîne de caractères, nous adoptons une nouvelle représentation des mots du langage $L(\mathcal{G})$ sous forme d'un arbre, appelé *terme de De Bruijn*.

Définition : Un terme de De Bruijn s'obtient à partir d'un arbre d'analyse $a \in \mathcal{A}$ en remplaçant toute feuille de l'arbre a , disons $V\ v$ avec $v \in L(\mathcal{G}_0)$, par une feuille étiquetée par l'entier $u = 1 + \ell$, où $\ell \in \mathbb{N}$ est le nombre de nœuds internes de la forme $F\ (v', _)$ avec $v' \in L(\mathcal{G}_0) \setminus \{v\}$, qui existent entre ladite feuille $V\ v$ et le plus proche nœud interne de la forme $F\ (v, _)$ parmi ses ascendants dans l'arbre d'analyse a .

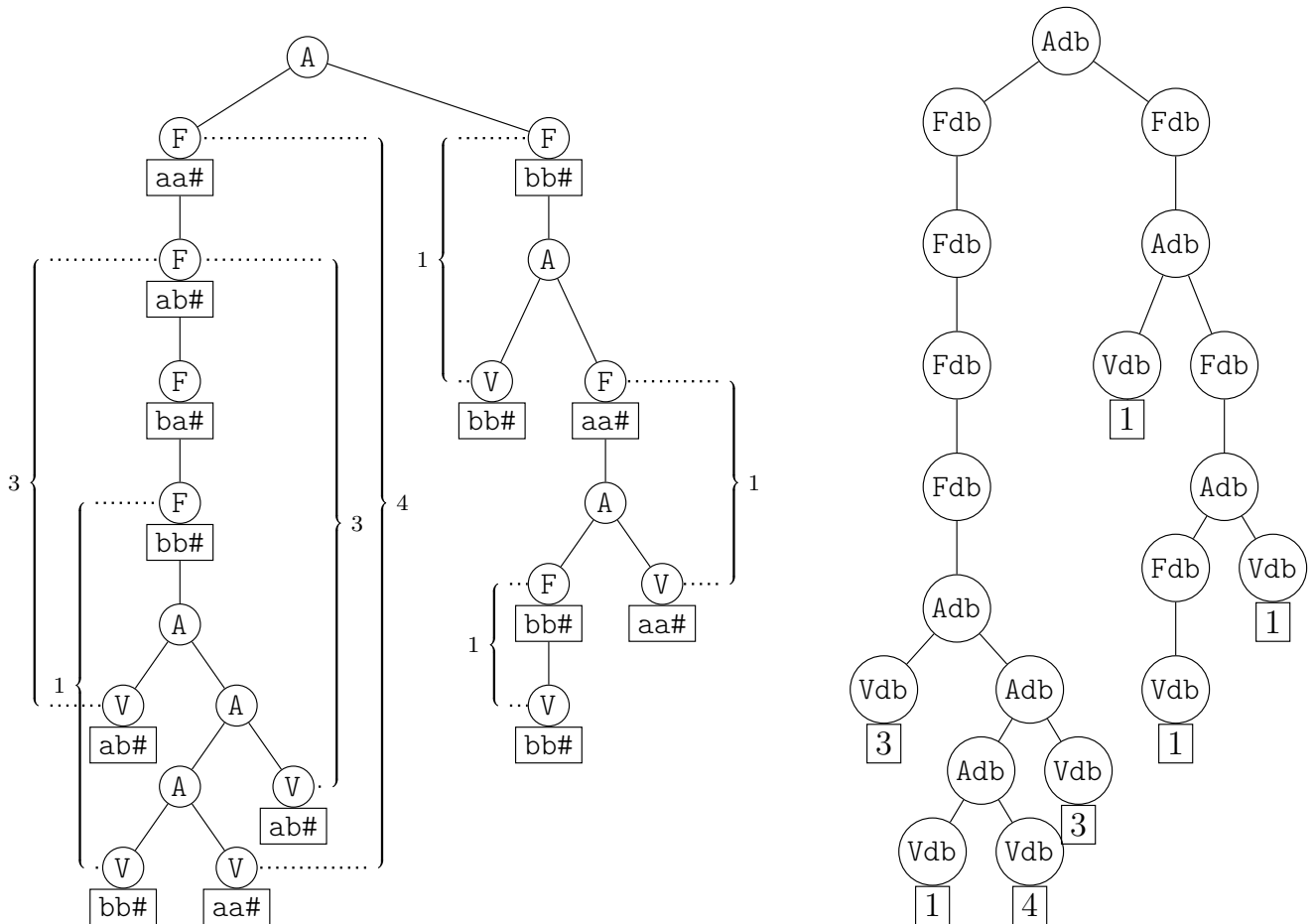
Nous déclarons un nouveau type

type tdb = Vdb **of** int | Adb **of** (tdb * tdb) | Fdb **of** tdb

Nous notons $\mathcal{T}$ l'ensemble des termes de De Bruijn.

Voici un exemple de construction d'un terme de De Bruijn (à droite) à partir d'un arbre d'analyse (à gauche) du mot

(aa#->ab#->ba#->bb#->(ab#((bb#aa#)ab#))bb#->(bb#aa#->(bb#->bb#aa#))) :



Seconde épreuve d'informatique MPI 2023

□ 30 – Dessiner le terme de De Bruijn qui représente le mot $\lceil n \rceil$.

□ 31 – Écrire une fonction OCaml `ada_of_tdb (t : tdb) : ada` dont la valeur de retour est un arbre d'analyse clos associé au terme de De Bruijn t .

Définition : Le *codage binaire* des termes de De Bruijn est l'application $t \in \mathcal{T} \mapsto \hat{t} \in \{0, 1\}^*$ définie par induction structurelle avec les règles d'inférence suivantes sur l'ensemble des termes de De Bruijn :

- si le terme $t \in \mathcal{T}$ est de la forme $\text{Vdb } u$, avec $u \in \mathbb{N}^*$, alors $\hat{t}$ est la chaîne de caractères $11 \dots 10$ (avec le symbole 1 répété u fois).
- si le terme $t \in \mathcal{T}$ est de la forme $\text{Adb } (t_1, t_2)$, t_1 et t_2 étant deux termes de De Bruijn, alors $\hat{t}$ est la chaîne de caractères $01\hat{t}_1\hat{t}_2$.
- si le terme $t \in \mathcal{T}$ est de la forme $\text{Fdb } t_1$, où t_1 est un terme de De Bruijn, alors $\hat{t}$ est la chaîne de caractères $00\hat{t}_1$.

□ 32 – Soient n un entier naturel et t le terme de De Bruijn associé au mot $\lceil n \rceil$ et obtenu à la question 30. Calculer la longueur $|\hat{t}|$ de la chaîne de caractères qui encode t .

□ 33 – Vérifier que le codage binaire des termes de De Bruijn est une application injective.

Nous utilisons le type `string` avec les caractères `'0'` et `'1'` afin de représenter des codages binaires de termes de De Bruijn.

□ 34 – Écrire une fonction OCaml `decode (z : string) : tdb` dont la valeur de retour est l'unique terme de De Bruijn t tel que $\hat{t} = z$ si un tel terme t existe et qui lève une exception `SyntaxError` sinon.

Il est possible de s'appuyer sur la fonction `charlist_of_string` précédemment présentée.

4.3 Interpréteur et décompresseur de De Bruijn

Dans cette sous-section, nous construisons un interpréteur, c'est-à-dire une procédure qui évalue les expressions appartenant au langage $L(\mathcal{G})$ et en déduisons une nouvelle description du mot $y_0 = "1000000\dots"$ relative à un décompresseur approprié.

Naïvement, lorsque nous rencontrons un sous-mot de la forme $w = (v \rightarrow w_b \ w_a)$ avec $v \in L(\mathcal{G}_0)$ et $(w_a, w_b) \in (L(\mathcal{G}))^2$, nous aimerions que w soit équivalent à un mot tiré de w_b dont les occurrences de la variable v ont été remplacées par w_a . Autrement dit, lorsqu'un arbre d'analyse est de la forme $A \ (F \ (v, b), a)$, nous voudrions construire un nouvel arbre à partir de $b \in \mathcal{A}$ et dans lequel les apparitions de $v \in L(\mathcal{G}_0)$ sont devenues des sous-arbres $a \in \mathcal{A}$.

Seconde épreuve d'informatique MPI 2023

Définition : Soient $(a, b) \in (\mathcal{A})^2$ deux arbres d'analyse et $v \in L(\mathcal{G}_0)$ une variable. La *substitution de la variable v par l'arbre a dans l'arbre b* est l'application $[v \leftarrow a] : \mathcal{A} \rightarrow \mathcal{A}$ définie par induction structurale avec les règles d'inférence suivantes :

- Si l'arbre $b \in \mathcal{A}$ est de la forme $V \ v_1$, avec $v_1 \in L(\mathcal{G}_0)$, alors

$$[v \leftarrow a](b) = \begin{cases} a & \text{si } v = v_1 \\ b & \text{si } v \neq v_1. \end{cases}$$

- Si l'arbre $b \in \mathcal{A}$ est de la forme $A \ (b_1, b_2)$, où $b_1 \in \mathcal{A}$ et $b_2 \in \mathcal{A}$ sont deux arbres d'analyse, alors

$$[v \leftarrow a](b) = A \ (b_1', b_2')$$

où $b_1' = [v \leftarrow a](b_1)$ et $b_2' = [v \leftarrow a](b_2)$.

- Si l'arbre $b \in \mathcal{A}$ est de la forme $F \ (v_1, b_1)$, avec $v_1 \in L(\mathcal{G}_0)$ et $b_1 \in \mathcal{A}$, alors

$$[v \leftarrow a](b) = \begin{cases} b & \text{si } v = v_1 \\ F \ (v_1, b_1') & \text{avec } b_1' = [v \leftarrow a](b_1) \quad \text{si } v \neq v_1 \text{ et } v_1 \notin VL(a) \\ F \ (v_1', b_1') & \text{avec } b_1' = [v \leftarrow a]([v_1 \leftarrow v_1'](b_1)) \quad \text{sinon} \end{cases}$$

où $v_1' \in L(\mathcal{G}_0)$ est une variable inédite qui n'appartient pas à $VL(a) \cup VL(b) \cup \{v, v_1\}$.

Nous supposons déjà programmée une fonction `new_string : unit -> string` inspirée de la section 3 qui permet, si besoin, d'engendrer une variable de $L(\mathcal{G}_0)$ jamais encore utilisée.

□ 35 – Écrire une fonction OCaml `substitute (v : string) (by_a : ada) (in_b : ada) : ada` qui substitue la variable v par l'arbre a dans l'arbre b .

Définition : La *réduction en un pas* est l'application $\triangleright : \mathcal{A} \rightarrow \mathcal{A}$ définie par induction structurale avec les règles d'inférence suivantes :

- Si l'arbre $a \in \mathcal{A}$ est de la forme $V \ v$, où $v \in L(\mathcal{G}_0)$, alors la valeur $\triangleright(a)$ n'est pas définie.
- Si l'arbre $a \in \mathcal{A}$ est de la forme $A \ (a_1, a_2)$, où $a_1 \in \mathcal{A}$ et $a_2 \in \mathcal{A}$ sont deux arbres d'analyse, alors

$$\triangleright(a) = \begin{cases} A \ (a_1, a_2') & \text{avec } a_2' = \triangleright(a_2) & \text{si } a_1 \text{ est de la forme } V \ v \\ a' & \text{avec } a' = [v \leftarrow a_2](a_{11}) & \text{si } a_1 \text{ est de la forme } F \ (v, a_{11}) \\ A \ (a_1', a_2) & \text{avec } a_1' = \triangleright(a_1) & \text{sinon.} \end{cases}$$

- Si l'arbre $a \in \mathcal{A}$ est de la forme $F \ (v, a_1)$, où $v \in L(\mathcal{G}_0)$ et $a_1 \in \mathcal{A}$, alors

$$\triangleright(a) = F \ (v, a_1') \text{ où } a_1' = \triangleright(a_1).$$

□ 36 – Écrire une fonction OCaml `reduce_one_step (a : ada) : ada` qui implémente la réduction en un pas $\triangleright$. Lorsque `reduce_one_step` rencontre un cas non défini, une exception `NoReduction` est levée.

□ 37 – Écrire une fonction OCaml `interpret (a : ada) : ada` qui applique répétitivement la réduction en un pas $\triangleright$ à l'arbre d'analyse a jusqu'à ce que la réduction ne soit plus définie. La valeur de retour est le dernier arbre d'analyse rencontré.

 Seconde épreuve d'informatique MPI 2023

Nous admettons que, pour tout entier naturel non nul n , si π est le mot de $L(\mathcal{G})$

$$\pi = (a\# \rightarrow ((a\#a\#)a\#) \lceil n \rceil),$$

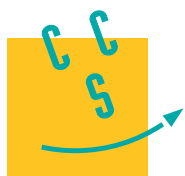
l'expression `interpret (parse pi)` produit une incarnation de l'entier $n^{(n^n)}$. Nous notons $\mathcal{B}$ le décompresseur défini par

```
let decompB (z : string) : string =
  "string_of_int␣(int_of_ada␣(interpret␣(ada_of_tdb␣(decode␣" ^ z ^ "))))"
```

□ 38 – Proposer une méthode pour obtenir une chaîne de caractères z_0 , formée uniquement de 0 et de 1, telle que `decompB z0` a pour valeur de retour la chaîne de caractères $y_0 = "1000000\dots"$ et en calculer la longueur. En déduire que la complexité de Kolmogoroff de y_0 par rapport au décompresseur $\mathcal{B}$ vérifie

$$K_{\mathcal{B}}(y_0) \leq 70.$$

FIN DE L'ÉPREUVE



CONCOURS CENTRALE•SUPÉLEC

Informatique

MPI

2023

4 heures

Calculatrice autorisée

Le sujet est composé de deux parties indépendantes, la première utilise le langage C et la seconde le langage OCaml. Toutes deux traitent de graphes.

La première se concentre sur un problème classique, le problème du voyageur de commerce. Elle étudie d'abord sa complexité, en le comparant notamment au problème très proche du cycle et du chemin hamiltonien, puis en le réduisant à 3-SAT, et en donnant un résultat sur l'approximabilité du problème. On propose ensuite une heuristique, l'algorithme de Christofides, qui repose sur différentes notions : arbres couvrants, couplages, circuit eulérien. On montrera que, dans une variante du problème où les distances vérifient l'inégalité triangulaire, l'algorithme de Christofides est une approximation de facteur $3/2$. Dans cette première partie, les graphes seront représentés par des matrices d'adjacence en C.

La deuxième partie s'intéresse à des processus d'édition sur des arbres non racinés dont les feuilles sont étiquetées, qui sont notamment utilisés en bio-informatique pour représenter l'évolution des espèces. On étudiera en particulier l'espace induit sur ces arbres par ces processus d'édition et on représentera cet espace par un graphe. On montrera notamment que le graphe induit par l'un de ces processus d'édition possède un cycle hamiltonien. On utilisera pour cette partie le langage OCaml pour implémenter des arbres par une définition récursive et des graphes par liste d'adjacence.

I Problème du voyageur de commerce

On considère des graphes non orientés $G(V, E)$ où V est l'ensemble des sommets et E l'ensemble des arêtes. On notera $n = |V|$ le nombre de sommets.

Un *chemin* est une suite de sommets reliés par des arêtes. On dit qu'un chemin *pass*e par un sommet si ce sommet appartient au chemin. Un *circuit* est un chemin qui commence et se termine au même sommet. Un *chemin hamiltonien* est un chemin qui passe une et une seule fois par chaque sommet du graphe. Un *circuit hamiltonien* est un circuit qui passe par chaque sommet une et une seule fois.

Le *problème du voyageur de commerce* consiste, étant donnée une liste de villes toutes reliées entre elles, à trouver le circuit le plus court qui passe une et une seule fois par chacune des villes.

Plus formellement, on considère un graphe complet non orienté, dont les arêtes sont étiquetées avec des nombres entiers strictement positifs, appelés *poids*, et on cherche le circuit passant par chacun des sommets du graphe qui minimise la somme des poids des arêtes. On appellera *poids d'un circuit* la somme des poids des arêtes empruntées par ce circuit. Une solution au problème du voyageur de commerce est un circuit hamiltonien de poids minimal.

Dans cette partie, on représente les graphes en C par des matrices d'adjacence. Le poids d'une arête est représenté par un `int`, une arête absente étant représentée par un 0. On représente un graphe par une structure de données avec deux attributs : son nombre de sommets `V` et un pointeur `adj` vers sa matrice d'adjacence de taille $V \times V$. Les sommets sont associés aux entiers de 0 à $V-1$.

```
struct Graphe {
    int V;
    int* adj;
};
```

Pour accéder au poids de l'arête entre le sommet `i` et le sommet `j` du graphe `G`, on pourra utiliser l'expression `G.adj[i * G.V + j]`.

On pourra par la suite utiliser les deux fonctions suivantes, qui sont supposées travailler en temps constant :

```
struct Graphe alloue_graphe(int V) {
    int* adj = malloc(V * V * sizeof(int));
    struct Graphe g = {.V = V, .adj = adj};
    return g;
}

void libere_graphe(struct Graphe g) {
    free(g.adj);
}
```

La fonction `alloue_graphe` prend comme argument un nombre V et renvoie un graphe qui possède V sommets et dont la matrice d'adjacence est allouée sur le tas, grâce à la fonction `malloc`. La fonction `libere_graphe` libère la mémoire du tas occupée par la matrice d'adjacence d'un graphe dont on n'a plus besoin.

On définit également une structure `Chemin` qui représente un chemin par un attribut `longueur` et un attribut `l_sommets`, pointeur vers un tableau à `longueur` éléments. Si C est un chemin et k un entier naturel strictement plus petit que $C.\text{longueur}$, alors $C.l_sommets[k]$ est le k -ième sommet du chemin C . De même que pour les graphes, on définit aussi des fonctions permettant d'allouer un chemin et de libérer un chemin dont on n'a plus besoin, et qui sont supposées travailler en temps constant.

```
struct Chemin {
    int longueur;
    int* l_sommets;
};

struct Chemin alloue_chemin(int longueur) {
    int* l_sommets = malloc(longueur * sizeof(int));
    struct Chemin c = {.longueur = longueur, .l_sommets = l_sommets};
    return c;
}

void libere_chemin(struct Chemin c) {
    free(c.l_sommets);
}
```

I.A – Prise en main du problème et appartenance à NP

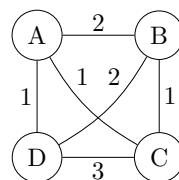


Figure 1 Exemple d'instance du problème du voyageur de commerce

Q 1. Donner une solution du problème du voyageur de commerce sur l'exemple de la figure 1 en précisant le poids du circuit trouvé.

Q 2. Donner le nombre de circuits hamiltoniens sur un graphe complet de n sommets. Donner un exemple de pondération pour que chacun de ces circuits ait un poids minimal pour le problème du voyageur de commerce.

Q 3. Écrire une fonction

```
int poids_chemin(struct Graphe g, struct Chemin c);
```

qui prend en arguments un graphe et un chemin et renvoie le poids de ce chemin. Donner la complexité de cette fonction.

Q 4. Le problème du voyageur de commerce est un problème d'optimisation ; à l'aide d'un seuil, transformer ce problème en un problème de décision. Montrer que ce nouveau problème, que nous appellerons par la suite « problème de décision du voyageur de commerce », appartient à la classe de complexité NP.

I.B – Étude de la complexité

Le *problème du chemin hamiltonien* consiste, étant donné un graphe non orienté et deux sommets a et b , à déterminer s'il existe un chemin hamiltonien commençant en a et finissant en b . Le *problème du chemin hamiltonien orienté* consiste, étant donné un graphe orienté et deux sommets a et b , à déterminer s'il existe un chemin hamiltonien commençant en a et finissant en b . Le *problème du circuit hamiltonien* consiste, étant donné un graphe non orienté, à déterminer s'il existe un circuit hamiltonien dans ce graphe.

I.B.1) NP-complétude

Q 5. Montrer que le problème du chemin hamiltonien se réduit au problème du circuit hamiltonien.

Q 6. Montrer que le problème du circuit hamiltonien se réduit au problème de décision du voyageur de commerce.

Q 7. Montrer que le problème du chemin hamiltonien orienté se réduit au problème du chemin hamiltonien.

Le problème 3-SAT consiste à déterminer la satisfaisabilité d'une formule logique sous forme normale conjonctive avec exactement 3 littéraux : pour n clauses C_i et m variables y_k , déterminer s'il existe une valuation des y_k qui permette de rendre vraie la formule $C_1 \wedge C_2 \wedge \dots \wedge C_n$ avec $C_i = x_{i,1} \vee x_{i,2} \vee x_{i,3}$ sachant que, pour chaque i et chaque p , il existe un k tel que $x_{i,p} = y_k$ ou $x_{i,p} = \neg y_k$. On admet que le problème 3-SAT est NP-complet.

On va maintenant montrer que 3-SAT se réduit au problème du chemin hamiltonien orienté.

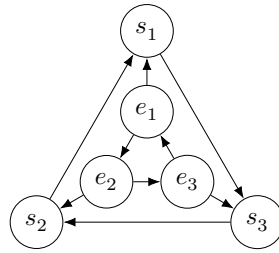


Figure 2 Graphe A

Q 8. On considère le graphe A (figure 2). Montrer qu'il existe un chemin entrant par e_1 et passant par tous les sommets pour ressortir en s_1 . Puis qu'il existe un chemin entrant par e_1 et sortant par s_1 et un chemin entrant par e_2 et sortant par s_2 , tels que chaque sommet soit visité par un et un seul des deux chemins. Même question, mais avec trois chemins, pour e_1, e_2, e_3 et s_1, s_2, s_3 .

On se donne une instance du problème 3-SAT, pour n clauses C_i et m variables y_k : $C_1 \wedge C_2 \wedge \dots \wedge C_n$ avec $C_i = x_{i,1} \vee x_{i,2} \vee x_{i,3}$ et $x_{i,p} = y_k$ ou $x_{i,p} = \neg y_k$. On veut donc savoir s'il existe une valuation des y_k pour que la formule soit vraie.

On construit alors le graphe orienté G de la manière suivante :

- pour chaque variable y_k on crée un sommet v_k ;
- on ajoute un sommet supplémentaire v_{m+1} ;
- pour chaque clause C_i on ajoute une copie du graphe A , notée A_i ;
- pour chaque variable y_k , on note $C_{k_1}, \dots, C_{k_\ell}$ les clauses dans lesquelles y_k apparaît en positif. On relie alors v_k à A_{k_1} par un arc allant de v_k vers e_p dans A_{k_1} lorsque y_k est en position p dans C_{k_1} c'est-à-dire lorsque $C_{k_1} = x_{k_1,1} \vee x_{k_1,2} \vee x_{k_1,3}$ et $x_{k_1,p} = y_k$. On relie ensuite la sortie s_p de A_{k_1} à l'entrée de A_{k_2} correspondant à la position de y_k dans C_{k_2} et ainsi de suite jusqu'au dernier dont on relie la sortie à v_{k+1} . On appelle G_k^+ le sous-graphe constitué du sommet v_k , des graphes $A_{k_1}, A_{k_2}, \dots, A_{k_\ell}$ et du sommet v_{k+1} , ainsi que des arcs que l'on vient d'ajouter entre eux en considérant y_k et les clauses dans lesquelles il apparaît positivement ;
- on crée de même des arcs pour chaque variable y_k et chaque clause dans laquelle y_k apparaît en négatif. On note G_k^- le sous-graphe correspondant entre v_k et v_{k+1} .

Q 9. Montrer que pour toute valuation de la formule il existe un chemin hamiltonien orienté de v_1 à v_{m+1} dans le graphe G .

Q 10. Montrer, en une dizaine de lignes au maximum, que pour chaque chemin hamiltonien orienté de v_1 à v_{m+1} il existe bien une valuation.

Q 11. En déduire que le problème du circuit hamiltonien et le problème de décision du voyageur de commerce sont NP-complets.

I.B.2) Approximation

Soit $\varepsilon > 0$, on va montrer que, si $P \neq NP$, il n'existe pas de $1 + \varepsilon$ approximation pour le problème du voyageur de commerce. Un algorithme est une $1 + \varepsilon$ approximation à un problème d'optimisation d'un poids p lorsque la solution proposée de poids p se compare toujours à la solution optimale p^* par $p < (1 + \varepsilon)p^*$.

Soit G un graphe non orienté à n sommets, on considère le graphe complet G' , de mêmes sommets que G , avec des poids aux arêtes obtenus en donnant un poids de 1 aux arêtes de G et un poids de $n(1 + \varepsilon) + 1$ aux arêtes qui ne sont pas dans G .

Q 12. Montrer que si G possède un circuit hamiltonien, alors G' possède un circuit de poids n .

Q 13. Montrer que si G ne possède pas de circuit hamiltonien alors toute solution pour l'instance de voyageur de commerce est de poids au moins $n(2 + \varepsilon)$.

Q 14. En déduire que, si $P \neq NP$, il n'existe pas de $1 + \varepsilon$ approximation au problème du voyageur de commerce.

I.C – Algorithme de Christofides

On va proposer une heuristique pour le problème du voyageur de commerce, l'algorithme de Christofides, et on va montrer que, sous certaines conditions sur le graphe en entrée, cette heuristique constitue un algorithme d'approximation. L'algorithme prend en argument un graphe G et procède comme suit :

- calculer un arbre couvrant de poids minimal T de G ;
- en notant I l'ensemble des sommets de degré impair dans T , calculer un couplage parfait M de poids minimum dans le sous-graphe de G induit par les sommets de I , $G|_I$;
- construire H le multigraphe ayant pour sommet les sommets de G et comme arêtes les arêtes de M et celles de T ;

- trouver un cycle eulérien dans H ;
- transformer le cycle eulérien en circuit hamiltonien en supprimant les éventuels sommets vus plusieurs fois.

Dans la suite, on étudie plus précisément certaines étapes de cet algorithme, avant de proposer une implémentation de cet algorithme.

I.C.1) Arbre couvrant

Un arbre couvrant est un sous-graphe connexe sans cycle d'un graphe avec les mêmes sommets. On appelle poids de l'arbre la somme des poids des arêtes de cet arbre. On rappelle que l'algorithme de Kruskal est un algorithme glouton qui vise à construire un arbre couvrant de poids minimal en considérant les arêtes par poids croissant et en ajoutant chaque arête si elle ne crée pas de cycle.

Pour cela, on va représenter une arête par une structure de données avec trois attributs : **s1** et **s2** donnent les sommets reliés par l'arête et **p** son poids :

```
struct Arete {
    int s1;
    int s2;
    int p;
};
```

Q 15. Écrire une fonction

```
struct Arete* liste_aretes(struct Graphe g);
```

qui prend en argument un graphe complet g et alloue et renvoie un tableau contenant les $\frac{g \cdot V \times (g \cdot V - 1)}{2}$ arêtes du graphe.

On dispose d'une fonction

```
void tri_aretes(struct Arete a[], int k);
```

qui prend en arguments un tableau d'arêtes et sa longueur et trie le tableau par ordre croissant de poids. La complexité de cette fonction est en $\mathcal{O}(k \ln(k))$.

Q 16. Écrire une fonction

```
struct Graphe kruskal(struct Graphe g);
```

implémentant l'algorithme de Kruskal qui renvoie un graphe représentant l'arbre couvrant de poids minimal du graphe donné en argument. On mettra des 0 lorsque l'arête est absente et des 1 lorsqu'elle est présente. Donner la complexité de la fonction `kruskal`.

Q 17. Montrer la correction de cet algorithme, c'est-à-dire l'optimalité de la solution proposée pour le problème d'arbre couvrant de poids minimal.

I.C.2) Couplage

On appelle couplage d'un graphe un ensemble d'arêtes qui n'ont pas de sommets en commun. Un couplage est parfait si tous les sommets du graphe appartiennent à une arête du couplage.

Q 18. Écrire une fonction

```
int degre(struct Graphe g, int i);
```

qui prend en arguments un graphe et l'indice d'un sommet et renvoie le degré de ce sommet.

Q 19. Écrire une fonction

```
int* sommets_impairs(struct Graphe g, int* nb_sommets);
```

qui prend en arguments un graphe g et un pointeur vers un entier. Cette fonction alloue sur le tas un tableau, le remplit avec les numéros des sommets de degré impair et le renvoie. Par ailleurs, elle renseigne l'entier pointé par `nb_sommets` avec le nombre des sommets de degré impair.

Q 20. Montrer l'existence d'un couplage parfait de poids minimal dans $G|_I$.

On dispose des deux fonctions suivantes :

```
struct Graphe graphe_induit(struct Graphe g, int nb_sommets, int* liste_sommets);
struct Graphe couplage(struct Graphe g);
```

La fonction `graphe_induit` renvoie le graphe induit dans un graphe g donné par un nombre et un tableau de sommets comme ceux de la question 19.

La fonction `couplage` renvoie un couplage parfait de poids minimal s'il existe, sous la forme d'un graphe représentant ce couplage, avec des 0 lorsque l'arête est absente et 1 lorsque l'arête est présente.

On supposera ces deux fonctions de complexité polynomiale.

I.C.3) Cycle eulérien

On appelle cycle eulérien un circuit qui passe par chaque arête une et une unique fois. On admet qu'un graphe possède un cycle eulérien si et seulement si les degrés des sommets de ce graphe sont pairs et que le graphe est connexe. Un multigraphe est un graphe dans lequel il peut exister plusieurs arêtes reliant un même couple de sommets.

Q 21. Montrer que le multigraphe H , défini dans l'introduction de la sous-partie I.C, possède un cycle eulérien.

On dispose des trois fonctions suivantes :

```
struct Multigraphe multigraphe(struct Graphe g1, struct Graphe g2);
struct Chemin eulerien(struct Multigraphe h);
void libere_multigraphe(struct Multigraphe h);
```

La fonction `multigraphe` prend en arguments deux graphes sur les mêmes sommets et renvoie le multigraphe obtenu en considérant les arêtes des deux graphes. La fonction `eulerien` renvoie un circuit eulérien d'un multigraphe sous la forme d'un chemin. La fonction `libere_multigraphe` libère la mémoire du tas utilisée par un multigraphe. Toutes trois sont supposées de complexité polynomiale.

Q 22. Écrire une fonction

```
struct Chemin euler_to_hamilton(struct Chemin c);
```

qui transforme un cycle eulérien c du multigraphe H en un circuit hamiltonien du graphe G en supprimant les doublons, et renvoie le chemin représentant la suite des sommets.

I.C.4) Implémentation

Q 23. Sur l'exemple de la figure 1, réaliser les différentes étapes de l'algorithme. On ne demande pas de détailler les étapes pour trouver un couplage et un cycle eulérien.

Q 24. Écrire une fonction

```
struct Chemin christofides(struct Graphe g);
```

qui implémente l'algorithme de Christofides, en prenant soin de libérer la mémoire allouée sur le tas qui n'est plus utilisée.

Q 25. Justifier que la fonction `christofides` renvoie bien un circuit hamiltonien.

Q 26. Montrer que la fonction `christofides` est de complexité polynomiale.

I.C.5) Preuve de l'approximation

On va maintenant montrer que cet algorithme est une $3/2$ -approximation pour le problème du voyageur de commerce, dans le cas où les poids des arêtes vérifient l'inégalité triangulaire, c'est-à-dire que pour tout sommet u, v, w les poids des arêtes vérifient, en notant $c_{i,j}$ le poids de l'arête entre des sommets i et j : $c_{u,v} \leq c_{u,w} + c_{w,v}$.

On note U une solution optimale et $c(U)$ son poids.

Q 27. Montrer que $c(T) \leq c(U)$, où $c(T)$ est le coût de l'arbre couvrant minimal.

Q 28. Montrer que $c(M) \leq 0.5c(U)$ où $c(M)$ est le coût du couplage parfait minimal.

Q 29. Montrer que la solution construite par l'algorithme est une $3/2$ -approximation de U .

Q 30. Sans la supposition de l'inégalité triangulaire, cette solution est-elle toujours une $3/2$ -approximation ? Proposer un contre-exemple ou une justification.

II Espaces d'arbres

Dans cette partie on considère des arbres binaires non racinés avec des feuilles étiquetées. Un *arbre binaire non raciné* est un graphe connexe sans cycle dont les nœuds sont de degrés 1 ou 3. Un *arbre binaire raciné* est un graphe connexe sans cycle dont les nœuds sont de degrés 1 ou 3 sauf exactement un nœud qui est de degré 2. Les nœuds de degré 1 sont appelés des *feuilles*, les nœuds de degré 2 ou 3 des *nœuds internes* et l'unique nœud de degré 2 d'un arbre binaire raciné est appelé sa *racine*. Un graphe réduit à un unique nœud est considéré comme un arbre raciné dont le nœud est à la fois feuille et racine. Les arêtes d'un arbre sont appelées des *branches*, les branches reliant des sommets de degré 2 ou 3 *branches internes*. On étiquette ensuite les feuilles d'un arbre à n feuilles par des entiers distincts entre 1 et n . On s'intéresse à des opérations d'édition qui transforment un arbre à n feuilles en un second arbre à n feuilles.

La première opération, notée SPR, pour *Subtree Prune and Regraft* (ou découpe et greffe d'un sous-arbre), prend en entrée 2 branches e et f de l'arbre. La branche e est supprimée, créant deux arbres racinés par les sommets à chaque extrémité de e , notons ces arbres T_1 (qui contient la branche f) et T_2 (qui ne la contient pas). On divise ensuite la branche f pour créer un nouveau nœud v de degré 2. Le sous-arbre raciné T_2 est alors rattaché par sa racine à v par une nouvelle branche ; v devient alors de degré 3. On obtient alors un arbre raciné, avec une racine de degré 2, que l'on supprime en reliant directement ses deux voisins pour obtenir un arbre non raciné.

On s'intéresse également à une autre opération, notée NNI, pour *Nearest Neighbor Interchange* (ou échange entre plus proches voisins). Cette opération prend une branche interne de l'arbre e et échange entre eux deux des quatre sous-arbres incidents à cette branche. C'est-à-dire, en notant a et b les extrémités de la branche interne e , a a deux sommets voisins qui ne sont pas b , a_1 et a_2 , et de même pour b , b_1 et b_2 . On choisit alors un des a_i et un des b_j à échanger, par exemple a_1 et b_1 , on supprime l'arête reliant a à a_1 et b à b_1 et on relie a à b_1 et b à a_1 .

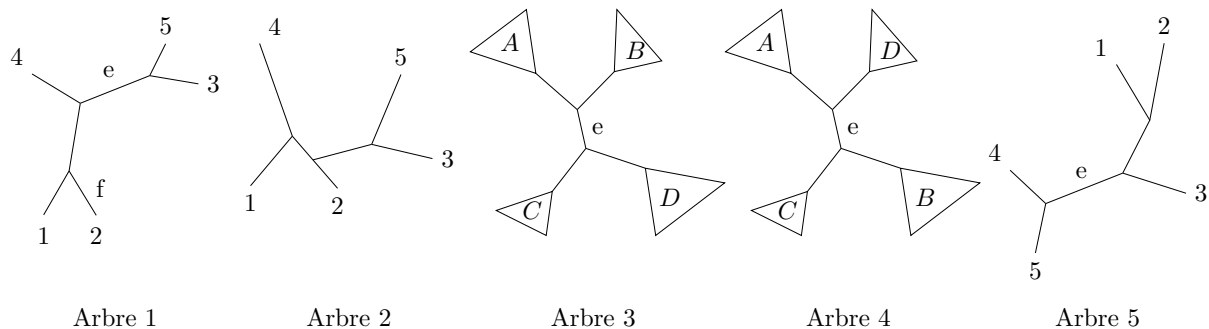


Figure 3 Exemples d'arbres non racinés

La figure 3 présente quelques exemples d'arbres non racinés :

- l'arbre 1 est un arbre binaire non raciné avec ses feuilles étiquetées ;
- l'arbre 2 est obtenu à partir de l'arbre 1 par un SPR en coupant la branche e et en la greffant sur la branche f ;
- dans l'arbre 3, où les triangles représentent des sous-arbres racinés, la branche interne e sépare 4 sous-arbres racinés A, B, C et D ;
- l'arbre 4 est obtenu à partir de l'arbre 3 par un mouvement NNI sur la branche e en échangeant les sous-arbres B et D ;
- l'arbre 5 est obtenu à partir de l'arbre 1 par un NNI sur la branche e en échangeant le sous-arbre contenant la feuille 5 et le sous-arbre contenant les feuilles 1 et 2.

On étudie l'espace des arbres binaires non racinés étiquetés de n feuilles $B(n)$ suivant ces deux processus d'édition. On définit $G_{\text{NNI}}(n)$ le graphe dont les sommets sont les arbres de $B(n)$ et dans lequel une arête relie deux arbres si l'on peut passer de l'un à l'autre par un mouvement NNI. On définit de même $G_{\text{SPR}}(n)$ avec les mouvements SPR. On va notamment s'intéresser à la structure de $G_{\text{NNI}}(5)$, écrire un programme permettant de construire $G_{\text{NNI}}(n)$ et montrer que $G_{\text{SPR}}(n)$ est hamiltonien, c'est-à-dire qu'il possède un circuit hamiltonien.

II.A – Prise en main

Q 31. Montrer que l'on peut passer de l'arbre 1 à l'arbre 2 par une opération NNI. Dessiner tous les voisins de l'arbre 1 dans $G_{\text{NNI}}(5)$.

Q 32. Donner le nombre de branches et de nœuds d'un arbre de $B(n)$, pour $n \geq 2$.

Q 33. On note $RB(n)$ l'ensemble des arbres binaires racinés à n feuilles étiquetées. Montrer que $|RB(n)| = (2n - 3)|B(n)|$ et que $|B(n)| = |RB(n - 1)|$. En déduire le nombre d'arbres binaires $B(n)$ en fonction de n .

Q 34. Tracer $G_{\text{NNI}}(4)$.

Q 35. Combien de voisins un arbre de $B(n)$ a-t-il dans $G_{\text{NNI}}(n)$?

On appelle arbre chenille un arbre binaire non raciné dans lequel il existe un chemin passant une et une seule fois par chaque nœud interne.

Q 36. Montrer que l'on peut passer de tout arbre de $B(n)$ à un arbre chenille, en effectuant seulement des mouvements NNI. En déduire que $G_{\text{NNI}}(n)$ est connexe.

Q 37. Montrer que tout mouvement SPR peut se décomposer en mouvements NNI, et que tous les mouvements NNI sont des mouvements SPR. En déduire que $G_{\text{SPR}}(n)$ est connexe.

II.B – Étude de $G_{\text{NNI}}(5)$

Q 38. Combien de sommets possède $G_{\text{NNI}}(5)$ et quel est le degré de ces sommets ?

Q 39. Montrer que tout arbre de $G_{\text{NNI}}(5)$ fait partie de cycles de longueur 5, de deux cycles de longueurs 3, et que pour tout arbre deux arbres sont à distance 3 de cet arbre.

Q 40. Tracer $G_{\text{NNI}}(5)$.

II.C – Construction de $G_{\text{NNI}}(n)$

On va écrire en OCaml un programme qui construit $G_{\text{NNI}}(n)$. Pour représenter les arbres binaires (racinés ou non), nous allons utiliser la structure de données suivante :

```
type arbre = Feuille of int
           | Noeud of arbre * arbre ;;
```

Un arbre raciné est alors représenté par sa racine, qui peut être soit une feuille soit un nœud interne ayant un sous-arbre gauche et un sous-arbre droit.

Un arbre non raciné à n feuilles est représenté à partir d'une racine fictive ajoutée entre la feuille d'étiquette n et le nœud auquel elle est reliée.

Enfin, pour assurer l'unicité de la représentation informatique d'un arbre, et ainsi simplifier les programmes, on adopte la convention, pour chaque nœud interne, que la plus grande des étiquettes du sous-arbre droit est supérieure aux étiquettes du sous-arbre gauche.

Ainsi, l'arbre 1 de la figure 3 est représenté par :

```
Noeud (Noeud (Feuille 3,
             Noeud (Noeud (Feuille 1,
                          Feuille 2),
                          Feuille 4)),
      Feuille 5);;
```

On représente les graphes par liste d'adjacence, plus précisément par une liste de couples dont le premier élément est le nom d'un sommet et le second la liste d'adjacence de ce sommet.

```
type graphe = (arbre * arbre list) list;;
```

Q 41. Écrire une fonction **feuilles** qui prend en argument un arbre et renvoie la liste des étiquettes de ses feuilles.

Q 42. Écrire une fonction **degres** qui prend en argument un graphe implémenté par liste d'adjacence et renvoie la liste des degrés de ces nœuds.

Q 43. Écrire une fonction **egaux** qui teste si deux arbres sont égaux, au sens des arbres binaires non racinés étiquetés. Justifier que cette fonction est correcte.

Q 44. Étant donné une liste d'arbres et un arbre, écrire une fonction **appartient** qui teste si l'arbre fait partie de la liste.

Q 45. En remarquant que parmi les 4 sous-arbres à échanger pour un mouvement de NNI autour d'une branche interne, on peut en choisir un qui restera fixe, écrire une fonction **voisinsNNI** qui prend en argument un arbre et renvoie tous les arbres que l'on peut obtenir à partir de celui-ci par un mouvement NNI.

Q 46. Écrire une fonction **chenille** qui prend en argument un entier n et renvoie l'arbre chenille dans lequel les feuilles sont rangées de 1 à n .

Q 47. Écrire une fonction **insere** qui prend en arguments un arbre et un graphe et ajoute l'arbre à la liste des sommets du graphe.

Q 48. Écrire une fonction **relie** qui prend en arguments un graphe et deux arbres et qui rajoute au graphe une arête reliant les deux sommets, si elle n'est pas déjà présente.

Q 49. Écrire une fonction **grapheNNI** qui prend en argument un entier n et renvoie $G_{\text{NNI}}(n)$, en implémentant le graphe avec des listes d'adjacences. Justifier la correction et la terminaison de votre programme.

II.D – $G_{\text{SPR}}(n)$ est hamiltonien

On va démontrer par récurrence que $G_{\text{SPR}}(n)$ est hamiltonien, c'est-à-dire qu'il possède un circuit hamiltonien, comme défini en première partie.

On note S_i l'ensemble des arbres de taille $n+1$ obtenu à partir d'un arbre t_i de taille n en ajoutant une feuille étiquetée $n+1$ à une des branches de t_i . Pour ajouter une feuille à une branche, on supprime la branche et on crée un nouveau nœud, relié à la nouvelle feuille et aux deux nœuds qui étaient reliés par la branche.

Q 50. Montrer que les sous-graphes $G_{\text{SPR}}(n+1)_{|S_i}$, induits dans $G_{\text{SPR}}(n+1)$ par les sommets de S_i sont complets.

Q 51. Soient t_i et t_j deux sommets de $G_{\text{SPR}}(n)$ reliés entre eux, montrer qu'il existe des arêtes entre S_i et S_j dans $G_{\text{SPR}}(n+1)$.

Q 52. Démontrer par récurrence que $G_{\text{SPR}}(n)$ est hamiltonien.

• • • FIN • • •

SESSION 2023



MPI5IN

ÉPREUVE MUTUALISÉE AVEC E3A-POLYTECH**ÉPREUVE SPÉCIFIQUE - FILIÈRE MPI**

INFORMATIQUE**Durée : 4 heures**

N.B. : le candidat attachera la plus grande importance à la clarté, à la précision et à la concision de la rédaction. Si un candidat est amené à repérer ce qui peut lui sembler être une erreur d'énoncé, il le signalera sur sa copie et devra poursuivre sa composition en expliquant les raisons des initiatives qu'il a été amené à prendre.

RAPPEL DES CONSIGNES

- Utiliser uniquement un stylo noir ou bleu foncé non effaçable pour la rédaction de votre composition ; d'autres couleurs, excepté le vert, peuvent être utilisées, mais exclusivement pour les schémas et la mise en évidence des résultats.
 - Ne pas utiliser de correcteur.
 - Écrire le mot *FIN* à la fin de votre composition.
-

Les calculatrices sont interdites.

Le sujet est composé de trois parties, toutes indépendantes.

Partie I - Palindromes

En langue française, "ressasser" est le mot palindrome le plus long, tandis qu'il semble que "saippua-kauppias" soit le plus long mot palindrome au monde, désignant un marchand de savon en Finlande. L'objet de cette partie est de compter le nombre de palindromes présents dans un mot donné.

Soit Σ un alphabet fini contenant au moins deux lettres. On note $u = u_0 \cdots u_{n-1}$ un *mot* sur Σ , composé de n lettres $u_i \in \Sigma, i \in \llbracket 0, n-1 \rrbracket$. La longueur de u est notée $|u|$. Pour $0 \leq i < j \leq n$, on note $u[i, j]$ le mot $u_i \cdots u_{j-1}$. L'ensemble des mots construits sur Σ et contenant le mot vide ϵ est noté Σ^* .

Définition 1 (Miroir)

Soit $u \in \Sigma^*$. Le *miroir* de $u = u_0 \cdots u_{n-1}$, noté $\bar{u}$, est le mot $\bar{u} = u_{n-1} \cdots u_0$. Par convention $\bar{\epsilon} = \epsilon$.

Définition 2 (Palindrome)

$u \in \Sigma^*$ est un *palindrome* si et seulement si $u = \bar{u}$.

Par convention, le mot vide ϵ n'est pas considéré comme un palindrome.

On dira qu'un palindrome u est *pair* (respectivement *impair*) lorsque sa longueur $|u|$ est paire (resp. impaire).

On recherche donc dans cette partie le nombre de palindromes facteurs d'un mot $u \in \Sigma^*$ (comptés avec les multiplicités éventuelles), soit le cardinal de l'ensemble $\{(i, j), 0 \leq i < j \leq |u|, u[i, j] = \overline{u[i, j]}\}$. Dit autrement, on recherche le nombre de palindromes contenus dans u .

Q1. Si $\Sigma = \{a, b\}$, donner le nombre de palindromes contenus dans le mot $u = babb$.

En parcourant naïvement les lettres d'un mot u donné, on peut proposer un algorithme en pseudo-code permettant de compter le nombre palindromes de u (**algorithme 1**).

Algorithme 1 - Décompte naïf du nombre de palindromes contenus dans un mot donné.

Entrées : Un mot u .

Sorties : Le nombre de palindromes contenus dans u .

début

```

    nb = 0
    pour i=0 à |u|-1 faire
        pour j=0 à |u|-1 faire
            estPalindrome=True
            pour k=i à j-1 faire
                si u[i] ≠ u[j-k-1] alors
                    estPalindrome=False
                Sortir du pour k
            si estPalindrome alors
                nb = nb+1
    retourner nb
```

Q2. Évaluer la complexité au pire des cas de l'**algorithme 1** en fonction de $|u|$.

On souhaite bien sûr améliorer cette première idée. Pour ce faire, on utilise tout d'abord le paradigme de la programmation dynamique.

Pour $u \in \Sigma^*$, on définit un tableau de booléens P de taille $(|u|+1) \times (|u|+1)$, $P[i][j]$ étant vrai si $u[i, j]$ est un palindrome. On a donc pour tout $i \in \llbracket 0, |u|-1 \rrbracket$, $P[i][i+1] = \text{True}$.

Q3. Soit $u[i, j]$ un mot. À quelles conditions sur u_i , u_{j-1} et $u[i+1, j-1]$ le mot $u[i, j]$ est-il un palindrome ?

Q4. En déduire une relation de récurrence vérifiée par les coefficients de P .

Q5. Écrire un algorithme de programmation dynamique en pseudo-code résolvant le problème. Évaluer sa complexité.

On insère maintenant entre chaque paire de lettres de u , ainsi qu'au début et à la fin du mot, un symbole spécial noté $\#$. On appelle ce nouveau mot $u^\#$. Ainsi, le mot $u = abba$ se transforme en $u^\# = \#a\#b\#b\#a\#$.

Q6. Montrer que les palindromes de $u^\#$, s'ils existent, sont tous impairs.

Q7. Soit v un palindrome de longueur $2k + 1$ de u , $k \in \mathbb{N}$. On construit le mot $u^\#$. Donner les deux palindromes correspondants à v dans $u^\#$.

Q8. Même question si v est un palindrome de longueur $2k$ de u .

Q9. En déduire une stratégie de recherche de tous les palindromes de u .

On voit que les palindromes impairs sont importants. On va donc construire un algorithme de recherche de ce type de palindrome.

Définition 3 (Rayon d'un palindrome)

Soient $u \in \Sigma^*$ et $i \in \llbracket 0, |u| - 1 \rrbracket$. On dit qu'il existe un *palindrome centré en i de rayon $\rho > 0$* si :

- (i). $i - \rho \geq 0$,
- (ii). $i + \rho + 1 \leq |u|$,
- (iii). $u[i - \rho, i + \rho + 1]$ est un palindrome.

Le *rayon maximal* $\hat{\rho}_i$ d'un palindrome centré en i est le plus grand rayon d'un palindrome centré en i .

Par exemple, si $u = abbababab$ et $i = 4$, alors b est un palindrome centré en 4 de rayon 0, aba est un palindrome centré en 4 de rayon 1, $babab$ est un palindrome centré en 4 de rayon 2 et c'est le plus grand, donc $\hat{\rho}_4 = 2$.

Q10. En remarquant qu'un palindrome impair est centré sur une lettre (ou un symbole spécial dans le cas de $u^\#$), proposer un algorithme en pseudo-code permettant de compter le nombre de palindromes impairs d'un mot u . Évaluer sa complexité.

Q11. Soient $u \in \Sigma^*$, $i \in \llbracket 0, |u| - 1 \rrbracket$ et $0 < j \in \llbracket 1, \hat{\rho}_i \rrbracket$. En exploitant les différentes symétries, montrer qu'il existe un palindrome centré en $i + j$ de rayon $\min(\hat{\rho}_i - j, \hat{\rho}_{i-j})$. En déduire $\hat{\rho}_{i+j} \geq \min(\hat{\rho}_i - j, \hat{\rho}_{i-j})$. Préciser à quelle condition il y a égalité.

En utilisant cette remarque, on développe un algorithme, dit de Manacher, qui construit un tableau d'entiers T permettant de compter le nombre de palindromes d'un mot u . Plus précisément, pour chaque position $i \in \llbracket 1, |u| - 1 \rrbracket$, $T[i]$ indique le rayon maximal $\hat{\rho}_i$, donc tel que la sous-chaîne de $i - \rho_i$ à $i + \rho_i + 1$ est un palindrome. L'algorithme 2 consiste à incrémenter $T[i]$ jusqu'à trouver le plus grand palindrome $u[i - T[i], i + T[i] + 1]$ centré en i .

Algorithme 2 - Algorithme de Manacher

Entrées : Un mot u

Sorties : Un tableau T .

début

Initialiser un tableau T de $|u|$ cases, initialisées à 0
 $k=0$

pour $i = 1$ **à** $|u| - 1$ **faire**

$j = i - k$

si $T[k] \geq j$ **alors**

$T[i] = \min(T[k - j], T[k] - j)$

tant que $(i - (T[i] + 1)) \geq 0$ **ET** $(i + T[i] + 1) < |u|$ **ET** $(u[i - (T[i] + 1)] = u[i + T[i] + 1])$ **faire**

$T[i] = T[i] + 1$

$k = i$

retourner T

Q12. Comment trouver à partir de cet algorithme le nombre de palindromes de u ?

Q13. Quelle est la complexité de cet algorithme ? Justifier.

Partie II - Traversée de rivière

Cette partie comporte des questions nécessitant un **code OCaml**.

Dans une vallée des Alpes, un passage à gué fait de cailloux permet de traverser la rivière. Deux groupes de randonneurs arrivent simultanément sur les berges gauche et droite de cette rivière et veulent la traverser. Le chemin étant très étroit, une seule personne peut se trouver sur chaque caillou de ce chemin (**figure 1**). Un randonneur sur la berge de gauche peut avancer d'un caillou (vers la droite sur la **figure 1**) et sauter par dessus le randonneur devant lui (un caillou à droite) si le caillou où il atterit est libre. De même, chaque randonneur de la berge de droite peut avancer d'un caillou (vers la gauche sur la **figure 1**) et sauter par dessus le randonneur devant lui, dans la mesure où le caillou sur lequel il atterit est libre. Une fois engagés, les randonneurs ne peuvent pas faire marche arrière. De plus, pour simplifier, on suppose qu'une fois tous les randonneurs sur le chemin, il ne reste qu'un caillou de libre.



Figure 1 - Les randonneurs et le chemin de cailloux

Le chemin de cailloux est défini par un tableau d'entiers :

```
type chemin_caillou = int array
```

Dans ce tableau, un randonneur venant de la berge de gauche est représenté par un 1, un randonneur issu de la berge de droite par un 2 et un caillou libre par un 0.

- Q14.** Écrire une fonction de signature `caillou_vide : chemin_caillou -> int` qui détermine la position du caillou inoccupé.
- Q15.** Écrire une fonction de signature `echange : chemin_caillou -> int -> int -> chemin_caillou` qui permute les valeurs codées sur deux cailloux. Le tableau d'entiers initial représentant le chemin n'est pas modifié. On pourra utiliser ici la fonction `copy` du module `Array`.
- Q16.** Écrire une fonction de signature `randonneurG_avance : chemin_caillou -> bool` qui teste si, parmi les randonneurs venant de la berge de gauche, il en existe un qui puisse avancer (vers la droite).
- Q17.** Écrire une fonction de signature `randonneurG_saute : chemin_caillou -> bool` qui teste si, parmi les randonneurs venant de la berge de gauche, il en existe un qui puisse sauter (vers la droite) au-dessus d'un randonneur.

On supposera dans la suite les fonctions de signature `randonneurD_avance : chemin_caillou -> bool` et `randonneurD_saute : chemin_caillou -> bool` écrites de manière similaire pour les randonneurs venant de la berge de droite.

Q18. Écrire une fonction de signature `mouvement_chemin : chemin_caillou -> chemin_caillou list` qui, en fonction de l'état du chemin, calcule l'état suivant après les opérations suivantes (si elles sont permises) :

- (i). déplacement d'un randonneur venant de la berge de gauche,
- (ii). déplacement d'un randonneur venant de la berge de droite,
- (iii). saut d'un randonneur venant de la berge de gauche,
- (iv). saut d'un randonneur venant de la berge de droite.

On donne la syntaxe OCaml pour créer une liste de N entiers $i : \text{List.init } N \text{ (fun } x \rightarrow i)$.
Par exemple, `List.init 5 (fun x -> 2);;` renvoie `[2; 2; 2; 2; 2]`.

Q19. Écrire une fonction de signature `passage : int -> int`, utilisant la question précédente, telle que l'appel `passage nG nD` résout le problème de passage de nG randonneurs venant de la berge de gauche et nD randonneurs venant de la berge de droite. Par exemple, `passage 3 2` permet de passer de `[1; 1; 1; 0; 2; 2]` à `[2; 2; 0; 1; 1; 1]`.

Partie III - Calcul d'une coupe minimum d'un graphe

Un agent secret a pour mission de perturber le réseau de communications ennemi en coupant certains fils dans le réseau. Ayant peu de moyens, l'agent a pour consigne de couper le moins de fils possibles pour accomplir cette tâche. Le réseau de communications est modélisé à l'aide d'un *multigraphe* non orienté $G = (S, A)$, où S est l'ensemble des sommets du graphe et A l'ensemble de ses arêtes. Résoudre le problème de l'agent secret, c'est rechercher une *coupe minimum* dans G .

Définition 4 (Multigraphe)

Un multigraphe est un graphe dans lequel des couples de mêmes sommets peuvent être reliés par plus d'une arête.

Dans toute la suite, on considérera les multigraphes sans arêtes du type (s, s) (boucles).

Définition 5 (Coupe)

Une coupe d'un multigraphe non orienté $G = (S, A)$ est une partition non triviale (S_1, S_2) de S , c'est-à-dire telle que $S_1 \cup S_2 = S$, $S_1 \cap S_2 = \emptyset$ et $S_1, S_2 \neq \emptyset$. On dira que les arêtes reliant S_1 à S_2 sont les arêtes de la coupe.

La *taille* d'une coupe (S_1, S_2) est définie par $|(S_1, S_2)| = |\{(s, t) \in A, s \in S_1, t \in S_2\}|$. Autrement dit, c'est le nombre d'arêtes de G qui relient S_1 et S_2 .

Définition 6 (Coupe minimum)

Une coupe minimum est une coupe de taille minimale.

Pour trouver une coupe minimum d'un multigraphe $G = (S, A)$, on pourrait énumérer l'ensemble des coupes possibles (il y en a $2^{|S|}$...), ou encore utiliser un algorithme de flot (le plus efficace étant en $O(|S|^3)$). Nous proposons dans la suite un algorithme probabiliste, l'algorithme de Karger, basé sur la notion de contraction d'arête.

III.1 - Contraction d'arête

Soient $G = (S, A)$ un multigraphe et $a = (s, t) \in A$ une arête de G de sommets s et t . *Contracter* l'arête a consiste à :

- (i). créer un nouveau sommet $u = (st)$ dans S . u est un *supersommet* du nouveau graphe,
- (ii). pour toute arête (r, s) ou (r, t) , $r \in S$, ajouter une arête (r, u) à A . Dans le cas où (r, s) et (r, t) existent, deux arêtes reliant r à u sont créées.
- (iii). Supprimer de A toutes les arêtes ayant s ou t comme extrémité.
- (iv). Supprimer s et t de S .

Un supersommet peut donc être vu comme un ensemble de deux sommets du graphe initial, obtenu après une contraction d'arête.

Le multigraphe obtenu est appelé *graphe contracté* et est noté G/st . Si plusieurs arêtes relient s à t dans G , contracter l'une d'entre elles supprime toutes les autres du graphe G/st . Dans toute la suite, on notera G_0 le graphe initial, avant transformations par contractions.

On considère le graphe G_0 de la **figure 2**.

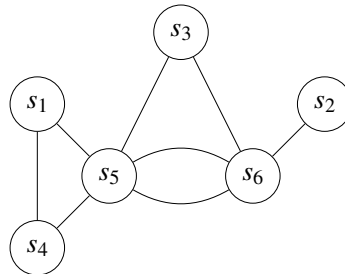


Figure 2 - Graphe G_0 exemple

Q20. Donner le graphe obtenu par contraction d'une arête (s_5, s_6) .

Après plusieurs contractions, un supersommet u du graphe résultant contient un sous-ensemble $S_u \subseteq S$ de sommets de G .

Q21. Soit u un supersommet et $s, t \in S_u$. Montrer qu'il existe un chemin Γ entre s et t dans G_0 tel que chaque arête de Γ a été contractée.

Q22. Montrer qu'en contractant une arête s, t dans un multigraphe G , la taille d'une coupe minimum du graphe contracté G/st est au moins égale à la taille d'une coupe minimum de G .

Q23. Montrer que la taille d'une coupe minimum de G/st est strictement plus grande que celle d'une coupe minimum de G si et seulement si l'arête (s, t) est une arête de toutes les coupes minimum de G .

III.2 - Premier algorithme

On construit un premier algorithme qui essaye de trouver une coupe minimum en contractant aléatoirement une arête de G , jusqu'à ce qu'il ne reste que deux sommets (**algorithme 3**).

Algorithme 3 - Algorithme de Karger.

Karger(G, n)

Entrées : $G = (S, A)$ multigraphe non orienté, $n \in \mathbb{N}$

Sorties : Une coupe minimum de G .

début

pour $i = |S|$ à n **en décrémentant de 1 faire**

 Tirer aléatoirement (loi uniforme) une arête $a = (s, t) \in A$

$G = G/st$

// Appel

Karger($G, 2$)

Q24. Appliquer l'**algorithme 3** au graphe de la **figure 2**, en contractant successivement (s_5, s_6) , $((s_5 s_6), s_2)$, (s_1, s_4) et $((s_5 s_6 s_2), s_3)$. Chaque graphe contracté sera représenté. La coupe calculée est-elle une coupe minimum ?

D'après la **Q23**, tant que l'on ne contracte pas une arête faisant partie de toutes les coupes minimum de G , alors l'**algorithme 3** trouvera une coupe minimum. On cherche alors la probabilité de ne jamais contracter une telle arête.

Q25. Soit $d(s)$ le degré d'un sommet $s \in S$. Montrer que $\sum_{s \in S} d(s) = 2|A|$ (lemme des poignées de main).

Q26. En déduire qu'une coupe minimum a une taille d'au plus $2|A|/|S|$.

Soit $C = (S_1, S_2)$ une coupe minimum de G . L'objet des questions **Q27** et **Q28** est de minorer la probabilité que l'**algorithme 3** renvoie C . Pour cela, on remarque que cet algorithme ne renvoie pas C si et seulement si lors d'une itération on choisit une arête qui traverse C , c'est-à-dire telle qu'un sommet est dans S_1 et l'autre dans S_2 .

Q27. Quelle est la probabilité maximum de choisir une arête qui traverse C ?

Q28. En déduire que la probabilité $P_{|S|}$ que l'**algorithme 3** renvoie C est supérieure ou égale à $\frac{2}{|S|(|S| - 1)}$.

Q29. Déduire de la question précédente qu'un multigraphe non orienté $G = (S, A)$ possède au plus $\frac{|S|(|S| - 1)}{2}$ coupes minimum.

III.3 - Deuxième algorithme

Le résultat précédent n'est pas satisfaisant d'un point de vue complexité. On peut cependant l'améliorer facilement en utilisant une technique d'*amplification* : on itère l'**algorithme 3** plusieurs fois et on retourne la valeur minimale obtenue (**algorithme 4**).

Algorithme 4 - Algorithme de Karger amplifié.

KargerAmplifie(G, N)

Entrées : $G = (S, A)$ multigraphe non orienté, $N \in \mathbb{N}$.

Sorties : Une coupe minimum de G .

début

```

     $t = \infty$ 
    pour  $i = 1$  à  $N$  faire
         $X = \text{Karger}(G, 2)$ 
        si  $|X| < t$  alors
             $t = |X|$ 
             $C = X$ 
    retourner  $C$ 

```

Q30. Montrer que la probabilité maximale que l'**algorithme 4** ne trouve pas une coupe minimum est égale à $\left(1 - \frac{2}{|S|(|S| - 1)}\right)^N$.

Q31. On rappelle que pour tout $x \in \mathbb{R}$, $1 + x \leq e^x$. Montrer que la probabilité que l'**algorithme 4** renvoie une coupe minimum est supérieure à $1 - e^{2N/|S|(|S|-1)}$. En déduire, pour $c > 0$ fixé, la plus petite valeur de N telle que cette probabilité soit supérieure à $1 - \frac{1}{N^c}$.

Q32. Les algorithmes proposés dans cette partie sont des algorithmes probabilistes. Sont-ils de type Monte Carlo ou Las Vegas ? Justifier la réponse.

III.4 - Implémentation en langage C

Cette partie comporte des questions de programmation qui seront abordées en utilisant **exclusivement le langage C**. Les codes seront commentés de manière pertinente.

On suppose qu'un graphe est codé à l'aide de l'ensemble de ses arêtes, chaque arête étant définie par ses deux sommets, représentés par des entiers.

Q33. Définir en C des types structurés *Arete* et *Graphe*. Pour ce dernier, on s'attachera à avoir un accès direct au nombre de sommets et au nombre d'arêtes. On rappelle la définition d'un type structuré par `struct nom_s {type1 champ1;...;typen champn;} et ensuite typedef struct nom_s nom;.`

Pour tout u supersommet, les S_u forment une partition de S . Pour coder ces sous-ensembles, on a donc naturellement recours à la structure de données Unir & Trouver (Union-Find), qui va permettre de trouver rapidement à quel sous-ensemble S_u un sommet $s \in S$ appartient (Trouver) et de fusionner deux supersommets S_u et S_v déjà construits (Unir).

On suppose donc disposer :

- (i). d'un type `subset`, décrivant un supersommet d'un graphe contracté.

```
struct subset
{
    int parent; // recherche par compression de chemin pour Trouver
    int rang;   // Union par rang.
};
typedef struct subset subset;
```

- (ii). d'une fonction de prototype `int Trouver(subset subsets[], int s)`, qui retourne le numéro du supersommet dans lequel le sommet s se trouve (par compression de chemin),
- (iii). d'une fonction de prototype `void Unir(subset subsets[], int Su, int Sv)` qui fusionne les deux supersommets S_u et S_v (union par rang) et maintient à jour la liste des supersommets.

Il n'est pas demandé d'écrire ces deux dernières fonctions.

- Q34.** Écrire une fonction de prototype `int contracteArete(Graphe G, subset subsets[], Arete a)` qui contracte l'arête a . On veillera à ne pas contracter l'arête si ses deux extrémités sont dans le même supersommet. La fonction renvoie 0 si aucune arête est contractée et -1 sinon.
- Q35.** Écrire une fonction de prototype `int compteAretesCoupe(Graphe G, subset subsets[])` qui compte le nombre d'arêtes du graphe contracté final G résultant de l'**algorithme 3**. `subsets` est la partition des sommets du graphe initial S calculée par l'**algorithme 3**.
- Q36.** En déduire une fonction de prototype `int kargerMinCut(Graphe G0)` qui renvoie la taille de la coupe calculée par l'**algorithme 3** sur le graphe G_0 . Les supersommets initiaux sont les $|S|$ sommets de G_0 , chacun ayant un rang nul et un numéro parent égal au numéro du sommet. Pour effectuer un tirage aléatoire uniforme, on utilisera la fonction `int rand()`; de la librairie `stdlib.h` qui renvoie un nombre aléatoire entre 0 et `RAND_MAX` ≥ 32767 . `RAND_MAX` est une constante définie dans `stdlib.h`.
- Q37.** Donner la complexité au pire des cas de cet algorithme.

FIN

**ECOLE POLYTECHNIQUE
ECOLES NORMALES SUPERIEURES**

CONCOURS D'ADMISSION 2024

JEUDI 18 AVRIL 2024

08h00 - 12h00

FILIERE MPI - Epreuve n° 7

INFORMATIQUE C (XULSR)

Durée : 4 heures

***L'utilisation des calculatrices n'est pas autorisée pour
cette épreuve***

Égalités et différences

Le sujet comporte 17 pages, numérotées de 1 à 17 et une annexe.

Vue d'ensemble du sujet.

Ce sujet traite du problème de satisfiabilité de formules construites par la conjonction de contraintes d'égalité et de différence (dis-égalité) entre expressions. Ce problème se présente, par exemple, dans la compilation de programmes, quand on cherche à réduire des expressions complexes (nécessitant beaucoup de calculs) vers des expressions plus simples, en utilisant les propriétés des opérateurs apparaissant dans ces expressions. Par exemple, si x est une variable entière, $(x * 2) / 2 == x$ est une contrainte d'égalité entre deux expressions écrites en C. De même, si y et z sont des variables booléennes, l'égalité $(y || (y \&\& z)) == y$ permet de simplifier l'expression C écrite à gauche de l'égalité.

Il est important de savoir si un ensemble donné de contraintes d'égalité ou de différence est satisfiable. Dans la suite, P_L *est le problème suivant* : « **Étant donnée une formule F , conjonction de contraintes de la classe L , décider si F est satisfiable** ».

Ce sujet propose d'étudier la complexité du problème P_L et les algorithmes pour le résoudre dans le cas de quatre classes de contraintes :

La partie I considère la classe L_1 où les contraintes ont la forme $x \leftrightarrow y$ ou $x \leftrightarrow \neg z$ avec x , y et z des variables propositionnelles (c'est-à-dire, ayant des valeurs booléennes).

La partie II concerne la classe L_2 incluant les contraintes de la classe L_1 et celles de la forme $x \leftrightarrow (y \wedge z)$ (entre autres) avec x , y et z des variables propositionnelles. Les algorithmes des deux premières parties seront codés en OCaml.

La partie III s'intéresse à la classe L_3 des contraintes de la forme $x = y$ ou $x \neq y$ avec x et y des variables entières.

Enfin, la partie IV concerne les contraintes d'égalité et de différence entre les expressions construites à partir de variables entières et de fonctions à deux arguments. Les algorithmes des deux dernières parties seront codés en C.

Les parties III et IV peuvent être traitées indépendamment de parties I et II. Il est recommandé de traiter les parties I et II de manière séquentielle. De même pour les parties III et IV.

Notations

Les contraintes seront construites en utilisant un ensemble $\mathbb{X}$ de n variables notées X_i avec $1 \leq i \leq n$. Une valeur de vérité est un élément de $\mathbb{B} \stackrel{\text{def}}{=} \{0, 1\}$ où 1 représente le vrai et 0 le faux. Une valeur entière k est un élément de $\mathbb{Z}$ et sa valeur absolue est notée $|k|$. Étant donnés $k, m \in \mathbb{Z}$ avec $k \leq m$, l'intervalle $[k; m]$ est l'ensemble des $i \in \mathbb{Z}$ tels que $k \leq i \leq m$.

Logique propositionnelle. On utilisera les formules de la logique propositionnelle construites à partir de l'ensemble de variables propositionnelles $\mathbb{X}$, les constantes propositionnelles $\perp$ (faux) et $\top$ (vrai), les connecteurs logiques classiques $\neg$ (négation), $\wedge$ (conjonction), $\vee$ (disjonction). On définit $F \rightarrow G$ (implication) par $(\neg F \vee G)$, et $F \leftrightarrow G$ (équivalence) par $(F \rightarrow G) \wedge (G \rightarrow F)$. Pour les connecteurs $\wedge$ et $\vee$, on utilise leur forme k -aire avec $k \geq 0$. Par exemple, $X_1 \wedge X_2 \wedge \dots \wedge X_k$ est une formule utilisant une conjonction k -aire. Par convention, une conjonction 0-aire est $\top$, et une disjonction 0-aire est $\perp$; une conjonction (resp. disjonction) unaire est identique à son opérande.

Un **littéral** ℓ est soit une variable X_i soit sa négation $\neg X_i$. Une formule est en **forme normale conjonctive** (CNF) si c'est une conjonction de disjonctions de littéraux. Une formule est en forme k -CNF avec $k > 0$, si elle est en forme CNF et le nombre de littéraux différents dans chaque disjonction est au plus k . Par exemple, la formule $(X_2 \vee X_3 \vee \neg X_4) \wedge (\neg X_3 \vee X_4) \wedge X_1$ est en forme 3-CNF.

On rappelle qu'une formule propositionnelle F peut être **satisfaite** ou non par une valuation $v : \mathbb{X} \rightarrow \mathbb{B}$, qui assigne une valeur de vérité à chaque variable. On note $v \models F$ la relation de satisfaction. Une formule F' est **conséquence logique** de F , noté $F \models F'$, quand, pour toute valuation v telle que $v \models F$, on a aussi $v \models F'$. Deux formules sont **logiquement équivalentes** lorsque chacune est conséquence logique de l'autre (ou, de façon équivalente, quand elles sont satisfaites par les mêmes valuations). Une formule est une **tautologie** si elle est satisfaite par toutes les valuations. Une formule est une **contradiction** (antilogie) ou contradictoire si elle n'est satisfaite par aucune valuation. Une formule F est **satisfiable** s'il existe une valuation v telle que $v \models F$.

L'annexe rappelle certaines règles de la déduction naturelle.

Complexité. Par la complexité d'un algorithme A on entend le nombre d'opérations élémentaires (comparaison, addition, soustraction, multiplication, division, affectation, test, etc.) nécessaires à l'exécution de A **dans le pire cas**. Lorsque la complexité dépend d'un ou plusieurs paramètres $k_0, \dots, k_r$, on dit que A a une complexité en $O(f(k_0, \dots, k_r))$ s'il existe une constante $C > 0$ telle que, pour toutes les valeurs de $k_0, \dots, k_r$ suffisamment grandes (c'est-à-dire plus grandes qu'un certain seuil), pour toute instance du problème de paramètres $k_0, \dots, k_r$ la complexité est au plus $C \times f(k_0, \dots, k_r)$. De même, on dit que A a une complexité en $\Theta(f(k_0, \dots, k_r))$ s'il existe deux constantes $C_1 > 0$ et $C_2 > 0$ telles que, pour toutes les valeurs de $k_0, \dots, k_r$ suffisamment grandes, la complexité de A est au moins $C_1 \times f(k_0, \dots, k_r)$ et au plus $C_2 \times f(k_0, \dots, k_r)$. Dans tout ce sujet, on note « log » le logarithme à base 2, et on utilisera exclusivement ce logarithme.

Programmation. Dans les questions de programmation en C et OCaml, on n'utilise que les fonctions qui sont incluses dans la bibliothèque standard du langage.

Pour le code écrit en C, on suppose que les en-têtes `stdbool.h`, `stdio.h`, `stdlib.h` et `assert.h` ont été inclus.

On rappelle quelques opérations sur les listes en OCaml.

- **List.iter** *f lst* applique la fonction *f*: 'a -> unit à chaque élément de la liste *lst* de type 'a list. Si la complexité de *f* est en $O(1)$, alors la complexité de **List.iter** est en $O(k)$, où *k* est la longueur de *lst*.
- **List.map** *f lst* renvoie la liste de type 'b list obtenue en appliquant la fonction *f*: 'a -> 'b à chaque élément de la liste *lst* de type 'a list. Si la complexité de *f* est en $O(1)$, alors la complexité de **List.map** est en $O(k)$, où *k* est la longueur de *lst*.
- **List.mem** *e lst* renvoie **true** si et seulement si *e* est un élément de *lst*. Si la complexité du test d'égalité est en $O(1)$, la complexité de **List.mem** est en $O(k)$, où *k* est la longueur de *lst*.
- **List.exists** *f lst* renvoie **true** si et seulement s'il existe au moins un élément *e* de *lst* (liste de type 'a list) tel que *f e* renvoie **true** avec *f*: 'a -> bool. Si la complexité de *f* est en $O(1)$, alors la complexité de **List.exists** est en $O(k)$, où *k* est la longueur de *lst*.
- **List.for_all** *f lst* renvoie **true** si et seulement si tous les éléments de *lst* (de type 'a list) satisfont le prédicat *f*: 'a -> bool. Si la complexité de *f* est en $O(1)$, alors la complexité de **List.for_all** est en $O(k)$, où *k* est la longueur de *lst*.
- **List.filter** *f lst* renvoie la liste de tous les éléments de *lst* (de type 'a list) qui satisfont le prédicat *f*: 'a -> bool. Si la complexité de *f* est en $O(1)$, alors la complexité de **List.filter** est en $O(k)$, où *k* est la longueur de *lst*.

On rappelle quelques opérations sur les tableaux en OCaml.

- **Array.length** *tab* renvoie la longueur du tableau *tab* en temps $O(1)$.
- **Array.make** *k v* crée un tableau de *k* éléments, tous initialisés à *v*, en temps $O(k)$.
- La valeur à la case numéro *i* du tableau *tab* est *tab.(i)*. Les cases sont numérotées à partir de zéro et l'accès est fait en temps $O(1)$.
- L'affectation de la case *i* du tableau *tab* avec la valeur *v* s'écrit *tab.(i) <- v*; sa complexité est en $O(1)$.
- **Array.copy** *tab* crée une copie du tableau *tab* en temps $O(k)$ avec *k* la longueur du tableau en argument.

Partie I. Équivalences de littéraux

Dans cette partie, on se place dans le cadre de la logique propositionnelle. On définit L_1 comme la classe des contraintes de la forme $\ell \leftrightarrow \ell'$ avec ℓ et ℓ' des littéraux. Une formule F est dite construite sur L_1 , notation $F \in \mathcal{F}(L_1)$, si elle est une conjonction de contraintes de L_1 . *Dans cette partie, on considère que la variable X_1 est toujours interprétée à 1, donc qu'elle représente la valeur de vérité vrai.* La formule F_0 définie ci-dessous est dans $\mathcal{F}(L_1)$:

$$F_0 \stackrel{\text{def}}{=} (X_3 \leftrightarrow \neg X_2) \wedge (X_4 \leftrightarrow X_3) \wedge (X_3 \leftrightarrow X_4) \wedge (X_1 \leftrightarrow \neg X_5)$$

Question 1. Donner une formule logiquement équivalente à F_0 en forme 2-CNF. ┘

Question 2. Donner une réduction en temps linéaire du problème P_{L_1} au problème de satisfiabilité des formules en 2-CNF. ┘

Mise en œuvre en OCaml. Une contrainte de L_1 est représentée par un couple d'entiers (i, j) avec $i, j \in [1; 2n]$. Pour tout $i \in [1; 2n]$, le littéral représenté par i , noté $\text{lit}(i)$, est X_i si $i \in [1; n]$ et $\neg X_{i-n}$ si $i \in [n+1; 2n]$. Par la convention sur l'interprétation de X_1 , l'entier $i = 1$ (resp. $i = n+1$) représente la valeur de vérité 1 (resp. 0). Enfin, une formule dans $\mathcal{F}(L_1)$ est représentée par la liste des couples correspondant aux contraintes dont la formule est conjonction. Par convention, la liste vide représente $\top$. Le nombre de variables, n , est une variable globale entière. Ce codage est donné par les déclarations OCaml ci-dessous :

```
1 type contr1 = int * int    (* contraintes L1 *)
2 type eform1 = contr1 list  (* formules L1 *)
3 exception Bad_literal of int
4 exception Exn_unsat
```

Les exceptions déclarées ci-dessus seront utilisées dans les fonctions de cette partie et de la partie suivante.

Question 3. Écrire les fonctions suivantes :

- `var_of_lit: int -> int` renvoie l'index de la variable apparaissant dans le littéral représenté par l'argument i . Par exemple, pour $n = 5$, la fonction renvoie 3 si l'argument est 3, et 2 si l'argument est 7. Si $i \notin [1; 2n]$ alors la fonction exécute `raise (Bad_literal i)`.
- `neg_lit: int -> int` renvoie, pour un argument i , l'entier j tel que $\text{lit}(j)$ est logiquement équivalent à $\neg \text{lit}(i)$; si $i \notin [1; 2n]$ alors la fonction exécute `raise (Bad_literal i)`.
- `eform1_is_wf: eform1 -> bool` renvoie `true` si et seulement si l'argument est « bien formé », c'est-à-dire qu'il contient uniquement des entiers qui représentent des littéraux sur l'ensemble de variables $\mathbb{X}$.

Dans la suite de ce sujet, on suppose que tous les entiers utilisés par les valeurs de type `contr1` et `eform1` représentent des littéraux sur l'ensemble de variables $\mathbb{X}$. Par conséquent, l'exception `Bad_literal` ne doit pas être levée. ┘

Graphe d'une formule. Le *graphe associé* à $F \in \mathcal{F}(L_1)$, noté $G_F = (V_F, E_F)$, est un graphe non orienté dont les sommets V_F sont les entiers dans l'intervalle $[1; 2n]$. Les arêtes E_F sont les ensembles $\{i, j\}$ tels que $i, j \in V_F$, $i \neq j$ et F contient soit la contrainte $\text{lit}(i) \leftrightarrow \text{lit}(j)$, soit la contrainte $\text{lit}(i') \leftrightarrow \text{lit}(j')$ avec i' et j' égaux respectivement à $\text{neg_lit}(i)$ et $\text{neg_lit}(j)$. Par exemple, pour $n = 5$, la contrainte $(X_3 \leftrightarrow X_4)$ est représentée par les arêtes $\{3, 4\}$ et $\{8, 9\}$. On note que les contraintes de F de la forme $\ell \leftrightarrow \ell$ ne sont pas représentées par E_F .

Question 4. On suppose $n = 5$. Pour chacune des formules suivantes, donner le graphe associé :

$$\begin{aligned} F_1 &\stackrel{\text{def}}{=} (X_3 \leftrightarrow X_4) \wedge (X_4 \leftrightarrow X_2) \wedge (X_3 \leftrightarrow \neg X_2) \wedge (X_5 \leftrightarrow X_2) \\ F_2 &\stackrel{\text{def}}{=} (\neg X_4 \leftrightarrow X_2) \wedge (X_2 \leftrightarrow \neg X_3) \wedge (X_4 \leftrightarrow \neg X_5) \wedge (X_3 \leftrightarrow X_5) \end{aligned}$$

┘

On suppose disposer d'une bibliothèque munie d'un type **graph** et qui code des graphes non orientés. Les sommets sont représentés par des entiers. Les opérations suivantes sont disponibles pour la construction et l'exploration des graphes :

- **g_create** : **int** -> **graph** renvoie une valeur représentant un graphe sans arêtes et ayant k sommets, où k est donné en argument, supposé strictement positif. Les sommets sont numérotés de 1 à k . La complexité de cette opération est en $O(k)$.
- **g_add_edge** : **graph** -> **int** -> **int** -> **unit** ajoute au graphe donné comme premier argument l'arête entre les sommets donnés en 2ème et 3ème arguments. Si les sommets en argument ne sont pas déclarés dans le graphe ou si l'arête existe déjà, alors cette fonction ne change pas le graphe. Sinon, le graphe est modifié en place. La complexité de cette opération est en $O(k)$, avec k le nombre de sommets du graphe.
- **g_nb_vertex** : **graph** -> **int** renvoie le nombre de sommets du graphe en argument. La complexité de cette opération est en $O(1)$.
- **g_succ** : **graph** -> **int** -> **int list** renvoie la liste des sommets adjacents du sommet en second argument dans le graphe donné en premier argument. Si le sommet en argument n'existe pas, la liste renvoyée est vide. La complexité de cette opération est en $O(1)$.

Question 5. Écrire une fonction **eform1_to_graph**: **eform1** -> **graph** qui renvoie le graphe associé à la formule F représentée par l'argument. Si F contient une contrainte de la forme $\ell \leftrightarrow \neg \ell$, alors la fonction lève l'exception **Exn_unsat**. ┘

Lorsqu'un appel à **eform1_to_graph lst** lève l'exception **Exn_unsat**, la formule représentée par **lst** contient toujours une contrainte contradictoire. Cependant, une formule peut être une contradiction sans que l'exception soit levée, comme c'est le cas de la formule $(X_1 \leftrightarrow X_2) \wedge (X_2 \leftrightarrow \neg X_1)$.

Question 6. Donner et justifier soigneusement la complexité de **eform1_to_graph** en fonction de n et de la longueur p de la liste en argument. ┘

Soient $F \in \mathcal{F}(L_1)$ une formule et **g** la représentation de G_F , le graphe associé à F . On note par $CC(G_F)$ l'ensemble des ensembles de sommets des composantes connexes de G_F . Soit m la taille de $CC(G_F)$, c'est-à-dire, le nombre de composantes connexes de G_F . On représente $CC(G_F)$ par un tableau **c** de $2n + 1$ entiers, qui associe à chaque $i \in [1; 2n]$, sommet de **g**, un entier dans l'intervalle $[1; m]$. Ainsi, **c.(i)** identifie de manière unique la composante connexe du sommet **i**. La case de **c** à la position 0 ne sera pas utilisée, mais elle permet d'utiliser directement les indices des variables comme positions de **c**. Par convention, **c.(0)** est 0.

Question 7. Écrire une fonction `g_cc_array: graph -> (int array * int)` qui renvoie le couple (c, m) , où c est le tableau des composantes connexes du graphe en argument et où m est le nombre de composantes connexes. ┘

Question 8. Démontrer que si i et j sont des sommets distincts appartenant à la même composante connexe de G_F , alors $F \models \text{lit}(i) \leftrightarrow \text{lit}(j)$. ┘

Question 9. Soit $C = \{i_1, \dots, i_q\} \in CC(G_F)$ l'ensemble des sommets d'une composante connexe de G_F . Soit C' l'ensemble de sommets $\{j_1, \dots, j_q\}$ avec $j_k = \text{neg_lit}(i_k)$ pour tout $k \in [1; q]$. Démontrer que C' est élément de $CC(G_F)$. ┘

Question 10. Démontrer que F est une contradiction si et seulement s'il existe une composante connexe C de G_F et un sommet i de C tels que $\text{neg_lit}(i)$ est un sommet de C . ┘

Question 11. Écrire une fonction `cc_is_sat: int array -> bool` qui a comme argument c , le tableau représentant les composantes connexes de G_F et qui renvoie `true` si et seulement si F est satisfiable. ┘

Représentation d'une valuation. Une valuation v de $\mathbb{X}$ est représentée par un tableau v de $n + 1$ valeurs 0 ou 1 tel que $v.(i)$ est 1 si et seulement si $v(X_i) = 1$. En particulier, $v.(1)=1$ par convention sur X_1 . La case à la position 0 n'est pas utilisée, mais elle permet d'utiliser les indices des variables comme positions du tableau ; par convention, $v.(0)$ est 0.

Question 12. Écrire une fonction `eform1_sat: eform1 -> (int array) option` qui a comme argument une liste `lst` représentant une formule $F \in \mathcal{F}(L_1)$. L'appel `eform1_sat lst` renvoie `None` si F est une contradiction. Si F est satisfiable, l'appel renvoie `Some(v)` avec v un tableau qui représente une valuation qui satisfait F . La complexité en temps de l'algorithme doit être en $O(p \times n)$ où p est la longueur de la liste `lst` en argument. Justifier que votre algorithme a cette complexité en temps. ┘

Partie II. Équivalences avec une conjonction de deux littéraux

On reste dans le cadre de la logique propositionnelle, mais on considère une nouvelle classe de contraintes. La classe L_2 contient les contraintes de la forme $\ell \leftrightarrow (\ell' \wedge \ell'')$ où ℓ, ℓ', ℓ'' sont des littéraux. Pour identifier les contraintes des classes L_1 et L_2 , on appelle *i-contrainte* une contrainte de la classe L_i avec $i \in \{1, 2\}$. Une formule F est dite construite sur L_2 , notation $F \in \mathcal{F}(L_2)$, si elle est une conjonction de 2-contraintes. **On garde l'hypothèse que X_1 est toujours interprétée à 1.** Un exemple de formule de $\mathcal{F}(L_2)$ est :

$$F_3 \stackrel{\text{def}}{=} (X_1 \leftrightarrow (X_2 \wedge \neg X_4)) \wedge (\neg X_2 \leftrightarrow (X_2 \wedge X_3)) \wedge (X_4 \leftrightarrow (\neg X_1 \wedge X_5))$$

Question 13. Donner une réduction en temps polynomial du problème P_{L_2} au problème de satisfiabilité des formules en 3-CNF. $\lrcorner$

Question 14. Donner une dérivation formelle (preuve en déduction naturelle) du séquent suivant :

$$X_2 \rightarrow (\neg X_2 \wedge X_3), (\neg X_2 \wedge X_3) \rightarrow X_2 \vdash \neg X_2.$$

Les règles de la déduction naturelle à utiliser dans cette preuve sont rappelées dans l'annexe. $\lrcorner$

En utilisant la convention sur l'interprétation de X_1 , toute 1-contrainte $\ell \leftrightarrow \ell'$ est logiquement équivalente à la 2-contrainte $\ell \leftrightarrow (\ell' \wedge X_1)$. À l'inverse, certaines 2-contraintes sont logiquement équivalentes à des formules de $\mathcal{F}(L_1)$. Par exemple, les équivalences logiques suivantes sont vraies pour tout couple de littéraux ℓ et ℓ' (les preuves ne sont pas demandées) :

- | | | |
|--|-------------------------------------|-----|
| $\ell \leftrightarrow (X_1 \wedge \ell')$ équivalent à $\ell \leftrightarrow \ell'$ | par 1 élément neutre de $\wedge$ | (1) |
| $\ell \leftrightarrow (\neg X_1 \wedge \ell')$ équivalent à $\ell \leftrightarrow \neg X_1$ | par 0 élément absorbant de $\wedge$ | (2) |
| $\ell \leftrightarrow (\ell' \wedge \ell')$ équivalent à $\ell \leftrightarrow \ell'$ | par idempotence de $\wedge$ | (3) |
| $\ell \leftrightarrow (\ell' \wedge \neg \ell')$ équivalent à $\ell \leftrightarrow \neg X_1$ | par contradiction | (4) |
| $X_1 \leftrightarrow (\ell \wedge \ell')$ équivalent à $(X_1 \leftrightarrow \ell) \wedge (X_1 \leftrightarrow \ell')$ | par définition de $\wedge$ | (5) |
| $\ell \leftrightarrow (\neg \ell \wedge \ell')$ équivalent à $(\neg X_1 \leftrightarrow \ell) \wedge (\neg X_1 \leftrightarrow \ell')$ | admis | (6) |

Dans la suite de cette partie, on dénote par $\mathbb{E}(\mathbb{X})$ toutes les équivalences définies sur l'ensemble des littéraux par les équivalences de (1) à (6) et par leur variantes obtenues en utilisant la symétrie de la conjonction. Par exemple, « $\ell \leftrightarrow (\ell' \wedge X_1)$ équivalent à $\ell \leftrightarrow \ell'$ » est également une équivalence de $\mathbb{E}(\mathbb{X})$, grâce à la symétrie de la conjonction appliquée à (1).

Mise en œuvre en OCaml. Une 2-contrainte $\ell \leftrightarrow (\ell' \wedge \ell'')$ est représentée par un enregistrement avec trois champs entiers. Les entiers de ces enregistrements prennent des valeurs dans $[1; 2n]$ et représentent des littéraux avec la même convention de codage qu'en première partie. Une formule dans $\mathcal{F}(L_2)$ est représentée par une liste d'enregistrements; la liste vide représente $\top$. Ce codage est donné par les types OCaml suivants :

```

1 type contr2 = { ll: int; lr1: int; lr2: int } (* contraintes L2 *)
2 type eform2 = contr2 list                    (* formules L2 *)
```

Une valeur e de type `contr2` représente la 2-contrainte $e.l1 \leftrightarrow (e.lr1 \wedge e.lr2)$.

Dans la suite de cette partie, on suppose que toutes les valeurs des types `contri` et `eformi` avec $i \in \{1, 2\}$ sont bien formées, c'est-à-dire que les entiers qu'elles contiennent représentent des littéraux sur $\mathbb{X}$.

Question 15. Écrire la fonction `contr2_simplify_eq5: contr2 -> eform1 option` qui applique l'équivalence (5) pour simplifier la 2-contrainte en argument. L'appel `contr2_simplify_eq5 e2` :

- renvoie **None** si la simplification ne peut pas être faite avec (5) ;
- lève l'exception **Exn_unsat** (déclarée en partie I) si la simplification peut être faite mais la 2-contrainte représentée par `e2` est une contradiction ;
- enfin, renvoie **Some** `[(1,e2.lr1);(1,e2.lr2)]` si la simplification peut être effectuée sans détecter de contradiction. ┐

Valuation partielle. Une valuation partielle est une fonction $v : \mathbb{X} \rightarrow \{-1, 0, 1\}$ telle que $v(X_1) = 1$. Pour tout $i \neq 1$, si $v(X_i) = b \in \mathbb{B}$ alors on dit que v assigne X_i à b . Si $v(X_i) = -1$ alors on dit que X_i n'est pas assignée par v . On étend la notion de valuation partielle aux littéraux et aux formules comme suit :

- $v(\neg X_i) = \neg b$ si $v(X_i) = b$ avec $b \in \mathbb{B}$, et $v(\neg X_i) = -1$ si $v(X_i) = -1$.
- Soit la 1-contrainte $\ell \leftrightarrow \ell'$. Si $v(\ell), v(\ell') \in \mathbb{B}$ alors $v(\ell \leftrightarrow \ell') = 1$ si $v(\ell) = v(\ell')$ et 0 sinon. Si au moins une des valeurs $v(\ell)$ ou $v(\ell')$ est -1 , alors $v(\ell \leftrightarrow \ell') = -1$.
- Pour la conjonction de deux littéraux $\ell \wedge \ell'$, $v(\ell \wedge \ell') = -1$ si au moins une des valeurs $v(\ell)$ ou $v(\ell')$ est -1 ; sinon la valeur de $v(\ell \wedge \ell')$ est 1 si et seulement si $v(\ell) = 1$ et $v(\ell') = 1$.
- Soit la 2-contrainte $\ell \leftrightarrow (\ell' \wedge \ell'')$. Si $v(\ell), v(\ell' \wedge \ell'') \in \mathbb{B}$ alors $v(\ell \leftrightarrow (\ell' \wedge \ell'')) = 1$ si $v(\ell) = v(\ell' \wedge \ell'')$ et 0 sinon. Si au moins une des valeurs $v(\ell)$ ou $v(\ell' \wedge \ell'')$ est -1 , alors $v(\ell \leftrightarrow (\ell' \wedge \ell'')) = -1$.

La **valeur d'une formule** $F_i \in \mathcal{F}(L_i)$ où $i \in \{1, 2\}$ pour une valuation partielle v , notation $v(F_i)$, est -1 s'il existe une contrainte e de F_i telle que $v(e) = -1$; sinon, $v(F_i) = 1$ si pour toute contrainte e de F_i on a $v(e) = 1$, et $v(F_i) = 0$ sinon.

Une valuation partielle v est **compatible** avec une valuation v' si pour tout $X_i \in \mathbb{X}$ tel que $v(X_i) \in \mathbb{B}$ on a $v'(X_i) = v(X_i)$.

Une valuation partielle v est représentée en OCaml par un tableau `v` de taille $n + 1$ dont les éléments appartiennent à $\{-1, 0, 1\}$. Comme pour les valuations, la case de position 0 n'est pas utilisée et, par convention, `v.(0)=0`.

Question 16. Écrire la fonction `eform1_of_valp: int array -> eform1` qui renvoie une formule F de $\mathcal{F}(L_1)$ telle que F encode les assignations de la valuation partielle `v` en argument. Ainsi, un appel à `eform1_of_valp v` renvoie la liste de couples (i, j) tels que $i > 0$, `v.(i)  $\neq -1$` et si `v.(i) = 1` alors $j = 1$ et si `v.(i) = 0` alors $j = n + 1$.

Soit F la formule représentée par le résultat de l'appel à `eform1_of_valp v`. Démontrez que, pour toute valuation v' on a $v' \models F$ si et seulement si `v` est compatible avec v' . ┐

Une 2-contrainte peut être simplifiée en présence d'une valuation partielle v en une formule de $\mathcal{F}(L_1)$. Par exemple, si $v(X_2) = 1$ alors la 2-contrainte $X_2 \leftrightarrow (\ell' \wedge \ell'')$ a la même valeur que la 1-contrainte $(X_1 \leftrightarrow \ell') \wedge (X_1 \leftrightarrow \ell'')$ pour la valuation partielle v . On suppose donnée une fonction `contr2_simplify_valp: contr2 -> int array -> eform1 option` qui applique les équivalences de $\mathbb{E}(\mathbb{X})$ pour simplifier la 2-contrainte `e2` en premier argument en tenant compte de

la valuation partielle vp donnée comme second argument. Plus précisément, si $vp(e2)=0$, alors la fonction lève l'exception **Exn_unsat**. Si aucune simplification ne s'applique, alors le résultat est **None**. Si $e2$ est simplifiable, le résultat est **Some(1st)** avec $1st$ une liste représentant une formule F'_1 de $\mathcal{F}(L_1)$ obtenue par simplification de $e2$ telle que $vp(e2) = vp(F'_1)$. La complexité de `contr2_simplify_valp` est en $O(1)$.

Algorithme. Les questions suivantes vous guident pour prouver la correction et la terminaison de l'algorithme de décision pour P_{L_2} qui est mis en œuvre dans la fonction `eform2_sat` de la figure 1.

On cherche à démontrer que la fonction `eform2_sat` est correcte, c'est-à-dire qu'un appel à `eform2_sat 1st0` doit renvoyer **None** si la formule $F_0 \in \mathcal{F}(L_2)$ représentée par `1st0` est une contradiction. Sinon, l'appel doit renvoyer **Some(v)** où v est un tableau qui représente une valuation qui satisfait F_0 . L'algorithme mis en œuvre pour `eform2_sat` est un algorithme de type retour sur trace (*backtracking*) dont le principe est de réduire la formule $F_0 \in \mathcal{F}(L_2)$ en une formule de $\mathcal{F}(L_1)$ afin d'appliquer `eform1_sat`, l'algorithme polynomial de la partie I. Pour effectuer cette réduction, `eform2_sat` appelle à la ligne 5 la fonction récursive `eform2_sat_aux` dont les arguments sont une liste `12` représentant une formule $F_2 \in \mathcal{F}(L_2)$, une liste `11` représentant une formule $F_1 \in \mathcal{F}(L_1)$ et une valuation partielle vp .

La correction de `eform2_sat_aux` est un lemme important de la preuve de correction de `eform2_sat`. Un appel à `eform2_sat_aux 12 11 vp` est correct s'il renvoie un tableau qui représente une valuation v telle que $v \models F_2 \wedge F_1$ et vp est compatible avec v , si une telle valuation existe ; sinon, l'appel doit lever l'exception **Exn_unsat**.

La mise en œuvre de `eform2_sat_aux` cherche à compléter la valuation partielle vp en assignant des valeurs à certaines variables (non assignées) qui apparaissent dans les littéraux de F_2 et qui permettent de simplifier F_2 en une formule de $\mathcal{F}(L_1)$. Pour savoir si une 2-contrainte $e2$ de `12` peut être simplifiée en présence de vp , on appelle `contr2_simplify_valp e2 vp` (ligne 15). Si $e2$ se simplifie, l'argument `11` accumule les formules de $\mathcal{F}(L_1)$ obtenues par la simplification et `eform2_sat_aux` continue la simplification du reste des contraintes de `12` (ligne 17). Si la contrainte $e2$ est une contradiction en présence de vp , l'exception **Exn_unsat** est levée par `contr2_simplify_valp e2 vp`. Si $e2$ ne peut pas être simplifiée (ligne 19), alors `eform2_sat_aux` explore les simplifications de la contrainte $e2$ obtenues en assignant la variable du littéral `e2.l1r1` à chaque valeur de $\mathbb{B}$. L'exploration d'une assignation est faite grâce à un appel récursif à `eform2_sat_aux` avec le paramètre représentant la valuation partielle mis à jour avec l'assignation choisie.

Quand toutes les 2-contraintes de F_2 ont été simplifiées, c'est-à-dire quand la liste `12` est vide (ligne 10), `eform2_sat_aux` appelle `eform1_sat` avec l'argument `(11'@11)` où la liste `11'` est obtenue par l'appel à `eform1_of_valp vp`. Si l'appel à `eform1_sat (11'@11)` renvoie **Some(v)**, alors le résultat de `eform2_sat_aux` est v . Sinon, `eform2_sat_aux` lève l'exception **Exn_unsat**. Ceci a comme effet de revenir dans un appel précédent (sur la pile d'appels) de `eform2_sat_aux` pour explorer une autre assignation (lignes 25–28) ou, si toutes les assignations ont été explorées, de terminer l'appel à `eform2_sat_aux 12 11 vp` par l'exception **Exn_unsat**.

Question 17. Démontrer que tout appel `eform2_sat 1st0` termine. ┘

Question 18. Démontrer que la fonction `eform2_sat` est correcte en supposant que la fonction `eform2_sat_aux` est correcte. ┘

Dans les trois questions suivantes, on cherche à démontrer les principaux cas de la correction de `eform2_sat_aux`.

```

1  let rec eform2_sat (lst:eform2) : (int array) option =
2    try
3      let vinit = Array.make (n+1) (-1) in (
4        vinit.(0) <- 0; vinit.(1) <- 1;
5        Some (eform2_sat_aux lst [] vinit) )
6    with Exn_unsat -> None
7
8  and eform2_sat_aux (l2:eform2) (l1:eform1) (vp:int array) : int array =
9    match l2 with
10   | [] -> let l1' = eform1_of_valp vp in
11           let r = eform1_sat (l1'@l1) in
12           (match r with None -> raise Exn_unsat | Some v -> v)
13
14   | e2::l2' ->
15     match contr2_simplify_valp e2 vp with
16     | Some l1' -> (* simplification *)
17       eform2_sat_aux l2' (l1'@l1) vp
18
19     | None -> (* branchement sur valeur de e2.lr1 *)
20       let xlr1 = var_of_lit e2.lr1 in
21       try
22         let vp1 = Array.copy vp in (
23           vp1.(xlr1) <- 1;
24           eform2_sat_aux l2 l1 vp1 )
25       with Exn_unsat ->
26         let vp0 = Array.copy vp in (
27           vp0.(xlr1) <- 0;
28           eform2_sat_aux l2 l1 vp0 )

```

FIGURE 1 – Mise en œuvre OCaml de l'algorithme de décision pour P_{L_2}

Question 19. Démontrez la correction de `eform2_sat_aux` quand son premier argument est la liste vide, on supposant que les fonctions `eform1_of_valp` et `eform1_sat` sont correctes. ┘

Question 20. Soit un appel `eform2_sat_aux l2 l1 vp` tel que `l2` est `e2::l2'` et `contr2_simplify_vp e2 vp` renvoie `Some l1'`. Démontrez la correction d'un tel appel en supposant que l'appel récursif à `eform2_sat_aux` de la ligne 17 est correct. ┘

Question 21. Soit un appel `eform2_sat_aux l2 l1 vp` tel que `l2` est `e2::l2'` et `contr2_simplify_vp e2 vp` renvoie `None`. Démontrez la correction d'un tel appel en supposant que les appels récursifs aux lignes 24 et 28 sont corrects. ┘

Question 22. Donner la complexité de `eform2_sat` en fonction de la longueur k de la liste en argument et du nombre de variables n . ┘

Partie III. Égalités et différences entre entiers

Dans cette partie, le domaine d'interprétation des variables est $\mathbb{Z}$. La classe L_3 est constituée de contraintes de la forme $X_i = X_j$ et $X_i \neq X_j$, avec $X_i, X_j \in \mathbb{X}$. Une formule F est dite construite sur L_3 , notation $F \in \mathcal{F}(L_3)$, si elle est une conjonction de contraintes de L_3 . Une valuation est une fonction $v : \mathbb{X} \rightarrow \mathbb{Z}$ qui assigne une valeur dans $\mathbb{Z}$ à chaque variable. Une valuation v **satisfait** une contrainte $X_i = X_j$ (resp. $X_i \neq X_j$) si et seulement si $v(X_i) = v(X_j)$ (resp. $v(X_i) \neq v(X_j)$). Une valuation v **satisfait** une formule $F \in \mathcal{F}(L_3)$ si et seulement si v satisfait chaque contrainte de F . Les notions de conséquence logique, équivalence logique, tautologie et antilogie sont définies de manière similaire à la logique propositionnelle.

Le problème P_{L_3} peut être réduit (en temps polynomial en n et en la taille de la formule) au problème de satisfiabilité d'une formule propositionnelle. Dans cette partie, on étudie un algorithme polynomial en n et en la taille de la formule pour P_{L_3} .

Mise en œuvre en langage C. On propose de représenter les formules de $\mathcal{F}(L_3)$ par des listes simplement chaînées acycliques de type `eform3_t*`. Chaque cellule `c` (de type `eform3_t`) d'une telle liste représente une contrainte $e \in L_3$ grâce à trois données : un booléen `c.is_eq` qui est vrai si et seulement si e est une égalité, et deux entiers positifs `c.xi` et `c.xj` qui représentent les indices des variables utilisées par e . La formule $\top$ est représentée par la liste vide de valeur `NULL`. On suppose que le nombre n de variables dans $\mathbb{X}$ est déclaré comme une variable globale entière. Enfin, on suppose que toute structure `eform3_t` est bien formée, c'est-à-dire que tous les entiers qu'elle contient appartiennent à l'intervalle $[1; n]$.

Question 23. Donner la déclaration du type `eform3_t`. ┘

Question 24. Écrire la fonction `void eform3_print(eform3_t *lst)` qui affiche sur la sortie standard chaque contrainte (par exemple, sous la forme `X_1 != X_4`) de la formule représentée par `lst`, une contrainte par ligne ; si `lst` est `NULL`, la fonction affiche `true` sur une ligne. ┘

Classes d'équivalence avec « unir et trouver ». On utilise une structure `unir et trouver` (*union-find*) pour résoudre le problème P_{L_3} . On suppose disposer d'une bibliothèque C qui fournit une implémentation d'une structure `unir et trouver` sur des ensembles d'entiers. Cette bibliothèque exporte les déclarations de types et de fonctions suivantes :

```

1  /* Rappel: definition dans stdlib.h
2  typedef unsigned int size_t; */
3
4  struct uf_s {          /* type unir et trouver (union-find) */
5      size_t nelem;      /* nombre d'elements de l'ensemble */
6      int *parent;      /* tableau de taille nelem = relation parent */
7      ...               /* champs utiles pour operations en O(log n) */
8  };
9  typedef struct uf_s uf_t;
10 uf_t *uf_create(size_t sz);
11 int uf_find(uf_t *cs, int i);
12 void uf_union(uf_t *cs, int i1, int i2);

```

Dans ce qui suit, on supposera les propriétés suivantes de cette implémentation :

- `uf_create(s)` renvoie un pointeur `cs` vers une structure unir et trouver allouée dans le tas et qui représente une partition de l'ensemble $\{0, \dots, s-1\}$ où chaque élément de l'ensemble est seul dans sa classe d'équivalence ; `s` donne la valeur du champ `cs->nelem`.
- `uf_find(cs, i)` renvoie la classe de `i`, c'est-à-dire un entier entre 0 et `cs->nelem-1` qui est le représentant de cette classe.
- `uf_union(cs, i1, i2)` fusionne les classes de `i1` et `i2` en modifiant la structure `*cs`.

Si `cs` est un pointeur valide vers une structure `uf_t` avec `cs->nelem==s`, le champ `cs->parent` pointe vers un tableau de `s` entiers ayant des valeurs dans l'intervalle $[0; s-1]$; ce tableau encode la relation représentant d'une classe d'équivalence. Si `cs->parent[i]==j` alors `i` est dans la classe de `j`. Le tableau `cs->parent` est initialisé par `uf_create` de sorte que `cs->parent[i]==i`. Les opérations ci-dessus sur la structure unir et trouver ont une complexité en $\Theta(s)$ pour `uf_create` et en $O(\log s)$ pour `uf_find` et `uf_union`.

Pour notre problème, *l'ensemble de valeurs à partitionner sera* $[0; n]$, c'est-à-dire l'ensemble d'indices de variables dans $\mathbb{X}$ auquel on ajoute la valeur 0. La valeur 0 ne sera pas utilisée, mais elle permet d'utiliser directement les indices des variables comme positions du tableau `cs->parent`.

Question 25. Écrire la fonction `uf_t *eform3_to_uf(eform3_t *lst)` qui renvoie une valeur de type `uf_t*` représentant les classes d'équivalence sur $[0; n]$ définies par les *contraintes d'égalité* de la formule représentée par `lst`. On suppose que `lst` n'est pas vide. Les contraintes de différence de `lst` seront ignorées dans cette fonction. ┘

Algorithme de décision. D'après la question précédente, la structure unir et trouver permet donc de représenter les contraintes d'égalité d'une formule de $\mathcal{F}(L_3)$, mais pas les contraintes de différence.

Question 26. Écrire une fonction `bool eform3_sat_int(eform3_t *lst)` qui renvoie `true` si et seulement si toutes les contraintes de différence de la formule représentée par `lst` peuvent être satisfaites en respectant les contraintes d'égalité de `lst`. Si `lst` est `NULL`, la fonction renvoie `true`. Justifier la correction de la fonction `eform3_sat_int`, c'est-à-dire, que tout appel à cette fonction renvoie `true` si et seulement si la formule représentée par son argument est satisfiable. ┘

Question 27. On suppose que `lst` (de type `eform3_t*`) représente une formule avec un nombre de contraintes d'égalité égal à e et un nombre de contraintes de différence égal à d . Donner la complexité de l'appel à `eform3_sat_int(lst)` en fonction de n , e et d . ┘

Calculer une valuation. Dans la suite de cette partie, on se propose de calculer, si elle existe, une valuation $v : \mathbb{X} \rightarrow \mathbb{Z}$ qui satisfait une formule de $\mathcal{F}(L_3)$.

Question 28. Écrire une fonction `int uf_classes(uf_t *cs)` qui renvoie le nombre de classes d'équivalence de la structure unir et trouver pointée par `cs` ; si `cs` est `NULL` alors la fonction renvoie 0. Cette fonction doit avoir une complexité en $O(n)$. ┘

Question 29. Écrire une fonction `int *uf_valuation(uf_t *cs)` qui renvoie un pointeur `v` vers un tableau de $n + 1$ entiers, alloué dans le tas. Soit k le résultat de l'appel de `uf_classes(cs)`. On suppose $k \geq 2$. Pour tout $i \in [1; n]$, `v[i]` est une valeur entière dans l'intervalle $[1; k]$ qui

est associée *de façon unique* à (chaque élément de) la classe d'équivalence de i . La case à la position 0 de v n'est pas utilisée, mais elle permet d'utiliser les indices des variables comme positions de v ; par convention, $v[0] = 0$. La fonction `uf_valuation` doit avoir une complexité en $O(n \log n)$. ┘

Soit $F \in \mathcal{F}(L_3)$ une formule représentée par une liste `lst` différente de `NULL` telle que `eform3_sat_int(lst)` renvoie `true`. Soient `cs` le résultat de l'appel à `eform3_to_uf(lst)` et v le résultat de l'appel à `uf_valuation(cs)`.

Question 30. Démontrer que v pointe vers un tableau qui représente une valuation qui satisfait la formule F représentée par `lst`. ┘

Question 31. Si F ne contient pas de contraintes de différence, alors quel est le nombre minimal de valeurs que peut avoir l'image d'une valuation v qui satisfait F ? Donner un exemple d'une telle valuation.

La même question si F contient une seule contrainte de différence. ┘

Question 32. Lorsque cela est possible, on souhaite construire une valuation v telle que v satisfait la formule F et l'image de v est un ensemble de taille 2. Quel algorithme sur les graphes peut être utilisé pour obtenir ce résultat et sur quel graphe? Vous devez définir formellement le graphe en entrée de l'algorithme et donner la complexité de l'algorithme. ┘

Partie IV. Égalités et différences d'expressions

Soit $\mathbb{F}$ un ensemble de m symboles de fonction notés f_i ($1 \leq i \leq m$). Chaque symbole $f_i \in \mathbb{F}$ représente une fonction de $\mathbb{Z} \times \mathbb{Z}$ dans $\mathbb{Z}$. Comme dans la partie III, on interprète les variables X_i de $\mathbb{X}$ dans l'ensemble $\mathbb{Z}$. Les symboles de fonctions de $\mathbb{F}$ **ne sont pas interprétés**, c'est-à-dire qu'ils n'ont pas de définition. Les seules propriétés connues de ces fonctions sont données sous la forme de contraintes d'égalité ou de différence. Ces contraintes sont celles de la classe L_4 définie ci-dessous.

Les contraintes de la classe L_4 utilisent des termes. Un **terme** t est défini de manière inductive : t est

- soit X_i , une variable de $\mathbb{X}$,
- soit $f_i(t_1, t_2)$, l'application d'une fonction $f_i \in \mathbb{F}$ sur deux termes t_1 et t_2 , appelés aussi **sous-termes directs** de t .

Le **symbole racine** du terme X_i (resp. $f_j(t_1, t_2)$) est X_i (resp. f_j). On note $\mathbb{T}(\mathbb{X}, \mathbb{F})$ l'ensemble des termes construits avec les variables de $\mathbb{X}$ et les symboles de fonctions $\mathbb{F}$.

Ces termes peuvent être représentés par des arbres binaires. La figure 2 représente les arbres binaires correspondants aux termes :

$$t_1 \stackrel{\text{def}}{=} f_1(f_2(X_1, X_2), X_2) \quad \text{et} \quad t_2 \stackrel{\text{def}}{=} f_3(X_1, f_2(X_1, X_2)).$$

Chaque noeud de ces arbres est étiqueté en partie haute par un nom unique u_i appelé identifiant, et en partie basse par le symbole racine du terme. L'arc à gauche (resp. droite) d'un noeud correspond au premier (resp. second) sous-terme direct et est dessiné avec un trait en pointillé (resp. plein).

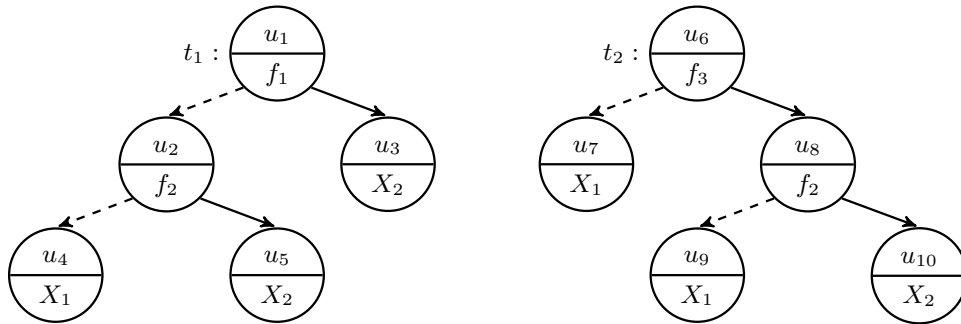


FIGURE 2 – Arbres des termes t_1 (à gauche) et t_2 (à droite)

L'ensemble des sous-termes d'un terme t , noté $\mathcal{T}(t)$, est défini de manière inductive par

$$\mathcal{T}(X_i) \stackrel{\text{def}}{=} \{X_i\} \quad \text{et} \quad \mathcal{T}(f_i(t_1, t_2)) \stackrel{\text{def}}{=} \{f_i(t_1, t_2)\} \cup \mathcal{T}(t_1) \cup \mathcal{T}(t_2).$$

Question 33. Donner $\mathcal{T}(t_1)$ et $\mathcal{T}(t_2)$ pour les termes t_1 et t_2 définis ci-dessus. ┐

La classe L_4 contient les contraintes de la forme $t_1 = t_2$ ou $t_1 \neq t_2$ avec t_1 et t_2 des termes. Une formule F de $\mathcal{F}(L_4)$ est une conjonction de contraintes L_4 . L'ensemble des sous-termes d'une contrainte e de la forme $t_1 = t_2$ ou $t_1 \neq t_2$ est $\mathcal{T}(e) \stackrel{\text{def}}{=} \mathcal{T}(t_1) \cup \mathcal{T}(t_2)$. L'ensemble des sous-termes d'une formule F de $\mathcal{F}(L_4)$, noté $\mathcal{T}(F)$, est défini comme l'union des ensembles des sous-termes de toutes les contraintes de F .

Mise en œuvre en langage C. Afin de concevoir un algorithme de décision pour $P(L_4)$, nous représentons *les contraintes d'égalité* d'une formule de $\mathcal{F}(L_4)$ par une structure de données appelée *e-graphe*. Cette structure combine un graphe orienté acyclique avec une structure unir et trouver. Les définitions de types C suivantes codent une structure de e-graphe :

```

1 struct tnode_s { /* type des noeuds des arbres representant les termes */
2     char *label; /* symbole racine */
3     int li; /* position dans tset du premier sous-terme direct ou 0 */
4     int ri; /* position dans tset du second sous-terme direct ou 0 */
5 };
6 typedef struct tnode_s tnode_t;
7
8 struct egraph_s { /* type e-graphe representant une formule  $L_4$  */
9     size_t tset_sz; /* longueur de tset */
10    tnode_t *tset; /* debut tableau des termes de la formule */
11    int *pre; /* relation sous-terme -- terme */
12    uf_t *uf; /* unir et trouver pour les classes des termes */
13 };
14 typedef struct egraph_s egraph_t;

```

Dans ce qui suit, on supposera les propriétés suivantes de la mise en œuvre de la structure `egraph_t`. Si `g` est une variable de type `egraph_t` représentant les contraintes d'égalité d'une formule $F \in \mathcal{F}(L_4)$, `g.tset` code l'ensemble des sous-termes $\mathcal{T}(F)$ comme suit :

- `g.tset` est un pointeur vers le debut d'un tableau de taille `g.tset_sz`;
- `g.tset[0]` est initialisée à la valeur `{.label=NULL, .li=0, .ri=0}` (premier champ à `NULL` et les suivants à 0) de `tnode_t`, valeur qu'on appelle aussi *terme vide*;
- un terme est représenté par une valeur `t` de type `tnode_t` tel que `t.label` est une chaîne de caractères qui décrit le symbole racine du terme, et `t.li` (resp. `t.ri`) est la position du premier (resp. second) sous-terme direct dans le tableau `g.tset`.

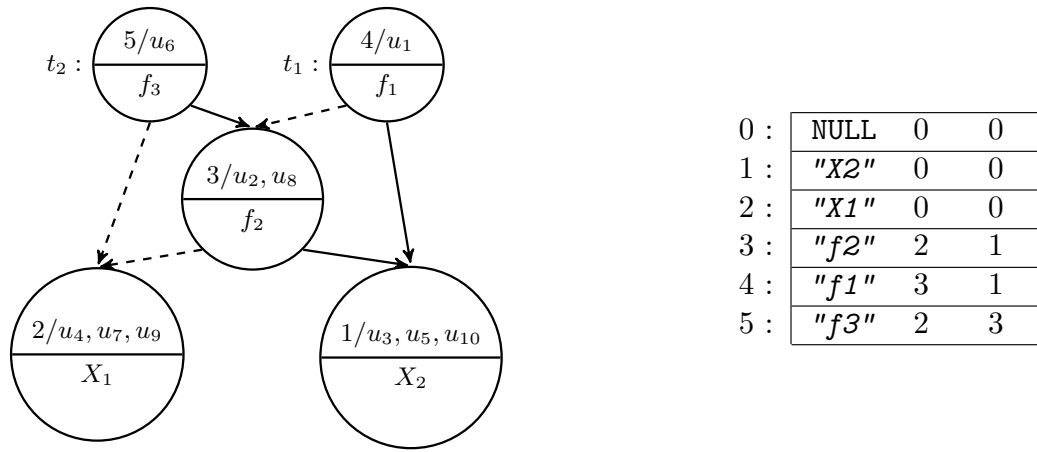
Un invariant du tableau `g.tset` est le suivant :

tout terme de $\mathcal{T}(F)$ est représenté par une unique position du tableau `g.tset`.

Cet invariant correspond à une factorisation des termes communs et donc à la transformation de l'ensemble des arbres correspondant aux termes en un graphe orienté acyclique.

Par exemple, considérons la formule $t_1 = t_2$ utilisant les termes représentés dans la figure 2. La partie droite de la figure 3 représente le tableau `g.tset` de l'ensemble des sous-termes de la formule (une valeur `tnode_t` par ligne) et le terme vide. La partie gauche de la figure 3 représente le tableau `g.tset` sous la forme d'un graphe orienté acyclique : les noeuds contiennent en partie haute la position du terme dans le tableau `g.tset` et, pour la lisibilité de la transformation, l'ensemble des identifiants u_i des noeuds correspondants dans les arbres de la figure 2. La partie basse d'un noeud donne le symbole racine.

La composante `g.pre` de l'e-graphe contient la relation prédécesseur dans le graphe dirigé acyclique représenté par `g.tset`. Cette relation est donnée sous la forme d'une matrice linéarisée en un tableau de `g.tset_sz2` valeurs dans l'ensemble $\{0, 1\}$. Si `g.pre[i*g.tset_sz + j]` vaut 1 alors le terme `g.tset[i]` a comme sous-terme direct le terme `g.tset[j]`. La valeur 0 dénote que `g.tset[j]` n'est pas sous-terme direct de `g.tset[i]`.

FIGURE 3 – Codage de $\mathcal{T}(t_1 = t_2)$ sous forme de graphe (à gauche) et de tableau (à droite)

La composante $\mathbf{g.uf}$ de l'e-graphe $\mathbf{g}$ code les classes d'équivalence définies par les contraintes d'égalités entre les termes. La définition du type `uf_t` est la même que celle donnée dans la partie III. Chaque terme est représenté dans l'ensemble des éléments de $\mathbf{g.uf}$ par la position du noeud racine du terme dans le tableau $\mathbf{g.tset}$. Par conséquent, un autre invariant de la structure d'e-graphe est le suivant :

$\mathbf{g.uf} \rightarrow \text{nelem}$ est égal à $\mathbf{g.tset_sz}$.

Question 34. Écrire la fonction `bool term_is_diff(egraph_t *g, int tid1, int tid2)` qui teste si la contrainte $t_1 \neq t_2$ peut être satisfaite simultanément aux contraintes d'égalité représentées par $\mathbf{g}$. Les paramètres `tid1` et `tid2` sont les positions dans $\mathbf{g.tset}$ des noeuds racine représentant respectivement t_1 et t_2 . ┘

Congruence. Soient $F \in \mathcal{F}(L_4)$ et $\mathbf{g}$ un e-graphe tel que $\mathbf{g.tset}$ représente $\mathcal{T}(F)$, l'ensemble des sous-termes de F . On définit la relation d'équivalence $\equiv_{\mathbf{g}}$ sur $\mathcal{T}(F)$ par : $t \equiv_{\mathbf{g}} t'$ si et seulement si t et t' sont représentés par des positions de $\mathbf{g.tset}$ qui sont dans la même classe d'équivalence de $\mathbf{g.uf}$. On définit la relation $\mathcal{C}_{\mathbf{g}}$ sur $\mathcal{T}(F)$ comme suit :

*pour tout $f_i \in \mathbb{F}$ et pour tout quadruplet de termes $(t_1, t_2, t_3, t_4) \in \mathcal{T}(F)^4$,
si $t_1 \equiv_{\mathbf{g}} t_3$ et $t_2 \equiv_{\mathbf{g}} t_4$, alors $(f_i(t_1, t_2), f_i(t_3, t_4)) \in \mathcal{C}_{\mathbf{g}}$.*

Question 35. Écrire la fonction `bool term_is_congruent(egraph_t *g, int tid1, int tid2)` qui renvoie `true` si et seulement si les termes représentés aux positions `tid1` et `tid2` dans $\mathbf{g.tset}$ sont dans la relation $\mathcal{C}_{\mathbf{g}}$. Donner et justifier soigneusement la complexité de `term_is_congruent` en fonction de $\mathbf{g.tset_sz}$, si on suppose que le test d'égalité d'étiquettes (symboles de $\mathbb{X}$ ou $\mathbb{F}$) se fait en temps constant. ┘

Algorithme de décision. Soient $F \in \mathcal{F}(L_4)$ et $\mathbf{g}$ un e-graphe tel que $\mathbf{g}$ représente les termes et les contraintes d'égalité de F ainsi que deux termes t_1 et t_2 , pas nécessairement dans $\mathcal{T}(F)$. On cherche à modifier $\mathbf{g}$ pour qu'il représente $F \wedge (t_1 = t_2)$. On appelle cette opération `merge(g, t1, t2)` pour faire la différence avec `uf_union`. En effet, cette opération ne peut pas se limiter à appeler `uf_union` pour unir les classes d'équivalence des positions représentant les termes t_1 et t_2 . Elle doit en plus tester si $(t'_1, t'_2) \in \mathcal{C}_{\mathbf{g}}$ pour tous termes t'_1 et t'_2 qui incluent respectivement t_1 et t_2 comme sous-termes directs mais *ne sont pas déjà dans la même*

classe d'équivalence. Si $(t'_1, t'_2) \in \mathcal{C}_g$, alors t'_1 et t'_2 doivent être mis dans la même classe d'équivalence. Cette dernière relation est propagée aux termes qui contiennent t'_1 et t'_2 et ainsi de suite, tant qu'on trouve des couples de termes dans la relation $\mathcal{C}_g$ mais pas dans $\equiv_g$.

Question 36. Écrire une fonction récursive `void merge(egraph_t *g, int tid1, int tid2)` qui ajoute la contrainte d'égalité entre les termes aux positions `tid1` et `tid2` dans `g.tset`, selon le processus décrit ci-dessus. Si `tid1` et `tid2` sont déjà dans la même classe d'équivalence, `merge` termine immédiatement. ┘

Question 37. Donner le nombre maximum d'appels récursifs qu'un appel à `merge` peut engendrer en fonction de `g.tset_sz`. ┘

Question 38. On suppose que le graphe orienté acyclique représenté par `g.tset` contient a arêtes. Donner la complexité en temps de l'opération `merge` en fonction de `g.tset_sz` et a . ┘

_____ *Fin du sujet.* _____

Annexe : Preuves en déduction naturelle

Un séquent $\Gamma \vdash F$ est composé d'un ensemble de formules Γ et d'une formule F .

Le sous-ensemble suivant de règles de la déduction naturelle peut être utilisé dans la preuve demandée à la question 14.

Introduction	Elimination
$\overline{\Gamma, F \vdash F} \text{ ax}$	
$\overline{\Gamma \vdash \top} \top_I$	$\frac{\Gamma \vdash \perp}{\Gamma \vdash F} \perp_E$
$\frac{\Gamma \vdash F'_1 \quad \Gamma \vdash F'_2}{\Gamma \vdash F'_1 \wedge F'_2} \wedge_I$	$\frac{\Gamma \vdash F_1 \wedge F_2}{\Gamma \vdash F_1} \wedge^1_E \quad \frac{\Gamma \vdash F_1 \wedge F_2}{\Gamma \vdash F_2} \wedge^2_E$
$\frac{\Gamma, F' \vdash F}{\Gamma \vdash F' \rightarrow F} \rightarrow_I$	$\frac{\Gamma \vdash F' \rightarrow F \quad \Gamma \vdash F'}{\Gamma \vdash F} \rightarrow_E$
$\frac{\Gamma, F \vdash \perp}{\Gamma \vdash \neg F} \neg_I$	$\frac{\Gamma \vdash \neg F \quad \Gamma \vdash F}{\Gamma \vdash \perp} \neg_E$

**ECOLE POLYTECHNIQUE
ECOLES NORMALES SUPERIEURES**

CONCOURS D'ADMISSION 2024

**MARDI 16 AVRIL 2024
14h00 - 18h00**

**FILIERES MP-MPI - Epreuve n° 4
INFORMATIQUE A (XULSR)**

Durée : 4 heures

***L'utilisation des calculatrices n'est pas autorisée pour
cette épreuve***

*Cette composition ne concerne qu'une partie des candidats de la
filière MP, les autres candidats effectuant simultanément la
composition de Physique et Sciences de l'Ingénieur.
Pour la filière MP, il y a donc deux enveloppes de Sujets pour
cette séance.*

Arbres équilibrés et géométrie algorithmique

Ce sujet traite de l'implémentation d'ensembles finis par des arbres équilibrés dits AVL et de leur application à un problème de géométrie algorithmique classique, la localisation d'un point dans le plan.

Ce sujet est conçu pour être traité linéairement. Les parties I et II introduisent les arbres AVL et leur construction et la partie III les utilise pour la localisation d'un point dans le plan. Pour traiter une question, on peut admettre le résultat de toute question précédente.

Langage OCaml. Ce sujet utilise notamment les tableaux d'OCaml. On peut créer un tableau avec la fonction `Array.make`. L'appel de `Array.make n x` crée un tableau de taille n dont toutes les cases contiennent la valeur x . Les cases d'un tableau sont numérotées à partir de 0. La fonction `Array.length` renvoie la taille d'un tableau. Pour un tableau `tab`, on accède à l'élément d'indice i avec `tab.(i)` et on le modifie avec `tab.(i) <- v`.

Dans le code OCaml fourni dans le sujet, on utilise la construction `assert false` pour signaler un point de code inatteignable, c'est-à-dire un cas de figure qui n'est pas censé se produire si les préconditions de la fonction sont respectées.

Pour les questions de programmation, il n'est pas demandé de justifier la correction du code donné en réponse.

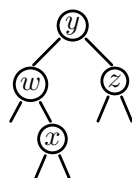
Complexité. Par *complexité en temps* d'un algorithme A on entend le nombre d'opérations élémentaires (comparaison, addition, soustraction, multiplication, division, affectation, test, etc.) nécessaires à l'exécution de A dans le cas le pire. Par *complexité en espace* d'un algorithme A on entend l'espace mémoire minimal nécessaire à l'exécution de A dans le cas le pire. Lorsque la complexité dépend d'un ou plusieurs paramètres $\kappa_0, \dots, \kappa_{r-1}$, on dit que A a une complexité en $O(f(\kappa_0, \dots, \kappa_{r-1}))$ s'il existe une constante $C > 0$ telle que, pour toutes les valeurs de $\kappa_0, \dots, \kappa_{r-1}$ suffisamment grandes (c'est-à-dire plus grandes qu'un certain seuil), pour toute instance du problème de paramètres $\kappa_0, \dots, \kappa_{r-1}$, la complexité est au plus $C \cdot f(\kappa_0, \dots, \kappa_{r-1})$. De même, on dit que A a une complexité en $\Theta(f(\kappa_0, \dots, \kappa_{r-1}))$ s'il existe deux constantes $C_1 > 0$ et $C_2 > 0$ telles que, pour toutes les valeurs de $\kappa_0, \dots, \kappa_{r-1}$ suffisamment grandes, la complexité de A est au moins $C_1 \cdot f(\kappa_0, \dots, \kappa_{r-1})$ et au plus $C_2 \cdot f(\kappa_0, \dots, \kappa_{r-1})$.

Dans tout ce sujet, on note « $\log$ » le logarithme à base 2. On utilisera exclusivement ce logarithme.

Partie I. Préliminaires

On cherche à construire une structure de données pour représenter des sous-ensembles finis d'un ensemble U donné. L'ensemble U est supposé muni d'un ordre strict total noté $<$, c'est-à-dire que, pour tous $x, y \in U$, on a $x < y$ ou $x = y$ ou $y < x$. On va utiliser des arbres binaires de recherche équilibrés pour représenter des sous-ensembles de U .

Arbres binaires. Un *arbre binaire* est défini récursivement de la manière suivante : soit comme l'arbre vide, noté $\langle \rangle$, qui ne contient aucun nœud ; soit comme un *nœud*, noté $\langle \ell, x, r \rangle$ avec un sous-arbre gauche ℓ , un élément $x \in U$ à la racine et un sous-arbre droit r . On dessine un arbre tel que $\langle \langle \rangle, w, \langle \langle \rangle, x, \langle \rangle \rangle \rangle, y, \langle \langle \rangle, z, \langle \rangle \rangle \rangle$ de la manière suivante



(A)

avec la racine (ici y) en haut, reliée aux sous-arbres gauche et droit dessinés en-dessous. Les sous-arbres vides ne sont pas dessinés, mais les liaisons vers les sous-arbres vides le sont.

On note $n(t)$ le nombre de nœuds et $h(t)$ la hauteur d'un arbre binaire t , définis par

$$n(\langle \rangle) = h(\langle \rangle) = 0, \quad n(\langle \ell, x, r \rangle) = 1 + n(\ell) + n(r) \quad \text{et} \quad h(\langle \ell, x, r \rangle) = 1 + \max(h(\ell), h(r))$$

On notera ici que, pour des raisons purement techniques, la hauteur de l'arbre vide est fixée à 0 contrairement au programme qui la fixe à -1 . Ainsi, pour l'arbre A pris en exemple ci-dessus, on a $n(A) = 4$ et $h(A) = 3$.

Le *parcours infixe* d'un arbre binaire est une séquence d'éléments de U définie récursivement de la manière suivante :

- le parcours infixe de l'arbre vide $\langle \rangle$ est la séquence vide ;
- le parcours infixe d'un arbre $\langle \ell, x, r \rangle$ est la concaténation, dans cet ordre, du parcours infixe de l'arbre ℓ , de l'élément x et du parcours infixe de l'arbre r .

Ainsi, le parcours infixe de l'arbre précédent est la séquence w, x, y, z .

Un arbre binaire t est un *arbre binaire de recherche* si la séquence de ses éléments donnée par un parcours infixe est triée par ordre strictement croissant. En particulier, un élément n'apparaît qu'au plus une fois dans cette séquence. Tous les arbres binaires manipulés dans ce sujet seront toujours des arbres binaires de recherche.

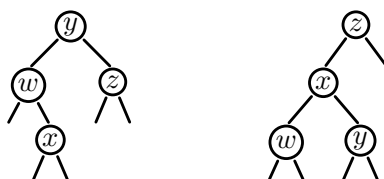
Question 1. Combien y a-t-il d'arbres binaires de recherche contenant exactement les trois éléments x, y et z , avec $x < y < z$? Les dessiner.

Question 2. Montrer qu'un arbre t est un arbre binaire de recherche si et seulement si t est l'arbre vide ou si t est un arbre $\langle \ell, x, r \rangle$ avec ℓ et r des arbres binaires de recherche où tous les éléments de ℓ sont strictement inférieurs à x et tous les éléments de r sont strictement supérieurs à x .

Arbres AVL. Pour équilibrer nos arbres binaires de recherche, on va imposer une condition supplémentaire : pour tout nœud $\langle \ell, x, r \rangle$, on a

$$|h(\ell) - h(r)| \leq 1 \tag{1}$$

De tels arbres sont dits AVL (du nom de leurs auteurs, Adelson-Velsky et Landis). Ainsi, dans les deux arbres ci-dessous,



celui de gauche est un AVL mais celui de droite n'en est pas un (en supposant $w < x < y < z$ par ailleurs).

Question 3. Combien y a-t-il d'arbres AVL contenant exactement les trois éléments x , y et z , avec $x < y < z$? Les dessiner. Même question avec les quatre éléments $w < x < y < z$. Justifier les réponses.

Question 4. Montrer que pour tout AVL t , on a $h(t) \leq 2 \log(n(t) + 1)$. Indication : on pourra considérer N_h , le nombre minimal de nœuds d'un AVL de hauteur h , et minorer cette quantité en fonction de h .

Mise en œuvre en OCaml. On suppose que les éléments de U sont représentés par un type OCaml `elt`, qui n'est pas précisé pour l'instant. Pour deux éléments x et y donnés, on peut tester si $x = y$ avec `eq x y` et tester si $x < y$ avec `lt x y`.

```
val eq: elt -> elt -> bool
val lt: elt -> elt -> bool
```

Un AVL est représenté par une valeur du type OCaml suivant :

```
type tree = E | N of int * tree * elt * tree
```

Le constructeur `E` représente l'arbre binaire vide $\langle \rangle$. Une valeur `N( $z, \ell, x, r$ )` représente l'arbre binaire non vide $\langle \ell, x, r \rangle$, avec l'*invariant* que la valeur z du premier champ est toujours égale à la hauteur de l'arbre, i.e. $z = h(\langle \ell, x, r \rangle)$. On se donne une fonction `height` pour obtenir la hauteur d'un arbre :

```
let height t = match t with E -> 0 | N(h, _, _, _) -> h
```

On note que cette fonction s'évalue en temps constant. Pour maintenir l'invariant, on se donne une fonction `node` pour construire un nouveau nœud :

```
let node l x r = N(1 + max (height l) (height r), l, x, r)
```

On note que cette fonction s'évalue également en temps constant. Dans tout ce qui suit, on utilisera toujours la fonction `node` pour construire un nœud et jamais le constructeur `N` directement. On utilisera le constructeur `N` uniquement dans les motifs de filtrage.

Question 5. Écrire une fonction `mem: elt -> tree -> bool` qui prend en arguments un élément x et un arbre AVL t et qui détermine si x apparaît dans t . La complexité doit être en $O(\log n(t))$ et on demande de la justifier.

Partie II. Construire des arbres AVL

Maintenir la propriété d'arbre AVL pendant la construction des arbres binaires de recherche demande un certain effort. On propose ici de concentrer cet effort dans une unique fonction, `join`, dont le code est donné figure 1. La fonction `join` reçoit en paramètres deux arbres AVL l et r et un élément x . Elle suppose que les éléments de l (resp. r) sont strictement plus petits (resp. plus grands) que x . Elle renvoie un arbre AVL contenant l'union des éléments de l , de r et du singleton $\{x\}$.

Si les hauteurs de l et r satisfont déjà à la propriété d'AVL, alors il suffit d'appeler `node` (ligne 4). Sinon, on appelle `join_right` (resp. `join_left`) selon que le déséquilibre vient de l (ligne 2) ou de r (ligne 3). La fonction `join_right` procède récursivement, en descendant le long de la branche droite de l'arbre l (lignes 8–11) jusqu'à trouver un sous-arbre de hauteur

acceptable (lignes 4–7). Si besoin, des rotations sont effectuées localement avec les deux fonctions `rotate_left` et `rotate_right` (en haut de la figure 1). Ces deux fonctions implémentent la notion usuelle de rotation dans les arbres binaires de recherche, illustrée juste en-dessous de leur code. La fonction `join_left` est symétrique et son code est omis. Dans les raisonnements sur la fonction `join`, on pourra se contenter de considérer seulement le cas où `join_right` est appelée, en considérant l'autre cas symétrique.

On prendra le temps de bien lire et de bien comprendre le code de la fonction `join`. L'objectif des questions suivantes est de se persuader que `join` fonctionne correctement, ce qui est plus subtile qu'il n'y paraît.

Question 6. Montrer que tout appel à `join`, avec deux arguments `l` et `r` qui sont des AVL, termine et ne peut pas « planter », au sens où la ligne 3 de `rotate_left`, la ligne 3 de `rotate_right` et la ligne 12 de `join_right` ne sont jamais atteintes.

Question 7. Montrer que, lorsque la fonction `join_right` est appelée avec un AVL `l` de hauteur h et un AVL `r` de hauteur $\leq h - 2$, alors elle renvoie

- soit un arbre de hauteur h ;
- soit un arbre de hauteur $h + 1$ avec un sous-arbre gauche de hauteur $h - 1$ et un sous-arbre droit de hauteur h .

Indication : pour chaque cas qu'on choisit de distinguer dans la preuve, on pourra se contenter d'un schéma donnant la forme de l'arbre renvoyé annoté avec les hauteurs pertinentes.

Question 8. Montrer que la fonction `join`, appliquée à deux AVL `l` et `r`, renvoie bien un arbre AVL dont la hauteur est au plus $1 + \max(h(l), h(r))$.

Question 9. Montrer que la fonction `join` a une complexité en $O(|h(l) - h(r)|)$.

Insertion d'un élément. Pour insérer un nouvel élément dans un arbre AVL, on se donne la fonction `insert` (voir figure 2). Elle utilise une fonction auxiliaire, `split`, dont le code est également donné.

Question 10. Expliquer ce que fait la fonction `split` appliquée à un AVL `t` et un élément `y`.

Question 11. Illustrer les étapes de l'insertion de l'élément `c` dans un AVL contenant les éléments `a`, `b` et `d`, avec $a < b < c < d$.

Question 12. Montrer que la fonction `split`, lorsqu'elle est appliquée à un AVL `t` et un élément `y`, renvoie deux arbres AVL dont la hauteur n'excède pas celle de `t`.

Question 13. Montrer que la fonction `insert`, lorsqu'elle est appliquée à un élément `x` et un arbre AVL `t`, renvoie bien un arbre AVL et a une complexité en $O(\log n(t))$.

```

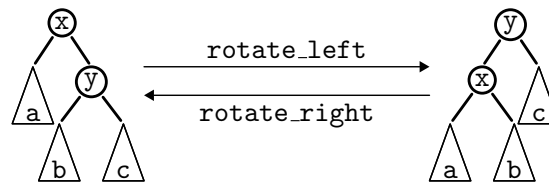
1 let rotate_left t = match t with
2   | N(_, a, x, N(_, b, y, c)) -> node (node a x b) y c
3   | _ -> assert false

```

```

1 let rotate_right t = match t with
2   | N(_, N(_, a, x, b), y, c) -> node a x (node b y c)
3   | _ -> assert false

```



```

1 let rec join_right (l: tree) (x: elt) (r: tree) : tree =
2   match l with
3   | N(_, ll, lx, lr) ->
4     if height lr <= height r + 1 then
5       let t = node lr x r in
6       if height t <= height ll + 1 then node ll lx t
7       else rotate_left (node ll lx (rotate_right t))
8     else
9       let t = join_right lr x r in
10      let t' = node ll lx t in
11      if height t <= height ll + 1 then t' else rotate_left t'
12   | E -> assert false

```

```

1 let rec join_left (l: tree) (x: elt) (r: tree) : tree =
2   ...

```

```

1 let join (l: tree) (x: elt) (r: tree) : tree =
2   if height l > height r + 1 then join_right l x r
3   else if height r > height l + 1 then join_left l x r
4   else node l x r

```

FIGURE 1 – Fonction join sur les AVL.

```

1 let rec split (t: tree) (y: elt) : tree * bool * tree =
2   match t with
3     | E -> E, false, E
4     | N(_, l, x, r) ->
5       if eq y x then l, true, r
6       else if lt y x then let ll, b, lr = split l y in ll, b, join lr x r
7       else let rl, b, rr = split r y in join l x rl, b, rr

1 let insert (x: elt) (t: tree) : tree =
2   let l, _, r = split t x in
3   join l x r

```

FIGURE 2 – Fonctions split et insert sur les AVL.

Question 14. Soit n un entier positif ou nul. On construit un tableau a de n arbres AVL de la manière suivante :

```

let a = Array.make n E
let () = for i = 1 to n - 1 do a.(i) <- insert (i-1) a.(i-1) done

```

Calculer le nombre total $f(n)$ d'entiers (possiblement répétés) stockés dans l'ensemble des arbres du tableau a . Montrer que la complexité en espace de la construction de a est en $O(g(n))$ pour une fonction g telle que $g(n) = o(f(n))$. Expliquer pourquoi il n'y a pas de contradiction.

Supprimer un élément. On souhaite maintenant écrire une fonction `remove` pour supprimer un élément dans un arbre AVL, qui ait une complexité logarithmique comme la fonction `insert`.

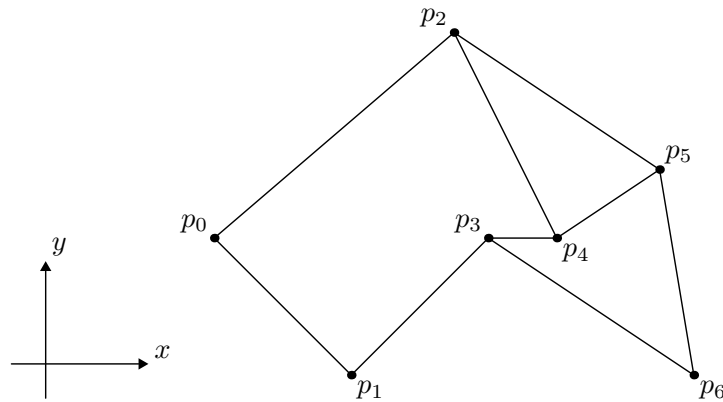
Question 15. Écrire une fonction `split_last: tree -> tree * elt` qui prend en argument un arbre AVL t , supposé non vide, et qui renvoie une paire (u, x) où x est le plus grand élément de t et u un arbre AVL contenant les éléments de t autres que x . La complexité doit être $O(h(t))$. On ne demande pas de la justifier.

Question 16. Écrire une fonction `join2: tree -> tree -> tree` qui prend en arguments deux arbres AVL ℓ et r , en supposant les éléments de ℓ strictement plus petits que ceux de r , et qui renvoie un arbre AVL t contenant l'union des éléments de ℓ et de r . La complexité doit être $O(h(\ell) + h(r))$. On ne demande pas de la justifier.

Question 17. Écrire une fonction `remove: elt -> tree -> tree` qui prend en arguments un élément x et un AVL t , et qui renvoie un arbre AVL contenant les éléments de t privés de x . (Si x n'apparaît pas dans t , le résultat contient les mêmes éléments que t .) La complexité doit être $O(\log n(t))$. On ne demande pas de la justifier.

Partie III. Application : localisation d'un point dans le plan

On considère une subdivision du plan définie par un ensemble fini de polygones. Ces polygones peuvent partager des sommets et des arêtes. Les arêtes ne se croisent pas. Voici un exemple avec 7 points et trois polygones :



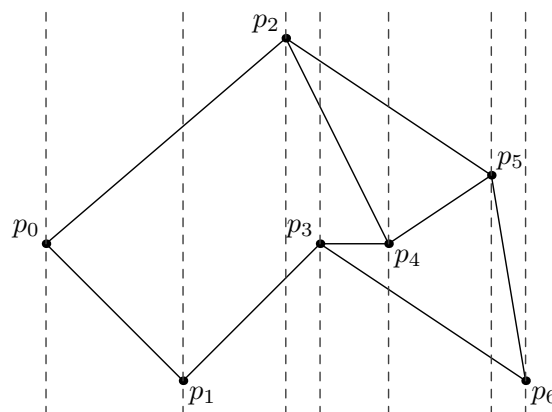
Le plan est ici divisé en quatre zones : l'intérieur de chaque polygone et la zone extérieure. Notre objectif est de construire une structure de données pour représenter cette subdivision, afin de répondre ensuite efficacement à la question « étant donné un point, dans quelle zone se trouve-t-il ? ». On s'intéresse notamment à la complexité de cette structure (temps et espace pour sa construction) et à la complexité de son utilisation ensuite. Ces complexités seront calculées en fonction du nombre total de sommets, noté N dans la suite. On admet que le nombre d'arêtes est en $O(N)$.

Notations. Pour un point p , on note $x(p)$ son abscisse et $y(p)$ son ordonnée. Les N sommets sont notés $p_0, p_1, \dots, p_{N-1}$ et ordonnés de la gauche vers la droite. On suppose qu'il n'y a pas deux sommets sur la même abscisse, c'est-à-dire

$$x(p_0) < x(p_1) < \dots < x(p_{N-1})$$

Une arête e est un couple (p_i, p_j) avec $0 \leq i < j < N$. On note $start(e)$ son extrémité gauche, c'est-à-dire le point p_i , et $end(e)$ son extrémité droite, c'est-à-dire le point p_j . On note $xmin(e)$ l'abscisse de p_i et $xmax(e)$ l'abscisse de p_j . L'arête e s'étend donc de l'abscisse $xmin(e)$ à l'abscisse $xmax(e)$.

Approche. Notre approche consiste à diviser le plan en $N + 1$ tranches verticales, délimitées par les abscisses des N points. Sur notre exemple, on a donc 8 tranches :



La tranche 0 correspond aux abscisses $]-\infty, x(p_0)]$, la tranche 1 aux abscisses $]x(p_0), x(p_1)]$, la tranche 2 aux abscisses $]x(p_1), x(p_2)]$, etc., et la tranche N aux abscisses $]x(p_{N-1}), +\infty[$. Chaque tranche est traversée par un certain nombre d'arêtes. On note que, dans chaque tranche, les segments d'arêtes qui traversent cette tranche peuvent être ordonnés verticalement (car, on le rappelle, les arêtes ne se croisent pas). Ainsi, dans la tranche 3 ci-dessus, on a, de bas en haut, l'arête (p_1, p_3) , l'arête (p_2, p_4) puis enfin l'arête (p_2, p_5) .

L'idée est alors de trier les tranches selon les abscisses, puis, dans chaque tranche, de trier les arêtes la traversant selon les ordonnées. Ainsi, étant un point du plan à localiser, il suffira d'une recherche dichotomique pour déterminer la tranche contenant le point puis d'une seconde recherche dichotomique pour déterminer entre quelles arêtes se trouve le point (et, par conséquent, dans quel polygone se trouve le point, le cas échéant).

Question 18. On note T_i le nombre d'arêtes qui traversent la tranche i . (Dans l'exemple ci-dessus, on a $T_0 = 0$, $T_1 = T_2 = 2$, $T_3 = 3$, etc.) Montrer que la quantité $\sum T_i$ peut être en $\Theta(N^2)$.

Mise en œuvre en OCaml. On utilise des nombres flottants pour les coordonnées des points. On introduit les deux types suivants pour les points et les arêtes, respectivement.

```
type point = float * float
type edge = point * point (* avec x1 < x2 *)
```

On se donne deux fonctions pour récupérer l'abscisse gauche et l'abscisse droite d'une arête :

```
let xmin ((x, _), _) = x
let xmax (_, (x, _)) = x
```

Pour représenter chaque tranche, on va utiliser un arbre AVL dont les éléments sont des arêtes, c'est-à-dire le type `tree` introduit plus haut dans le sujet avec

```
type elt = edge
```

Une tranche est alors représentée par le type suivant,

```
type slab = { xleft: float; xright: float; tree: tree }
```

où `xleft` est l'abscisse du bord gauche de la tranche, `xright` l'abscisse du bord droit et `tree` l'arbre AVL contenant les arêtes qui traversent cette tranche, ordonnées de bas en haut.

Question 19. Écrire une fonction `lt: edge -> edge -> bool` qui prend en arguments deux arêtes e_1 et e_2 , supposées traversant une même tranche, et qui détermine si e_1 est strictement en dessous de e_2 .

Construction des tranches. On représente l'ensemble des tranches par un tableau `slabs`: `slab` array de taille $N + 1$. Il est trié selon les abscisses, c'est-à-dire

```
-∞ = slabs.(0).xleft < slabs.(0).xright = x(p0) = slabs.(1).xleft
                        < slabs.(1).xright = x(p1) = slabs.(2).xleft
                        < ...
                        < slabs.(N).xright = +∞
```

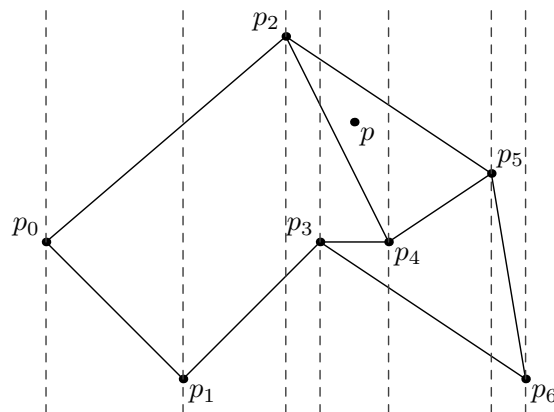
(Le type `float` inclut des valeurs `neg_infinity` pour $-\infty$ et `infinity` pour $+\infty$.) On construit les arbres AVL de chaque tranche de la gauche vers la droite, avec l'algorithme suivant :

- L'arbre de la tranche 0 est vide.
- Pour construire l'arbre de la tranche i , pour $1 \leq i \leq N$,
 - on retire les arêtes qui se terminent au point p_{i-1} ;
 - on ajoute les arêtes qui démarrent au point p_{i-1} .

En particulier, l'arbre de la dernière tranche est vide.

Question 20. Justifier que l'on peut construire, à partir de la liste des arêtes, le tableau `slabs` en temps et en espace $O(N \log N)$. On ne demande pas d'écrire le code, mais on demande d'être précis quant aux structures de données et algorithmes utilisés et on demande d'être convaincant quant à la complexité. On ne demande pas de vérifier que tous les sommets ont des abscisses différentes.

Localisation d'un point. Enfin, on peut utiliser le tableau `slabs` pour localiser un point p donné. On fait l'hypothèse que ce point n'est sur aucune arête, et est donc en particulier différent des N points p_i . Si on prend l'exemple suivant



alors le point p est d'abord localisé dans la tranche 4 puis, dans cette tranche, localisé entre les arêtes (p_2, p_4) et (p_2, p_5) .

Question 21. Écrire une fonction `find_slab: slab array -> float -> slab` qui prend en arguments les tranches et l'abscisse du point p recherché et renvoie la tranche dans laquelle se trouve le point p . La complexité doit être $O(\log N)$. On ne demande pas de la justifier.

Question 22. Écrire une fonction `find: slab array -> point -> answer` qui prend en arguments les tranches et un point p et renvoie la zone dans laquelle le point p se trouve, sous la forme d'une valeur du type

`type answer = Outside | Between of edge * edge`

où `Outside` signifie que le point p est à l'extérieur de tout polygone et `Between( $e_1, e_2$ )` signifie que le point p se trouve à l'intérieur d'un polygone, entre les arêtes e_1 et e_2 . La complexité doit être $O(\log N)$ et on demande de la justifier.

* *
*

ECOLES NORMALES SUPERIEURES

CONCOURS D'ADMISSION 2024

**VENDREDI 19 AVRIL 2024
14h00 - 18h00**

FILIERES MP et MPI

Epreuve n° 10

INFO-FONDAMENTALE (ULSR)

Durée : 4 heures

***L'utilisation des calculatrices n'est pas
autorisée pour cette épreuve***

Couverture minimale d'un graphe

Le sujet comporte 12 pages, numérotées de 1 à 12.

Début de l'épreuve.

Ce sujet est consacré au calcul d'un paramètre de graphe appelé **couverture minimale**, défini comme étant le plus petit nombre de sommets d'un **ensemble couvrant**. On étudiera plusieurs façon de le calculer, ce qui nous permettra de faire un tour d'horizon de quelques techniques algorithmiques modernes.

Ce sujet est constitué de 6 parties qui peuvent être traitées relativement indépendamment ; cependant, les concepts introduits dans une partie peuvent être réutilisés dans des parties ultérieures.

- Dans la première partie (page 3) on découvre progressivement certaines propriétés des couvertures minimales, leurs valeurs sur quelques familles de graphes simples et leurs liens avec d'autres paramètres classiques de la théorie des graphes.
 - Dans la deuxième partie (page 4) on développe deux algorithmes efficaces qui calculent un ensemble couvrant minimum d'un arbre.
 - Dans la troisième partie (page 5) on étudie deux heuristiques naturelles calculant un ensemble couvrant s'avérant loin d'être minimum, pour finalement découvrir un algorithme simple calculant une approximation de la couverture minimale.
 - Dans la quatrième partie (page 7) on découvre la complexité paramétrée.
 - Dans la cinquième partie (page 9) on continue notre exploration de la complexité paramétrée en introduisant la notion de noyau.
 - Dans la sixième partie (page 10) on découvre la puissance de l'optimisation linéaire, permettant une autre approximation de la couverture minimale ainsi qu'un noyau plus petit que celui de la partie V.
-

Notations et définitions

- Un **graphe** (non-orienté) G est la donnée d'un ensemble fini $V(G)$ de **sommets** et d'un ensemble $E(G)$ dont les éléments, appelés **arêtes**, sont des paires de sommets de V . On notera le fait que deux sommets u et v sont reliés par une arête par $\{u, v\} \in E(G)$. On dit que u et v sont les **extrémités** de l'arête $\{u, v\}$ et que u et v sont **adjacents** dans G .
- Une conséquence de cette définition est que tous les graphes considérés sont **simples**, c'est à dire sans boucle ($\{u, u\} = \{u\}$ n'est pas une arête), et avec au plus une arête entre deux sommets.
- Soit v un sommet d'un graphe G . Le **voisinage** $N(v)$ de v est l'ensemble des sommets adjacents à v , c'est-à-dire $\{u \in V(G) \mid \{u, v\} \in E(G)\}$. Le **degré** $d(v)$ est le cardinal de $N(v)$.
- Un **chemin** $P = (v_1, \dots, v_n)$ dans un graphe G est une liste de sommets deux à deux distincts tels que pour tout $1 \leq i < n$, v_i et v_{i+1} sont adjacents dans G .
- Un **ensemble couvrant** d'un graphe G est un ensemble de sommets X tel que toute arête de G a au moins une extrémité dans X . Le nombre de sommets d'un **ensemble couvrant minimum** (c'est-à-dire, un ensemble couvrant minimisant le nombre de sommets) de G est appelé **couverture minimale** de G et noté $\tau(G)$.
- Soit G un graphe et X un ensemble de sommets de G . On dénote par $G - X$ le graphe sur les sommets $V(G) \setminus X$ avec comme arêtes $\{\{u, v\} \in E(G) \mid u \notin X \text{ et } v \notin X\}$.
- Par **complexité** (en temps) d'un algorithme ALG , on entend le nombre d'opérations élémentaires nécessaires à l'exécution de ALG dans le pire des cas.
- Lorsque la complexité en temps dépend d'un ou plusieurs paramètres $x_1, \dots, x_p$, on dit qu'elle est en $O(f(x_1, \dots, x_p))$ s'il existe une constante $C > 0$ telle que, pour toutes les valeurs de $x_1, \dots, x_p$, suffisamment grandes (c'est-à-dire, plus grandes qu'un certain seuil), la complexité est au plus $C \times f(x_1, \dots, x_p)$. La complexité est dite **linéaire** si $f(x_1, \dots, x_p) = x_1 + \dots + x_p$.
- Dans tout l'énoncé (sauf dans la partie II où on travaille sur des arbres et dans la partie III où on ne s'intéresse pas à la complexité précise des algorithmes), on fera l'hypothèse que les graphes sont encodés par leur matrice d'adjacence.

Partie I

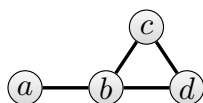
Un graphe est **complet** si tous les sommets sont deux à deux adjacents. On dénote par K_n le graphe complet sur n sommets $\{1, \dots, n\}$.

Un **ensemble indépendant** d'un graphe est un ensemble de sommets du graphe dont les sommets sont deux à deux non-adjacents. On note $\alpha(G)$ le nombre maximum de sommets d'un ensemble indépendant de G . Un **couplage** est un ensemble d'arêtes deux à deux disjointes, c'est à dire ne partageant aucun sommet. Le nombre maximum d'arêtes dans un couplage de G est noté $\mu(G)$.

Un graphe est **biparti** si ses sommets peuvent être partitionnés en deux ensembles indépendants. Il est **biparti complet** s'il admet une partition en deux ensembles indépendants telle que chaque sommet d'une partie est adjacent à chaque sommet de l'autre partie. Étant donné deux entiers naturels a et b , on dénote par $K_{a,b}$ le graphe biparti complet dont les parties de la partition en deux ensembles indépendants ont respectivement a et b sommets (disons, les sommets étant numérotés de 1 à a dans la première partie et de $a+1$ à $a+b$ dans la deuxième).

On dénote par P_n le **graphe chemin** sur $n \geq 1$ sommets, c'est-à-dire le graphe dont les sommets sont $\{1, \dots, n\}$ et les arêtes $\{\{1, 2\}, \{2, 3\}, \dots, \{n-1, n\}\}$. Un **graphe cycle** est le graphe obtenu en ajoutant une arête entre les deux extrémités d'un graphe chemin. On dénote par C_n le graphe cycle sur $n \geq 2$ sommets.

Question I.1. Donner les valeurs de $\tau(G)$, $\alpha(G)$ et $\mu(G)$ pour le graphe G représenté ci-dessous. Justifier.



Question I.2. Donner la valeur de $\tau(K_n)$, $\tau(P_n)$, $\tau(C_n)$ et $\tau(K_{a,b})$ pour tous entiers naturels non nuls a, b, n . Justifier soigneusement les réponses.

Question I.3. Montrer que pour tout graphe G , $\tau(G) = |V(G)| - \alpha(G)$.

Question I.4. Montrer que pour tout graphe G , $\tau(G) \geq \mu(G)$.

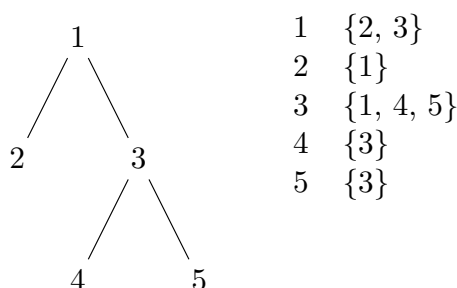
Aucun algorithme polynomial n'est connu pour calculer un ensemble couvrant minimum d'un graphe. Dans la question suivante, on demande un algorithme exponentiel.

Question I.5. Donner une description à haut niveau (sans détailler les lignes de pseudo-code) d'un algorithme calculant un ensemble couvrant minimum d'un graphe G en temps $O(2^{|V(G)|} \cdot |V(G)|^2)$.

Partie II

Un **arbre** est un graphe connexe sans cycle. Un sommet de degré ≤ 1 d'un arbre est appelé une **feuille**. Dans cette partie, on va développer deux algorithmes efficaces pour calculer un ensemble couvrant minimum d'un arbre.

Dans cette partie, contrairement au reste du sujet, les arbres ne sont pas représentés par leur matrice d'adjacence mais par un tableau T dont les éléments sont des ensembles de sommets : pour chaque sommet $1 \leq i \leq n$ (où n est le nombre de sommets de l'arbre), $T[i]$ est une représentation de l'ensemble des sommets adjacents à i . On supposera que la structure de données utilisée pour représenter l'ensemble $T[i]$ permet en temps $O(1)$ les opérations suivantes : renvoyer le nombre d'éléments de l'ensemble ; rechercher si un élément donné appartient à l'ensemble ; ajouter un nouvel élément à l'ensemble ; retirer un élément existant de l'ensemble. Par exemple, voici un arbre sur les sommets $\{1, \dots, 5\}$ et sa représentation sous forme de tableau dont les éléments sont les ensembles de sommets adjacents :



Question II.1. Soient T un arbre et $\{u, v\}$ une arête de T telle que v est une feuille de T . Montrer qu'il existe un ensemble couvrant minimum de T qui contient le sommet u , mais pas v .

Question II.2. En utilisant la question précédente, proposer un algorithme (en pseudo-code) qui reçoit un arbre T en entrée, et renvoie un ensemble couvrant minimum de T , en enlevant progressivement des arêtes à l'arbre. La complexité de cet algorithme doit être linéaire en le nombre n de sommets de l'arbre. Justifier de la complexité de l'algorithme.

Question II.3. Proposer un autre algorithme (en pseudo-code) qui renvoie le nombre de sommets d'un ensemble couvrant minimum d'un arbre, cette fois basé sur la programmation dynamique. On pourra commencer par **enraciner** l'arbre, c'est-à-dire choisir un sommet arbitraire qui jouera le rôle de racine de l'arbre. Pour chaque sommet $v \in V(G)$, on note T_v le sous-arbre de T enraciné en v ; pour calculer le résultat final, l'algorithme devra d'abord calculer en programmation dynamique pour chaque $v \in V(G)$ les trois valeurs suivantes :

- $D_{\in}(v)$ = nombre de sommets d'un ensemble couvrant minimum de T_v contenant v ;
- $D_{\notin}(v)$ = nombre de sommets d'un ensemble couvrant minimum de T_v ne contenant pas v ;
- $D(v)$ = nombre de sommets d'un ensemble couvrant minimum de T_v .

On ne demande pas la preuve de la correction, mais il faut donner et prouver sa complexité en fonction du nombre n de sommets de l'arbre.

Partie III

Dans cette partie, plutôt que d'essayer de calculer un ensemble couvrant minimum, on va chercher à en calculer un dont le cardinal est proche du minimum.

Formalisons cette idée. Soit ALG un algorithme prenant un graphe en entrée et renvoyant un ensemble couvrant (pas forcément minimum). On note $ALG(G)$ le nombre de sommets de l'ensemble couvrant renvoyé par ALG appliqué à G . On note $OPT(G)$ le nombre de sommets d'un ensemble couvrant minimum de G . On dit que ALG est une ***c-approximation*** (pour un nombre réel $c \geq 1$) si pour tout graphe G , on a :

$$\frac{ALG(G)}{OPT(G)} \leq c.$$

Noter que les algorithmes que nous allons considérer n'ont pas une façon unique de s'exécuter sur un graphe donné. Typiquement, à la ligne 3 de l'algorithme HEURISTIQUE ci-dessous, il peut y avoir beaucoup de façon différentes de choisir le sommet v . Dans ce cas, on considère que $HEURISTIQUE(G)$ est le résultat de la pire exécution possible, c'est à dire celle renvoyant le plus grand ensemble couvrant parmi tous les ensembles couvrants qu'elle peut potentiellement renvoyer.

On considère un premier algorithme calculant un ensemble couvrant :

Algorithme 1 HEURISTIQUE(G)

```

1:  $S \leftarrow \emptyset$ 
2: while  $G$  a des arêtes do
3:   Choisir un sommet  $v$  de  $G$  tel que  $d(v) \geq 1$ 
4:    $S \leftarrow S \cup \{v\}$ 
5:    $G \leftarrow G - \{v\}$ 
6: return  $S$ 
```

Question III.1. Montrer que l'ensemble S calculé par HEURISTIQUE est bien un ensemble couvrant.

Question III.2. Montrer que, pour tout entier $n \geq 2$, il existe un graphe G à n sommets tel que

$$\frac{HEURISTIQUE(G)}{OPT(G)} \geq n - 1.$$

L'heuristique ci-dessus pouvant donner des résultats catastrophiques, on propose de l'améliorer en choisissant le sommet de degré maximum, donc celui couvrant le plus d'arêtes.

Algorithme 2 GLOUTON(G)

```

1:  $S \leftarrow \emptyset$ 
2: while  $G$  a des arêtes do
3:   Choisir un sommet  $v$  de degré maximum dans  $G$ 
4:    $S \leftarrow S \cup \{v\}$ 
5:    $G \leftarrow G - \{v\}$ 
6: return  $S$ 
```

GLOUTON renvoie un ensemble couvrant pour les mêmes raisons que HEURISTIQUE.

On va définir une série de graphes bipartis G_n avec bipartition A_n et B_n comme suit. A_n est un ensemble de n sommets. B_n est partitionné en n sous-ensembles $B_n^1, \dots, B_n^n$ ayant les propriétés suivantes pour $k = 1, \dots, n$:

- (i) $|B_n^k| = \lfloor \frac{n}{k} \rfloor$,
- (ii) chaque sommet de B_n^k a exactement k voisins dans A_n ,
- (iii) deux sommets de B_n^k n'ont pas de voisins communs dans A_n .

Question III.3. Dessiner (une réalisation possible de) G_4 et donner les valeurs de $\text{OPT}(G_4)$ et de $\text{GLOUTON}(G_4)$. On ne demande pas de justifier.

Question III.4. En utilisant la construction ci-dessus, montrer que pour toute constante c , GLOUTON n'est pas une c -approximation.

Ainsi, GLOUTON ne donne pas non plus une bonne approximation. On va maintenant considérer l'algorithme suivant.

Algorithme 3 APPROX(G)

```

1:  $S \leftarrow \emptyset$ 
2: while  $G$  a des arêtes do
3:   Choisir une arête  $\{u, v\}$  de  $G$ 
4:    $S \leftarrow (S \cup \{u\}) \cup \{v\}$ 
5:    $G \leftarrow (G - \{u\}) - \{v\}$ 
6: return  $S$ 
```

Question III.5. Montrer que APPROX renvoie un ensemble couvrant, en s'appuyant sur la question I.3.

Question III.6. Montrer que APPROX est une 2-approximation.

Question III.7. Donner une suite infinie de graphes justifiant que, pour tout $c < 2$, APPROX n'est pas une c -approximation.

Partie IV

Dans cette partie, on cherche à répondre à la question suivante : *Étant donné un graphe G et un entier k , est-ce que $\tau(G) \leq k$?* On rappelle que G est représenté par sa matrice d'adjacence.

Le but est de trouver un algorithme qui, étant donné un graphe G et un entier k , décide si $\tau(G) \leq k$ en temps $O(f(k) \times |V(G)|^c)$ où f est une fonction (arbitraire, potentiellement exponentielle en son argument) et $c \geq 0$ une constante. Un algorithme avec une telle complexité est dit **FPT** pour *fixed-parameter tractable* ou *tractable à paramètre fixé*.

On va commencer modestement. Noter que l'algorithme demandé à la question ci-dessous n'est pas FPT.

Question IV.1. Donner une description à haut niveau (sans détailler les lignes de pseudo-code) d'un algorithme qui, étant donné un graphe G et un entier k , décide si $\tau(G) \leq k$ en temps $O(|V(G)|^{k+2})$.

Le reste de cette partie est dédié à la conception de notre premier algorithme FPT.

Question IV.2. Soit G un graphe et $\{u, v\} \in E(G)$. Montrer l'équivalence suivante :

$$\tau(G) \leq k \iff \tau(G - \{u\}) \leq k - 1 \text{ ou } \tau(G - \{v\}) \leq k - 1.$$

Question IV.3. En s'aidant de la question précédente, donner un algorithme récursif (en pseudo-code) qui, étant donné un graphe G et un entier k , décide si $\tau(G) \leq k$ en temps $O(2^k \times |V(G)|^2)$. Justifier la correction et la complexité de l'algorithme. On rappelle qu'avec la représentation en matrice d'adjacence, il est possible de supprimer l'ensemble des arêtes dont un sommet u est une extrémité en $O(|V(G)|)$.

On va maintenant légèrement modifier l'algorithme de la question précédente pour améliorer sa complexité.

Question IV.4. Donner une description précise des graphes de degré maximum 2. En déduire une description à haut niveau (sans détailler les lignes de pseudo-code) d'un algorithme en $O(|V(G)|)$ pour décider si un graphe de degré maximum 2 a un ensemble couvrant avec au plus k sommets.

On rappelle que, étant donné un sommet v , $N(v)$ est l'ensemble des voisins de v et on définit $N[v] = N(v) \cup \{v\}$, le *voisinage fermé* de v .

Question IV.5. Soit $v \in V$. Montrer que si $d(v) \geq 3$, alors

$$\tau(G) \leq k \iff \tau(G - \{v\}) \leq k - 1 \text{ ou } \tau(G - N[v]) \leq k - d(v).$$

Question IV.6. En s'aidant de la question précédente, donner un algorithme récursif (en pseudo-code) qui décide si $\tau(G) \leq k$ en temps $O(1.4656^k \times |V(G)|^2)$. Justifier la correction et la complexité de l'algorithme. On utilisera le fait que la fonction $f(k)$ définie récursivement comme suit :

$$f(i) = \begin{cases} f(i-1) + f(i-3) & \text{si } i \geq 3 \\ 1 & \text{sinon.} \end{cases}$$

satisfait $f(k) \leq 1.4656^k$.

Partie V

Comme dans la partie précédente, on cherche à répondre à la question suivante : *Étant donné un graphe G et un entier k , est-ce que $\tau(G) \leq k$?*

On va s'intéresser à la notion de noyau. On dit que le problème de l'ensemble couvrant a un **noyau de taille** $f(k)$ (pour une certaine fonction croissante f) s'il existe un algorithme qui étant donné une instance (G, k) renvoie en temps polynomial (en $|V(G)|$) une instance (G', k') tel que :

- (i) $\tau(G) \leq k$ si et seulement si $\tau(G') \leq k'$,
- (ii) si $\tau(G) \leq k$, alors $|V(G')| \leq f(k)$ et
- (iii) $k' \leq k$.

Question V.1. Montrer que si le problème de l'ensemble couvrant a un noyau de taille $f(k)$, alors il existe un polynôme P tel qu'on peut décider si $\tau(G) \leq k$ en temps $O\left(P(|V(G)|) + f(k)^{k+2}\right)$.

Le reste de la partie est dédié à la conception d'un algorithme montrant que le problème de l'ensemble couvrant a un noyau de taille $k^2 + k$.

Question V.2. Montrer que si un sommet v de G a degré 0, alors $\tau(G) \leq k$ si et seulement si $\tau(G - \{v\}) \leq k$.

Question V.3. Montrer que si un sommet v de G a degré au moins $k + 1$, alors $\tau(G) \leq k$ si et seulement si $\tau(G - \{v\}) \leq k - 1$.

On considère maintenant l'algorithme qui, étant donné (G, k) , applique les deux règles suivantes (dans n'importe quel ordre) tant que c'est possible. On note (G', k') le couple obtenu.

Règle 1 : si un sommet v a degré 0, supprimer v .

Règle 2 : si un sommet v a degré au moins $k + 1$, supprimer v , et diminuer k de 1.

On remarque à l'aide des deux questions précédentes que $\tau(G) \leq k$ si et seulement si $\tau(G') \leq k'$.

Question V.4. Donner la complexité de cet algorithme en fonction de $|V(G)|$ (on rappelle que G est représenté par sa matrice d'adjacence).

Question V.5. Montrer que pour tout sommet v de G' , on a dans G' : $1 \leq d(v) \leq k'$. En déduire que si $\tau(G') \leq k'$, alors $|V(G')| \leq k' + k'^2$ et $E(G') \leq k'^2$.

Question V.6. Conclure que le problème de l'ensemble couvrant a un noyau de taille $k^2 + k$.

Partie VI

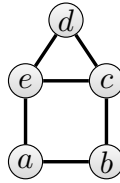
Soit $G = (V, E)$ un graphe avec $V = \{v_1, \dots, v_n\}$. Un ensemble de sommets S peut être représenté par un vecteur $x = (x_{v_1}, x_{v_2}, \dots, x_{v_n}) \in \{0, 1\}^n$ tel que, pour $i = 1, \dots, n$, $x_{v_i} = 1$ si $v_i \in S$ et $x_{v_i} = 0$ si $v_i \notin S$.

On introduit une variable x_v pour chaque sommet de G . Observer que les ensembles couvrant de G correspondent précisément aux solutions du système d'équations suivant :

$$\begin{cases} x_u + x_v \geq 1 \text{ pour tout } \{u, v\} \in E(G); \\ x_v \in \{0, 1\} \text{ pour tout } v \in V(G). \end{cases} \quad (1)$$

Ainsi, une solution minimisant $\sum_{v \in V(G)} x_v$ correspond à un ensemble couvrant minimum. Pour un graphe G donné, on nomme $\text{OLNEEC}(G)$ (pour **optimisation linéaire en nombre entier pour ensemble couvrant**) le système d'équations (1) associé à G .

Question VI.1. Écrire le système d'équations $\text{OLNEEC}(G)$ pour le graphe ci-dessous.



En relaxant la contrainte $x_v \in \{0, 1\}$ par $0 \leq x_v \leq 1$, on obtient le système d'équations suivant nommé $\text{OLEC}(G)$ (pour **optimisation linéaire pour ensemble couvrant**) :

$$\begin{cases} x_u + x_v \geq 1 \text{ pour tout } \{u, v\} \in E(G); \\ 0 \leq x_v \leq 1 \text{ pour tout } v \in V(G). \end{cases} \quad (2)$$

On peut considérer que $x_u = \frac{1}{3}$ (par exemple) correspond à mettre $\frac{1}{3}$ du sommet u dans la solution. Une solution à ce système d'équations s'appelle un **ensemble couvrant fractionnaire** de G et sa **taille** vaut $\sum_{v \in V(G)} x_v$. Une solution minimisant $\sum_{v \in V(G)} x_v$ est un **ensemble couvrant fractionnaire minimum** de G , et sa taille est noté $\tau_f(G)$. Il est clair que pour tout graphe G ,

$$\tau_f(G) \leq \tau(G)$$

Question VI.2. Parmi les vecteurs $(x_a, x_b, x_c, x_d, x_e)$ suivants, dites lesquels sont des ensembles couvrants fractionnaires du graphe de la question VI.1 et lesquels ne le sont pas. Donner leur taille quand ils le sont, et expliquer précisément la raison quand ils ne le sont pas.

$$\left(\frac{1}{2}, \frac{1}{2}, \frac{1}{2}, \frac{1}{2}, \frac{1}{2}\right), \quad \left(\frac{1}{3}, \frac{1}{3}, \frac{2}{3}, \frac{2}{3}, \frac{1}{3}\right), \quad (1, 1, 0, 1, 0), \quad (1, 0, 1, 1, 0).$$

Question VI.3. Montrer que pour tout graphe G , on a $\tau_f(G) \leq \frac{|V(G)|}{2}$. En déduire qu'il n'existe pas de constante A telle que $\tau(G) - \tau_f(G) < A$ pour tout graphe G .

Soit G un graphe. Soit $(x_v)_{v \in V(G)} \in [0, 1]^n$ un ensemble couvrant fractionnaire minimum de G . On partitionne les sommets de G de la façon suivante :

$$V_0 = \left\{v : x_v < \frac{1}{2}\right\} \quad V_{\frac{1}{2}} = \left\{v : x_v = \frac{1}{2}\right\} \quad V_1 = \left\{v : x_v > \frac{1}{2}\right\}$$

Question VI.4. Montrer que V_0 est un ensemble indépendant (c'est à dire qu'aucune arête n'a ses deux extrémités dans V_0), et qu'il n'y a pas d'arête ayant une extrémité dans V_0 et l'autre dans $V_{\frac{1}{2}}$.

Question VI.5. Montrer que $V_{\frac{1}{2}} \cup V_1$ est un ensemble couvrant de G et que $|V_{\frac{1}{2}} \cup V_1| \leq 2\tau(G)$. On admettra qu'il existe un algorithme en temps polynomial permettant de trouver un ensemble couvrant fractionnaire minimum. En déduire un algorithme polynomial qui donne une 2-approximation pour le problème de l'ensemble couvrant minimum.

Le but des 3 prochaines questions est de montrer qu'il existe un ensemble couvrant minimum inclus dans $V_{\frac{1}{2}} \cup V_1$. Soit S^* un ensemble couvrant minimum de G , et soit $S = (V_{\frac{1}{2}} \cap S^*) \cup V_1$.

Question VI.6. Montrer que S est un ensemble couvrant de G .

On va perturber la solution $(x_v)_{v \in V(G)}$ comme suit. On pose

$$\varepsilon = 0 \text{ si } V_0 \cup V_1 = \emptyset \quad \text{et} \quad \varepsilon = \min \left\{ \left| x_v - \frac{1}{2} \right| : v \in V_0 \cup V_1 \right\} \text{ sinon}$$

et pour tout $v \in V(G)$, on définit :

$$y_v = \begin{cases} x_v - \varepsilon & \text{si } v \in V_1 \setminus S^*; \\ x_v + \varepsilon & \text{si } v \in V_0 \cap S^*; \\ x_v & \text{dans tous les autres cas.} \end{cases}$$

Question VI.7. Montrer que $(y_v)_{v \in V(G)}$ est un ensemble couvrant fractionnaire de G .

Question VI.8. Montrer que S est un ensemble couvrant minimum de G .

On va maintenant étudier l'algorithme suivant pour déterminer, étant donné un graphe G et un entier k , si $\tau(G) \leq k$.

Algorithme 4 OLNOYAU(G, k)

- 1: Calculer un ensemble couvrant fractionnaire minimum $(x_v)_{v \in V(G)}$ à l'aide d'un algorithme en temps polynomial.
 - 2: Définir les ensembles V_0 , $V_{\frac{1}{2}}$ et V_1 comme précédemment.
 - 3: **if** $\sum_{v \in V(G)} x_v > k$ **then return** NON
 - 4: **if** $|V_{\frac{1}{2}}| > 2k$ **then return** NON
 - 5: Résoudre l'instance $(G - V_0 - V_1, k - |V_1|)$ en utilisant un algorithme par force brute.
-

Question VI.9. Montrer que si l'algorithme renvoie NON aux lignes 3 ou 4, alors $\tau(G) > k$.

Question VI.10. Montrer que $\tau(G) \leq k$ si et seulement si $\tau(G - V_0 - V_1) \leq k - |V_1|$. Conclure que le problème de l'ensemble couvrant a un noyau de taille $2k$.

Fin du sujet.

A2024 – INFO I MPI

**ÉCOLE DES PONTS PARISTECH,
ISAE-SUPAERO, ENSTA PARIS,
TÉLÉCOM PARIS, MINES PARIS,
MINES SAINT-ÉTIENNE, MINES NANCY,
IMT ATLANTIQUE, ENSAE PARIS,
CHIMIE PARISTECH - PSL.**

**Concours Mines-Télécom,
Concours Centrale-Supélec (Cycle International).**

CONCOURS 2024**PREMIÈRE ÉPREUVE D'INFORMATIQUE**

Durée de l'épreuve : 3 heures

L'usage de la calculatrice et de tout dispositif électronique est interdit.

Cette épreuve concerne uniquement les candidats de la filière MPI.

L'énoncé de cette épreuve comporte 9 pages de texte.

Si, au cours de l'épreuve, un candidat repère ce qui lui semble être une erreur d'énoncé, il le signale sur sa copie et poursuit sa composition en expliquant les raisons des initiatives qu'il est amené à prendre.

Les sujets sont la propriété du GIP CCMP. Ils sont publiés sous les termes de la licence
Creative Commons Attribution - Pas d'Utilisation Commerciale - Pas de Modification 3.0 France.

Tout autre usage est soumis à une autorisation préalable du Concours commun Mines Ponts.



Préliminaires

Présentation du sujet

L'épreuve est composée d'un problème unique comportant 33 questions divisées en trois sections. L'objectif du problème est d'extraire un *sous-graphe le plus dense* d'un graphe non orienté quelconque, c'est-à-dire repérer un sous-ensemble de sommets riche en arêtes. Le calcul d'un sous-graphe dense possède des applications considérables dans le domaine de la fouille des données : qu'il s'agisse de détection de communautés dans des réseaux sociaux, de repérage d'attaques par pollupostage (ou *spamming*) ou d'identification de complexes protéiques au sein de données biologiques massives pour ne citer que quelques exemples.

Dans la première section (page 1), nous résolvons un problème de minimisation de bordure dans un multigraphe orienté à l'aide d'une technique de *chemins augmentants*. Dans la deuxième section (page 5), nous étudions une technique de *réduction du problème* de construction d'un sous-graphe le plus dense au problème de la première section. Dans la troisième section (page 8), nous étudions un *algorithme d'approximation* plus rapide.

Dans tout l'énoncé, un même identificateur écrit dans deux polices de caractères différentes désignera la même entité, mais du point de vue mathématique avec la police en italique (par exemple n , $n_{\max}$ ou n_1) et du point de vue informatique avec celle en romain avec espacement fixe (par exemple `n`, `NMAX` ou `n1`).

Travail attendu

Pour répondre à une question, il est permis de réutiliser le résultat d'une question antérieure, même sans avoir réussi à établir ce résultat.

Selon les consignes, il faudra coder des fonctions à l'aide du langage de programmation C exclusivement, en reprenant le prototype de fonction fourni par le sujet, ou en pseudo-code (c-à-d. dans une syntaxe souple mais conforme aux possibilités offertes par le langage C). Inclure les entêtes tels que `<assert.h>`, `<stdbool.h>`, etc. n'est pas demandé.

Quand l'énoncé demande de coder une fonction, sauf indication explicite de l'énoncé, il n'est pas nécessaire de justifier que celle-ci est correcte ou de tester que des préconditions sont satisfaites.

Le barème tient compte de la clarté et de la concision des programmes : nous recommandons de choisir des noms de variables intelligibles ou encore de structurer de longs codes par des blocs ou par des fonctions auxiliaires dont on décrit le rôle. Lorsqu'une réponse en pseudo-code est permise, seule la logique de programmation est évaluée, même dans le cas où un code en C a été fourni en guise de réponse.

1 Entourage de plus petite bordure

Dans cette section, nous travaillons dans un *multigraphe orienté*, c'est-à-dire un couple (W, F, μ) où W est un ensemble fini, appelé ensemble de sommets, F est une partie de $W \times W$, appelée ensemble d'arcs, et $\mu : F \rightarrow \mathbb{N}^*$ est une application, appelée multiplicité des arcs.

Les démonstrations ou justifications pourront utiliser la notion de *multi-ensemble* sans formalité excessive. L'intersection, notée $\cap$, se calcule en considérant le minimum des multiplicités des éléments; l'union disjointe, notée $\sqcup$, se calcule en considérant la somme des multiplicités des éléments; le cardinal se calcule en sommant toutes les multiplicités.

1.1 Réseaux

Définitions : Nous appelons *réseau* tout quintuplet $H = (W, F, \mu, s, t)$ tel que

- W est un ensemble fini, appelé ensemble de *sommets*,
- F est un sous-ensemble de couples de $W \times W$, appelé ensemble d'*arcs*,
- μ est une application $F \rightarrow \mathbb{N}^*$, l'entier naturel $\mu(e)$ étant appelé la *multiplicité* de l'arc e ,
- s est un sommet particulier de W , appelé la *source*,
- t est un sommet particulier de W , appelé le *puits*.

Indication C : Par convention, nous numérotons les sommets d'un réseau $H = (W, F, \mu, s, t)$ par les entiers entre 0 et $n + 1$, où $n + 2$ est le cardinal de W . La source s porte le numéro 0 ; le puits t porte le numéro $n + 1$.

Nous préparons deux fichiers, `network.h` et `network.c`. Le premier fichier contient les déclarations suivantes.

```

1.  /* Debut du fichier network.h */
2.
3.  #ifndef NETWORK_H
4.  #define NETWORK_H
5.
6.  const unsigned int NMAX = 102;
7.
8.  struct network_s {
9.      unsigned int n;
10.     unsigned mu[NMAX][NMAX];
11. };
12. typedef struct network_s network;
13.
14. void nw_init(unsigned int n);
15. void nw_add(int u, int v);
16. void nw_remove(int u, int v);
17. void nw_disconnect(void);
18.
19. void _arr_init(int *T);
20. bool _bfs_tree(void);
21. void _residue(void);
22.
23. #endif
24.
25. /* Fin du fichier network.h */
```

□ 1 – Expliquer le rôle des lignes 3, 4 et 23 dans le fichier `network.h`.

Le second fichier contient initialement les définitions suivantes et sera complété par les fonctions de la section 1.

Première épreuve d'informatique MPI 2024

```

26.  /* Debut du fichier network.c */
27.
28.  network H;
29.  int A[NMAX];
30.
31.  /* Fin du fichier network.c */

```

Pour tout couple $(u, v) \in W \times W$, le champ $\mu[u][v]$ de la variable globale H vaut 0 si (u, v) n'est pas un arc du réseau et vaut $\mu((u, v))$ si (u, v) est un arc.

☐ 2 – Expliquer comment faire interagir les fichiers `network.h` et `network.c`.

☐ 3 – Écrire une fonction C `void nw_init(unsigned int n)` ainsi spécifiée :

Précondition : L'entier naturel n est strictement inférieur à la constante globale $n_{\max}$.

Effet : La précondition est vérifiée par une assertion.

Postcondition : La variable globale H contient le réseau vide avec $n + 2$ sommets et aucun arc.

☐ 4 – Écrire une fonction C `void nw_add(int u, int v)` qui ajoute une occurrence de l'arc (u, v) au réseau (W, F, μ, s, t) stocké dans la variable globale H , autrement dit, qui incrémente la multiplicité $\mu[u][v]$ d'une unité.

☐ 5 – Écrire une fonction C `void nw_remove(int u, int v)` qui retire une occurrence de l'arc (u, v) dans le réseau stocké dans la variable globale H et qui se défend, par le truchement d'une assertion, du cas où l'arc à supprimer n'existe pas.

☐ 6 – Écrire une fonction C `void _arr_init(int *T)` ainsi spécifiée :

Précondition : Le pointeur T désigne un tableau d'entiers formé de $n + 2$ cases, où $n + 2$ est le nombre de sommets du réseau stocké dans la variable globale H .

Postcondition : Toutes les cases du tableau désigné par le pointeur T valent -1 .

Nous utilisons le terme *chemin* pour signifier systématiquement un chemin simple dans le graphe orienté (W, F) , c'est-à-dire une suite d'arcs $(e_1, \dots, e_t)$ de F distincts qui se succèdent. Le sommet initial d'un chemin s'appelle la *source*, le sommet final s'appelle le *puits*.

☐ 7 – Écrire une fonction C `bool _bfs_tree(void)` ainsi spécifiée :

Précondition : La variable globale H contient un réseau (W, F, μ, s, t) à $n + 2$ sommets.

Effet : La fonction calcule une arborescence $\mathcal{A}$ de plus courts chemins selon un parcours en largeur dans le graphe (W, F) et l'écrit dans la variable globale A .

Postcondition : On a $A[s] = s$ et, pour tout autre sommet $v \in W$ accessible depuis s , la case $A[v]$ contient le prédécesseur du sommet v dans l'arborescence $\mathcal{A}$. Enfin, toute autre case de A vaut -1 .

Valeur de retour : Booléen `true` si le puits t est accessible depuis la source s dans l'arborescence $\mathcal{A}$. Booléen `false` sinon.

1.2 Déconnexion de la source et du puits

Définition : Nous disons qu'un ensemble de chemins $\Pi = \{\pi_1, \dots, \pi_k\}$ de source commune s et de puits commun t est *réalisable* dans le réseau $H = (W, F, \mu, s, t)$ si, pour tout arc $e \in F$, les chemins de Π utilisent moins de $\mu(e)$ fois l'arc e , autrement dit

$$\forall e \in F, \quad \text{Card}(\{j \in \llbracket 1, k \rrbracket \text{ tel que } e \in \pi_j\}) \leq \mu(e).$$

Définition : Étant donné un ensemble réalisable de chemins $\Pi = \{\pi_1, \dots, \pi_k\}$ de source commune s et puits commun t dans un réseau $H = (W, F, \mu, s, t)$, nous appelons *réseau résiduel de H par rapport à Π* , et nous notons $\text{Res}(H, \Pi)$, le réseau obtenu à partir de H en effectuant, pour tout indice j compris entre 1 et k et pour tout arc $e = (u, v) \in \pi_j$, la suppression d'une occurrence de l'arc e et l'ajout d'une occurrence de l'arc opposé $\bar{e} = (v, u)$.

□ 8 – Écrire une fonction C `void _residue(void)` ainsi spécifiée :

Précondition : La variable globale H contient un réseau (W, F, μ, s, t) . L'ensemble des arcs $\{(A[v], v); 0 < v \leq n + 1 \text{ avec } A[v] > 0\}$ tiré de la variable globale A forme une arborescence $\mathcal{A}$ de racine $s = 0$ atteignant le puits t dans le graphe (W, F) .

Postcondition : La variable H contient le réseau résiduel $\text{Res}((W, F, \mu, s, t), \{\alpha\})$ où α est l'unique chemin de source s et de puits t dans l'arborescence $\mathcal{A}$.

□ 9 – Nous appliquons k fois la fonction `_residue` à un réseau H en mettant à jour la variable globale A et notons $\alpha_1, \dots, \alpha_k$ les chemins utilisés entre la source s et le puits t . Montrer qu'il est toujours possible de trouver un ensemble réalisable Π de k chemins dans H tels que le réseau obtenu des applications de `_residue` coïncide avec le réseau résiduel par rapport à Π , autrement dit tel que l'on a

$$\text{Res}(\dots \text{Res}(\text{Res}(H, \{\alpha_1\}), \{\alpha_2\}), \dots, \{\alpha_k\}) = \text{Res}(H, \Pi).$$

□ 10 – Écrire une fonction C `void nw_disconnect(void)` ainsi spécifiée :

Précondition : La variable globale H contient un réseau (W, F, μ, s, t) .

Effet : Tant qu'il existe un chemin de source s et de puits t , le réseau H est transformé en le réseau résiduel $\text{Res}((W, F, \mu, s, t), \{\alpha\})$, où α est un plus court chemin de s vers t .

Postcondition : Si la fonction se termine, il n'existe pas de chemin de la source vers le puits dans le réseau H . De plus, l'ensemble des arcs $\{(A[v], v); 0 \leq v \leq n + 1 \text{ avec } A[v] > 0\}$ tiré de la variable globale A forme une arborescence $\mathcal{A}$ dans le réseau résiduel dans laquelle la racine est s et le sommet t n'est pas accessible.

□ 11 – Démontrer que la fonction `nw_disconnect` se termine en introduisant un variant de boucle inspiré par la question 9.

1.3 Optimalité

Définition : Soit $H = (W, F, \mu, s, t)$ un réseau et S un sous-ensemble de sommets de W . Nous appelons *bordure*, et notons ∂S , l'ensemble d'arcs allant de S vers son complémentaire :

$$\partial S = \{(u, v) \in F \text{ avec } u \in S \text{ et } v \notin S\}.$$

Nous étendons la notion de multiplicité à des ensembles d'arcs en posant :

$$\mu(\partial S) = \sum_{e \in \partial S} \mu(e).$$

Première épreuve d'informatique MPI 2024

Définition : Nous appelons *entourage* dans le réseau $H = (W, F, \mu, s, t)$ tout sous-ensemble de sommets $S \subseteq W$ contenant la source s mais pas le puits t .

□ 12 – Soient Π un ensemble réalisable de k chemins dans un réseau $H = (W, F, \mu, s, t)$ et $S \subseteq W$ un entourage dans H . Établir la relation

$$k \leq \mu(\partial S).$$

Nous fixons pour les quatre prochaines questions un ensemble réalisable Π_0 de k_0 chemins dans un réseau H tel que, dans le réseau résiduel $\text{Res}(H, \Pi_0)$, il n'existe plus aucun chemin entre la source s et le puits t . Nous notons S_0 l'entourage dans H composé des sommets accessibles dans $\text{Res}(H, \Pi_0)$ depuis la source s .

□ 13 – Montrer que, pour tout arc $e = (u, v) \in F$ tel que $u \in S_0$ et $v \notin S_0$, toutes les $\mu(e)$ occurrences de l'arc e sont utilisées par un chemin de l'ensemble Π_0 .

□ 14 – Montrer qu'aucune occurrence d'un arc $(v, u) \in F$ tel que $v \notin S_0$ et $u \in S_0$ n'est utilisée par un chemin de l'ensemble Π_0 .

□ 15 – Démontrer l'égalité

$$k_0 = \mu(\partial S_0).$$

Définition : Nous disons qu'un entourage S est *de plus petite bordure* s'il minimise la multiplicité $\mu(\partial S)$. Nous notons $\sigma(H)$ la multiplicité de la bordure d'un tel entourage.

□ 16 – Démontrer la relation

$$k_0 = \sigma(H)$$

et en déduire que, si la variable globale H contient un réseau, après exécution de la fonction `nw_disconnect`, l'ensemble $\{v \in W ; A[v] \geq 0\}$ est un entourage de plus petite bordure de H , à savoir un entourage de multiplicité $\sigma(H)$.

2 Algorithme de Goldberg

2.1 Densité d'un graphe

Pour tout ensemble fini X , la notation $\binom{X}{2}$ désigne l'ensemble des paires (non ordonnées) à valeur dans X , c'est-à-dire l'ensemble

$$\binom{X}{2} = \{\{u, v\} \text{ avec } (u, v) \in X^2 \text{ tel que } u \neq v\}.$$

Définition : Un *graphe non orienté* est un couple $G = (V, E)$ où V est un ensemble fini, appelé ensemble de sommets, et $E \subseteq \binom{V}{2}$ est un ensemble de paires, appelé ensemble d'arêtes. Pour tout ensemble de sommets $X \subseteq V$, nous notons $G_X = (X, E_X)$ le graphe $G_X = (X, E \cap \binom{X}{2})$, appelé *graphe induit par X sur G* .

Première épreuve d'informatique MPI 2024

Indication C : Par convention, nous numérotons les sommets de tout graphe non orienté $G = (V, E)$ par les entiers entre 1 et n , où n est le cardinal de V . Nous représentons un graphe par un tableau de listes d'adjacence doublement chaînées en adoptant les déclarations de type suivantes.

```

32.  struct link_s {
33.      int node;
34.      struct link_s *prev;
35.      struct link_s *next;
36.  };
37.  typedef struct link_s link;
38.
39.  struct graph_s {
40.      unsigned int n;
41.      link **neighbours;
42.  };
43.  typedef struct graph_s graph;

```

Le champ `neighbours` est un tableau alloué dynamiquement de longueur $n + 1$ et dont la première case n'est pas utilisée. Chaque arête est stockée sous forme de deux arcs.

□ 17 – Écrire une fonction C `graph *gr_create(unsigned int n)` dont la valeur de retour est un graphe non orienté vide à n sommets.

□ 18 – Écrire une fonction C `void gr_add(graph *G, int u, int v)` ayant pour effet d'insérer une arête $\{u, v\}$ dans le graphe non orienté G .

Définition : Nous notons $\deg_G(v)$ le *degré* d'un sommet v dans le graphe non orienté $G = (V, E)$:

$$\deg_G(v) = |\{u \in V \text{ tel que } \{u, v\} \in E\}| = \sum_{\substack{u \in V \text{ t.q.} \\ \{v, u\} \in E}} 1.$$

□ 19 – Écrire une fonction C `int degree(graph *G, int v)` dont la valeur de retour est le degré $\deg_G(v)$ du sommet v dans le graphe non orienté G .

□ 20 – Écrire une fonction C `int edges(graph *G)` dont la valeur de retour est le nombre d'arêtes dans le graphe non orienté G .

Définition : Nous appelons densité du graphe $G = (V, E)$ le rapport

$$\delta(G) = \frac{m}{n}$$

où n est le cardinal de V et m est le cardinal de E .

□ 21 – Écrire une fonction C `double density(graph *G)` dont la valeur de retour est la densité $\delta(G)$ du graphe non orienté G .

Définition : Le problème d'optimisation du *sous-graphe le plus dense d'un graphe non orienté* G consiste à trouver un ensemble de sommets non vide X tels que la densité $\delta(G_X)$ du sous-graphe induit $G_X = (X, E_X)$ est maximale et calculer cette densité maximale

$$\rho(G) = \max_{\substack{X \subseteq V; \\ X \neq \emptyset}} \delta(G_X).$$

2.2 Oracle pour le problème de décision

Nous nous intéressons au problème de décision suivant :

Étant donné un graphe non orienté G à n sommets et un entier naturel r , existe-t-il un sous-graphe induit $G_X = (X, E_X)$ de densité $\delta(G_X)$ strictement supérieure au seuil $\frac{r}{n^2}$? Autrement dit, a-t-on $\rho(G) > \frac{r}{n^2}$?

Définition : Étant donné un graphe non orienté $G = (V, E)$ avec n sommets et m arêtes ainsi qu'un entier naturel r , nous appelons *réseau associé au graphe G et au seuil r* le réseau $H = (W, E, \mu, s, t)$ où

(α) L'ensemble W est la réunion $W = V \cup \{s, t\}$, les sommets source s et puits t étant de nouveaux sommets n'appartenant pas à V .

(β) L'ensemble d'arcs F comprend :

- pour tout sommet $v \in V$, un arc (s, v) , de multiplicité mn^2 .
- pour toute arête $\{u, v\} \in E$, un arc (u, v) et un arc (v, u) , de multiplicité n^2 .
- pour tout sommet $u \in V$, un arc (u, t) , de multiplicité $mn^2 - n^2 \deg_G(u) + 2r$.

□ 22 – Écrire une fonction C `void reduce(graph *G, int r)` ayant pour effet d'initialiser la constante globale H , de type `network`, par le réseau associé au graphe G et au seuil r .

□ 23 – Soient $G = (V, E)$ un graphe non orienté et $X \subseteq V$ une partie non vide des sommets. Montrer que la densité du graphe induit $G_X = (X, E_X)$ satisfait la relation

$$\delta(G_X) = \frac{\sum_{v \in X} \deg_G(v) - \sum_{\substack{u \in X, v \in V \setminus X \\ \text{t.q. } \{u, v\} \in E}} 1}{2|X|}.$$

□ 24 – Soient $G = (V, E)$ un graphe non orienté avec n sommets et m arêtes, $X \subseteq V$ une partie non vide des sommets, H le réseau associé au graphe G et à un seuil r et S l'entourage $S = \{s\} \cup X$ dans H . Montrer que la multiplicité de la bordure de S satisfait l'égalité

$$\mu(\partial S) = mn^3 + 2|X| \left(r - n^2 \delta(G_X) \right)$$

et que

$$\mu(\partial\{s\}) = mn^3.$$

□ 25 – Soit H le réseau associé au graphe G et au seuil r et soit $\sigma(H)$ la multiplicité de la bordure d'un entourage de plus petite bordure. Montrer les équivalences suivantes :

$$\begin{cases} \sigma(H) = mn^3 & \Leftrightarrow \rho(G) \leq \frac{r}{n^2}, \\ \sigma(H) < mn^3 & \Leftrightarrow \rho(G) > \frac{r}{n^2}. \end{cases}$$

□ 26 – Écrire une fonction C `bool oracle(graph *G, int r)` dont la valeur de retour est le booléen `true` si $\rho(G) > \frac{r}{n^2}$ et `false` si $\rho(G) \leq \frac{r}{n^2}$. En justifier le principe. On pourra s'intéresser au contenu du tableau global `A` après exécution de la fonction `nw_disconnect`.

2.3 Recherche dichotomique

Nous considérons l'ensemble de couples d'entiers

$$\Delta = \left\{ (m', n') \in \mathbb{N}^2 \text{ avec } 0 \leq m' \leq m \text{ et } 1 \leq n' \leq n \right\}.$$

□ 27 – Soient deux couples (m_1, n_1) et (m_2, n_2) appartenant à Δ . Montrer que,

$$\text{si } \frac{m_1}{n_1} \neq \frac{m_2}{n_2}, \quad \text{alors} \quad \left| \frac{m_1}{n_1} - \frac{m_2}{n_2} \right| \geq \frac{1}{n^2}.$$

□ 28 – Soient $G = (V, E)$ un graphe non orienté à n sommets et X une partie non vide de V . Montrer que s'il n'existe aucun sous-graphe induit de densité supérieure ou égale à $\delta(G_X) + \frac{1}{n^2}$, alors le sous-graphe induit G_X est un sous-graphe de densité maximale $\rho(G)$.

□ 29 – Écrire une fonction C efficace `void binary_search(graph *G)` qui, pour tout graphe $G = (V, E)$, modifie le tableau global `A` de sorte que l'ensemble de sommets $X = \{v \in V ; A[v] \geq 0\}$ vérifie

$$\delta(G_X) = \rho(G).$$

Justifier brièvement le principe de fonctionnement de la fonction.

□ 30 – Nous supposons avoir écrit l'ensemble des fonctions de la section 2 dans un fichier `densest_subgraph.c`. Expliquer comment faire interagir le fichier `densest_subgraph.c` avec les fichiers `network.h` et `network.c` de la section 1.

3 Algorithme de Charikar

Dans le cadre d'un compromis entre temps d'exécution et justesse du calcul d'optimisation, nous nous intéressons à l'algorithme suivant.

Algorithme 1 : Recherche gloutonne d'une densité élevée

Entrées : Graphe $G = (V, E)$.

Sorties : Sous-graphe induit à grande densité

```

1  $\Gamma \leftarrow G$  ;                               /* Plus dense graphe rencontré */
2  $G_n \leftarrow G$  où  $n = |V|$ 
3 pour  $i$  de  $n$  à 2 (cas inclus) faire
4   si  $\delta(G_i) > \delta(\Gamma)$  alors
5      $\Gamma \leftarrow G_i$ 
6   Identifier le sommet  $v_i$  qui minimise le degré  $\deg_{G_i}$ .
7   Former le graphe  $G_{i-1}$  en supprimant de  $G_i$  le sommet  $v_i$  et les
   arêtes incidentes à  $v_i$ .
8 retourner  $\Gamma$ 
```

3.1 Analyse d'un facteur d'approximation

Définition : Nous appelons *orientation* d'un graphe non orienté $G = (V, E)$ tout graphe orienté $\hat{G} = (V, \hat{E})$ tel qu'il existe une bijection $\omega : E \rightarrow \hat{E}$ avec $\omega(\{u, v\}) = (u, v)$ pour tout arc $(u, v) \in \hat{E}$. Le *degré entrant* d'un sommet v dans un graphe orienté $\hat{G} = (V, \hat{E})$ est

$$\deg_{\hat{G}}^{-}(v) = \text{Card} \left(\left\{ u \in V \text{ tel que } (u, v) \in \hat{E} \right\} \right) = \sum_{\substack{u \in V \text{ t.q.} \\ (u, v) \in \hat{E}}} 1.$$

□ 31 – Établir la majoration suivante entre la densité d'un graphe non orienté quelconque G et le degré entrant maximum d'une orientation quelconque $\hat{G}$ de G

$$\delta(G) \leq \max_{v \in V} \deg_{\hat{G}}^{-}(v).$$

Nous exécutons l'algorithme 1 sur un certain graphe non orienté G . Dans les questions qui suivent, nous numérotions les sommets de G selon la même numérotation que celle de l'algorithme. Nous notons ω_0 la bijection qui oriente l'arête $\{v_i, v_{i'}\}$ dans le sens $(v_{\min(i, i')}, v_{\max(i, i')})$ et $\hat{G}_0$ l'orientation associée. Autrement dit, nous orientons toute arête, au moment de sa suppression, vers le sommet incident qui a conduit à sa suppression.

□ 32 – Établir, pour tout i compris entre 1 et n , l'inégalité

$$\deg_{\hat{G}_0}^{-}(v_i) \leq 2\delta(G_i)$$

et en déduire que l'algorithme 1 est un algorithme d'approximation dont on précisera le facteur d'approximation.

3.2 Implémentation optimale

□ 33 – Détailler une structure de données qui permette d'exécuter l'algorithme 1 en temps $O(n + m)$ où $n = |V|$ et $m = |E|$. Il est suggéré de nantir la structure de données **graph** d'une table qui maintient, pour tout entier d compris entre 0 et n , la collection des sommets de degré d ainsi que d'autres éléments que l'on jugera opportuns.

Une réponse décrite en pseudo-code ou en langage naturel est admise mais doit être suffisamment précise pour s'assurer que la complexité est belle et bien linéaire.

FIN DE L'ÉPREUVE

A2024 – INFO II MPI

**ÉCOLE DES PONTS PARISTECH,
ISAE-SUPAERO, ENSTA PARIS,
TÉLÉCOM PARIS, MINES PARIS,
MINES SAINT-ÉTIENNE, MINES NANCY,
IMT ATLANTIQUE, ENSAE PARIS,
CHIMIE PARISTECH - PSL.**

**Concours Mines-Télécom,
Concours Centrale-Supélec (Cycle International).**

CONCOURS 2024**DEUXIÈME ÉPREUVE D'INFORMATIQUE**

Durée de l'épreuve : 4 heures

L'usage de la calculatrice et de tout dispositif électronique est interdit.

Cette épreuve concerne uniquement les candidats de la filière MPI.

L'énoncé de cette épreuve comporte 12 pages de texte.

Si, au cours de l'épreuve, un candidat repère ce qui lui semble être une erreur d'énoncé, il le signale sur sa copie et poursuit sa composition en expliquant les raisons des initiatives qu'il est amené à prendre.

Les sujets sont la propriété du GIP CCMP. Ils sont publiés sous les termes de la licence Creative Commons Attribution - Pas d'Utilisation Commerciale - Pas de Modification 3.0 France.

Tout autre usage est soumis à une autorisation préalable du Concours commun Mines Ponts.



Préliminaires

L'épreuve est composée d'un problème unique, comportant 36 questions. Le problème est divisé en quatre sections. Dans la première section (page 1), nous introduisons une forme paresseuse de listes, les *flots de données*, et construisons quelques fonctions de base utiles tout au long du sujet. Dans la deuxième section (page 4), nous nous intéressons à un *algorithme de détermination à la volée du cardinal* d'un flot, dont l'inspiration est probabiliste. Dans la troisième section (page 7), nous nous appuyons sur des raisonnements de logique pour analyser le *chiffrement par flot*. Dans la quatrième section (page 9), nous traitons quelques questions de *combinatoire énumérative* à l'aide d'une grammaire ou simplement d'un procédé manuel de dérécursification.

Les sections 2, 3 et 4 peuvent être vues comme des exercices indépendants qui ne dépendent que de la première section.

Dans tout l'énoncé, un même identificateur écrit dans deux polices de caractères différentes désigne la même entité, mais du point de vue mathématique pour la police en italique (par exemple n) et du point de vue informatique pour celle en romain avec espacement fixe (par exemple `n`).

Des rappels portant sur des extraits du manuel de documentation de OCaml et sur les règles de la déduction naturelle sont fournis en annexe.

Travail attendu

Pour répondre à une question, il est permis de réutiliser le résultat d'une question antérieure, même sans avoir réussi à établir ce résultat.

Il faudra coder des fonctions à l'aide du langage de programmation OCaml exclusivement, en reprenant l'en-tête de fonctions fourni par le sujet, sans s'obliger à recopier la déclaration des types. Il est permis d'utiliser la totalité du langage OCaml mais il est recommandé de s'en tenir aux fonctions les plus courantes afin de rester compréhensible. Quand l'énoncé demande de coder une fonction, sauf demande explicite, il n'est pas nécessaire de justifier que celle-ci est correcte ou de tester que des préconditions sont satisfaites.

Le barème tient compte de la clarté et de la concision des programmes : nous recommandons de choisir des noms de variables intelligibles ou encore de structurer de longs codes par des blocs ou par des fonctions auxiliaires dont on décrit le rôle.

1. Flots de données

1.1. Motivation

En langage OCaml, le type natif `'a list` est défini comme point fixe de l'équation

```
1. type 'a list = [] | (::) of 'a * 'a list
```

Ce type permet de définir des listes finies tout comme certaines listes infinies, ainsi que l'expose l'exemple suivant, qui est parfaitement licite :

```
2. let list_finite = [ 1; 2; 3 ]           (* liste finie a 3 elements *)
3. let rec list_ones = 1 :: list_ones     (* liste infinie toute a 1 *)
```

Seconde épreuve d'informatique MPI 2024

□ 1 – Rendre compte, sous la forme d'un croquis constitué de maillons chaînés entre eux, de l'organisation en mémoire des variables `list_finite` et `list_ones`. Dire quelle région de la mémoire d'un programme est utilisée pour ce stockage.

Les termes de la suite de Fibonacci $(F_n)_{n \in \mathbb{N}}$ obéissent aux relations :

$$F_0 = 0, \quad F_1 = 1, \quad \text{et} \quad \forall n \in \mathbb{N}, F_{n+2} = F_n + F_{n+1}.$$

Nous nous enhardissons à définir la liste des termes de la suite de Fibonacci $(F_n)_{n \in \mathbb{N}}$ en écrivant la déclaration récursive suivante :

```
4. let list_fibo : int list =
5.   let rec list_fibo_with_internal_state (a:int) (b:int) : int list =
6.     a :: (list_fibo_with_internal_state b (a+b))
7.   in
8.     list_fibo_with_internal_state 0 1
```

Cependant, à l'évaluation de cette expression par un interpréteur (ou *toplevel* ou encore *REPL*), le système renvoie le message d'erreur suivant :

```
9. Stack overflow during evaluation (looping recursion?).
```

□ 2 – Traduire en français le mot *stack*. Expliquer le message d'erreur en présentant le déroulement des premières étapes de création de la variable `list_fibo` par l'interpréteur OCaml.

Nous recopions le code fautif dans un fichier `fibo.ml` et — *horresco referens* — nous nous apprêtons à appeler successivement depuis un terminal les commandes `ocamlopt fibo.ml -o fibo` pour la compilation de la source vers un exécutable puis `./fibo` pour l'exécution.

□ 3 – Dire si l'une des deux étapes, compilation ou exécution, renvoie une erreur et, le cas échéant, préciser laquelle des étapes.

1.2. Définition des flots de données

Définition : Un *flot de données* est une suite, finie ou infinie, d'éléments de même type.

Indication OCaml : Afin de représenter les flots de données, nous fixons le type suivant :

```
10. type 'a stream = Nil | Cons of 'a * (unit -> 'a stream)
```

Le constructeur `Nil` qualifie le flot vide. Définir la suite infinie constante égale à 1 en tant que `int stream` demeure possible. Nous parvenons de surcroît à déclarer la suite de Fibonacci en tant que `int stream` sans encombre. En voici le code :

```
11. let rec (ones : int stream) = Cons (1, fun () -> ones)
12.
13. let fibo : int stream =
14.   let rec fibo_with_internal_state a b =
15.     Cons (a, fun () -> fibo_with_internal_state b (a+b))
16.   in
17.     fibo_with_internal_state 0 1
```

 Seconde épreuve d'informatique MPI 2024

- 4 – Écrire une constante OCaml `integers : int stream` dont la valeur de retour est le flot des entiers naturels énumérés par ordre croissant.
- 5 – Écrire une fonction OCaml `range (a:int) (b:int) : int stream` dont la valeur de retour est le flot des entiers compris entre l'entier a inclus et l'entier b exclu énumérés par ordre croissant.
- 6 – Soit $(\sigma_n)_{n \in \mathbb{N}}$ une suite quelconque de chaînes de caractères. Dire s'il existe forcément un flot de donnée, de type `string stream`, qui encode la suite $(\sigma_n)_{n \in \mathbb{N}}$ et, le cas échéant, en préciser la construction en langage OCaml ou en démontrer l'inexistence.

1.3. Quelques manipulations de base des flots de données

- 7 – Écrire une fonction OCaml `hd (u : 'a stream) : 'a` dont la valeur de retour est le premier élément du flot de données u et qui lève une exception `Failure "hd"` si le flot u est vide.
- 8 – Écrire une fonction OCaml `tl (u : 'a stream) : 'a stream` dont la valeur de retour est le flot de données u privé de l'élément de tête et qui lève une exception `Failure "tl"` si le flot u est vide.
- 9 – Écrire une fonction OCaml `of_list (l : 'a list) : 'a stream` dont la valeur de retour est le flot des éléments de la liste ℓ .
- 10 – Écrire une fonction OCaml `iter (f : 'a -> unit) (t : int) (u : 'a stream) : unit` qui applique la fonction f aux t premières valeurs existantes du flot de données u si l'entier t est positif et ne fait rien sinon.
- 11 – Écrire une fonction OCaml `map (f : 'a -> 'b) (u : 'a stream) : 'b stream` dont la valeur de retour est le flot constitué de l'application de la fonction f aux éléments de u .
- 12 – Écrire une fonction OCaml `zip (w : 'a stream array) : 'a array stream` qui transforme un tableau de flots de données $w = [(u_n)_{n \in \mathbb{N}}, (v_n)_{n \in \mathbb{N}}, \dots]$ en un flot de données tabulaire $([u_n, v_n, \dots])_{n \in \mathbb{N}}$. Si l'un des flots de w est vide à partir d'un certain rang, la valeur de retour est aussi vide à partir de ce rang.
Par exemple, `zip [|ones; range 0 100 ; fibo|]` désigne le flot fini des tableaux d'entiers de longueur trois $([1, k, F_k])_{0 \leq k < 100}$.
- 13 – Écrire une fonction OCaml `intertwin (u : 'a stream) (v : 'a stream) : 'a stream` dont la valeur de retour est le flot obtenu en prenant alternativement des éléments du flot u et du flot v , en cessant de prendre des éléments d'un flot une fois que celui-ci est vide. Par exemple, `intertwin ones fibo` désigne le flot d'entiers $1, F_0, 1, F_1, 1, F_2, 1, F_3, \dots$, tandis que `intertwin (range 0 2) fibo` désigne le flot d'entiers $0, F_0, 1, F_1, F_2, F_3, F_4, \dots$.

Seconde épreuve d'informatique MPI 2024

Nous nous proposons d'écrire une fonction `product` (`u1 : 'a stream`) (`u2 : 'b stream`) : (`'a * 'b`) `stream` dont la valeur de retour est un flot qui énumère tous les couples tirés du flot u_1 et du flot u_2 dans un ordre non spécifié. Nous écrivons le code suivant :

```
18. let rec product_wrong u1 u2 =
19.     match u1,u2 with
20.     | _, Nil | Nil, _ -> Nil
21.     | Cons(hd1, tl_begetter1), Cons(hd2, tl_begetter2) ->
22.         let part1 = product_wrong (tl_begetter1 ()) u2 in
23.         let part2 = map (fun y -> (hd1, y)) (tl_begetter2 ()) in
24.         Cons((hd1, hd2), fun () -> intertwin part1 part2)
```

□ 14 – Montrer que l'appel `product_wrong u1 u2` ne se termine pas toujours mais qu'il suffit de déplacer quelques mots au sein du code pour le rendre correct.

1.4. Filtre d'un flot

□ 15 – À titre préliminaire, rappeler brièvement la signification de l'entier 0 dans l'instruction `return 0` qui termine usuellement la fonction `main` en langage C ou dans l'instruction `exit 0` du langage OCaml.

□ 16 – Énoncer le théorème de Turing relatif au problème de l'arrêt.

Pour répondre à la question 17, on pourra admettre sans justification l'existence d'une *machine universelle à chronomètre*, c'est-à-dire une fonction OCaml `run : string -> int -> int` qui se termine toujours et telle que :

- si la chaîne de caractères x est le code source d'une expression OCaml et si l'exécution du code x se termine sans erreur en moins de n étapes élémentaires de calcul, alors `run x n` a pour valeur de retour l'entier 0;
- sinon, `run x n` a pour valeur de retour l'entier 1.

□ 17 – Décrire la construction ou bien réfuter l'existence d'une fonction OCaml `filter (f : 'a -> bool) (u : 'a stream) -> 'a stream` dont la valeur de retour est formé des éléments du flot u satisfaisant le prédicat f , l'ordre des éléments de u étant préservé.

Par exemple, `let is_negative x = x < 0 in filter is_negative ones` vaut `Nil`.

2. Estimation du cardinal d'un flot de données

Définition : Nous appelons *cardinal* d'un flot de données le cardinal de l'ensemble des valeurs distinctes appartenant à ce flot. Ce cardinal peut être *fini* même si le flot est infini.

Dans cette section, nous cherchons à estimer le cardinal s d'un flot de chaînes de caractères $(u_n)_{n \in \mathbb{N}}$ de cardinal fini avec peu de mémoire et en supposant que $0 \leq s < 2^{62}$.

Seconde épreuve d'informatique MPI 2024

Définition : Nous appelons *premier bit activé* d'un entier $h \in \llbracket 0, 2^{62} - 1 \rrbracket$, et notons $\varrho(h)$, le plus grand entier i tel que les $i - 1$ bits les plus à droite dans la représentation binaire de h sont tous nuls. Par convention, nous fixons $\varrho(0) = 63$.

Par exemple, nous avons $\varrho(1) = \varrho(\overline{00001}_{(2)}) = 1$, $\varrho(6) = \varrho(\overline{00110}_{(2)}) = 2$, ou encore $\varrho(8) = \varrho(\overline{01000}_{(2)}) = 4$.

□ 18 – Écrire une fonction OCaml `ffs (h:int) : int` dont la valeur de retour est le premier bit activé $\varrho(h)$ de l'entier h . En anglais, `ffs` désigne *find first set*.

Indication OCaml : Nous disposons d'une fonction de hachage $\mathcal{H}$ à valeur dans l'intervalle entier $\llbracket 0, 2^{62} - 1 \rrbracket$ que nous réalisons à l'aide de `String.hash : string -> int`.

Soit s un entier naturel non nul. Nous tirons s chaînes de caractères aléatoirement et notons $(H_j)_{1 \leq j \leq s}$ leur haché par la fonction de hachage $\mathcal{H}$. Pour tout indice j compris entre 1 et s , nous supposons que chaque variable aléatoire H_j est distribuée selon une loi uniforme dans l'intervalle entier $\llbracket 0, 2^{62} - 1 \rrbracket$. Nous posons

$$R = \max_{1 \leq j \leq s} \varrho(H_j).$$

Une analyse mathématique, que nous admettons, montre alors que les variables aléatoires $\varrho(H_j)$ sont tirées selon des lois géométriques et que la variable R est proche de $\log_2 s$, si bien que la quantité 2^R est une bonne approximation de s .

Soit $(u_n)_{n \in \mathbb{N}}$ un flot de chaînes de caractères de cardinal s , l'entier s étant inconnu. Nous proposons deux algorithmes qui estiment le cardinal s par l'observation des t premiers termes du flot, en supposant que les s valeurs distinctes apparaissent parmi les t premiers termes.

— Algorithme naïf : garnir à la volée une table de hachage avec les chaînes $(u_n)_{n \leq t}$, puis renvoyer le compte des éléments présents dans la table. En voici le code :

```

25. let cardinality_nf (t:int) (u: string stream) : float =
26.   let htb = Hashtbl.create 1 in
27.   iter (fun x-> Hashtbl.add htb x ()) t u;
28.   float_of_int (Hashtbl.length htb)

```

où l'on réutilise la fonction `iter` de la question 10 et des fonctions du module `Hashtbl` rappelées en annexe.

— Algorithme de Flajolet-Martin, inspiré par les observations ci-dessus. En voici le code :

```

29. let cardinality_fm (t:int) (u: string stream) : float =
30.   let r = ref 0 in
31.   let update x = r := max !r (ffs (String.hash x)) in
32.   iter update t u;
33.   float_of_int (1 lsl !r) (* calcule 2^r en flottant *)

```

□ 19 – Comparer la complexité en espace des algorithmes `cardinality_nf` et `cardinality_fm`.

Nous rappelons que la moyenne harmonique de m réels $z_1, \dots, z_m$ strictement positifs se calcule par la formule

$$\text{Harm}(z_1, \dots, z_m) = m \cdot \left(\sum_{j=1}^m \frac{1}{z_j} \right)^{-1}.$$

Seconde épreuve d'informatique MPI 2024

□ 20 – Écrire une fonction OCaml `harmonic_mean` (`z : float array`) : `float` dont la valeur de retour est la moyenne harmonique des termes du tableau `z`.

La suite de chaînes de caractères $(u_n)_{n \in \mathbb{N}}$ et l'entier m étant fixés, pour tout entier i compris entre 0 et $m - 1$ et pour tout entier naturel t , nous notons U_t^i l'ensemble de chaînes de caractères

$$U_t^i = \{u_n \text{ où } n \leq t \text{ et } \mathcal{H}(u_n) \equiv i \pmod{m}\}.$$

Afin d'obtenir une estimation de cardinal du flot $(u_n)_{n \in \mathbb{N}}$ plus robuste et de diminuer la variabilité associée à l'unicité du point de mesure, nous proposons une amélioration de la fonction `cardinality_fm` qui utilise un effet de moyenne.

— Algorithme HyperLogLog : Nous divisons les premiers termes du flot $(u_n)_{n \in \mathbb{N}}$ en m sous-flots disjoints $U_t^0, \dots, U_t^{m-1}$. Nous estimons le cardinal de chaque sous-flot comme le fait l'algorithme de Flajolet-Martin et obtenons des valeurs $(2^{r_i})_{0 \leq i \leq m-1}$, où

$$r_i = \max_{u \in U_t^i} \varrho(\mathcal{H}(u)/m),$$

qui, intuitivement, forment chacune une estimation grossière du quotient s/m . Nous calculons la moyenne harmonique $\text{Harm}(2^{r_0}, \dots, 2^{r_{m-1}})$, qui, intuitivement, constitue une estimation robuste du quotient s/m .

Une analyse mathématique poussée confirme notre intuition. Il s'avère que le cardinal s est très bien approché par la quantité

$$s \simeq \alpha_m \cdot m \cdot \text{Harm}(2^{r_0}, \dots, 2^{r_{m-1}}) \quad (\clubsuit)$$

où α_m est une constante, valant ici $\alpha_m = \left(m \int_0^\infty \left(\log_2 \left(\frac{2+u}{1+u} \right) \right)^m du \right)^{-1}$. On utilisera l'équation $(\clubsuit)$ sans la démontrer durant l'épreuve.

Indication OCaml : Nous fixons les constantes globales pour le reste de la section 2.

```
34. let m = 16
35. let alpha16 = 0.6731 (* calcul numerique admis *)
```

L'algorithme HyperLogLog se présente alors sous la forme

```
1. let cardinality_hll (t:int) (u : string stream) : float =
2.   let r = Array.make m 0 in
3.   iter (hll_update r) t u;
4.   mean_estimate r
```

où les fonctions `hll_update` et `mean_estimate` seront écrites aux questions 21 et 22.

□ 21 – Écrire une fonction OCaml `hll_update` (`r : int array`) (`x : string`) : `unit` ainsi spécifiée :

Précondition : Il existe un flot de chaînes de caractères $(u_n)_{n \in \mathbb{N}}$, des entiers m et t tels que $x = u_t$ et le tableau `r` contient $\left(\max_{u \in U_{t-1}^i} \varrho(\mathcal{H}(u)/m) \right)_{0 \leq i \leq m-1}$.

Postcondition : Le tableau `r` contient $\left(\max_{u \in U_t^i} \varrho(\mathcal{H}(u)/m) \right)_{0 \leq i \leq m-1}$.

Seconde épreuve d'informatique MPI 2024

- 22 – Écrire une fonction OCaml `mean_estimate (r : int array) : float` ainsi spécifiée
Précondition : Le tableau r contient les valeurs $(r_i)_{0 \leq i \leq m-1}$ avec

$$r_i = \max_{u \in U_t^i} \varrho(\mathcal{H}(u)/m)$$

pour un certain flot de chaînes de caractères $(u_n)_{n \in \mathbb{N}}$ et un certain entier m .

Valeur de retour : Estimation du cardinal du flot $(u_n)_{n \in \mathbb{N}}$ d'après l'équation ($\clubsuit$).

Nous nous plaçons finalement dans la situation où deux flots de chaînes de caractères $(u_n)_{n \in \mathbb{N}}$ et $(v_n)_{n \in \mathbb{N}}$ nous parviennent simultanément. Nous chargeons deux fils d'exécution auxiliaires distincts de construire indéfiniment leurs tableaux respectifs de maxima de bit activés tandis que le fil d'exécution principal déclenche toute les cinq secondes l'affichage du cardinal. Voici une ébauche de code :

```

36. let cardinality_threadedhll u v =
37.   let r_u = Array.make m 0 in
38.   let f1 () = iter (hll_update r_u) Int.max_int u in
39.   let _ = Thread.create f1 () in
40.
41.   let r_v = Array.make m 0 in
42.   let f2 () = iter (hll_update r_v) Int.max_int v in
43.   let _ = Thread.create f2 () in
44.
45.   while true do
46.     let r = Array.init m (fun i -> max r_u.(i) r_v.(i)) in
47.     print_float (mean_estimate r);
48.     Thread.delay 5. (* mise en pause de 5 secondes *)
49.   done

```

dans laquelle `Int.max_int` représente le plus grand entier représentable.

- 23 – Identifier les portions de code sujettes à des courses critiques. Indiquer un mécanisme qui permet de corriger l'ébauche de code.

3. Chiffrement par flot de booléens

Dans cette partie, nous nous appuyons sur des variables booléennes $(x_j)_{j \leq k}$ et sur les constantes $\top$ (tautologie) et $\perp$ (antilogie) pour former des formules, à l'aide des trois connecteurs $\neg$ (négation), $\wedge$ (conjonction) et $\vee$ (disjonction). Pour toutes formules de logique ϕ et ψ , nous notons $X(\phi, \psi)$ la nouvelle formule

$$X(\phi, \psi) = (\phi \vee \psi) \wedge (\neg\phi \vee \neg\psi).$$

Les règles d'inférence de la déduction naturelle sont rappelées dans l'annexe B.

- 24 – Soit ϕ une formule. Construire des arbres de preuve qui démontrent les séquents

$$X(\phi, \phi) \vdash \perp \quad \text{et} \quad \perp \vdash X(\phi, \phi)$$

à partir des règles d'inférence de la déduction naturelle.

Seconde épreuve d'informatique MPI 2024

□ 25 – Soit ψ une formule. Construire des arbres de preuve qui démontrent les séquents

$$X(\perp, \psi) \vdash \psi \quad \text{et} \quad \psi \vdash X(\perp, \psi).$$

Nous admettons que, pour toutes formules de logique ϕ et ψ , il existe des arbres de preuve qui démontrent les séquents $X(X(\phi, \phi), \psi) \vdash \psi$ et $\psi \vdash X(X(\phi, \phi), \psi)$.

Définitions : Nous notons $\mathbb{B} = \{V, F\}$ l'ensemble des valeurs de vérité. Nous introduisons la fonction booléenne $\oplus : \mathbb{B} \times \mathbb{B} \rightarrow \mathbb{B}$ telle que $\oplus(x, y)$ est l'évaluation de la formule $X(x, y)$.

□ 26 – Justifier que les fonctions booléennes $\mathbb{B} \times \mathbb{B} \rightarrow \mathbb{B}$ suivantes $(x, y) \mapsto \oplus(\oplus(x, x), y)$ et $(x, y) \mapsto y$ sont égales.

Nous admettons, plus généralement, que la fonction booléenne $\oplus : \mathbb{B} \times \mathbb{B} \rightarrow \mathbb{B}$ est commutative et associative. Pour tout entier $m \geq 1$ et pour tous booléens $(b_i)_{1 \leq i \leq m}$, la somme $\bigoplus_{i=1}^m b_i$ est définie comme $\oplus(\dots \oplus (\oplus(b_1, b_2), b_3), \dots, b_m)$ mais peut être calculée dans n'importe quel ordre. La somme vide vaut, par convention, la valeur de vérité F .

Définition : Nous appelons *registre à décalage à rétroaction linéaire* (ou encore *LFSR* d'après l'anglais *linear feedback shift register*) de longueur m , de germe $(b_i)_{0 \leq i < m} \in \mathbb{B}^m$ et de connexion $C \subseteq \llbracket 0, m-1 \rrbracket$, le flot de booléens $(b_n)_{n \in \mathbb{N}}$ défini par la relation de récurrence

$$\forall n \in \mathbb{N}, \quad b_{n+m} = \bigoplus_{i \in C} b_{n+i}$$

Indication OCaml : Nous utilisons le type `bool array` pour représenter le germe et le type `int list` pour représenter la connexion d'un LFSR. La longueur s'obtient avec `Array.length`.

□ 27 – Écrire une fonction OCaml `lfsr (g:bool array) (c:int list) : bool stream` dont la valeur de retour est le flot de données constitué par le LFSR de germe g et de connexion c .

Indication OCaml : Nous représentons les formules de logique à l'aide du type

```
50.   type formula = | Var of int
51.                   | Not of formula
52.                   | And of formula * formula
53.                   | Or  of formula * formula
```

Pour toute formule de logique ϕ définie à partir des variables booléennes $(x_j)_{1 \leq j \leq k}$ et pour tout k -upplet de valeurs de vérité $b = (b^1, \dots, b^k) \in \mathbb{B}^k$, nous notons $\phi(b^1, \dots, b^k)$ l'évaluation de la formule ϕ en b .

□ 28 – Écrire une fonction OCaml `logical_map (phi : formula) (w : bool stream array) : bool stream` ainsi spécifiée :

Précondition : La formule ϕ ne dépend que des variables $x_1, \dots, x_k$. Le tableau w est de longueur k . Nous notons $w = \left[\left(b_n^{(1)} \right)_{n \in \mathbb{N}}, \dots, \left(b_n^{(k)} \right)_{n \in \mathbb{N}} \right]$ son contenu.

Valeur de retour : Flot des évaluations booléennes $\left(\phi \left(b_n^{(1)}, \dots, b_n^{(k)} \right) \right)_{n \in \mathbb{N}}$.

Dans la question 29, deux personnages, Alice et Bob, se sont accordés, lors de leur tout premier tête-à-tête et à l'abri de toute écoute par un tiers, au sujet de k germes et de k connexions de LFSR ainsi qu'au sujet d'une formule de logique $\psi_0(x_1, \dots, x_k)$. Nous notons $(b_n^{(1)})_{n \in \mathbb{N}}, \dots, (b_n^{(k)})_{n \in \mathbb{N}}$ les k flots de données booléens associés aux LFRS. À présent, Alice souhaite communiquer à Bob la suite de bits $c = (c_n)_{n \in \mathbb{N}}$ via un canal public. Elle transmet à Bob la suite des évaluations $(X(\psi_0(b_n^{(1)}, \dots, b_n^{(k)}), c_n))_{n \in \mathbb{N}}$.

□ 29 – Dire si Bob peut toujours parvenir à reconstituer le flot $c = (c_n)_{n \in \mathbb{N}}$ et, le cas échéant, expliquer comment.

4. Énumération d'objets combinatoires

4.1. Flot des mots de Dyck

Soit Σ l'alphabet binaire $\Sigma = \{a, b\}$. Nous nous intéressons à la constante OCaml dyck définie par le code suivant

```
54. let rec dyck =
55.   let f (x,y) = "a" ^ x ^ "b" ^ y in
56.   Cons("", fun () -> map f (product dyck dyck))
```

où la fonction product a été introduite à la question 14. Nous notons $\mathcal{D} \subseteq \Sigma^*$ le langage constitué des mots appartenant au flot de données dyck.

□ 30 – Exhiber une grammaire non contextuelle $\mathcal{G}$ telle que le langage $L(\mathcal{G})$ engendré par la grammaire $\mathcal{G}$ coïncide avec le langage $\mathcal{D}$.

□ 31 – Énoncer une propriété de la grammaire $\mathcal{G}$ de la question 30 qui permet d'établir que le flot de chaînes de caractères dyck n'énumère jamais deux fois le même mot. La démontrer.

4.2. Flot des permutations d'un tableau

Nous partageons une variante d'un algorithme dû à B. R. Heap sous la forme du pseudo-code suivant. Dans ce pseudo-code, les positions de tableau sont numérotées à partir de l'indice 0.

Seconde épreuve d'informatique MPI 2024

Algorithme 1 : Algorithme de B. R. Heap**Entrées** : Entier k , tableau t de longueur $\geq k$.**Effet** : Effectue des échanges en place dans le tableau t et des affichages de t .**Sortie** : Aucune valeur de retour.

```

1 si  $k = 1$  alors
2   | Afficher le tableau  $t$ .
3 sinon
4   | pour  $i = 0$  à  $k - 1$  (inclus) faire
5     | Exécuter  $\text{Heap}((k - 1), t)$ .
6     | si  $k$  est pair alors
7       | Échanger  $t[i]$  et  $t[k - 1]$ 
8     | sinon
9       | Échanger  $t[0]$  et  $t[k - 1]$ 

```

□ 32 – Soient k un entier et t un tableau de longueur supérieure à k . Dénombrer le nombre d'appels $A(k)$ à la fonction « afficher » lorsque l'on exécute $\text{Heap}(k, t)$.

□ 33 – Soient k un entier et t un tableau de longueur supérieure à k . Démontrer, pour tout entier naturel non nul k , la proposition $\mathcal{P}(k)$ suivante :

Après l'exécution de $\text{Heap}(k, t)$, si k est un entier impair, alors le tableau t est inchangé ; en revanche, si k est un entier pair, alors les k premiers coefficients du tableau t ont subi une permutation circulaire vers la droite : autrement dit, le nouvel état du tableau est

$$(t[k - 1] \ t[0] \ t[1] \ \dots \ t[k - 3] \ t[k - 2] \ t[k] \ \dots \ t[n - 1])$$

□ 34 – Soient k un entier et t un tableau de longueur supérieure à k . Démontrer que l'exécution de $\text{Heap}(k, t)$ affiche, pour toute permutation des k premiers alvéoles du tableau t , le tableau dans lequel les k premiers alvéoles ont été permutés et dans lequel les derniers alvéoles ont été laissés inchangés.

□ 35 – Écrire une fonction OCaml `pivot (k:int) (i:int) (t:'a array) : unit` dont l'effet est d'appliquer au tableau t les opérations des lignes 6 à 9 de l'algorithme 1.

□ 36 – Écrire une fonction OCaml `permstream_of_array (t:'a array) : 'a array stream` dont la valeur de retour est un flot de données fini constitué de toutes les permutations du tableau t énumérées dans l'ordre d'affichage de l'algorithme de Heap et qui utilise efficacement la mémoire.

A. Annexe : rappels de programmation

(D'après <https://v2.ocaml.org/api/index.html>)

Opérations générales : Le module Stdlib, chargé par défaut, offre les fonctions suivantes :

- `max : 'a -> 'a -> 'a`
Return the greater of the two arguments. The result is unspecified if one of the arguments contains the float value nan.
- `float_of_int : int -> float`
Convert an integer to floating-point.
- `(lsl) : int -> int -> int`
`n lsl m` shifts `n` to the left by `m` bits.

Opérations sur les tableaux : Le module Array offre les fonctions suivantes :

- `make : int -> 'a -> 'a array`
`make n x` returns a fresh array of length `n`, initialized with `x`.
- `init : int -> (int -> 'a) -> 'a array`
`init n f` returns a fresh array of length `n`, with element number `i` initialized to the result of `f(i)`.
- `exists : ('a -> bool) -> 'a array -> bool`
`exists f [|a1; ...; an|]` checks if at least one element of the array satisfies the predicate. That is, it returns `(f a1) || (f a2) || ... || (f an)`.

Opérations sur les tables de hachage : Le module Hashtbl offre les fonctions suivantes :

- `create : int -> ('a, 'b) t`
`Hashtbl.create n` creates a new, empty hash table, with initial size `n`.
- `add : ('a, 'b) t -> 'a -> 'b -> unit`
`Hashtbl.add t k d` adds a binding of key `k` to data `d` in table `t`.
- `length : ('a, 'b) t -> int`
`Hashtbl.length tt` returns the number of bindings in `t`. It takes constant time. Multiple bindings are counted once each.

Opérations sur les fils d'exécution : Le module Thread offre les fonctions suivantes :

- `create : ('a -> 'b) -> 'a -> t`
`Thread.create f x` creates a new thread of control, in which the function application `f(x)` is executed concurrently with the other threads of the domain. The application of `Thread.create` returns the handle of the newly created thread.
- `delay : float -> unit`
`delay d` suspends the execution of the calling thread for `d` seconds. The other program threads continue to run during this time.

Opérations sur les verrous : Le module Mutex offre les fonctions suivantes :

- `create : unit -> t`
Return a new mutex.
- `lock : t -> unit`
Lock the given mutex. Only one thread can have the mutex locked at any time. A thread that attempts to lock a mutex already locked by another thread will suspend until the other thread unlocks the mutex.
- `unlock : t -> unit`
Unlock the given mutex. Other threads suspended trying to lock the mutex will restart. The mutex must have been previously locked by the thread that calls `Mutex.unlock`.

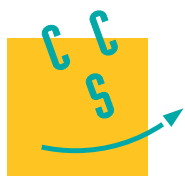
B. Annexe : règles de la déduction naturelle

Dans les tableaux suivants, la lettre Δ désigne un ensemble de formules de logique ; les lettres A , B et C désignent des formules de logique.

Axiome
$\frac{}{\Delta, A \vdash A} \text{ (ax)}$

	Introduction	Élimination
$\top$	$\frac{}{\Delta \vdash \top} \text{ (}\top\text{i)}$	
$\perp$		$\frac{\Delta \vdash \perp}{\Delta \vdash A} \text{ (}\perp\text{e)}$
$\wedge$	$\frac{\Delta \vdash A \quad \Delta \vdash B}{\Delta \vdash A \wedge B} \text{ (}\wedge\text{i)}$	$\frac{\Delta \vdash A \wedge B}{\Delta \vdash A} \text{ (}\wedge\text{e)}$ $\frac{\Delta \vdash A \wedge B}{\Delta \vdash B} \text{ (}\wedge\text{e)}$
$\vee$	$\frac{\Delta \vdash A}{\Delta \vdash A \vee B} \text{ (}\vee\text{i)}$ $\frac{\Delta \vdash B}{\Delta \vdash A \vee B} \text{ (}\vee\text{i)}$	$\frac{\Delta \vdash A \vee B \quad \Delta, A \vdash C \quad \Delta, B \vdash C}{\Delta \vdash C} \text{ (}\vee\text{e)}$
$\neg$	$\frac{\Delta, A \vdash \perp}{\Delta \vdash \neg A} \text{ (}\neg\text{i)}$	$\frac{\Delta \vdash A \quad \Delta \vdash \neg A}{\Delta \vdash \perp} \text{ (}\neg\text{e)}$

FIN DE L'ÉPREUVE



CONCOURS CENTRALE•SUPÉLEC

Informatique

MPI

2024

4 heures

Calculatrice autorisée

*Le jeu de go***I Introduction au jeu de go****I.A – Présentation du sujet**

Ce sujet s'intéresse au jeu de go et à différentes facettes de la construction d'un programme jouant au jeu. La partie 1 présente des règles simplifiées du jeu de go qui seront utilisées tout au long de ce sujet, et qui seront particulièrement utiles aux parties 2 et 5. Plusieurs fonctions permettant de mener une partie de go seront à implémenter en langage **C** dans la partie 2. Les parties suivantes se penchent davantage sur la dimension stratégique du jeu. La partie 3 s'intéresse à l'estimation d'un coup pertinent à partir d'une base de données, en utilisant l'algorithme des **k plus proches voisins**, puis en mettant en place une **table de hachage** contenant des coups localement similaires, en langage **C**. Cette recherche de coup se poursuit en partie 4 à travers une **Recherche Arborescente de Monte-Carlo** utilisant des **fils d'exécutions concurrents**, en **OCaml**. Enfin, la partie 5 conclut ce sujet en montrant que décider si une position est gagnante ou non dans un jeu de go généralisé est un problème **NP-dur**.

I.B – Présentation et règles du jeu de go

Le jeu de *go* est un jeu de stratégie opposant deux adversaires, qui placent à tour de rôle des pierres sur un plateau quadrillé appelé *goban*. Ce plateau est modélisé par une grille de d lignes par d colonnes, avec d un entier appelé la *dimension* du goban. Dans ce sujet nous nous intéresserons principalement à des gobans standards de dimension $d = 19$. Chacune des d^2 intersections du goban est identifiée par sa *position*, qui est un couple (*ligne, colonne*) représentant son emplacement sur le plateau.

Les deux joueurs sont nommés d'après la couleur des pierres qu'ils placent sur le goban : Noir, qui joue en premier et place des pierres noires, et Blanc qui place des pierres blanches. À chaque tour, un joueur place une pierre de sa couleur sur une intersection *libre* du goban, c'est-à-dire sans pierre. Un coup correspond donc simplement à l'ajout d'une pierre sur une intersection libre, les joueurs ne peuvent pas déplacer une pierre déjà posée.

Nous appellerons *configuration* du goban, ou simplement *goban*, l'état du plateau à un moment de la partie, c'est-à-dire la donnée des positions et couleurs de toutes les pierres placées sur le goban. La figure 1 illustre différentes configurations progressives au cours d'une partie possible de go.

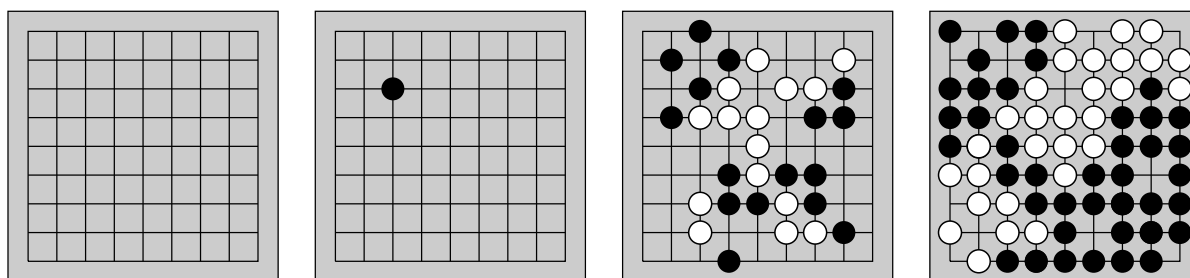


Figure 1 Un goban vide de dimension 9, un premier coup de Noir, une configuration du jeu à un moment intermédiaire, et une configuration à la fin de la partie dans laquelle chaque intersection est contrôlée par un des deux joueurs.

Chaque intersection possède plusieurs *intersections voisines* qui sont les intersections orthogonalement adjacentes à cette dernière (et non diagonalement adjacentes). Une intersection a donc au plus 4 intersections voisines, celles sur les extrémités du goban pouvant en avoir moins. Par extension, les intersections voisines d'une pierre sont celles de l'intersection où elle est placée.

Deux pierres d'une même couleur sont dites *connectées* si elles se trouvent sur des intersections voisines. Nous appellerons *groupe* de pierres une composante connexe pour cette relation de connexion. Chaque groupe possède un nombre de *libertés* qui est défini comme le nombre d'intersections libres qui sont voisines d'une pierre du groupe. Une intersection voisine de plusieurs pierres du groupe n'est comptée qu'une seule fois.

Un groupe de pierres auquel on vient de supprimer la dernière liberté est *capturé*, et les pierres de ce groupe sont alors immédiatement retirées du plateau. Ces notions de libertés et de capture sont illustrées dans la figure 2.

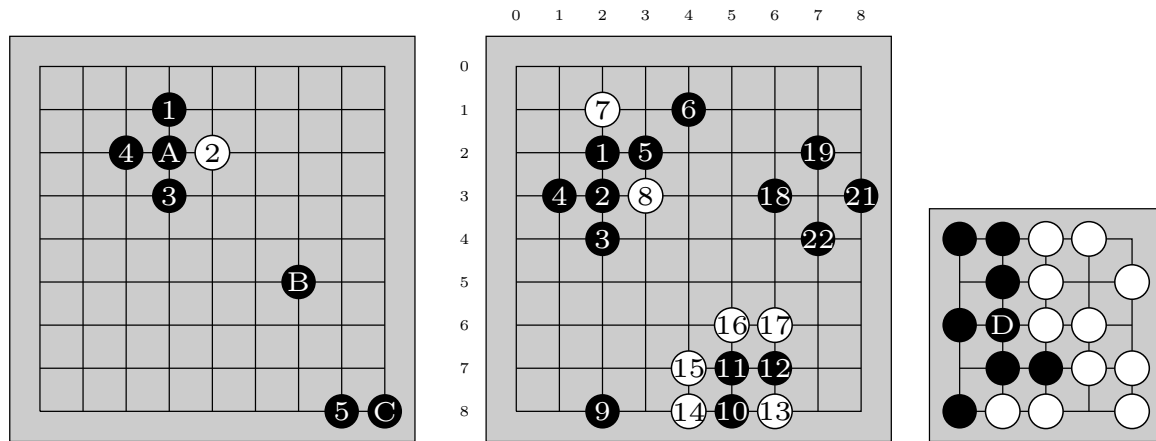


Figure 2 À gauche : la pierre A a une pierre sur chacune de ses 4 *intersections voisines*. La pierre B n'a pas de pierres sur ses 4 intersections voisines. La pierre C a une pierre sur une de ses deux intersections voisines. Au milieu : les pierres 1, 2, 3, 4 et 5 forment un *groupe* de pierres noires, la pierre 6 ne fait pas partie de ce groupe, ni les pierres blanches 7 et 8. Ce groupe a 7 *libertés*. La pierre 9 a 3 libertés. Le groupe de pierre 10, 11, 12 a une seule liberté, si Blanc joue en (7, 7) alors il *capture* immédiatement ce groupe, et les pierres 10, 11, 12 sont retirées du plateau. L'intersection (3, 7) est *contrôlée* par noir. À droite : un exemple de goban de dimension 5 en fin de partie.

Cette règle de capture permet d'aboutir à des configurations avec moins de pierres sur le goban. Afin de s'assurer que la partie termine, une règle supplémentaire interdit de jouer un coup qui fasse revenir à une configuration du goban déjà vue durant la partie.

Lorsqu'une pierre est placée, on commence par regarder si elle permet de capturer des pierres adverses, et dans ce cas on retire immédiatement ces pierres capturées du goban. On regarde ensuite si le groupe de la pierre placée possède des libertés. Si elle ne possède aucune liberté, le coup est interdit. Cependant, si elle a permis de capturer des pierres, son groupe a nécessairement au moins une liberté. Il est donc possible de jouer sur une position sans libertés, mais uniquement dans le cas où cela permet de capturer des pierres à l'adversaire.

Une intersection est dite *contrôlée* par un joueur si une de ses pierres est placée dessus, ou si l'intersection est vide mais que toutes les intersections voisines contiennent des pierres de sa couleur.

La partie s'arrête lorsque toutes les intersections sont contrôlées par un des deux joueurs. Une telle configuration du goban est alors dite *finale*. Le *score* de chaque joueur est défini comme le nombre d'intersections contrôlées par ce dernier. Le joueur avec le score le plus élevé remporte la partie.

I.C – Généralités sur le jeu de go

Q 1. Compter le nombre de libertés du groupe auquel appartient la pierre D dans le goban de la figure 2 Droite. Donner le score de Blanc, celui de Noir et le joueur gagnant de cette partie.

Q 2. Montrer qu'il y a toujours un gagnant dans une configuration finale sur un goban standard de dimension 19.

II Implémentation du jeu de go

L'objectif de cette première partie est d'implémenter diverses fonctions permettant de manipuler le goban pour mener une partie de go. Ces fonctions sont à écrire en langage C. Dans tout le sujet, il sera supposé que les entêtes suivantes ont été incluses :

```
#include <assert.h>
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
```

Afin de représenter et manipuler un goban, nous utiliserons la structure de données suivante pour représenter une configuration.

```
1 struct goban_s {
2     int d;
3     int* m;
4 };
5 typedef struct goban_s goban;
```

Le champ `d` est ici la dimension du goban, et `m` est un pointeur vers un tableau unidimensionnel d'entiers de taille $d \times d$ représentant les pierres placées. Une intersection libre est représentée par l'entier 0, une pierre noire par l'entier 1 et une pierre blanche par l'entier 2. Initialement, le tableau est donc rempli de 0.

Q 3. Écrire une fonction `goban_initialisation(int d)` prenant en argument un entier `d` et qui renvoie un goban de dimension `d` avec un tableau `m` alloué dynamiquement, représentant un goban vide.

La position d'une intersection du goban sera représentée par une structure de couple d'entiers, contenant la ligne i et la colonne j de la position, avec $0 \leq i, j < d$.

```
1 struct couple_int {
2     int i;
3     int j;
4 };
5 typedef struct couple_int position;
```

Une position pourra être initialisée avec la fonction suivante :

```
1 position pos(int i, int j) {
2     position p = {.i = i, .j = j};
3     return p;
4 }
```

Un goban pourra être lu et modifié grâce aux deux fonctions ci-dessous, en supposant que la position correspond à une intersection valide du goban.

```
1 int lire(goban g, position p) {
2     return g.m[p.i * g.d + p.j];
3 }
4
5 void ecrire(goban g, position p, int couleur) {
6     g.m[p.i * g.d + p.j] = couleur;
7 }
```

Les questions suivantes vont permettre d'isoler un groupe de pierres. Pour cela, nous représentons les intersections voisines d'une intersection donnée grâce à la structure suivante :

```
1 struct voisins_s {
2     int nb;
3     position t[4];
4 };
5 typedef struct voisins_s voisins;
```

Le nombre de voisins de l'intersection est donné par le champ `nb`, et `t` est un tableau de positions de taille 4, dont les `nb` premières cases contiennent les positions des intersections voisines de l'intersection. La fonction incomplète suivante permet ce calcul des voisins :

```
1 voisins pos_voisins(goban g, position p) {
2     int i = p.i;
3     int j = p.j;
4     voisins v;
5     ...
6     if (...) {v.t[k] = pos(i - 1, j); k = k + 1;}
7     if (...) {v.t[k] = pos(i + 1, j); k = k + 1;}
8     if (...) {v.t[k] = pos(i, j - 1); k = k + 1;}
9     if (...) {v.t[k] = pos(i, j + 1); k = k + 1;}
10    v.nb = k;
11    return v;
12 }
```

Q 4. Compléter cette fonction `pos_voisins` en donnant l'instruction à écrire en ligne 5 ainsi que les conditions à écrire pour chacune des lignes 6, 7, 8 et 9 pour qu'elle renvoie, dans une structure `voisins` les positions des intersections voisines de la position donnée en entrée.

Q 5. Écrire une fonction `goban groupe(goban g, position p)` qui renvoie un nouveau goban dans lequel seules les pierres du groupe de la pierre en position `p` sont présentes. Expliquer par une phrase, avant le code de votre fonction, l'approche choisie. Il pourra être utile de commencer par implémenter une fonction auxiliaire récursive, dont la spécification sera clairement explicitée. Donner la complexité obtenue.

Une première tentative pour calculer le nombre de libertés du groupe de la pierre en position `p` a abouti sur la fonction suivante :

```

1  int liberte(goban g, position p) {
2      goban g1 = groupe(g, p);
3      int cpt = 0;
4      int couleur = lire(g, p);
5      for (int i = 0; i < g.d; i = i + 1) {
6          for (int j = 0; j < g.d; j = j + 1) {
7              position p2 = pos(i, j);
8              if (lire(g1, p2) == couleur) {
9                  voisins v = pos_voisins(g1, p2);
10                 for (int k = 0; k < v.nb; k = k + 1) {
11                     if (lire(g, v.t[k]) == 0) {
12                         cpt = cpt + 1;
13                     }
14                 }
15             }
16         }
17     }
18     return cpt;
19 }
```

Q 6. Proposer un jeu de quatre tests permettant de tester cette fonction `liberte`. Deux tests seront passés par la fonction, et deux montreront qu'elle n'est pas correcte. Les gobans en entrées pourront être représentés par des schémas. On prendra soin de justifier l'intérêt des tests proposés et d'indiquer les valeurs attendues.

Q 7. Proposer des modifications à apporter au code de la fonction `liberte` pour qu'elle renvoie bien le nombre de libertés, et qu'elle libère les structures créées pendant son appel.

Q 8. Écrire une fonction `void retire_groupe(goban g, position p)` prenant en argument un goban, une position et qui retire toutes les pierres du groupe de la pierre à cette position. Une complexité linéaire en la taille du groupe à retirer est attendue.

Q 9. Écrire une fonction `void joue(goban g, position p, int couleur)` prenant en argument un goban, une position et une couleur et qui modifie le goban en ajoutant une pierre de la couleur donnée à la position donnée. On prendra en compte les possibles pierres à retirer et on utilisera des assertions pour vérifier que le coup est valable. On ne vérifiera cependant pas si la configuration du goban a déjà été rencontrée au cours de la partie. Donner la complexité de cette fonction.

III Prédiction et évaluation d'un prochain coup

Dans cette partie, nous envisageons de réaliser un programme qui renvoie ou évalue un prochain coup pertinent à jouer depuis une configuration donnée du goban, afin de simuler un adversaire. La dimension stratégique du jeu de go est réputée pour être complexe, c'est pourquoi une première approche simple peut être de s'inspirer de coups joués par des joueurs humains professionnels.

Pour cela nous nous baserons sur une base de données de 160 000 parties de maîtres, jouées sur des gobans standards de dimension 19. Nous avons commencé par décomposer ces parties en 29.4 millions de triplets (configuration du goban, joueur dont c'est le tour, coup suivant choisi), que nous appellerons *observations*, et qui peuvent être redondantes.

Q 10. À partir des informations sur la base de données, donner une estimation de la profondeur du jeu de go, c'est-à-dire du nombre de coups moyen d'une partie de go. En utilisant la description du jeu ainsi que la profondeur, donner une estimation de la largeur du jeu de go, c'est-à-dire le nombre moyen de coups valables lors d'une partie.

Est-il envisageable de mettre en place un algorithme *min-max* pour trouver une stratégie gagnante à ce jeu ? Justifier.

III.A – *K plus proches voisins*

Afin de jouer en un temps raisonnable, dans cette sous-partie nous allons implémenter l'algorithme des *k plus proches voisins* pour prédire un prochain coup pertinent de manière supervisée. Pour cela, nous allons confronter la configuration d'entrée sur laquelle nous désirons trouver le prochain coup à toutes les configurations compatibles de la base de données, c'est-à-dire les configurations dont le coup suivant qui a été joué peut effectivement être réalisé. Le calcul d'une distance entre deux configurations du goban ne sera pas étudié ici, nous supposons que nous disposons d'un tableau non trié dans lequel ces n distances ont été préalablement calculées. Dans cette sous-partie, pour simplifier les algorithmes et les programmes nous nous intéresserons seulement à déterminer les k plus petites valeurs de ce tableau de distances non trié, sans se préoccuper de garder en mémoire les configurations auxquelles correspondent ces distances.

Q 11. Proposer une fonction `double* k_plus_petits(double* distances, int n, int k)` qui prend en entrée un tableau de n distances et un entier k et renvoie un tableau de k distances correspondant aux k plus petites valeurs du tableau en entrée. Il est attendu ici une complexité en $O(nk)$ dans le pire cas. La fonction pourra, si besoin, modifier le tableau donné en entrée. Justifier la complexité obtenue et prouver la correction de cet algorithme à l'aide d'un invariant de boucle.

Q 12. Proposer une structure de données abstraite qui permette d'améliorer la complexité asymptotique de la fonction `k_plus_petits`. Rappeler en une phrase une façon d'implémenter cette structure de données ainsi que la complexité obtenue.

Dans le cadre d'un adversaire jouant en temps réel, il est préférable d'avoir une borne précise sur le nombre de comparaisons qui seront réalisées. Nous supposons désormais que n est une puissance de 2 avec $n \geq 2$. Nous allons construire un algorithme permettant de trouver les indices des deux plus petits éléments d'un tableau de n distances. Il sera possible de supposer que les distances de ce tableau sont toutes différentes.

Nous appellerons comparatif un algorithme prenant en entrée un tableau de distances pour lequel la seule opération autorisée sur les distances du tableau est la comparaison entre deux distances du tableau.

Q 13. Montrer que tout algorithme comparatif renvoyant l'indice du plus petit élément d'un tableau devra réaliser la comparaison du plus petit élément et du second plus petit élément du tableau.

Q 14. Proposer un algorithme qui renvoie les indices des deux plus petits éléments d'un tableau de n distances en réalisant au plus $n + \log_2(n) - 2$ comparaisons. On pourra s'inspirer d'une approche de type diviser pour régner et commencer par comparer chaque élément d'indice $2i$ avec l'élément d'indice $2i + 1$, pour $0 \leq i < n/2$. Il est ensuite possible d'adapter cette approche à la sélection des k plus petits éléments.

III.B – *Hachage de Zobrist*

Le but de cette sous-partie est de présenter une autre approche utilisant la base de données des parties de maîtres, cette fois pour apprendre une fonction d'évaluation des prochains coups qui s'intéresse uniquement à l'état local du goban autour d'une position envisagée. Nous chercherons, dans la base de données, si des configurations localement identiques ont mené à un coup à cette position. Cette recherche sera implémentée de manière efficace dans cette partie grâce à un prétraitement de la base de données utilisant des tables de hachage et une fonction de hachage adaptée, nommée *hachage de Zobrist*.

Un état local sera modélisé par différents *zooms* autour d'une position, qui considèrent la disposition du plateau autour d'une intersection en ignorant le reste du goban, appelée *motif*. Nous noterons $Z_i(g, (x_0, y_0))$ le motif obtenu à partir du zoom i sur la position (x_0, y_0) du goban g comme étant les couleurs des pierres aux positions (x, y) telles que $|x - x_0| + |y - y_0| \leq i$, et avec $(x - x_0, y - y_0) \neq (0, 0)$. Par exemple, $Z_1(g, p)$ correspond à la configuration des intersections voisines de l'intersection de position p . Nous dénoterons par $(x - x_0, y - y_0)$ la position du goban (x, y) dans ce motif $Z_i(g, (x_0, y_0))$. Pour avoir toujours le même nombre d'éléments dans les motifs, une couleur supplémentaire 3 sera assignée aux positions qui dépassent les bords du goban. Les intersections libres seront représentées par 0, celles avec une pierre noire par 1 et celles avec une pierre blanche par 2.

Le but d'un zoom sur une position est d'évaluer un coup potentiel à cette position. Étant donné un motif centré sur une intersection libre sur laquelle il est possible de jouer, nous allons chercher dans la base de données si des configurations possédant ce même motif ont mené à un coup à cette position. Nous considérerons toujours que le centre du motif est une intersection vide.

Q 15. Donner une borne supérieure sur le nombre de motifs distincts pour un zoom i . Justifier qu'il est raisonnable de supposer que tout motif de zoom 1 a été rencontré au moins une fois dans la base de données. On pourra faire cette supposition dans le reste du sujet.

Les fonctions de cette partie sont à écrire en langage C, les configurations seront manipulées avec les mêmes structures que celles de la partie 2, et les fonctions qui y sont définies peuvent être librement réutilisées.

Pour savoir combien de fois un motif a conduit à un coup en son centre, nous allons construire et remplir une table de hachage, pour chaque niveau de zoom. La fonction de hachage associée assignera à chaque motif un entier non signé de 64 bits, de type `uint64_t`. Cette fonction de hachage particulière se base sur une table d'entiers `uint64_t` fixés, ayant autant de colonnes que de positions dans le motif, et quatre lignes pour les quatre

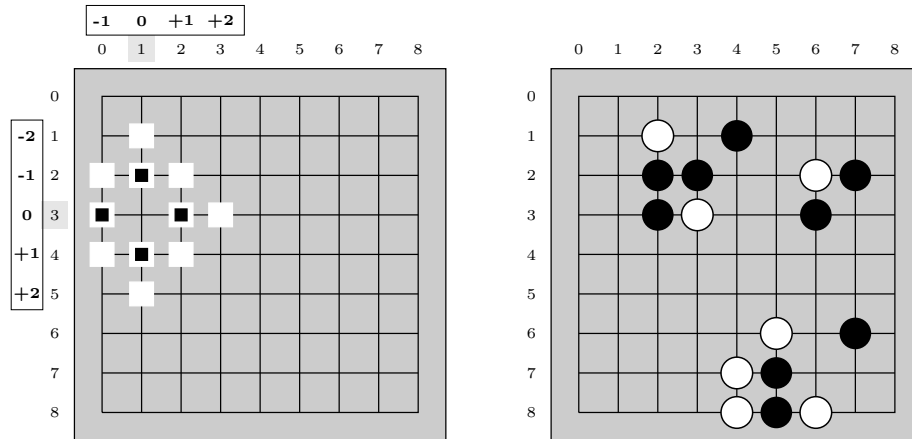


Figure 3 Gauche : l'étendue du zoom 1 (les 4 intersections avec un carré noir), et du zoom 2 (les 11 intersections avec des carrés blancs, et une intersection extérieure à la grille non représentée) sur la position (3,1). La case (2,1) est en position $(-1,0)$ dans les zooms sur la position (3,1).
Droite : un exemple de goban. Les motifs correspondant au zoom 1 en position (2,4) et au zoom 1 en position (3,7) sont identiques, ce motif est ici observé deux fois dans cette configuration. Si Noir choisit de jouer en (2,4) on dira alors que ce motif a été choisi une fois.

couleurs possibles (libre, noir, blanc et hors du goban). Afin de manipuler à la main un nombre raisonnable de bits, sur la table donnée en exemple en figure 4, nous utilisons des entiers codés sur 8 bits et non sur 64 bits. La valeur de hachage est alors calculée comme le *ou exclusif* (XOR) bit à bit, noté $\oplus$, des entiers des cases de toutes les combinaisons position-couleur se trouvant dans le motif. La table de vérité du XOR pour un bit est rappelée en figure 4.

Formellement, en notant A la fonction qui associe à un couple de couleur et de position dans le motif l'entier fixé associé dans la table, nous définissons la fonction de hachage d'un motif h par :

$$h(Z_i(g, (x_0, y_0))) = \bigoplus_{\substack{(x_u, y_u) \in \mathbb{Z}^2 \setminus \{(0,0)\} \\ \text{tels que } |x_u| + |y_u| \leq i}} A(g(x_0 + x_u, y_0 + y_u), (x_u, y_u))$$

a	b	a XOR b
0	0	0
1	0	1
0	1	1
1	1	0

Couleur et position	(0, +1)	(+1, 0)	(0, -1)	(-1, 0)
0 (vide)	00001111	11100111	00011000	01111011
1 (noir)	11110110	11111110	01101001	10011111
2 (blanc)	10100001	10001101	00110111	00110100
3 (au delà du bord)	10100000	01101010	10111011	11000010

Figure 4 Table de vérité du XOR, et une table d'entiers fixés sur 8 bits associés à des couples couleur et position dans le motif. Par exemple une pierre noire en position (+1,0) dans un motif est associée à 11111110.

Q 16. Dans le goban de la figure 3 Droite, et avec le tableau de valeurs sur 8 bits donné dans la figure 4, calculer la valeur de hachage du motif en position (7,6) pour le zoom 1. Expliquer comment il est possible de mettre à jour cette valeur en calculant seulement 2 XOR supplémentaires lorsque l'on joue une pierre blanche en (6,6).

Nous disposons désormais d'une fonction `uint64_t hachage(position p, goban g, int zoom)` qui calcule la valeur de hachage $h(Z_i(g, p))$ étant donnée une position p , un goban g et un zoom $zoom$.

De plus, en connaissant la valeur de hachage h_1 du motif $Z_i(g_1, p)$, nous disposons d'une fonction `uint64_t maj_hachage(position p, goban g1, goban g2, int zoom, uint64_t h1)` qui permet de calculer efficacement, en terme de nombre de XOR effectués, la valeur $h(Z_i(g_2, p))$ en mettant à jour $h(Z_i(g_1, p))$ à partir des différences entre les gobans g_1 et g_2 .

Nous nous intéressons maintenant uniquement aux choix des coups de Noir. Il nous faut dénombrer les motifs observés et les motifs choisis dans la base de données de parties. Pour un niveau de zoom fixé, nous stockerons ces valeurs dans une table de hachage, associant un motif à son nombre d'*observations* et de *choix*.

Pour un niveau de zoom fixé, dans une configuration du goban où c'est au tour de Noir de jouer, nous *observons* un motif par position libre de cette configuration. Noir va alors jouer dans une de ces positions libres, et on dit que le motif associé à la position libre où Noir joue est *choisi* (voir la figure 3).

Pour chaque niveau de zoom, les valeurs stockées dans sa table de hachage associée seront représentées par la structure suivante :

```

1  struct observation_s {
2      int n; //nombre d'observations du motif au total
3      int v; //nombre de fois ou ce motif a ete choisi
4  };
5  typedef struct observation_s observation;
```

Le champ `n` correspond au nombre d'observations du motif dans des gobans où c'était à Noir de jouer. Le champ `v` correspond au nombre de fois où le motif était observé au moment où c'était à Noir de jouer et où le coup de Noir a été de placer une pierre au centre de ce motif.

Nous définissons alors une structure de table de hachage composée d'un tableau d'observations, et d'un entier `nt` correspondant à sa taille.

```

1  struct table_de_hachage_s {
2      int nt;
3      observation* t;
4  };
5  typedef struct table_de_hachage_s t_hachage;
```

La table de hachage est initialisée avec $n = 0$ et $v = 0$, pour chaque case. Pour accéder à la valeur de la table associée à une clé, nous prendrons la clé modulo `nt`. Ce sujet ne s'intéresse pas aux collisions, et nous pourrions supposer qu'il n'y en a aucune.

Pour remplir la table de hachage, nous allons simuler la partie de maître configuration après configuration, en utilisant la fonction `maj_hachage` pour mettre à jour les valeurs de hachages. Au départ, nous partons donc de la configuration initiale du goban, et nous allons garder en mémoire les valeurs de hachage pour chaque position du goban, et les faire évoluer au fur et à mesure de la partie.

Q 17. Écrire une fonction `uint64_t* init_t_hash(int d, int zoom)` qui prend en entrée un entier `d` correspondant à la dimension d'un goban, et un entier `zoom` correspondant au niveau de zoom d'intérêt, et qui renvoie un tableau d'entiers non signés sur 64 bits alloué dynamiquement, de taille $d \times d$ où chaque case correspond à la valeur de hachage initiale sur cette position dans un goban vide.

On considère un goban `gN` où Noir doit jouer, le goban suivant `gB` après le coup de Noir, et le goban après le coup de Blanc, quand Noir doit jouer à nouveau `gNs`. La fonction `maj_coup`, donnée ci-dessous, met à jour dans toutes les positions possibles du goban, si nécessaire, le nombre d'observations des motifs en prenant en compte le goban `gN`, avec la fonction `maj_n`, le nombre de choix des motifs avec la fonction `maj_v`, en s'intéressant au coup choisi par Noir entre `gN` et `gB`. Elle prend notamment en entrée `tableau_hache` qui contient les valeurs de hachage des positions du goban `gN`. La fonction `maj_t_hache`, donnée ci-dessous, met à jour les valeurs de hachage associées aux positions avec la fonction `maj_h`, le tableau en entrée `tableau_hache` contient les valeurs de hachage des positions du goban `gN` et doit être à jour sur le goban `gNs` à l'issue de l'exécution de la fonction.

```

1  void maj_coup(t_hachage table, uint64_t* tableau_hache, goban gN, goban gB, int zoom) {
2      for (int i = 0; i < gN.d; i = i + 1) {
3          for (int j = 0; j < gN.d; j = j + 1) {
4              position p = pos(i, j);
5              maj_n(table, tableau_hache, gN, p);
6              maj_v(table, tableau_hache, gN, gB, p);
7          }
8      }
9  }
10
11 void maj_t_hache(uint64_t* tableau_hache, goban gN, goban gNs, int zoom) {
12     for (int i = 0; i < gN.d; i = i + 1) {
13         for (int j = 0; j < gN.d; j = j + 1) {
14             position p = pos(i, j);
15             maj_h(tableau_hache, gN, gNs, zoom, p);
16         }
17     }
18 }
```

Q 18. Écrire les fonctions


```
void maj_n(t_hachage table, uint64_t* tableau_hache, goban gN, position p),
void maj_v(t_hachage table, uint64_t* tableau_hache, goban gN, goban gB, position p),
void maj_h(uint64_t* tableau_hache, goban gN, goban gNs, int zoom, position p),
```

pour que les fonctions `maj_coup` et `maj_t_hache` mettent correctement à jour la table de hachage et le tableau des valeurs de hachage.

Une partie de go sera représentée par une structure de liste simplement chaînée, chaque coup étant représenté par le goban actuel, le joueur qui doit jouer, et un pointeur vers le reste de la partie. Nous supposons que les parties commencent par le goban vide associé au joueur d'indice 1 (Noir).

```
1 struct partie_s {
2     goban g;
3     int joueur;
4     struct partie_s* suivant;
5 };
6 typedef struct partie_s partie;
```

Q 19. En s'intéressant uniquement aux coups de Noir, écrire une fonction `t_hachage lit_partie(partie* lp, int zoom, t_hachage table)` qui prend en entrée une partie et met à jour pour le zoom donné et dans toutes les positions libres du goban le nombre d'occurrences des motifs, et le nombre de choix des motifs choisis, dans la table de hachage donnée en entrée (qui a déjà été initialisée et mise à jour sur d'autres parties). On pourra supposer que la partie contient au moins 3 coups.

Q 20. Étant donné un tableau des valeurs de hachage initialisé avec le goban initial, montrer qu'au plus $4c$ XOR sont nécessaires pour calculer toutes les valeurs de hachage d'un motif à cette position au cours d'une partie de c coups.

Nous allons représenter toutes les tables de hachages pour les différents niveaux de zoom allant de 1 à $n_z - 1$, inclus, dans une table de tables de hachage, représentée par la structure suivante :

```
1 struct table_table_de_hachage {
2     int nz;
3     t_hachage* t;
4 };
5 typedef struct table_table_de_hachage tt_hachage;
```

Nous construisons cette table de tables de hachages `tt` telle que `tt[i]` contienne la table de hachage associée au zoom i . La première case de ce tableau, `tt[0]`, ne sera ainsi pas utilisé.

Pour évaluer la pertinence d'un coup dans une configuration, nous allons chercher le plus grand zoom pour lequel le motif a été observé au moins une fois dans la base de données. La qualité estimée de ce coup est alors le nombre de fois où ce motif a été choisi sur le nombre de fois où il a été vu.

Q 21. Écrire une fonction booléenne `bool appartient(position p, goban g, t_hachage table, int zoom)` qui renvoie vrai si le motif en position p et avec le zoom `zoom` sur le goban g est présent dans la table de hachage, c'est-à-dire a été observé au moins une fois dans la base de données, et faux sinon.

Q 22. Écrire une fonction `float evaluate_zoom(position p, goban g, t_hachage table, int zoom)` qui renvoie la valeur associée au motif, c'est-à-dire le rapport entre le nombre d'observations et le nombre de choix.

Q 23. Écrire une fonction `float evaluation(position p, goban g, tt_hachage tt)` qui renvoie l'évaluation du coup en position p sur le goban g . Une fonction de complexité logarithmique en le nombre de zooms n_z est attendue. Donner une preuve de la terminaison et de la correction partielle de ce programme, et justifier sa complexité.

IV Recherche arborescente de Monte-Carlo

Dans cette partie nous nous intéressons à la construction d'un programme en OCaml jouant au go, en utilisant l'approche de Recherche Arborescente de Monte-Carlo. C'est notamment sur cette méthode que s'est basé AlphaGo pour vaincre le champion du monde de go en 2017. Nous supposons dans cette section que les parties de go finissent toujours dans une configuration finale, c'est-à-dire complètement contrôlée, sans égalité et que le goban est de dimension standard $d = 19$.

Les fonctions de cette partie sont à écrire en langage OCaml. Une annexe rappelant des éléments de langage OCaml, modules Thread et Mutex, module List, type enregistrement et fonction de conversion de type, est proposée en fin de partie.

Commençons par décrire quatre types pour représenter les joueurs, les gobans et les positions :


```

1  type joueur = Noir | Blanc
2  type intersection = Libre | Pierre_noire | Pierre_blanche
3  type goban = intersection array array
4  type position = int * int

```

Nous supposons que les fonctions suivantes ont été déjà implémentées et peuvent être librement utilisées.

- `liste_coup_prior : goban -> joueur -> (position * float) list` : prend un goban et un joueur et renvoie une liste, pour toutes les positions jouables dans le goban pour le joueur, de couples (`position`, `prior`) où `prior` est un flottant entre 0 et 1 correspondant à une estimation de la probabilité de victoire de Noir lorsque le joueur joue en position `position`.
- `strategie_default : goban -> joueur -> position` : prend un goban et un joueur et renvoie une position où jouer, en suivant une stratégie par défaut définie *a priori*, et qui peut contenir une part d'aléatoire. Cette fonction renvoie l'exception `Pas_de_coup_valide` lorsque le plateau est complètement contrôlé et qu'il n'y a plus de coup valide.
- `gagnant : goban -> joueur` : prend un goban complètement contrôlé et renvoie le joueur gagnant.
- `joue : goban -> position -> joueur -> goban` : prend un goban, une position et un joueur et renvoie le goban obtenu après que le joueur a joué en cette position. Le goban en entrée n'est pas modifié.

Q 24. Écrire une fonction de signature `js : joueur -> joueur` qui prend un joueur et renvoie le joueur suivant.

IV.A – Simulation de partie pour évaluer une configuration

Pour trouver un prochain coup pertinent, il est important de pouvoir évaluer si une configuration du goban est prometteuse ou non, ce qui est une tâche complexe. L'évaluation de Monte-Carlo permet d'estimer cette évaluation de goban, en jouant une partie depuis la configuration actuelle jusqu'au bout, et en choisissant les coups successifs grâce à une stratégie définie *a priori*. Celle-ci peut-être complètement aléatoire, ou basée sur des données d'apprentissage par exemple.

Q 25. Écrire une fonction de signature `sim_default : goban -> joueur -> joueur` qui prend un goban et un joueur, qui utilise la fonction `strategie_default` pour choisir des coups jusqu'à atteindre une configuration finale, et qui renvoie alors le joueur gagnant.

IV.B – Valeur d'action

Dans la recherche arborescente de Monte-Carlo, nous explorons l'arbre de toutes les parties possibles depuis le goban actuel, mais sans stocker explicitement cet arbre tout entier. De multiples explorations sont réalisées partant chacune de la racine de l'arbre et descendant jusqu'à une configuration finale, sans représentation explicite de l'arbre, en suivant des stratégies définies *a priori*. L'idée principale de la recherche arborescente de Monte-Carlo est d'ajouter à cette approche sans mémoire la construction d'un arbre partiel de jeu, dont les nœuds vont correspondre aux configurations souvent explorées, et pour lesquelles il sera possible d'enregistrer différentes informations :

- `n` : le nombre de visites du nœud, entier mutable
- `v` : le nombre de visites du nœud ayant mené à une victoire de Noir, entier mutable
- `prior` : la probabilité *a priori* de victoire de Noir donnée par la fonction `liste_coups_prior`, flottant
- `pos` : la position du coup associé au nœud, position
- `t` : un verrou, de type `t.Mutex`, qui sera utilisé pour mettre en place des parcours et mises à jour concurrentes de l'arbre.

Afin de stocker ces données, les nœuds de l'arbre seront étiquetés par le type enregistrement `etiquette`.

```

1  type etiquette =
2      {mutable n:int; mutable v:int; prior:float; pos:position; t:Mutex.t}

```

Enfin, nous définissons le type suivant pour l'arbre partiel :

```

1  type arbre = N of etiquette * arbre list

```

La première étape de la recherche arborescente de Monte-Carlo est une descente qui consiste à suivre un chemin dans l'arbre partiel qui semble prometteur jusqu'à atteindre une feuille. Pour un nœud a de l'arbre, nous notons n_a , v_a , et $prior_a$ les nombres de visites, de victoires et la prior, c'est-à-dire l'estimation à priori de victoire de Noir, du nœud a . Dans un nœud de l'arbre partiel n'étant pas une feuille, pour savoir quel enfant choisir nous définissons la quantité V_a^N appelée *valeur d'action* du nœud a pour Noir :

$$V_a^N = \begin{cases} \frac{v_a}{n_a} + \frac{\text{prior}_a}{1+n_a} & \text{si } n_a > 0 \\ \text{prior}_a & \text{sinon} \end{cases}$$

Cette valeur est un compromis entre la prior apprise par apprentissage et le taux de victoires obtenu lors des descentes dans l'arbre, l'importance de la prior décroissant avec l'augmentation du nombre de descentes dans l'arbre passant par le nœud. Une valeur d'action analogue peut être définie pour Blanc. En partant de la racine de l'arbre partiel, la feuille à explorer est atteinte en descendant le long d'une branche, en choisissant itérativement comme nœud suivant le fils de valeur d'action maximale pour le joueur qui doit jouer.

Q 26. Écrire une fonction de signature `etiq : arbre -> etiquette` qui prend un arbre et renvoie l'étiquette de cette arbre.

Q 27. Écrire une fonction de signature `valeurActionNoir : arbre -> float` qui prend un arbre et renvoie la valeur d'action pour Noir de cet arbre basée sur son étiquette. Les champs mutables de l'étiquette étant modifiables par d'autres fils d'exécution concurrents, une attention particulière sera donc accordée à l'utilisation du verrou du nœud lors des accès à ces champs.

Q 28. Donner la valeur à maximiser au tour de Blanc en suivant le même principe que pour la valeur d'action de Noir. Nous supposons par la suite que la fonction `valeurActionBlanc : arbre -> float` a été également implémentée.

Q 29. Écrire une fonction de signature `popargmax : 'a list -> ('a -> 'b) -> 'a * 'a list` qui prend en entrée une liste d'éléments de type 'a, 1, et une fonction f qui associe à un type 'a un type 'b tel que les éléments de type 'b soient comparables avec les opérateurs polymorphes de comparaison de OCaml (comme >), et qui renvoie le premier élément de 1 maximum pour f ainsi qu'une liste des autres éléments de 1. Informellement, nous sortons l'argument maximum de 1 pour f pour renvoyer cet argument maximum ainsi que la liste privée de ce dernier.

IV.C – Descente et remontée dans l'arbre

Pour affiner les évaluations des configurations stockées dans l'arbre partiel, la recherche arborescente de Monte-Carlo procède en trois étapes clés :

- une descente dans l'arbre partiel en choisissant à chaque nœud le fils de valeur d'action maximale pour le joueur associé, afin de sélectionner une feuille qui semble prometteuse ;
- une simulation de partie depuis cette feuille comme implémentée en partie IV.A, donnant un joueur vainqueur ;
- une remontée qui met à jour le nombre d'observations et de victoires pour Noir dans chaque nœud rencontré, le long de la branche reliant la feuille explorée à la racine de l'arbre.

Ces étapes pourront être exécutées en parallèles par plusieurs fils d'exécution. Il est donc nécessaire de faire attention à ce que deux fils ne rentrent pas en conflit d'écriture et de lecture pour un champ mutable d'une étiquette de l'arbre, ce qui justifie l'intérêt des verrous associés à chaque nœud.

De plus, pour décourager deux fils d'exécution de suivre la même branche lors de la descente, lorsque l'enfant d'un nœud est sélectionné par un fil d'exécution, ce fil va modifier l'étiquette de cet enfant pour lui ajouter des *défaites virtuelles*, ce qui baissera sa valeur d'action et donc son attractivité du point de vue des autres fils. Nous ajouterons 5 défaites virtuelles à un nœud s'il est sélectionné, en prenant soin d'enlever ces défaites virtuelles lors de la remontée.

Lors de la descente, nous empêcherons ainsi un fil d'exécution de commencer à traiter un nœud, (c'est-à-dire de chercher l'enfant de ce nœud de valeur d'action maximale) si un autre fil d'exécution est déjà en train de traiter ce nœud. Nous attendrons que le fil d'exécution en cours ait ajouté des défaites virtuelles à la branche qu'il a choisi de suivre avant de commencer le traitement.

Q 30. Écrire la fonction `descente_remontee_arbre : goban -> joueur -> arbre -> joueur` qui prend un goban, le joueur dont c'est le tour et un arbre de jeu, et qui réalise une descente, une simulation et une remontée de l'arbre. Cette fonction renverra le joueur gagnant, et lors de la remontée mettra à jour les champs mutables des nœuds rencontrés. Une attention particulière sera portée sur le fait que cette fonction puisse être exécutée par plusieurs fils d'exécution concurrents.

IV.D – Expansion de l'arbre

L'idée de la recherche arborescente de Monte-Carlo est d'alterner entre des étapes de descente, simulation et remontée de l'arbre pour améliorer les évaluations des nœuds, et des étapes d'expansion, où nous ajoutons de nouveaux nœuds à l'arbre partiel de jeu.

Q 31. Écrire une fonction de signature `expansion_feuille : goban -> joueur -> arbre list` qui prend un goban et un joueur, et qui renvoie la liste des enfants du nœud correspondant à ce goban et ce joueur. Il est attendu d'utiliser la fonction `liste_coup_prior` et d'initialiser les étiquettes des enfants avec 0 observation, 0 victoire et un nouveau verrou `Mutex.t`.

Une étape d'expansion consiste à étendre une feuille, qui sera sélectionnée en partant de la racine et en choisissant à chaque nœud le fils ayant été le plus visité.

Q 32. Écrire une fonction de signature `expansion : goban -> joueur -> arbre -> arbre` qui réalise une étape complète d'expansion, c'est-à-dire part de l'arbre et descend jusqu'à une feuille en choisissant à chaque étape le fils le plus visité, étend la feuille rencontrée en ajoutant tous ses fils, puis qui renvoie l'arbre partiel de jeu étendu.

IV.E – Algorithme complet

Pour pouvoir commencer à réaliser l'étape d'expansion, il est nécessaire d'attendre que tous les fils d'exécution réalisant des descentes et remontées aient terminé.

Q 33. Écrire une fonction de signature `attendre : Thread.t list -> unit` permettant d'attendre que tous les fils d'exécutions de la liste donnée en argument aient terminé leur exécution.

Q 34. Écrire une fonction de signature `etape_complete : int -> goban -> joueur -> arbre -> arbre` qui réalise une étape complète de l'algorithme, telle que `etape_complete nt g j a` lance `nt` fils d'exécution qui réalisent chacun une descente, simulation puis remontée de l'arbre `a` où la racine correspond au goban `g` et au joueur `j`, et une fois que tous les fils ont terminé, réalise une étape d'expansion avant de renvoyer l'arbre obtenu après cette expansion.

Q 35. Montrer que cette fonction termine. Il est notamment attendu de montrer qu'il n'y a pas d'interblocages dans les exécutions de la fonction `descente_remontee_arbre` en se référant au code proposé en question 30.

Pour estimer le coup à jouer, il reste alors à réaliser plusieurs étapes complètes, par exemple un nombre `ns` fixé à l'avance. Puis, à la fin de ces étapes, l'algorithme renvoie la position du coup correspondant au nœud le plus visité parmi les fils de la racine.

Le sous-arbre associé à ce coup sera ensuite utilisé comme base pour la suite de la partie, ce qui permet de ne pas repartir de zéro à chaque coup.

Q 36. Écrire une fonction de signature `recherche_arborescente : int -> int -> goban -> joueur -> arbre -> arbre * position` telle que `recherche_arborescente ns nt g j a` effectue `ns` étapes de simulations sur l'arbre `a` en partant du goban `g` avec le joueur `j`, chaque étape de simulations utilisant `nt` fils d'exécutions, avant de renvoyer le coup choisi, ainsi que l'arbre à conserver pour le coup suivant.

Éléments du langage OCaml

Module Thread

- `Thread.t` : Le type des fils d'exécution.
- `Thread.create : ('a -> 'b) -> 'a -> Thread.t` : `Thread.create funct arg` crée un nouveau fil d'exécution dans lequel l'application de `funct arg` est exécuté en même temps que les autres fils d'exécutions. L'application de `Thread.create` renvoie le fil d'exécution créée. Le nouveau fil termine quand l'application `funct arg` termine, soit normalement soit en remontant une exception `Thread.Exit` ou en remontant d'autres exceptions non rattrapées. Dans ce dernier cas, l'exception non rattrapée est affichée sur la sortie d'erreur standard, mais pas propagée au fil d'exécution parent. De la même manière le résultat de l'application `funct arg` est perdu et n'est pas directement accessible au fil d'exécution parent.
- `Thread.join : t -> unit` : `join th` suspend l'exécution du fil qui appelle cette fonction jusqu'à ce que le fil d'exécution `th` ait terminé.

Module Mutex

- `Mutex.t` : Le type des mutex.
- `Mutex.create : unit -> Mutex.t` : Renvoie un nouveau mutex.
- `Mutex.lock : Mutex.t -> unit` : Verrouille le mutex donné. Un seul fil d'exécution peut verrouiller le mutex à un moment donné. Un fil d'exécution tentant de verrouiller un mutex déjà verrouillé attendra jusqu'à ce que l'autre fil d'exécution déverrouille le mutex.
- `Mutex.unlock : Mutex.t -> unit` : Déverrouille le mutex donné. Les autres fils d'exécution en pause tentant de verrouiller le mutex reprennent. Le mutex doit avoir été verrouillé par le fil d'exécution qui appelle `Mutex.unlock`.

Module List

- `List.iter : ('a -> unit) -> 'a list -> unit` : `iter f [a1; ...; an]` applique la fonction `f` tour à tour à `[a1; ...; an]`. Cet application est équivalente à `f a1; f a2; ...; f an`.
- `List.map : ('a -> 'b) -> 'a list -> 'b list` : `map f [a1; ...; an]` applique la fonction `f` à `a1, ..., an`, et construit la liste `[f a1; ...; f an]` avec les résultats renvoyés par `f`.

Type enregistrement

- `type exemple = {mutable a : int; b int}` : permet de définir un type enregistrement.
- `let c = {a = 1; b = 3}` permet de définir une variable du type `exemple`.
- `c.a` permet d'accéder au champ `a` de `c`.
- `c.a <- 2` permet de modifier le champ mutable `a` du type enregistrement `c`.

Fonction de conversion de type

- `float_of_int : int -> float` : convertit un `int` en `float`.
- `int_of_float : float -> int` : convertit un `float` en `int`.

V Complexité du problème de la stratégie gagnante au go

Un problème classique qui survient lorsque l'on s'intéresse à des jeux est de se demander quel joueur possède une stratégie gagnante à partir d'une configuration donnée. Ce problème de décision sera étudié ici sous le nom de go généralisé.

Go généralisé

Entrée : Une configuration d'une partie de go généralisé, c'est-à-dire un goban de dimension d , avec N un entier arbitraire, sur lequel sont placées des pierres, et la donnée du joueur qui doit réaliser le prochain coup.

Sortie : Oui s'il existe une stratégie gagnante pour Blanc à partir de cette configuration, non sinon.

Il est possible de montrer que ce problème est NP-dur à partir d'une chaîne de réductions depuis le problème SAT, dont la dernière étape sera montrée dans cette partie.

Q 37. Rappeler la définition de la classe NP, et expliquer ce que signifie être NP-dur et NP-complet. Donner un exemple de problème NP-complet classique autre que SAT et 3SAT, sans prouver son appartenance à cette classe de complexité.

Dans cette partie nous considérons une variante du problème 3SAT, nommée *Rectilinear Planar 3SAT*, qui est également NP-dur. Tout comme pour 3SAT, les instances de ce problème de décision sont des formules sous forme normale conjonctive avec exactement trois littéraux par clauses, mais désormais chaque clause ne possède que des littéraux positifs, ou que des littéraux négatifs. De plus, la formule en question doit pouvoir se représenter graphiquement de manière planaire rectiligne comme illustré en figure 5, avec toutes les variables alignées horizontalement et les clauses reliées verticalement à leurs variables associées sans aucune intersection. Une telle représentation graphique est également donnée en entrée du problème avec la formule, et n'a donc pas besoin d'être trouvée.

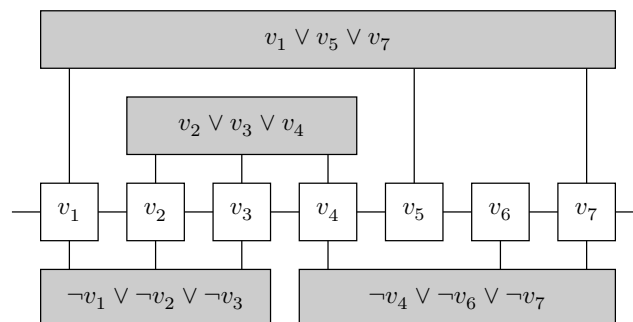


Figure 5 La formule $\varphi = (v_1 \vee v_5 \vee v_7) \wedge (v_2 \vee v_3 \vee v_4) \wedge (\neg v_1 \vee \neg v_2 \vee \neg v_3) \wedge (\neg v_4 \vee \neg v_6 \vee \neg v_7)$ représentée graphiquement sous forme planaire rectiligne. Il est à noter que sur une telle représentation graphique, certaines clauses peuvent en enjambrer d'autres. Ici, la clause $(v_1 \vee v_5 \vee v_7)$ enjambe la clause $(v_2 \vee v_3 \vee v_4)$.

Soit φ et sa représentation graphique planaire rectiligne une instance de Rectilinear Planar 3SAT. Nous allons construire une configuration du jeu de go généralisé telle que Blanc possède une stratégie gagnante si et seulement si φ est satisfiable. Pour cela, nous supposons qu'une grande quantité de pierres blanches se trouvent presque encerclées par des pierres noires, de sorte que l'issue de la partie soit uniquement déterminée par le fait que Blanc parvienne à relier ces pierres blanches à une structure permettant de sécuriser son groupe (figure 6), (d)), (auquel cas il sécurise un nombre de points suffisamment grand pour lui garantir la victoire), ou pas (auquel cas Noir encercle et capture toutes ces pierres blanches, et gagne). En s'affrontant pour cette capture décisive, les deux joueurs vont placer des pierres dans des *gadgets* reliés entre eux et à la réserve de pierres blanches par des tuyaux (figure 6, (b) et (c)).

Q 38. Expliquer pourquoi si Blanc parvient à relier sa réserve à un gadget (d) de la figure 6, alors Noir ne peut plus capturer les pierres blanches de la réserve.

Ce gadget à atteindre pour sécuriser le groupe de pierres sera nommé des *yeux*. En tentant à tout prix de relier sa réserve à des yeux, Blanc va progressivement relier sa réserve à des gadgets, qui sont des dispositions

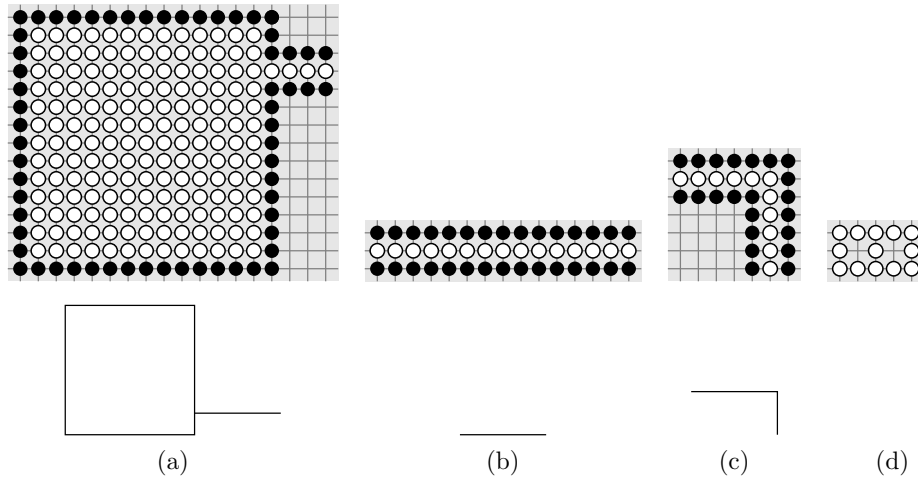


Figure 6 Un exemple de réserve de pierres blanches presque capturée par le joueur noir (a), ainsi que deux sections de tuyaux (b) et (c). Des représentations simplifiées sont données en bas. En (d) une configuration de 13 pierres blanches que Blanc va chercher à relier à sa réserve de pierres blanches pour les sécuriser.

locales de pierres qui vont permettre de simuler un comportement souhaité. Ces gadgets sont construits de sorte que le nombre de libertés du groupe de pierres blanches qui risquent d'être capturées soit toujours de 1, ou exceptionnellement de 2 dans le gadget (figure 7 (a)) ce qui permet à Blanc de faire un choix ponctuellement. Ce premier gadget (figure 7 (a)) permet de modéliser un choix pour Blanc, qui décidera s'il veut relier le tuyau du haut (supposé relié à la réserve de pierres) au tuyau de gauche ou au tuyau de droite. Nous pourrions supposer que c'est à Blanc de poser la première pierre dans chaque gadget.

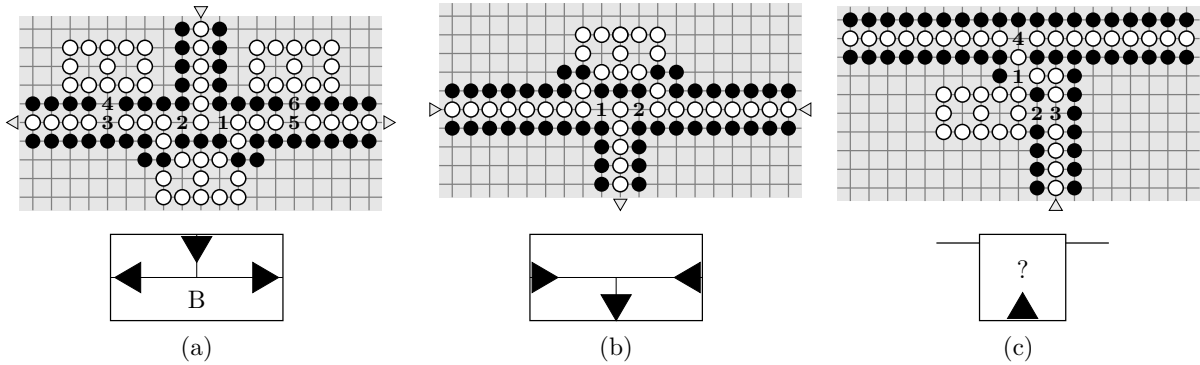


Figure 7 Trois gadgets qui seront utilisés pour la réduction, ainsi que leurs représentations simplifiées associées.

Q 39. Expliciter précisément les deux suites de coups possibles pour le gadget (a), et justifier à l'aide d'un arbre que si Blanc ou Noir s'écarte de l'une de ces deux stratégies, alors l'autre joueur peut arriver à ses fins en au plus deux coups.

Q 40. Décrire des modifications à apporter au gadget (a) pour obtenir un gadget représentant un branchement mais dans lequel c'est Noir qui décide si le tuyau de gauche ou de droite sera relié au tuyau du haut. On rappelle qu'on se place toujours dans le cas où Blanc joue en premier dans les gadgets.

Nous utiliserons, pour ce nouveau gadget, la même représentation simplifiée que le gadget (a) en remplaçant le B par un N.

Q 41. Que permet de réaliser le gadget (b) ? Justifier.

Q 42. Justifier que le gadget (c) permet, en arrivant par le bas, de tester si le tuyau transversal du haut a déjà été relié à la réserve de pierres blanches par le passé ou pas. On pourra notamment justifier que la partie se décidera sur ce gadget précis, et l'issue en sera uniquement déterminée par le fait que le tuyau transversal du haut ait été préalablement relié à la réserve ou non.

Nous nous intéressons désormais à l'assemblage de ces gadgets pour former un circuit, qui pourra être représenté en utilisant les notations simplifiées. Dans un premier temps, nous allons simuler la valuation de chaque variable par le fait d'avoir relié une section à la réserve ou non.

Q 43. Vérifier que pour chaque variable v_i , il existe une stratégie pour Noir qui permette qu'au plus un des deux tuyaux v_i ou $\neg v_i$ soit relié à la réserve de pierres blanches.

Q 44. Vérifier qu'il existe, pour toute valuation des variables, une stratégie permettant à Blanc de relier à la réserve de pierres blanches les sections associées à cette valuation.

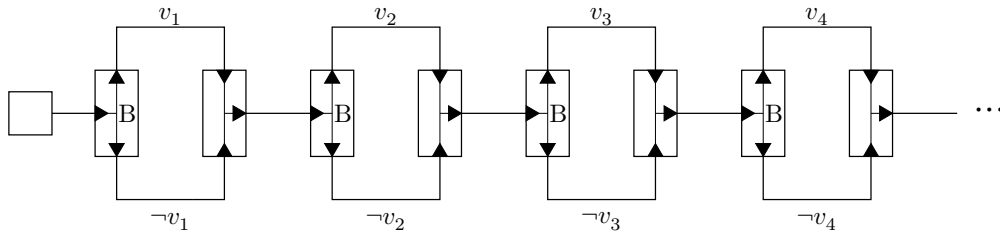


Figure 8 Un circuit représentant le choix d'une valuation, modélisée par le fait de choisir de relier le tuyau du haut ou le tuyau du bas à la réserve pour chaque variable.

Ainsi, Blanc a choisi de relier certaines sections à sa réserve de pierres blanches et la partie est encore en cours. Il reste désormais à déterminer si la valuation choisie par Blanc est un modèle de φ ou non. Pour cela, l'idée est de laisser Noir choisir la clause C qu'il souhaite dans φ . Ensuite, Blanc choisit d'accéder au littéral de son choix au sein de cette clause, afin de témoigner qu'elle est bien satisfaite. Le gadget suivant (figure 9) est une première tentative pour modéliser ce choix de Blanc, mais sa construction bloque le passage et empêche Noir d'accéder à d'éventuelles autres clauses enjambées, qui pourraient être représentées à l'intérieur dans la représentation planaire rectiligne de φ .

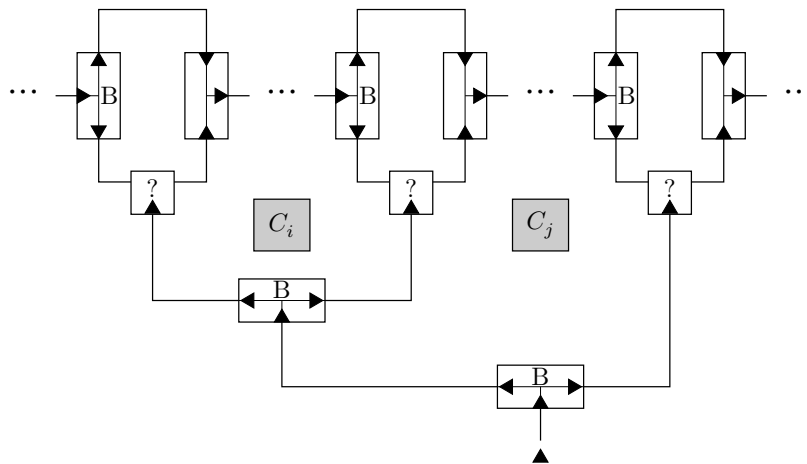


Figure 9 Un gadget permettant à Blanc de vérifier que le littéral de son choix est bien satisfait dans une clause, mais qui bloque le passage à d'autres clauses éventuelles comme C_i ou C_j qui peuvent être enjambées.

Q 45. Modifier le circuit de vérification de clause proposé, afin que Noir puisse en plus décider d'accéder à une clause intérieure s'il le souhaite, sans en modifier le comportement.

Q 46. En utilisant les questions précédentes, conclure en proposant une réduction complète de Rectilinear Planar 3SAT au jeu de go généralisé.

• • • FIN • • •

SESSION 2024



MPI5IN

ÉPREUVE MUTUALISÉE AVEC E3A-POLYTECH**ÉPREUVE SPÉCIFIQUE - FILIÈRE MPI**

INFORMATIQUE**Durée : 4 heures**

N.B. : le candidat attachera la plus grande importance à la clarté, à la précision et à la concision de la rédaction. Si un candidat est amené à repérer ce qui peut lui sembler être une erreur d'énoncé, il le signalera sur sa copie et devra poursuivre sa composition en expliquant les raisons des initiatives qu'il a été amené à prendre.

RAPPEL DES CONSIGNES

- Utiliser uniquement un stylo noir ou bleu foncé non effaçable pour la rédaction de votre composition ; d'autres couleurs, excepté le vert, bleu clair ou turquoise, peuvent être utilisées, mais exclusivement pour les schémas et la mise en évidence des résultats.
 - Ne pas utiliser de correcteur.
 - Écrire le mot *FIN* à la fin de votre composition.
-

Les calculatrices sont interdites.

Le sujet est composé de trois parties, toutes indépendantes.

Partie I - Algorithme Single Pass

Cette partie comporte des questions nécessitant un code en **langage C**.

On cherche à classer $n = 5$ documents textuels $doc_i, i \in \llbracket 1, n \rrbracket$ dans lesquels les termes T_1, T_2 et T_3 apparaissent un certain nombre de fois, ces occurrences étant décrites dans le tableau suivant :

	doc_1	doc_2	doc_3	doc_4	doc_5
T_1	1	2	0	1	1
T_2	3	1	1	3	0
T_3	3	0	0	1	1

Chaque document doc_i est donc représenté par un ensemble de $p = 3$ valeurs. On notera $d_i, i \in \llbracket 1, n \rrbracket$ un vecteur de $\mathbb{N}^3$, de composantes $d_{ij}, j \in \llbracket 1, p \rrbracket$, d_{ij} indiquant le nombre d'occurrences du terme T_j dans le document doc_i .

On cherche à voir si des textes traitent des mêmes thématiques, en faisant l'hypothèse que des textes sont sémantiquement proches si des termes communs apparaissent.

Q1. Recopier et remplir le tableau suivant en appliquant l'algorithme des k-moyennes avec $k = 2$ et d_2 et d_5 comme centres de classe initiaux. On utilisera la distance $\delta(d_i, d_j) = \sum_{\ell=1}^p |d_{i\ell} - d_{j\ell}|$. On notera de plus c_i les centres de classe et $\mathcal{A}_i$ les classes correspondantes.

Itération	c_1	c_2	$\mathcal{A}_1$	$\mathcal{A}_2$
1	d_2	d_5		
2				
3				

On souhaite maintenant traiter ce problème par une méthode dite "single pass" décrite dans l'**algorithme 1**.

Algorithme 1 - Algorithme Single Pass

Entrées : $(d_1 \dots d_n)$ les vecteurs des documents, θ un seuil appartient à $\mathbb{R}$.

Sorties : $(\mathcal{A}_1 \dots \mathcal{A}_j)$ les classes de centres $(c_1 \dots c_j)$

début

```

     $c_1 = d_1$  ; // Initialisation
     $\mathcal{A}_1 = \{d_1\}$  ;
     $j = 1$  ;
    pour  $i$  de 2 à  $n$  faire
        pour  $k$  de 1 à  $j$  faire
            Étape (i) Calculer  $\delta(d_i, c_k)$  ;
        si Étape (ii)  $\delta(d_i, c_k) > \theta \forall c_k$  alors
             $j = j + 1$  ; // Création d'une nouvelle classe
             $\mathcal{A}_j = \{d_i\}$  ;
             $c_j = d_i$  ;
        sinon
            // Indice du centre de classe le plus proche de  $d_i$  au sens de  $\delta$ 
            Étape (iii)  $\ell = \arg \min_{1 \leq k < i} (\delta(d_i, c_k))$  ;
             $\mathcal{A}_\ell = \mathcal{A}_\ell \cup \{d_i\}$  ; // Affectation de  $d_i$  à la classe  $\ell$ 
             $c_\ell = \frac{1}{|\mathcal{A}_\ell|} \sum_{d_i \in \mathcal{A}_\ell} d_i$  ; // Recalcul de  $c_\ell$ 

```

Q2. Appliquer l'**algorithme 1** avec $\theta = 5.0$. Détailler les résultats des étapes de l'algorithme.

On propose d'implémenter cet algorithme en langage C. À cet effet, on définit un type structuré vecteur permettant d'encoder les centres de classe et les textes.

```
struct vecteur_s {
    double *v;          // pointeur vers les coordonnées
    int taille;         // taille du vecteur
    int num_classe;     // classe du vecteur
};
typedef struct vecteur_s vecteur;
```

Puisque le nombre de centres de classe varie au cours de l'algorithme, on utilise une liste chaînée de vecteurs pour représenter l'ensemble des centres de classe.

```
struct noeud_s {
    vecteur *c;
    struct noeud_s *suivant;
};
typedef struct noeud_s noeud;
```

Q3. Écrire une fonction de prototype `void ajoutVecteur(vecteur *vec, noeud **tete)` permettant d'ajouter un vecteur `vec` en tête de la liste des centres de classe pointée par `tete`. On prendra soin de vérifier que l'allocation mémoire s'est bien passée.

On suppose dans la suite disposer :

- de la variable `vecteur *documents[nb_documents]` qui contient l'ensemble des documents, où pour tout $i \in \llbracket 0, \text{nb_documents} - 1 \rrbracket$, `documents[i]` est égal à d_{i+1} .
 - d'une fonction `void recalculCentre (vecteur *documents, noeud **tete, int l)` qui effectue le recalcul du centre c_l et met à jour le nœud correspondant dans la liste chaînée des centres de classe.
- Q4.** Écrire une fonction de prototype `double delta(vecteur *di, vecteur *c)` qui calcule la distance entre le document d_i et le centre de classe c (étape **(i)** de l'**algorithme 1**). On suppose que d_i et c ont la même taille.
- Q5.** Écrire une fonction de prototype `bool distmax(double dists[], int j, double theta)` qui réalise l'étape **(ii)** de l'**algorithme 1**. Le tableau `dists` contient les distances de d_i à tous les c_k : pour tout $k \in \llbracket 0, j - 1 \rrbracket$ l'élément `dists[k]` du tableau `dists` contient la distance de d_i à c_{k+1} . La fonction renvoie `true` si $\delta(d_i, c_k) > \theta \forall c_k$ et `false` sinon.
- Q6.** Écrire une fonction de prototype `int distmin(double dists[], int j)` qui réalise l'étape **(iii)** de l'**algorithme 1**. Le tableau `dists` contient les distances de d_i à tous les c_k comme dans la question précédente. La fonction renvoie l'entier l décrit dans l'algorithme.
- Q7.** À l'aide des questions précédentes et de la fonction `recalculCentre`, proposer une implémentation de l'**algorithme 1** sous la forme d'une fonction de prototype `noeud *algorithme1(vecteur *documents, int nb_documents, double theta)`. Évaluer la complexité de l'**algorithme 1**.

Partie II - Langages réguliers

Soit $\Sigma = \{a, b, c\}$ un alphabet. Σ^* est l'ensemble des mots de longueur finie sur l'alphabet Σ . Pour $u \in \Sigma^*$, $|u| \in \mathbb{N}$ désigne la longueur du mot u . On note $u_i, i \in \llbracket 1, |u| \rrbracket$ la i -ème lettre de u . Pour $k \in \llbracket 1, |u| \rrbracket$, on note $u[1, k]$ le préfixe de u de longueur k , c'est-à-dire le mot $u_1 \dots u_k$.

On définit sur Σ^* la relation symétrique $\mathcal{R}$ de la manière suivante : $u \mathcal{R} v$ s'il existe $w, z \in \Sigma^*$ tels que $u = wabz$ et $v = wbaz$. On note $\mathcal{R}^*$ la fermeture réflexive transitive de $\mathcal{R}$: $u \mathcal{R}^* v$ si $u = v$ ou s'il existe des mots $x_0 \dots x_n$ tels que :

- (i). $x_0 = u$,
- (ii). pour $i \in \llbracket 0, n-1 \rrbracket, i < \min(|u|, |v|), x_i \mathcal{R} x_{i+1}$,
- (iii). $x_n = v$.

Q8. Montrer que $\mathcal{R}^*$ est une relation d'équivalence.

Soit $u \in \Sigma^*$. La classe de u modulo $\mathcal{R}^*$ est l'ensemble des mots v vérifiant $v \mathcal{R}^* u$. Tous les mots de cette classe ont la même longueur. Ainsi, par exemple, $\{abcabb, abcbab, abcbba, bacabb, bacbab, bacbba\}$ est une classe modulo $\mathcal{R}^*$.

Q9. Montrer que les classes modulo $\mathcal{R}^*$ sont des langages réguliers.

Q10. Donner les classes de mots de longueur 3.

On définit la relation $\leq$ sur Σ telle que $a < b < c$ par : pour $u, v \in \Sigma^*$, $u \leq v$ si l'une des deux conditions suivantes est réalisée :

- (i). $u = v[1, |u|]$,
- (ii). il existe $i > 1$ tel que $u[1, i-1] = v[1, i-1]$ et $u_i < v_i$.

Q11. Montrer que la relation $\leq$ est réflexive et transitive. En déduire que $\leq$ est un ordre total sur Σ^* .

Soit $\mathcal{U}$ une classe modulo $\mathcal{R}^*$. Le *représentant* de $\mathcal{U}$ est le plus petit élément de cette classe pour l'ordre $\leq$. On note $\mathcal{L}$ l'ensemble des représentants des classes modulo $\mathcal{R}^*$.

Q12. Donner le représentant de la classe contenant le mot *bacbab*.

Q13. Donner une expression régulière du langage régulier $\mathcal{L}$.

Q14. Proposer un automate fini déterministe complet reconnaissant $\mathcal{L}$.

Partie III - Correspondance de Burge

La correspondance de Burge permet d'exprimer une bijection entre des graphes simples et des tableaux de Young semi-standards. Ces objets combinatoires ont de nombreuses applications, notamment dans l'étude de groupes symétriques et la géométrie algébrique. En théorie des graphes, ils permettent également de trouver, s'ils existent, les graphes simples dont les sommets sont contraints à avoir des degrés donnés.

Cette partie comporte des questions nécessitant un **code OCaml**. Pour ces questions, **les réponses ne feront pas appel aux fonctionnalités impératives du langage** (références, champs mutables, exceptions).

Notations

Dans toute la suite on notera :

- $[l] = \llbracket 1, l \rrbracket$ l'ensemble des entiers naturels de 1 à l ,
- $|E|$ le cardinal de l'ensemble E ,
- $G = ([l], A)$ un *graphe simple*, c'est-à-dire un graphe non orienté à l sommets et m arêtes, sans boucles ni arêtes multiples,
- $d_i = |\{j \in [l], (i, j) \in A\}|$ le degré du sommet $i \in [l]$ dans le graphe $G = ([l], A)$.

De plus, les sommets de G seront ordonnés par degrés décroissants, de sorte que $d_1 \geq d_2 \geq \dots \geq d_l \geq 0$.

Définition 1 (Suite de degrés d'un graphe)

Soit $G = ([l], A)$ un graphe simple. Si $d_i, i \in [l]$ est le degré du sommet i , avec $d_1 \geq d_2 \geq \dots \geq d_l \geq 0$, la *suite de degrés* de G est le l -uplet $(d_1, d_2, \dots, d_l)$.

III.1 - Partitions d'un entier

Définition 2 (Partition)

Une *partition* d'un entier $n \in \mathbb{N}^*$, notée $\lambda \vdash n$, est une suite décroissante $\lambda = (\lambda_1, \lambda_2, \dots, \lambda_l)$ de nombres entiers strictement positifs de somme n .

Par exemple, $n = 4$ a cinq partitions : (4) , $(3,1)$, $(2,2)$, $(2,1,1)$ et $(1,1,1,1)$.

Une partition $\lambda = (\lambda_1 \dots \lambda_l) \vdash n$ peut être vue comme une suite de degrés sous certaines conditions.

Q15. Montrer que si $\sum_{i=1}^l \lambda_i$ est impaire, alors la partition $\lambda \vdash n$ ne peut pas générer de graphe simple ayant la suite de degrés $(\lambda_1, \lambda_2, \dots, \lambda_l)$.

Dans la suite, on considérera que la somme des termes de la suite d'entiers $(\lambda_1, \lambda_2, \dots, \lambda_l)$ est paire. Même avec cette contrainte, cette suite peut ne pas pouvoir générer de graphe simple.

Définition 3 (Suite graphique)

Une suite d'entiers $d_1 \geq d_2 \geq \dots \geq d_l \geq 0$ est dite *graphique* s'il existe un graphe simple dont la suite des degrés est $(d_1, d_2, \dots, d_l)$.

Notons qu'une suite graphique vérifie par définition $d_1 \geq d_2 \geq \dots \geq d_l$.

Pour déterminer si une suite d'entiers donnée est graphique, on peut utiliser l'algorithme d'Havel-Hakimi, basé sur le théorème suivant :

Théorème 1 (Havel-Hakimi)

- (i). Pour tout $(d_1, \dots, d_l) \in \mathbb{N}^l$, si $(d_1, \dots, d_l)$ est graphique, alors $d_{d_1+1} > 0$ et toute permutation décroissante de $(d_2 - 1, d_3 - 1, \dots, d_{d_1+1} - 1, d_{d_1+2}, \dots, d_l)$ est graphique.
- (ii). Réciproquement, pour tout $(d_2, \dots, d_l) \in \mathbb{N}^{l-1}$, si $(d_2, \dots, d_l)$ est graphique, alors pour $d_1 \in \llbracket d_2 + 1, l - 1 \rrbracket$, la suite $(d_1, d_2 + 1, \dots, d_{d_1+1} + 1, d_{d_1+2}, \dots, d_l)$ est graphique.

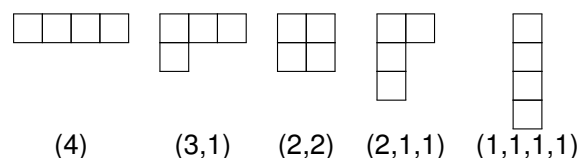
- Q16.** Montrer que si $(d_2 \cdots d_l)$ est graphique, alors il existe un graphe simple $G = (S, A)$ à l sommets, de sommet 1 de degré $d_1 \in \llbracket d_2 + 1, l - 1 \rrbracket$ tel que $(d_1, d_2 + 1 \cdots d_{d_1+1} + 1, d_{d_1+2}, d_{d_1+3}, \cdots d_l)$ soit la suite des degrés des sommets de G .
- Q17.** On suppose que $(d_1, d_2 \cdots, d_l)$ est graphique. Montrer qu'il existe $G = (S, A)$, $S = [l]$, le degré du sommet i étant d_i pour tout $i \in [l]$, tel que : $\forall j \in \llbracket 2, d_1 + 1 \rrbracket$, $(1, j) \in A$. Pour cela, on pourra raisonner par l'absurde et supposer que le sommet 1 est adjacent à un maximum de sommets dans $(2 \cdots d_1 + 1)$, mais pas à tous.
- Q18.** En déduire que si $d_{d_1+1} > d_{d_1+2}$, alors $(d_2 - 1, d_3 - 1 \cdots d_{d_1+1} - 1, d_{d_1+2}, d_{d_1+3} \cdots d_l)$ est graphique.
- Q19.** Déterminer si les suites $(4, 3, 3, 3, 3)$ et $(6, 4, 4, 2, 2, 1, 1)$ sont graphiques.
- Q20.** Écrire une fonction de signature `compare_entiers : int -> int -> int` qui compare deux entiers et telle que l'appel à `compare_entiers m n` renvoie 1 si $m < n$, -1 si $m > n$ et 0 sinon.
- Q21.** Écrire une fonction de signature `decr_n : n -> int list -> int list option` telle que pour tout $n \geq 0$ l'appel `decr_n n l` retourne `Some l'`, avec `l'` la liste `l` dont les n premiers éléments sont décrémentés, s'ils étaient tous supérieurs à 1 et `None` sinon.
- Q22.** Écrire une fonction récursive de signature `havel_hakimi : int list -> bool` qui retourne `true` si la liste passée en entrée, triée par ordre décroissant, est graphique et `false` sinon. À chaque étape utilisant le **théorème 1**, il sera nécessaire de réordonner par ordre décroissant la liste `list` construite, ce qui pourra être fait à l'aide de l'appel à `List.sort compare_entiers list`.
- Q23.** Donner deux graphes simples à $l = 5$ sommets ayant une même suite de degrés $(3, 2, 2, 2, 1)$. Ainsi, il n'existe pas de correspondance univoque entre suite de degrés et graphe simple.

III.2 - Diagramme et tableau de Young

Définition 4 (Diagramme de Young d'une partition)

Le diagramme de Young de forme λ , noté $Y(\lambda)$, d'une partition $\lambda = (\lambda_1, \lambda_2, \cdots, \lambda_l) \vdash n$ est un tableau de cases constitué de l lignes alignées à gauche, chaque ligne $i \in [l]$ ayant λ_i cases. Par convention, la ligne associée à λ_1 est la première ligne du tableau.

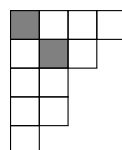
Par exemple, les diagrammes de Young des partitions de l'entier 4 sont



Définition 5 (Diagonale d'un diagramme de Young)

Soit $Y(\lambda)$ un diagramme de Young. La diagonale de $Y(\lambda)$ est définie par les r cases $(i, i), i \in [r]$.

Dans le tableau suivant, $r = 2$ et les cases de la diagonale sont grisées.



Définition 6 (Partition conjuguée)

Soit $Y(\lambda)$ un diagramme de Young associé à $\lambda \vdash n$. La partition conjuguée de λ , notée $\lambda^* = (\lambda_1^*, \cdots, \lambda_k^*)$, est la partition obtenue en énumérant le nombre de cases de $Y(\lambda)$ par colonne, en partant de la colonne de gauche.

On remarque immédiatement que pour tout j , $\lambda_j^* = |\{i, \lambda_i \geq j\}|$.

Q24. Soit $\lambda = (5, 4, 1, 1, 1, 1) \vdash 13$. Donner λ^* .

Définition 7 (Représentation de Frobenius d'un diagramme de Young)

Soient $(\lambda_1, \lambda_2, \dots, \lambda_l) \vdash n$ une partition, $Y(\lambda)$ son diagramme de Young et r la taille de sa diagonale. Pour $i \in [r]$, soient $\alpha_i = \lambda_i - i$ le nombre de cases à droite de la case (i, i) dans la i -ème ligne de $Y(\lambda)$ et $\beta_i = \lambda_i^* - i$ le nombre de cases en-dessous de la case (i, i) dans la i -ème colonne de $Y(\lambda)$. Alors $\alpha_1 > \alpha_2 \geq \dots \alpha_r \geq 0$, $\beta_1 > \beta_2 \geq \dots \beta_r \geq 0$ et la notation de Frobenius de λ est donnée par $\lambda = \begin{pmatrix} \alpha_1 & \alpha_2 & \dots & \alpha_r \\ \beta_1 & \beta_2 & \dots & \beta_r \end{pmatrix}$.

Q25. Soit $(4, 3, 2, 2, 1) \vdash 12$. Donner la représentation de Frobenius de $Y(\lambda)$.

Définition 8 (Tableau de Young)

Soit $\lambda \vdash n$ une partition de $n > 0$. Un λ -tableau de Young est un tableau obtenu en remplissant les cases d'un diagramme $Y(\lambda)$ par des entiers dans $[n]$.

Par exemple, pour $\lambda = (2, 1) \vdash 3$, les tableaux suivants sont des λ -tableaux

1	2	2	1	1	3	3	1	2	3	3	2	1	1	1	3
3		3		2		2		1		1		2		1	

Dans la suite, on notera $T(i, j)$ l'entier en position (i, j) dans le tableau T .

Définition 9 (Tableau de Young (semi)standard)

Un tableau de Young est dit *semi standard* si les éléments de chaque ligne (respectivement colonne) forment une suite croissante de gauche à droite (respectivement strictement croissante de haut en bas). Il est dit *standard* s'il est semi-standard et si les entiers de 1 à n n'apparaissent qu'une et une seule fois.

Dans les exemples précédents, les premier, troisième et septième tableaux sont semi standards. Les premier et troisième tableaux sont standards.

III.3 - Insertion de Schensted

Pour construire un tableau de Young semi-standard à partir d'une suite d'entiers $d_1, \dots, d_l$, on utilise une méthode d'insertion proposée par Schensted.

On cherche à insérer dans un tableau de Young semi standard T un entier k , de sorte à ce que le tableau créé (contenant une case de plus) soit toujours un tableau de Young semi standard. On note cette opération d'insertion $T \leftarrow k$.

L'entier k est inséré dans T en le comparant aux valeurs de la première ligne et en déplaçant le premier entier plus grand que k en lisant la ligne de gauche à droite. S'il n'y a pas de plus grand élément, k est ajouté à la fin de la ligne. Si un élément est déplacé, il est inséré dans la deuxième ligne et les lignes suivantes du tableau sont traitées de la même manière. L'**algorithme 2** décrit l'insertion de k dans la i -ème ligne de T .

1	1	2	2	4
2	3	3		
3				
4				

Soit T_0 le tableau de Young semi standard

Q26. Insérer la valeur 5 dans T_0 et donner le tableau de Young semi standard résultant. Même question pour l'insertion de la valeur 1 dans T_0 .

Q27. Énoncer une précondition de l'**algorithme 2** permettant de prouver sa correction, c'est-à-dire qu'il retourne bien un tableau de Young semi standard contenant k .

Pour coder les tableaux de Young, on choisit d'utiliser un type OCaml type `tableau = int list list` et on encode la structure par liste de lignes.

Algorithme 2 - Algorithme d'insertion d'un entier k dans la ligne i d'un tableau de Young semi standard

Insertion(T, k, i)

Entrées : un tableau de Young semi standard T , un entier k à insérer, i la ligne traitée

Sorties : un tableau de Young semi standard contenant k

début

si (la ligne i est vide) OU (k plus grand que l'élément le plus à droite de la ligne i) **alors**
 └ Ajouter k en bout de la ligne i de T

sinon

 Soit j le plus petit indice tel que $k < T(i, j)$

$p = T(i, j)$

$T(i, j) = k$

 └ **Insertion**($T, p, i + 1$)

Q28. Écrire une fonction récursive de signature `insereligne : int -> int list -> int*int list` qui, à partir d'un entier k et d'une ligne i d'un tableau de Young semi standard, retourne un couple (m, r) où

- $m = 0$ et r est la ligne i où k a été ajouté en queue, si k est plus grand que tous les éléments de la ligne i ,
- m , qui est dans la ligne i et r est la ligne i où m a été remplacé par k sinon.

Q29. En déduire une fonction récursive de signature `insertion : int -> tableau -> tableau` réalisant l'**algorithme 2**.

Q30. Écrire une fonction récursive de signature `construit_tableau : int list -> tableau` qui construit un tableau semi standard à partir d'une suite d'entiers.

L'**algorithme 2** permet de définir une position (s, t) , coordonnées de la case où k a été ajouté à T .

III.4 - Correspondance de Burge

Définition 10 (Tableau de Burge)

Soit $G = ([I], A)$ un graphe simple, $|A| = m$. Le *tableau de Burge* associé à G est défini par

$$\mathcal{B}_G = \begin{pmatrix} u_1 & u_2 & \cdots & u_m \\ v_1 & v_2 & \cdots & v_m \end{pmatrix}$$

où pour tout $i \in [m]$ (u_i, v_i) est une arête de G , avec $u_i > v_i$, le tableau étant formé de sorte que pour $i \in [m - 1]$, $u_i \leq u_{i+1}$ et si $u_i = u_{i+1}$ alors $v_i > v_{i+1}$.

Soit le graphe G de la **figure 1**.

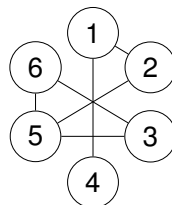


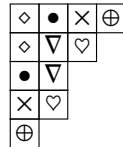
Figure 1 - Graphe exemple

Q31. Donner le tableau $\mathcal{B}_G$ correspondant au graphe de la **figure 1**.

On utilise alors $\mathcal{B}_G$ pour associer à G un diagramme de Young $Y(\lambda)$ représenté par une forme de Frobenius particulière, notée $\mathcal{F}_Y = \begin{pmatrix} \alpha_1 & \alpha_2 & \cdots & \alpha_r \\ \alpha_1 + 1 & \alpha_2 + 1 & \cdots & \alpha_r + 1 \end{pmatrix}$, où r est le nombre de cases de la diagonale principale du diagramme de Young $Y(\lambda)$ associé.

Q32. Montrer que, dans ce cas, pour tout $i \in [r]$ $\lambda_i^* = \lambda_i + 1$.

Ainsi, $Y(\lambda)$ est divisé en deux parties symétriques. La partie inférieure est constituée de toutes les cases qui se trouvent strictement sous la diagonale et la partie supérieure est constituée du reste. Chaque position dans la partie supérieure (inférieure) du diagramme de Young correspond à une position unique, appelée *position opposée*, dans la partie inférieure (supérieure) de $Y(\lambda)$. Par exemple, dans le diagramme de Young suivant, ayant comme représentation de Frobenius $\mathcal{F}_Y$, les positions opposées sont codées par le même symbole.

$$\mathcal{F}_Y = \begin{pmatrix} 3 & 1 \\ 4 & 2 \end{pmatrix}$$


Q33. Soit (s, t) une case. Donner la position opposée en fonction de s et t .

L'**algorithme 3** utilise alors la représentation $\mathcal{B}_G$ d'un graphe G pour construire un tableau de Young semi standard dont le diagramme correspondant $Y(\lambda)$ admet une représentation de Frobenius du type $\mathcal{F}_Y$. Dans cet algorithme, $T \leftarrow v_i$ insère la valeur v_i dans le tableau de Young semi standard T .

Algorithme 3 - Algorithme de Burge

Entrées : $\mathcal{B}_G$ de taille $m \times 2$

Sorties : tableau de Young semi standard T dont la représentation de Frobenius du diagramme correspondant est du type $\mathcal{F}_Y$

début

```

    T = tableau vide ; // Initialisation
    pour i de 1 à m faire
        (s, t) = T ← v_i
        Placer u_i dans la position opposée à (s, t)
```

Cet algorithme produit, à partir du graphe de la **figure 1**, le tableau :

1	1	2	3
2	3	5	
4	5		
5	6		
6			

Q34. Donner les tableaux de Young semi standard intermédiaires produits à chaque itération.

Q35. À quoi correspond le nombre d'apparitions de chaque entier contenu dans les cases du tableau de Young résultat de l'**algorithme 3** ?

Notons pour terminer qu'il est à l'inverse possible de produire un tableau bidimensionnel $\mathcal{B}_G$ (et donc un graphe G) à partir d'un tableau de Young semi standard $Y(\lambda)$ de type $\mathcal{F}_Y$.

FIN

**ECOLE POLYTECHNIQUE
ECOLES NORMALES SUPERIEURES**

CONCOURS D'ADMISSION 2025

**MARDI 15 AVRIL 2025
14h00 - 18h00
FILIERES MP-MPI - Epreuve n° 4
INFORMATIQUE A (XULSR)**

Durée : 4 heures

***L'utilisation des calculatrices n'est pas autorisée pour
cette épreuve***

*Cette composition ne concerne qu'une partie des candidats de la
filière MP, les autres candidats effectuant simultanément la composition
de Physique et Sciences de l'Ingénieur.*

*Pour la filière MP, il y a donc deux enveloppes de Sujets pour
cette séance.*

Arbres de classification d'arbres

Le sujet comporte 12 pages.

Vue d'ensemble du sujet.

Ce sujet porte sur la réalisation d'arbres d'identification des plantes basés sur des observations morphologiques. À partir d'une base de connaissance botaniste, il s'agit de réaliser un arbre dont les feuilles sont des diagnostics (typiquement des noms d'espèce) et les nœuds des *caractères morphologiques* (par exemple la couleur des fleurs). Un nœud a autant de fils que le caractère a de valeurs possibles (jaune, bleue, etc.).

Étant donnée une base de connaissance, il s'agit de construire un arbre dont la hauteur moyenne est la plus petite possible, ce qui revient à minimiser le nombre moyen d'observations nécessaires pour identifier une plante. La difficulté revient donc à choisir à chaque étape le caractère le plus *discriminant*, c'est-à-dire celui qui va le mieux séparer l'espace de recherche.

Pour tenir compte de la diversité au sein d'une espèce, nous adoptons une approche probabiliste basée sur un raisonnement bayésien : la valeur d'un caractère morphologique pour une espèce donnée sera probabiliste. Par exemple, la couleur des fleurs d'une certaine espèce pourra être bleue ou jaune, avec certaines probabilités.

La partie I introduit les éléments de raisonnement bayésien nécessaires à la construction des arbres. La partie II propose une approche par énumération pour générer un arbre qui minimise la hauteur moyenne. La partie III propose une heuristique basée sur l'entropie de Shannon pour éviter la construction de tous les arbres. La partie IV revient sur le calcul exact en proposant une optimisation permettant de réduire l'espace de recherche. Les parties III et IV sont indépendantes l'une de l'autre. Enfin, la partie V propose une approche logique pour étendre la notion de caractère. Cette dernière partie ne dépend que de la première.

Rappels de probabilités

Une **mesure** sur un ensemble fini X est une fonction $\varphi : X \rightarrow \mathbb{R}^+$. Sa **masse** est la quantité $\sum_{x \in X} \varphi(x)$. Une mesure de masse 1 est appelée une **distribution de probabilité**, ou simplement **distribution**. On note $D(X)$ l'ensemble des distributions sur l'ensemble fini X . Pour $x \in X$, la **dirac** en x , notée δ_x , est la distribution qui associe à x le poids 1, et zéro à tous les autres éléments. Étant donnés une famille de distributions $d_1, \dots, d_n \in D(X)$ et des poids $p_1, \dots, p_n \in [0; 1]$ tels que $\sum_{1 \leq i \leq n} p_i = 1$, on note $p_1 \cdot d_1 + \dots + p_n \cdot d_n$ la distribution $d \in D(X)$ telle que $d(x) = p_1 \times d_1(x) + \dots + p_n \times d_n(x)$ pour tout $x \in X$.

Si X est un ensemble, on notera $\mathcal{P}(X)$ l'ensemble de ses parties.

Rappels d'OCaml

Dans le reste du sujet, on pourra utiliser les fonctions suivantes de la bibliothèque standard OCaml pour les listes :

- `List.hd [e1;...;en]` renvoie `e1`, et `List.hd []` lève une exception.
- `List.tl [e1;...;en]` renvoie `[e2;...;en]`, et `List.tl []` lève une exception.
- `List.length l` renvoie la longueur (nombre d'éléments) de la liste `l`.
- `List.init n f` renvoie la liste `[f 0;...;f (n-1)]` pour `n` positif.
- `List.assoc k [(k1,v1);...;(kn,vn)]` renvoie le premier `vi` tel que `k = ki` s'il existe, et lève l'exception `Not_found` sinon.
- `List.find f [e1;...;en]` renvoie le premier `ei` tel que `f ei = true` s'il existe, et lève l'exception `Not_found` sinon.
- `List.map f [e1;...;en]` renvoie `[f e1;...;f en]`.
- `List.filter f l` renvoie la liste des éléments `e` de `l` tels que `f e = true`, dans l'ordre.
- `List.concat [l1;...;ln]` renvoie `l1@...@ln`, la concaténation des listes `li`.
- `List.concat_map f l` renvoie `List.concat (List.map f l)`.
- `List.fold_left f a0 [e1;...;en]` renvoie `an` où `ak+1 = f ak ek+1` pour $0 \leq k < n$.

Pour les tableaux, on pourra utiliser les fonctions suivantes :

- `Array.length a` renvoie la longueur (nombre d'éléments) du tableau `a`.
- `Array.init n f` renvoie le tableau `[f 0;...;f (n-1)]` pour `n` positif.
- `Array.of_list [e1;...;en]` renvoie le tableau `[e1;...;en]`.

On rappelle enfin que la fonction `log : float -> float` est disponible en OCaml pour calculer un logarithme naturel.

Partie I : Description probabiliste des plantes

On se donne un nombre de caractères $N \in \mathbb{N}^*$. Un **caractère** sera un entier $i \in \{0, 1, \dots, N-1\}$. Pour chaque caractère i , on se donne un ensemble fini V_i de **valeurs** possibles pour ce caractère.

Pour nos exemples, nous prendrons $N = 2$ et deux valeurs pour chaque caractère : $V_0 = \{4, 5\}$ représentera le nombre de pétales des fleurs, et $V_1 = \{\text{bleu}, \text{blanc}\}$ représentera leur couleur.

Nous utiliserons une représentation probabiliste des espèces pour modéliser la variabilité intra-espèce, ainsi que la variabilité des interprétations lors des observations : on peut trouver au sein d'une même espèce des individus à fleurs bleues et d'autres à fleurs blanches ; par ailleurs, une couleur bleue très claire pourra être interprétée comme bleue ou blanche en fonction de l'observateur. Ainsi, chaque espèce sera décrite par des valeurs possibles pour chaque caractère, organisées en distributions de probabilité : une **espèce** est un N -uplet $s = (s_0, \dots, s_{N-1})$ où $s_i \in D(V_i)$. Intuitivement, $s_i(v)$ représente la probabilité que la valeur v soit observée pour le caractère i chez un individu de l'espèce. On suppose donné un ensemble fini d'espèces $\mathcal{S}$.

Dans nos exemples, $\mathcal{S}$ sera composé des trois espèces suivantes, décrites en utilisant des diracs δ :

$$\begin{aligned} \text{myosotis} &= (\delta_5, 0, 8 \cdot \delta_{\text{bleu}} + 0, 2 \cdot \delta_{\text{blanc}}) \\ \text{lin} &= (\delta_5, \delta_{\text{bleu}}) \\ \text{gaillet} &= (\delta_4, \delta_{\text{blanc}}) \end{aligned}$$

On définit l'ensemble Ω suivant, dont les éléments modélisent des individus, décrits par leur espèce et les valeurs de leurs caractères :

$$\Omega \stackrel{\text{def}}{=} \{(s, v_0, \dots, v_{N-1}) \mid s \in \mathcal{S}, v_i \in V_i \text{ pour tout } i\}$$

On considérera l'espace probabilisable $(\Omega, \mathcal{P}(\Omega))$ obtenu en munissant Ω de la tribu discrète. On utilisera plusieurs lois de probabilité sur cet espace, définies en fonction d'un **état** $\sigma \in D(\mathcal{S})$: intuitivement, l'état indiquera la probabilité d'observer chaque espèce. La **loi de probabilité P^σ induite par un état σ** est définie comme l'unique loi satisfaisant l'équation suivante pour tout $(s, v_0, \dots, v_{N-1}) \in \Omega$:

$$P^\sigma(\{(s, v_0, \dots, v_{N-1})\}) = \sigma(s) \times \prod_{0 \leq i < N-1} s_i(v_i)$$

On notera S la variable aléatoire qui à $(s, v_0, \dots, v_{N-1}) \in \Omega$ associe s . Pour chaque caractère i , on notera C_i la variable aléatoire qui à $(s, v_0, \dots, v_{N-1}) \in \Omega$ associe v_i . On pourra ainsi écrire, par exemple, $P^\sigma(C_i = v \mid S = s)$, qui représente (dans l'état σ) la probabilité (conditionnelle) que le caractère i prenne la valeur v chez un individu de l'espèce s .

Préliminaires probabilistes. On établit tout d'abord quelques résultats élémentaires sur lesquels s'appuiera notre méthode de classification.

Question 1. On considère l'état $\sigma_0 = \{\text{gaillet} : 50\%, \text{myosotis} : 30\%, \text{lin} : 20\%\}$ sur les espèces de notre exemple. Quelle est la probabilité $P^{\sigma_0}(C_1 = \text{blanc})$ d'observer des fleurs blanches dans cet état ? Pour chaque espèce s , que vaut la probabilité $P^{\sigma_0}(S = s \mid C_1 = \text{blanc})$ d'observer un individu de l'espèce s sachant que celui-ci a des fleurs blanches ? Aucune justification n'est attendue.

Question 2. Étant donné un état $\sigma \in D(\mathcal{S})$, une espèce $s \in \mathcal{S}$, un caractère i et une valeur $v \in V_i$, exprimer simplement les probabilités $P^\sigma(S = s)$ et $P^\sigma(C_i = v)$, ainsi que $P^\sigma(C_i = v \mid S = s)$ quand elle est bien définie. Les réponses devront être justifiées.

Question 3. Étant donné un état $\sigma \in D(\mathcal{S})$, un caractère i , une valeur $v \in V_i$ et une espèce $s \in \mathcal{S}$, exprimer $P^\sigma(S = s \mid C_i = v)$. Sous quelle condition cette probabilité est-elle bien définie ?

Pour un état σ , un caractère i et une valeur $v \in V_i$, on définit **l'état mis à jour** $\sigma[i := v]$ comme la distribution qui à chaque espèce s associe $P^\sigma(S = s \mid C_i = v)$.

Question 4. Pour un état σ , deux caractères $i \neq j$ et des valeurs $v \in V_i$ et $v' \in V_j$, montrer que $P^\sigma(S = s \mid C_i = v, C_j = v') = P^{\sigma[i:=v]}(S = s \mid C_j = v')$ quand ces probabilités sont bien définies.

Le résultat précédent nous indique que, si l'on souhaite évaluer dans un état σ la probabilité d'observer une certaine espèce sachant que $C_i = v$ et $C_j = v'$, il nous suffit de calculer une probabilité conditionnée seulement par $C_j = v'$ mais dans l'état $\sigma[i := v]$. En allant plus loin, si l'on pose $\sigma' = \sigma[i := v]$ et $\sigma'' = \sigma'[j := v']$, on a $P^\sigma(S = s \mid C_i = v, C_j = v') = \sigma''(s)$. On admettra que ce résultat se généralise à un nombre arbitraire d'observations. Ainsi, il est possible de procéder à la reconnaissance de plantes en calculant des états mis à jours par des observations successives, jusqu'à atteindre un état de la forme δ_s ou ne plus pouvoir faire de nouvelle observation.

Choix des représentations pour le code. Un caractère sera naturellement représenté par un entier OCaml. On représentera aussi les valeurs par des entiers, en supposant que chaque V_i est de la forme $\{0, 1, \dots, k_i - 1\}$, ce qui revient à numéroté les valeurs possibles. On pose ainsi :

```
type caractere = int
type valeur = int
```

On suppose donné le nombre de valeurs pour chaque caractère sous la forme d'un tableau :

```
val num : int array (* num.(i) = k_i, i.e., V_i = {0, 1, ..., num.(i) - 1} *)
```

Enfin, on considèrera que la liste des caractères $[0; 1; \dots; N-1]$ est définie en variable globale à partir du tableau `num` :

```
let caracteres : caractere list = List.init (Array.length num) (fun i -> i)
```

Pour notre exemple avec deux caractères, on aura ainsi :

```
(* Nombre de valeurs possibles pour le nombre de pétales et pour leur couleur *)
let num = [|2; 2|]
```

```
(* Nombre de pétales *)
let nb_petales : caractere = 0
let quatre, cinq = 0, 1
```

```
(* Couleur *)
let couleur_petales : caractere = 1
let bleu, blanc = 0, 1
```

Une mesure μ sur X sera représentée par une liste de couples $(x, \mu(x))$ avec $x \in X$ et $\mu(x) > 0$, sans doublons. **En particulier, les éléments de poids nuls ne sont pas inclus dans la liste, et l'ordre n'a pas d'importance.**

```
type 'x mesure = ('x * float) list
let dirac v = [(v, 1.)]
```

Enfin, on codera comme suit les espèces et états :

```
type espece = { name: string; mesures: valeur mesure array }
type etat = espece mesure
```

Pour notre exemple, on aura ainsi :

```
let myosotis : espece =
  { name = "myosotis";
    mesures = [| dirac cinq; [(bleu, 0.8);(blanc, 0.2)] |] }
let lin : espece =
  { name = "lin";
    mesures = [| dirac cinq; dirac bleu |] }
let gaillet : espece =
  { name = "gaillet";
    mesures = [| dirac quatre; dirac blanc |] }
let sigma0 : etat = [(gaillet, 0.5);(myosotis, 0.3);(lin, 0.2)]
```

Question 5. Écrire les fonctions suivantes :

```
val proba_de      : 'x -> 'x mesure -> float
val somme_ponderee : 'x mesure -> ('x -> float) -> float
val repondere     : 'x mesure -> ('x -> float) -> 'x mesure
```

La première renvoie le poids d'un élément selon une mesure donnée. La seconde renvoie la somme d'une mesure μ pondérée par une fonction $f : X \rightarrow \mathbb{R}$, définie par $\sum_{x \in X} \mu(x)f(x)$. La troisième repondère une mesure sur X par une fonction $f : X \rightarrow \mathbb{R}$: le poids de $x \in X$ passe de $\mu(x)$ à $\mu(x)f(x)$.

Question 6. Écrire deux fonctions

```
val proba_espece : espece -> caractere -> valeur -> float
val proba_etat   : etat -> caractere -> valeur -> float
```

où `proba_espece s i v` renvoie la probabilité $P^{\delta_s}(C_i = v)$ et `proba_etat  $\sigma$  i v` renvoie $P^\sigma(C_i = v)$.

Question 7. Écrire une fonction

```
val reponse : etat -> caractere -> valeur -> etat
```

qui, étant donnés un état σ , un caractère i et une valeur $v \in V_i$, renvoie l'état $\sigma[i := v]$ si celui-ci est bien défini, et lève une exception sinon.

Partie II : Arbre optimal

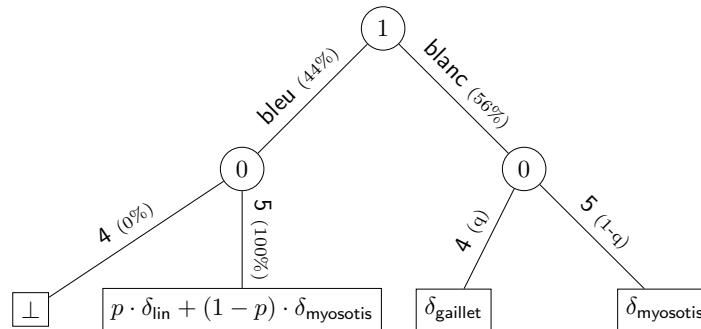
Dans cette seconde partie, on définit la notion d'arbre de décision, et on s'intéresse au calcul d'un arbre de décision optimal en un certain sens. On introduit les types de données suivants pour coder nos arbres :

```
type noeud = {caractere: caractere; enfants: (float * arbre) array}
and arbre =
  | Noeud of noeud      (* On pose une question sur un caractère. *)
  | Feuille of etat     (* On a atteint une feuille, on donne l'état. *)
  | Impossible          (* On a atteint une feuille impossible :
                        aucune espèce ne correspond aux réponses données. *)
```

Les valeurs de type `arbre` ne représentent pas toutes des arbres de décision. Une valeur t de type `arbre` est **un arbre de décision** pour un ensemble de caractères C et un état σ lorsque :

- L'arbre t n'est pas `Impossible`.
- Si t est de la forme `Feuille  $\sigma'$` , alors $\sigma' = \sigma$ et (au moins) l'une des deux conditions suivantes est satisfaite :
 - σ est une distribution dirac (il n'y a plus qu'une seule espèce possible) ;
 - $C = \emptyset$ (il n'y a plus de caractère disponible pour discriminer).
- Si t est de la forme `Noeud {caractere= $i$ ; enfants= $e$ }` alors i est un élément de C , e est un tableau à `num.(i)` éléments, et pour chaque valeur possible $0 \leq v < \text{num.}(i)$, $e.(v)$ est de la forme $(P^\sigma(C_i = v), t')$ avec :
 - si $P^\sigma(C_i = v) = 0$, alors t' est `Impossible` ;
 - sinon, t' est un arbre de décision pour l'ensemble de caractères $C \setminus \{i\}$ et l'état $\sigma[i := v]$.

Dans le contexte de l'exemple de la partie I, on représente graphiquement ci-dessous un arbre de décision pour l'ensemble complet de caractères $\{0, 1\}$ et l'état σ_0 de la question 1 (pour des valeurs de p et q non précisées) :



Les nœuds sont étiquetés par des caractères et les feuilles par des états ; la notation $\perp$ est utilisée pour représenter les cas impossibles où aucune espèce ne peut correspondre aux valeurs observées. Chaque arc est étiqueté par la valeur du caractère correspondant à l'enfant ainsi que la probabilité que cette valeur soit observée.

On définit la **hauteur moyenne** d'un arbre t , notée $h(t)$, comme suit :

- Si t est de la forme `Feuille  $\sigma$` ou `Impossible`, alors $h(t) = 0$.

— Si t est de la forme `Noeud {caractere=i;enfants=e}`, et si l'on note $(p_v, t_v) = \mathbf{e}.(v)$, alors :

$$h(t) = 1 + \sum_{0 \leq v < \text{num.}(i)} p_v \times h(t_v)$$

La hauteur moyenne correspond au nombre moyen de questions avant d'atteindre une feuille. Sur l'exemple précédent, la hauteur moyenne est de deux.

Question 8. Sur l'exemple de la question 1, énumérer tous les arbres de décision possibles pour les caractères $\{0, 1\}$ et l'état σ_0 , et donner leur hauteur moyenne — pour l'arbre déjà représenté ci-dessus, on pourra se contenter de préciser les valeurs de p et q . Quel est l'arbre de décision avec la hauteur moyenne minimale ?

Dans la suite de cette partie, on se propose de calculer naïvement un arbre de décision de hauteur moyenne minimale. On dira qu'un arbre t est **optimal pour C et σ** quand c'est un arbre de décision pour C et σ tel que $h(t)$ est minimale parmi les hauteurs moyennes de tous les arbres de décision pour C et σ . Dans un contexte où C et σ sont clairs, on dira simplement que t est optimal.

Question 9. Écrire une fonction

```
val choix_possibles : 'x list array -> 'x array list
```

qui renvoie la liste des tableaux obtenus en choisissant un élément dans chaque liste du tableau d'entrée. Par exemple :

```
choix_possibles [| [1; 2]; [3; 4; 5] |] = [
  [| 1; 3 |]; [| 1; 4 |]; [| 1; 5 |];
  [| 2; 3 |]; [| 2; 4 |]; [| 2; 5 |];
]
```

Question 10. Écrire une fonction qui, étant donné un état initial σ_0 , renvoie la liste de tous les arbres de décision possibles pour σ_0 et l'ensemble de tous les caractères :

```
val enumere : etat -> arbre list
```

On rappelle que l'ensemble $\{0, 1, \dots, N-1\}$ de tous les caractères est représenté par la liste `caracteres` déclarée en variable globale.

Question 11. Écrire une fonction

```
val argmin : 'x list -> ('x -> float) -> 'x
```

telle que `argmin l f` renvoie un élément de `l` qui minimise `f`, si la liste est non-vide. En déduire une fonction qui, étant donné un état initial σ_0 , renvoie un arbre de décision optimal pour σ_0 et l'ensemble de tous les caractères :

```
val arbre_optimal : etat -> arbre
```

Partie III : Algorithme glouton

Afin d'éviter l'énumération de tous les arbres de décision dans l'algorithme de la partie précédente, on cherche un algorithme glouton : l'idée générale est de décider localement du caractère à choisir en fonction de l'état courant, plutôt que d'envisager tous les choix possibles. Pour cela, on décide de prendre le caractère qui minimise l'**entropie moyenne**. Avant de définir cette quantité, on introduit l'entropie simple $H(\mu)$ d'une distribution $\mu \in D(X)$, qui est donnée par :

$$H(\mu) \stackrel{\text{def}}{=} - \left(\sum_{x \in X, \mu(x) \neq 0} \mu(x) \ln(\mu(x)) \right)$$

Question 12. Soit X un ensemble de cardinal $k \in \mathbb{N}$. Donner (en justifiant) les valeurs minimale et maximale de H sur $D(X)$ en fonction de k , ainsi que des distributions les atteignant.

Étant donné un caractère i et un état σ , on définit l'entropie moyenne de i dans l'état σ , notée $H_i(\sigma)$, comme l'espérance des entropies sur les distributions obtenues en faisant une observation sur le caractère i :

$$H_i(\sigma) \stackrel{\text{def}}{=} \sum_{v \in V_i} P^\sigma(C_i = v) H(\sigma[i := v])$$

Question 13. Écrire une fonction qui renvoie l'entropie moyenne d'un caractère dans un état donné :

```
val score_caractere : etat -> caractere -> float
```

Question 14. Écrire une fonction qui calcule un arbre de décision en suivant l'algorithme glouton qui construit l'arbre en choisissant à la racine le caractère minimisant l'entropie moyenne, et procède récursivement de la même façon pour les sous-arbres :

```
val glouton : etat -> arbre
```

On se propose maintenant de montrer que l'algorithme glouton n'est pas optimal. On considère une situation avec deux caractères, $V_0 = V_1 = \{\text{oui}, \text{non}\}$, et trois espèces a , b et c :

$$\begin{aligned} a &= (\delta_{\text{oui}}, 0, 5 \cdot \delta_{\text{oui}} + 0, 5 \cdot \delta_{\text{non}}) \\ b &= (\delta_{\text{non}}, \delta_{\text{oui}}) \\ c &= (\delta_{\text{non}}, \delta_{\text{non}}) \end{aligned}$$

On considère enfin l'état initial σ_0 suivant :

$$a : 20\%, \quad b : 40\%, \quad c : 40\%$$

L'espèce a est donc plus rare que les espèces b et c qui sont plus communes.

Question 15. Dessiner les arbres de décision possibles pour l'état σ_0 et les caractères $\{0, 1\}$, et déterminer l'arbre de décision optimal.

Question 16. Calculer les entropies moyennes $H_0(\sigma_0)$ et $H_1(\sigma_0)$. Que peut-on en conclure ? On admettra que $\ln(5) < \frac{12}{5} \ln(2)$.

Partie IV : Optimisation de l'algorithme naïf

On cherche à optimiser l'algorithme naïf de la partie II en évitant certains calculs inutiles. Pour permettre cela, on commence par changer la méthode de recherche d'un arbre optimal, en évitant de construire la liste de tous les arbres de décision possibles. La nouvelle méthode s'appuiera sur la fonction `arbre_optimal_avec_oracle` donnée en Figure 1. Le but de cette fonction est, étant donné un état σ et une liste de caractères C , de renvoyer $(t, h(t))$ où t est un arbre optimal pour C et σ . Il faut bien noter que cette fonction n'est pas récursive, mais qu'elle dispose d'un argument supplémentaire `oracle` dont les appels pourront correspondre à des appels récursifs : il s'agit de *réursion ouverte*.

Question 17. Donner des hypothèses (pré-conditions) aussi précises que possible sur les arguments C , σ et `oracle` de la fonction `arbre_optimal_avec_oracle`, sous lesquelles la fonction termine toujours et renvoie bien $(t, h(t))$ où t est un arbre optimal pour C et σ . On veillera à répondre à cette question après avoir réfléchi aux deux suivantes, où il faudra démontrer puis exploiter la correction de la fonction par rapport à la spécification énoncée ici.

Question 18. Démontrer que la fonction satisfait bien cette spécification, en veillant notamment à énoncer clairement l'invariant de boucle.

Question 19. Écrire une fonction, récursive cette fois-ci, et basée sur `arbre_optimal_avec_oracle`, qui renvoie un arbre de décision optimal pour un état et une liste de caractères donnés :

```
val arbre_optimal : etat -> caractere list -> arbre * float
```

Donner sa spécification et montrer qu'elle la satisfait.

Question 20. On cherche maintenant à optimiser notre algorithme en interrompant la génération d'un arbre dès que son score dépasse celui de l'`arbre_optimal` courant. Pour cela, écrire une nouvelle fonction

```
val arbre_optimal_opt : etat -> caractere list -> float -> (arbre * float) option
```

qui prend un paramètre supplémentaire représentant le score à ne pas dépasser, et renvoie `None` si tous les arbres de décision possibles sont de hauteur moyenne supérieure au score maximal, ou bien un arbre optimal de hauteur moyenne inférieure s'il en existe un. La preuve de correction n'est pas demandée.

```

1  let arbre_optimal_avec_oracle
2      (oracle : etat -> caractere list -> arbre * float)
3      (etat : etat)
4      (caracteres : caractere list) : arbre * float =
5  if caracteres = [] then
6      (Feuille etat, 0.)
7  else
8      let caracteres_a_tester = ref caracteres in
9      let score_optimal = ref infinity in
10     let arbre_optimal = ref None in
11     while !caracteres_a_tester <> [] do
12         let caractere = List.hd !caracteres_a_tester in
13         let score_total = ref 1. in
14         let enfants : (float * arbre) array =
15             Array.init num.(caractere) (fun valeur ->
16                 let p_valeur = proba_etat etat caractere valeur in
17                 let sous_arbre, score_sous_arbre =
18                     if p_valeur = 0. then Impossible, 0. else
19                         let etat' = reponse etat caractere valeur in
20                         let caracteres_restants =
21                             List.filter (fun c -> c <> caractere) caracteres
22                         in
23                             oracle etat' caracteres_restants
24                 in
25                 score_total := !score_total +. p_valeur *. score_sous_arbre;
26                 (p_valeur, sous_arbre))
27         in
28         if !score_total < !score_optimal then
29             (arbre_optimal := Some (Noeud {caractere=caractere;enfants=enfants});
30              score_optimal := !score_total);
31         caracteres_a_tester := List.tl !caracteres_a_tester
32     done;
33     match !arbre_optimal with
34     | Some a -> a, !score_optimal
35     | None -> assert false

```

FIGURE 1 – Fonction `arbre_optimal_avec_oracle`.

Partie V : Observations complexes et formules logiques

Pour aller plus loin, on se propose de représenter des observations complexes sous forme de formules logiques construites à partir des caractères, par exemple “la fleur est bleue et a cinq pétales”. La syntaxe de cette logique est la suivante :

$$\varphi ::= c_i \in V \mid \varphi_1 \wedge \dots \wedge \varphi_m \mid \varphi_1 \vee \dots \vee \varphi_m \quad (0 \leq i < N, V \subseteq V_i, m \geq 0)$$

Cela signifie que l'ensemble des formules est le plus petit ensemble contenant les formules **atomiques** de la forme $c_i \in V$ où $0 \leq i < N$ et $V \subseteq V_i$, et que si $\varphi_1, \dots, \varphi_m$ sont des formules pour $m \geq 0$, alors la **conjonction** $\varphi_1 \wedge \dots \wedge \varphi_m$ et la **disjonction** $\varphi_1 \vee \dots \vee \varphi_m$ sont aussi des formules. Une formule $c_i \in V$ signifie intuitivement que le caractère i prend une valeur dans l'ensemble $V \subseteq V_i$. On définit la formule $\perp$ (la formule fausse) comme la disjonction vide, et $\top$ (la formule vraie) comme la conjonction vide.

Par exemple, avec les ensembles V_0 et V_1 de l'exemple utilisé en partie I, $c_1 \in \{\text{bleu}\} \wedge c_0 \in \{5\}$ est une formule, correspondant à l'énoncé “la fleur est bleue et a cinq pétales”. Ou encore, si $V_i = \{0, 1, 2, 3, 4\}$, $c_i \in \{1\} \wedge c_i \in \{2, 3\}$ est une formule, correspondant à l'énoncé contradictoire “le caractère i vaut 1, et il vaut 2 ou 3”.

On définit la taille d'une formule φ , notée $|\varphi|$, comme suit :

$$\begin{aligned} |c_i \in V| &= 1 + |V| \\ |\varphi_1 \wedge \dots \wedge \varphi_m| &= 1 + m + \sum_{1 \leq i \leq m} |\varphi_i| \\ |\varphi_1 \vee \dots \vee \varphi_m| &= 1 + m + \sum_{1 \leq i \leq m} |\varphi_i| \end{aligned}$$

Dans un premier temps, nous définissons quand une situation certaine (dite déterministe) satisfait une certaine formule. On appelle **modèles déterministes** les éléments de $V_0 \times \dots \times V_{N-1}$. On notera $\vec{v} \in \vec{V}$ pour signifier que $\vec{v}$ est un tel modèle, et dans ce cas on notera v_i la i -ème composante de $\vec{v}$, ce qui revient à dire que $\vec{v} = (v_0, \dots, v_{N-1})$. On définit une relation de **satisfaction** entre les modèles déterministes et les formules, notée $\vec{v} \models \varphi$, comme suit :

$$\begin{aligned} \vec{v} \models c_i \in V & \quad \text{si et seulement si} \quad v_i \text{ appartient à l'ensemble } V \\ \vec{v} \models \varphi_1 \wedge \dots \wedge \varphi_m & \quad \text{si et seulement si} \quad \vec{v} \models \varphi_k \text{ pour tout } k \in \{1, \dots, m\} \\ \vec{v} \models \varphi_1 \vee \dots \vee \varphi_m & \quad \text{si et seulement si} \quad \vec{v} \models \varphi_k \text{ pour un } k \in \{1, \dots, m\} \end{aligned}$$

On étend ensuite cette relation de satisfaction au cadre probabiliste de la manière suivante. Un **modèle probabiliste** est un élément de $D(V_0) \times \dots \times D(V_{N-1})$. Un tel modèle associe une distribution de probabilité plutôt qu'une valeur déterminée à chaque caractère. On remarquera que cette modélisation suppose que les valeurs de deux caractères distincts sont probabilistes mais indépendantes l'une de l'autre. Étant donné une formule φ et un modèle probabiliste $(d_0, \dots, d_{N-1})$, on définit la probabilité que $\vec{d} = (d_0, \dots, d_{N-1})$ satisfasse φ , notée $P(\vec{d} \models \varphi)$, ainsi :

$$P(\vec{d} \models \varphi) \stackrel{\text{def}}{=} \sum_{\vec{v} \in \vec{V}, \vec{v} \models \varphi} \prod_{0 \leq i < N} d_i(v_i)$$

Question 21. Pour deux formules φ et ψ , montrer l'équivalence entre ces deux propositions :

- Pour tout modèle déterministe $\vec{v}$, $\vec{v} \models \varphi$ si et seulement si $\vec{v} \models \psi$.
- Pour tout modèle probabiliste $\vec{d}$, $P(\vec{d} \models \varphi) = P(\vec{d} \models \psi)$.

On dira que φ et ψ sont **équivalentes**, noté $\varphi \equiv \psi$, quand l'une ou l'autre des propositions précédentes est vraie.

Dans les questions qui suivent, on reprend le codage OCaml des distributions, et on représente directement les formules via le type suivant :

```
type formule =
  | Feuille of caractere * valeur list
  | Conjonction of formule list
  | Disjonction of formule list
```

On supposera de plus que les listes codant les ensembles de valeurs dans les formules atomiques sont sans doublons. On codera les modèles déterministes (resp. probabilistes) par des tableaux de N valeurs (resp. distributions). On notera enfin K le cardinal maximal des V_i :

$$K \stackrel{\text{def}}{=} \max_{0 \leq i < N} |V_i|$$

Question 22. On considère l'algorithme qui, étant donné un modèle probabiliste $\vec{d}$ et une formule φ , calcule $P(\vec{d} \models \varphi)$ en suivant naïvement sa définition. Quelle est la complexité temporelle de cet algorithme, en fonction de N , K et $|\varphi|$? Un programme OCaml détaillé n'est pas demandé, mais on veillera à en décrire les aspects nécessaires pour justifier l'analyse de complexité.

Dans la suite, nous allons élaborer une méthode de calcul de $P(\vec{d} \models \varphi)$ dont la complexité ne dépend pas de N . Cette méthode s'appuie sur des formes particulières de formules. Une **clause** est une conjonction de formules atomiques concernant des caractères différents. En reprenant les exemples précédents, $c_1 \in \{\text{bleu}\} \wedge c_0 \in \{5\}$ est une clause mais pas $c_i \in \{1\} \wedge c_i \in \{2, 3\}$.

Question 23. Décrire un algorithme qui, étant donné un modèle probabiliste $\vec{d}$ et une clause φ , calcule $P(\vec{d} \models \varphi)$ avec une complexité qui ne dépend que de K et $|\varphi|$, mais pas de N . Justifier la correction de l'algorithme et son analyse de complexité. Il n'est pas nécessaire de donner un programme OCaml détaillé.

Question 24. Étant données deux clauses φ et ψ , montrer que $\varphi \wedge \psi$ est équivalente à une clause que l'on notera $\varphi \bar{\wedge} \psi$.

Question 25. Montrer que pour toute formule φ il existe $m \geq 0$ et des clauses $\varphi_1, \dots, \varphi_m$ telles que $\varphi \equiv \varphi_1 \vee \dots \vee \varphi_m$.

Question 26. Décrire un algorithme qui, étant donné un modèle probabiliste $\vec{d}$ et une formule φ , calcule $P(\vec{d} \models \varphi)$ avec une complexité temporelle (potentiellement non-polynomiale) qui ne dépend que de K et $|\varphi|$ mais pas de N .

En pratique, un tel algorithme est utile car on va avoir des modèles avec de nombreux caractères mais un nombre de valeurs possibles restreint pour chaque caractère, et l'on s'intéressera à des formules de taille limitée.

Fin du sujet.

**ECOLE POLYTECHNIQUE
ECOLES NORMALES SUPERIEURES**

CONCOURS D'ADMISSION 2025

JEUDI 17 AVRIL 2025

08h00 - 12h00

FILIERE MPI - Epreuve n° 7

INFORMATIQUE C (XULSR)

Durée : 4 heures

***L'utilisation des calculatrices n'est pas autorisée pour
cette épreuve***

Couplages maximaux et parfaits

Le sujet comporte 18 pages, numérotées de 1 à 18.

Début de l'épreuve.

Complexité. Par la complexité d'un algorithme, on entend le nombre d'opérations élémentaires (comparaison, addition, soustraction, multiplication, division, affectation, test, etc.) nécessaires à l'exécution de celui-ci dans le pire cas. Lorsque la complexité dépend d'un ou plusieurs paramètres $k_0, \dots, k_r$, on dit que l'algorithme a une complexité en $O(f(k_0, \dots, k_r))$ s'il existe une constante $C > 0$ telle que, pour toutes les valeurs de $k_0, \dots, k_r$ suffisamment grandes (c'est-à-dire plus grandes qu'un certain seuil), pour toute instance du problème de paramètres $k_0, \dots, k_r$ la complexité est au plus $C \times f(k_0, \dots, k_r)$.

Fonctions utiles en OCaml. Dans les questions de programmation en OCaml, il est interdit d'utiliser des fonctions de la bibliothèque qui ne sont pas incluses dans la liste suivante.

- **Option.get**: `'a option -> 'a` renvoie `v` si l'argument de type `'a option` est égal à `Some v`, et lève une exception sinon. La complexité est en $O(1)$.
- **List.length**: `'a list -> int` renvoie la taille de la liste donnée en argument. La complexité est en $O(n)$, où n est la longueur de la liste donnée en argument.
- **List.hd**: `'a list -> 'a` renvoie le premier élément de la liste donnée en argument si celle-ci n'est pas vide, et lève une exception sinon. La complexité est en $O(1)$.
- **List.tl**: `'a list -> 'a list` renvoie la liste donnée en argument privée de son premier élément si la liste donnée en argument n'est pas vide, et lève une exception sinon. La complexité est en $O(1)$.
- **List.rev**: `'a list -> 'a list` renvoie la liste donnée en argument dans l'ordre inverse. La complexité est en $O(n)$, où n est la longueur de la liste donnée en argument.
- **List.map**: `('a -> 'b) -> 'a list -> 'b list` renvoie la liste de type `'b list` obtenue en appliquant la fonction de type `'a -> 'b` donnée en premier argument à chaque élément de la liste de type `'a list` donnée en second argument. Si la complexité de `f` est en $O(1)$, alors la complexité de `List.map f lst` est en $O(n)$, où n est la longueur de `lst`.
- **List.mem**: `'a -> 'a list -> bool` renvoie `true` si la valeur donnée en premier argument est un élément de la liste donnée en second argument, et renvoie `false` sinon. La complexité est en $O(n)$, où n est la longueur de la liste donnée en second argument.
- **List.assoc**: `'a -> ('a * 'b) list -> 'b` : cette fonction spécifique aux listes d'association est décrite en page 4.

Fonctions utiles en C. Dans les questions de programmation en C, on pourra supposer que les en-têtes `<stdbool.h>`, `<stdio.h>`, `<stdlib.h>`, et `<math.h>` ont été inclus. En particulier, on pourra utiliser la fonction `double log2(double x)` définie par l'entête `<math.h>`, qui calcule le logarithme en base 2.

Présentation du sujet. Ce sujet concerne les couplages dans les graphes non orientés. La partie I, à composer dans le langage OCaml, concerne la recherche d'un couplage maximal dans un graphe. On étudiera et implémentera un algorithme dû à Edmonds. Les parties II et III, à composer dans le langage C, concernent le problème de l'existence d'un couplage parfait dans un graphe. On y implémente des méthodes algébriques et probabilistes basées sur des calculs de déterminants. La partie II concerne l'implémentation d'un algorithme pour calculer le déterminant d'une matrice carrée dû à Mahajan et Vinay. La partie III exploite cet algorithme pour implémenter un algorithme probabiliste testant l'existence d'un couplage parfait dans un graphe non orienté.

Les parties I et II sont indépendantes. La partie III dépend de la partie II et de notions introduites en partie I.

Graphes non orientés. On travaillera dans les parties I et III sur des graphes non orientés et sans boucles. Un tel graphe G est défini par une paire d'ensembles finis (V, E) , où V est l'ensemble des sommets du graphe et E l'ensemble de ses arêtes. Une arête est une paire non-ordonnée $\{u, v\}$ d'exactly deux sommets $u, v \in V$. Ainsi, les arêtes sont non orientées ($\{u, v\} = \{v, u\}$) et il n'y a pas de boucles ($\{v\} = \{v, v\}$ n'est jamais une arête de G). Une arête $\{u, v\}$ est dite **incidente** aux sommets u et v . On supposera toujours que V est non vide.

Étant donné un graphe $G = (V, E)$:

- Un **chemin** est une suite finie de sommets $v_0, v_1, \dots, v_n$ avec $n \geq 0$ et telle que pour tout $i = 0, \dots, n-1$, on a $\{v_i, v_{i+1}\} \in E$. La **longueur** du chemin $v_0, v_1, \dots, v_n$ est n .
- Un chemin est **élémentaire** si ses sommets sont deux-à-deux distincts.
- Un **cycle** est un chemin $v_0, v_1, \dots, v_n$ de longueur ≥ 3 , avec $v_0 = v_n$ et tel que le chemin $v_1, \dots, v_n$ est élémentaire. Autrement dit, le seul sommet autorisé à apparaître plusieurs fois dans un cycle $v_0, v_1, \dots, v_n$ est le sommet $v_0 = v_n$, qui apparaît exactement deux fois (une fois au début et une fois à la fin).

Partie I : Algorithme d'Edmonds

Cette partie concerne l'algorithme d'Edmonds pour la recherche de couplage maximal.

Question 1. *Écrire une fonction `del: 'a list -> 'a list -> 'a list` qui prend en entrée deux listes `lst1` et `lst2` et renvoie la liste des éléments de `lst1` n'apparaissant pas dans `lst2`. On attend une complexité en $O(m_1 m_2)$ où m_1 est la longueur de `lst1` et m_2 est la longueur de `lst2`, sans la justifier.*

On suppose définie une fonction `join: 'a list -> 'a list -> 'a list` telle que `join lst1 lst2` est une liste sans répétitions contenant l'ensemble de tous les éléments apparaissant dans `lst1` et de tous les éléments apparaissant dans `lst2`.

Nous utilisons des listes d'association pour représenter les graphes. Les listes d'association implémentent une structure de dictionnaire ; il s'agit de listes de paires (a, b) , dont le premier élément a représente la clef et le second élément b représente la valeur associée à cette clef.

Il existe des fonctions OCaml permettant de travailler avec les listes d'association. Dans la suite, nous utiliserons seulement la fonction `List.assoc: 'a -> ('a * 'b) list -> 'b` qui prend en arguments une clef x et une liste d'association `lst`, et renvoie la valeur b associée à cette clef, c'est-à-dire l'élément b du premier couple (x, b) appartenant à `lst`, s'il en existe. La fonction lève une exception si x n'est pas une clef de la liste `lst`. La complexité est en $O(n)$, où n est la longueur de `lst`.

Question 2. *Écrire une fonction `keys: ('a * 'b) list -> 'a list` qui prend en entrée une liste d'association `lst` et qui renvoie la liste de ses clefs. On attend une complexité en $O(m)$ où m est la longueur de `lst`, sans la justifier.*

Nous représentons les graphes en utilisant le type OCaml suivant :

```
type graphe = (int * (int list)) list
```

Un graphe est représenté par une liste de paires (a, lst) où a est un sommet du graphe et `lst` est la liste d'adjacence de ce sommet. Dans la suite, on suppose toujours que :

- les sommets du graphe sont exactement les clefs de la liste d'association ;
- chaque clé de la liste d'association est unique.

Par exemple, le graphe

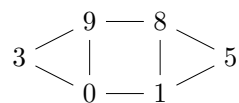
$$2 \text{ --- } 0 \text{ --- } 4$$

peut être représenté par la liste d'association `[(0, [2;4]); (4, [0]); (2, [0])]`.

Couplages

Un **couplage** C dans un graphe G est un ensemble (possiblement vide) d'arêtes de G sans intersections : un sommet de G ne peut appartenir à plus d'une arête dans C .

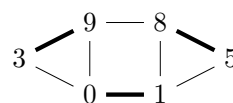
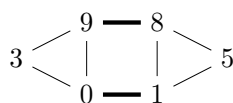
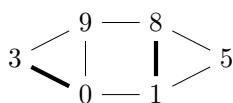
Considérons par exemple le graphe :



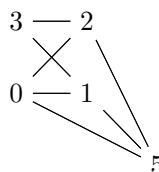
Les ensembles d'arêtes suivants sont des couplages dans ce graphe :

$$\{\{0, 3\}, \{1, 8\}\} \quad \{\{0, 1\}, \{8, 9\}\} \quad \{\{0, 1\}, \{5, 8\}, \{3, 9\}\}$$

Les arêtes d'un couplage sont représentées par des traits plus épais. Par exemple, les trois couplages ci-dessus sont représentés comme suit :



Question 3. On considère le graphe suivant :



Indiquer lesquels des ensembles suivants sont des couplages dans le graphe ci-dessus :

- (1) $\{\{1, 2\}, \{0, 5\}\}$,
- (2) $\{\{1, 3\}, \{0, 5\}\}$,
- (3) $\{\{0, 1\}, \{2, 3\}, \{0, 5\}\}$.

Un couplage est représenté par une liste de paires de sommets :

```
type couplage = (int * int) list
```

Dans toute la suite, une arête $\{a, b\}$ sera représentée par la paire $(\min a \ b, \max a \ b)$.

Étant donné un couplage C dans un graphe G , on dit qu'un sommet v de G est **couvert** par C s'il existe une arête dans le couplage C qui est incidente à v .

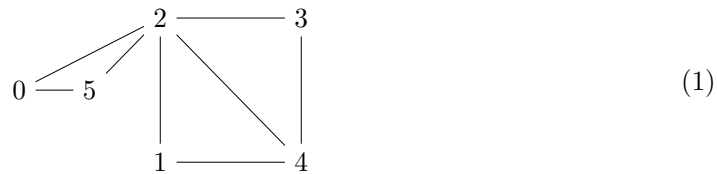
Question 4. Écrire une fonction `couverts: couplage -> int list` qui prend en entrée un couplage `cpl` et qui renvoie la liste des sommets couverts par `cpl`.

On attend une complexité en $O(m)$ où m est la longueur de la liste `cpl`, sans la justifier.

Couplages maximaux

Nous allons maintenant nous intéresser aux couplages maximaux, c'est-à-dire aux couplages ayant un nombre maximal d'arêtes. On remarque qu'un couplage dans un graphe à n sommets ne peut contenir plus de $\lfloor n/2 \rfloor$ arêtes.

Considérons le graphe suivant :



Le couplage $\{\{0, 5\}, \{2, 3\}, \{1, 4\}\}$ est maximal. Mais le couplage $\{\{0, 5\}, \{2, 4\}\}$ n'est pas maximal bien qu'il ne soit pas possible de lui adjoindre d'arête supplémentaire.

Question 5. Donner un autre couplage maximal dans le graphe ci-dessus.

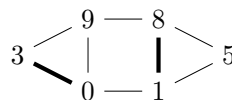
Chemins d'augmentation

L'algorithme d'Edmonds est basé sur la recherche de chemins d'augmentation. Étant donné un couplage C dans un graphe G , un **chemin d'augmentation** pour C (dans G) est un chemin élémentaire $v_0, \dots, v_n$ dans le graphe G tel que $n > 0$ et :

- v_0 et v_n ne sont pas couverts par C ;
- c'est un chemin **alternant** : pour tout $i = 0, \dots, n-2$, on a $\{v_i, v_{i+1}\} \in C$ si et seulement si $\{v_{i+1}, v_{i+2}\} \notin C$.

Un chemin d'augmentation P pour C permet de construire un nouveau couplage $C(P)$ constitué des arêtes de C qui ne sont pas dans P et des arêtes de P qui ne sont pas dans C .

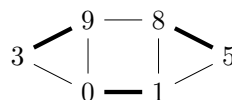
Par exemple, dans le graphe



le chemin suivant est un chemin d'augmentation :

$$9 \text{ --- } 3 \text{ --- } 0 \text{ --- } 1 \text{ --- } 8 \text{ --- } 5$$

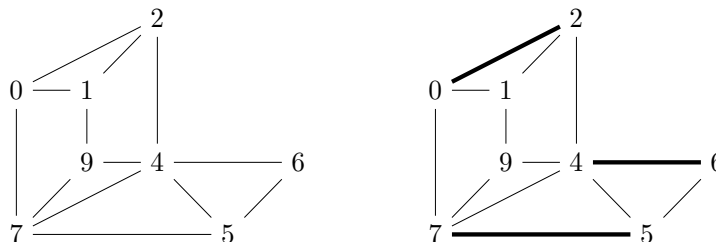
Le nouveau couplage obtenu à partir de ce chemin d'augmentation est le suivant :



Question 6. Considérons le graphe en (1) ci-dessus. Donner un chemin d'augmentation P

pour le couplage $C = \{\{0, 5\}, \{2, 4\}\}$ de manière à ce que le couplage $C(P)$ soit maximal.

Question 7. On considère le graphe et le couplage suivants :



Donner un chemin d'augmentation et le couplage augmenté correspondant.

Question 8. Soit P un chemin d'augmentation pour un couplage C dans un graphe G . Montrer que $C(P)$ est bien un couplage, et qu'il contient strictement plus d'arêtes que C .

En OCaml, un chemin est représenté par une liste de sommets, c'est-à-dire par une valeur de type `int list`.

Question 9. Écrire une fonction

```
separer : int list -> ((int * int) list) * ((int * int) list)
```

qui prend en entrée un chemin d'augmentation `chm` et renvoie deux listes `lstin` et `lstout` où `lstin` contient les arêtes composant le chemin `chm` qui appartiennent au couplage, et où `lstout` contient les arêtes composant le chemin `chm` qui n'appartiennent pas au couplage.

On attend une complexité en $O(m)$, où m est la longueur de la liste `chm`, sans la justifier.

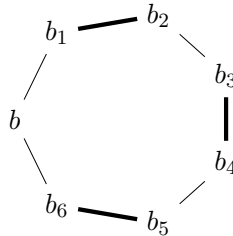
Question 10. Écrire une fonction `augmente: couplage -> int list -> couplage` qui prend en entrée un couplage `cpl` et un chemin d'augmentation `chm`, et qui renvoie le couplage `cpl(chm)`.

L'algorithme d'Edmonds, que nous allons voir ci-dessous, repose essentiellement sur le lemme de Berge. Ce résultat, que nous admettrons, énonce qu'un couplage C dans un graphe G est maximal si et seulement s'il n'existe pas de chemin d'augmentation pour C dans G .

Bourgeons

L'algorithme d'Edmonds va de plus effectuer des opérations de **contraction de bourgeons**. Soit C un couplage dans un graphe G . Un bourgeon est un cycle dans G de longueur impaire $2m + 1$ dont exactement m arêtes sont dans C . La **base** b d'un bourgeon B est le seul sommet

de B dont les deux arêtes adjacentes dans B ne sont pas dans C :



La contraction d'un bourgeon consiste à identifier tous les sommets de celui-ci. On représente un bourgeon B par une valeur de type `int list` qui contient exactement les sommets de B , dans un ordre quelconque, mais sans répétition. Par exemple, le bourgeon illustré ci-dessus peut être représenté par la liste $[b_1; b_6; b_2; b_5; b_3; b; b_4]$.

Étant donné un graphe $G = (V, E)$ et une liste de sommets $L = \ell_1, \dots, \ell_k$, le **graphe contracté** G/L est défini comme suit :

- L'ensemble des sommets du graphe G/L est $(V \setminus L) \cup \{w\}$ où w , **le nouveau sommet de G/L** , est un entier qui n'est pas un sommet de G .
- Les arêtes de G/L sont les arêtes de G entre sommets n'étant pas dans L , ainsi que les arêtes de la forme $\{u, w\}$ où u n'appartient pas à L mais est adjacent dans G à un sommet de L . L'ensemble des arêtes de G/L est donc défini comme $\{\{u, v\} \in E \mid u \notin L \text{ et } v \notin L\} \cup \{\{u, w\} \mid u \notin L \text{ et } u \text{ adjacent dans } G \text{ à un sommet de } L\}$

On suppose donnée une fonction `renomme: int list -> int list -> int -> int list` telle que `renomme lst rnm w` renvoie une liste obtenue en remplaçant dans `lst` tous les éléments de `rnm` par `w`, et en supprimant tous les doublons.

Question 11. Écrire une fonction `contracteG: graphe -> int list -> int -> graphe` qui prend en entrée un graphe `grph`, une liste de sommets `lst`, et le nom `w` du nouveau sommet, et qui renvoie le graphe contracté `grph/lst`.

Soit C un couplage dans un graphe G , et soit B un bourgeon. On définit un couplage C/B dans le graphe contracté G/B . En notant w le nouveau sommet de G/B , l'ensemble C/B a pour éléments :

- les arêtes $\{a, b\} \in C$ telles que $a \notin B$ et $b \notin B$;
- les arêtes $\{a, w\}$ telles que $a \notin B$ et telles qu'il existe un $b \in B$ avec $\{a, b\} \in C$.

On remarquera que C/B ne dépend que de C , B , et du nom w du nouveau sommet.

Question 12. Montrer que C/B est un couplage dans G/B .

Question 13. Écrire une fonction

`contracteC : couplage -> int list -> int -> couplage`

qui prend en entrée un couplage `cpl`, un bourgeon `brg`, et le nom `w` du nouveau sommet, et qui renvoie le couplage `cpl/brg`.

On admettra dans la suite qu'il existe un chemin d'augmentation pour le couplage C/B dans G/B si et seulement s'il existe un chemin d'augmentation pour le couplage C dans G .

Recherche de chemins d'augmentation

La recherche de chemins d'augmentation va utiliser des forêts, c'est-à-dire des listes d'arbres. Les types ci-dessous représentent des arbres et des forêts dont les nœuds sont étiquetés par des entiers. Ces entiers seront par la suite des sommets d'un graphe.

```
type arbre = N of int * foret
and foret = arbre list
```

Question 14. Écrire une fonction `find: foret -> int -> (int list)` option telle que `find foret v` renvoie `None` si l'entier `v` n'étiquette aucun des nœuds de `foret`, et telle que `find foret v` renvoie `Some c` sinon, où `c` est la liste des étiquettes le long d'un chemin allant d'une racine à un nœud d'étiquette `v` dans `foret`.

On attend une complexité en $O(n)$, où n est le nombre de nœuds de `foret`, sans la justifier.

Question 15. Écrire une fonction `extend: foret -> int -> int -> foret`, qui prend en arguments une forêt `foret` et deux entiers `u` et `v`, et qui renvoie une copie de `foret` dans laquelle pour chaque nœud étiqueté par `u`, on a créé un nouvel enfant étiqueté par `v`.

On attend une complexité en $O(n)$, où n est le nombre de nœuds de `foret`, sans la justifier.

La **profondeur** d'un nœud dans un arbre est définie inductivement comme suit : la racine est à profondeur 0, et pour $h \geq 0$, les nœuds à profondeur $h + 1$ sont les enfants des nœuds à profondeur h .

L'algorithme de recherche de chemins d'augmentation prend en entrée un graphe G et un couplage C dans G . Il manipule un ensemble de sommets T et peut être décrit comme suit.

- (1) On construit une forêt F dont les nœuds consistent exactement en une racine u pour chaque sommet u de G non couvert par C .
- (2) On fixe $T = \emptyset$.
- (3) Tant qu'il existe un sommet $u \notin T$ tel que u apparait à profondeur paire dans F :

On ajoute u à l'ensemble T .

Pour chaque voisin v de u dans G tel que $v \notin T$:

- (a) Si v n'est pas dans la forêt F , il est couvert par une arête $\{v, z\}$ de C , et on ajoute les arêtes $\{u, v\}$ et $\{v, z\}$ à F .
- (b) Si v est dans la forêt F :
 - (i) Si u, v apparaissent à profondeurs paires dans des arbres distincts de racines respectives r_u, r_v , alors on renvoie la suite des sommets de G vus dans F en allant de r_u à u (r_u et u inclus), puis en allant de v à r_v (v et r_v inclus).
 - (ii) Si u et v apparaissent dans le même arbre à profondeurs paires, alors on construit le bourgeon B correspondant, et on cherche un chemin d'augmentation pour le couplage C/B dans le graphe G/B . Si on en trouve un, alors on renvoie un chemin d'augmentation pour C dans G .

Question 16. Montrer que la suite de sommets renvoyée par l'algorithme dans le cas (3)(b)(i) est un chemin d'augmentation pour C dans G .

On admet dans toute la suite que l'algorithme ci-dessus renvoie un chemin d'augmentation pour C si et seulement s'il en existe un.

Nous allons étudier une implémentation OCaml de cet algorithme. On suppose données les fonctions suivantes.

- La fonction `frais: int list -> int` renvoie un entier n'appartenant pas à la liste donnée en argument.
- La fonction `prochain: foret -> int list -> int option` prend en arguments une forêt `f` et une liste de sommets `lst`. Un appel `prochain f lst` renvoie `Some d` où `d` est un entier apparaissant dans `f` à profondeur paire et n'appartenant pas à `lst` si un tel entier `d` existe. Sinon, `prochain f lst` renvoie `None`.
- La fonction `apparie: couplage -> int -> int` prend en arguments un couplage `cpl` et un sommet `u` couvert par le couplage, et renvoie le sommet apparié, c'est-à-dire l'unique `v` tel que $(\min u v, \max u v)$ appartient à `cpl`.
- La fonction `gonfle: graphe -> int list -> int list -> int -> int list` prend en arguments un graphe `grph`, un bourgeon `brg`, un chemin d'augmentation dans le graphe contracté `grph/brg`, et le nom du nouveau sommet `nv`, et renvoie le chemin d'augmentation correspondant dans `grph`.

La figure 1 page 11 représente une implémentation *incomplète* d'une fonction

`recherche : graphe -> couplage -> (int list) option`

Les questions ci-dessous demandent de donner le code de la ligne 15, des lignes 17–18 et des lignes 24–26, ainsi que d'implémenter la fonction `extrait` (ligne 20).

La fonction `recherche` prend en arguments un graphe et un couplage. Elle doit renvoyer `Some p` s'il existe un chemin d'augmentation `p`, et renvoyer `None` sinon. Certaines fonctions appelées dans le code de `recherche` peuvent lever des exceptions, mais on admettra que ces exceptions ne sont jamais levées si les données d'entrée sont bien formées.

Question 17. Donner le code devant apparaître à la ligne 15 de la figure 1.

Question 18. Donner le code devant apparaître dans le bloc aux lignes 17–18 de la figure 1.

Question 19. Écrire une fonction `extrait: int list -> int list -> int list` telle que l'appel `extrait cu cv` ligne 20 renvoie le bourgeon associé aux chemins `cu` et `cv`.

Question 20. Donner le code devant apparaître dans le bloc aux lignes 24–26 de la figure 1.

Question 21. Écrire une fonction `edmonds: graphe -> couplage` qui prend en entrée un graphe et qui renvoie un couplage maximal.

```

1  let rec recherche graphe couplage =
2      let nc = del (keys graphe) (couverts couplage) in
3      let foret = ref (List.map (fun x -> N(x,[])) nc) in
4      let rec aux_voisins u liste_voisins =
5          let cu = Option.get (find !foret u) in
6          match liste_voisins with
7          | [] -> None
8          | v :: tl ->
9              match find !foret v with
10             | None -> let z = apparie couplage v in
11                 foret := extend (extend !foret u v) v z;
12                 aux_voisins u tl
13             | Some cv when (List.length cv) mod 2 = 0
14                 (* nombre de sommets pair = profondeur impaire *)
15                 -> ...
16             | Some cv when (List.hd cu) <> (List.hd cv)
17                 -> ...
18                 ...
19             | Some cv
20                 -> let bourgeon = extrait cu cv in
21                     let nv = frais (keys graphe) in
22                     let ng = contracteG graphe bourgeon nv in
23                     let nc = contracteC couplage bourgeon nv in
24                     ...
25                     ...
26                     ...
27      in
28      let rec aux_sommets traitees =
29          match prochain !foret traitees with
30          | None -> None
31          | Some u -> let liste_voisins = del (List.assoc u graphe) traitees in
32              match aux_voisins u liste_voisins with
33              | None -> aux_sommets (u::traitees)
34              | Some ch -> Some ch
35      in
36      aux_sommets []

```

FIGURE 1 – Fonction recherche.

Partie II : Calculs de déterminants

Cette partie concerne l'algorithme de Mahajan-Vinay pour le calcul de déterminants de matrices carrées. Elle est à composer dans le langage C.

Préliminaires : matrices linéarisées

On travaille avec des matrices carrées de taille arbitraire indexées à partir de 0. On utilise une représentation linéarisée de ces matrices. Une matrice A de dimension $n \times n$ (c'est-à-dire avec n lignes et n colonnes) est représentée par un tableau A avec n^2 éléments de manière à ce que le coefficient $A_{i,j}$ de la matrice (indice de ligne i et indice de colonne j) corresponde à l'élément $A[i*n+j]$ du tableau.

On suppose dans la suite que les opérations arithmétiques ont une complexité en $O(1)$.

Question 22. Écrire des fonctions

```
int read_sqmatrix (int n, int *A, int i, int j)
void write_sqmatrix (int n, int *A, int i, int j, int val)
```

telles que

- `read_sqmatrix(n,A,i,j)` renvoie la valeur du coefficient $A_{i,j}$ où A est la matrice $n \times n$ représentée par A ;
- `write_sqmatrix(n,A,i,j,val)` modifie la valeur du coefficient $A_{i,j}$ pour qu'il soit égal à `val`.

On supposera que A est un tableau avec n^2 éléments et que $i, j \in \{0, \dots, n-1\}$.

On attend une complexité en $O(1)$ pour ces fonctions, sans la justifier.

Algorithme de Mahajan-Vinay

L'algorithme de Mahajan-Vinay est basé sur une formulation du déterminant d'une matrice A en termes de **suites de marches fermées** dans un graphe associé à A .

Soit A une matrice carrée de dimension $n \times n$. On associe à A son **graphe d'adjacence** $G(A)$. Le graphe $G(A)$ est **orienté et pondéré**. Il est défini comme suit :

- les sommets de $G(A)$ sont les entiers de 0 à $n-1$,
- les arcs sont les paires ordonnées $e = (i, j)$ telles que $A_{i,j} \neq 0$,
- le poids $\omega(e)$ d'un arc $e = (i, j)$ est égal à $A_{i,j}$.

Les sommets de $G(A)$ sont donc des entiers. L'ordre usuel sur ces entiers est utilisé dans la suite de cette partie.

Question 23. Dessiner le graphe d'adjacence de la matrice suivante :

$$\begin{pmatrix} 2 & 3 \\ -2 & 1 \end{pmatrix}$$

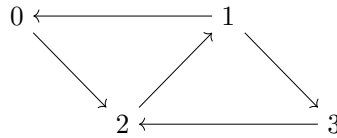
Une **marche fermée** dans le graphe $G(A)$ est une suite de sommets $C = v_0 \dots v_m$ avec $m > 0$ et qui satisfait les conditions suivantes :

- pour tout $i = 0, \dots, m-1$, il existe un arc (v_i, v_{i+1}) dans $G(A)$;
- le sommet $v_0 = v_m$ est le plus petit entier de la suite, appelé la **tête** de C , notée $t(C)$;
- le sommet v_0 n'apparaît qu'au début et à la fin de la suite : pour tout $i = 1, \dots, m-1$, on a $v_i \neq v_0$.

La **longueur** $|C|$ de la marche fermée C est le nombre d'arcs qui la composent. La longueur de $C = v_0 \dots v_m$ est donc égale à m . Le **poids** $\omega(C)$ d'une marche fermée C est le produit des poids des arcs (avec multiplicités) qui la composent :

$$\omega(C) = \prod_{i=0}^{m-1} \omega(v_i, v_{i+1}).$$

Exemple. Considérons le graphe suivant, dont les arcs ont tous poids 1 :



Dans ce graphe, les suites de sommets suivantes sont des marches fermées :

$$0 \rightarrow 2 \rightarrow 1 \rightarrow 0 \quad \text{et} \quad 0 \rightarrow 2 \rightarrow 1 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0.$$

On remarque qu'il est possible de passer plusieurs fois par le même sommet. Cependant la suite

$$0 \rightarrow 2 \rightarrow 1 \rightarrow 0 \rightarrow 2 \rightarrow 1 \rightarrow 0$$

n'est pas une marche fermée car la tête ne doit apparaître qu'au début et à la fin. La suite

$$2 \rightarrow 1 \rightarrow 0 \rightarrow 2$$

n'est pas non plus une marche fermée car la tête n'est pas le plus petit sommet.

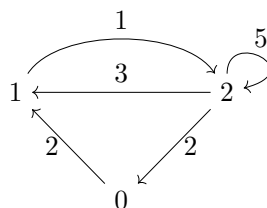
Question 24. Donner une marche fermée de longueur 3 et de tête 1 dans le graphe ci-dessus.

Une **suite de marches fermées** est une suite S de marches fermées $C_1, \dots, C_k$ telle que :

- les têtes sont croissantes : $t(C_1) < t(C_2) < \dots < t(C_k)$;
- le nombre total d'arcs (en comptant les multiplicités) est égal au nombre de sommets n de $G(A)$: $\sum_{i=1}^k |C_i| = n$.

Le **poids** d'une suite de marches fermées est le produit des poids des marches fermées qui la composent : $\omega(S) = \prod_{i=1}^k \omega(C_i)$. La **tête** de S , notée $t(S)$, est la tête de la première marche fermée de S : $t(S) = t(C_1)$.

Question 25. On considère le graphe suivant :



- (1) Donner les marches fermées de longueur au plus 3 et leurs poids.
- (2) Donner les suites de marches fermées et leurs poids.

On définit aussi le **signe** d'une suite de marches fermées. Si $S = C_1 \dots C_k$, alors on pose :

$$\text{sgn}(S) = (-1)^{n+k}.$$

On dit que S est une suite de marches fermées **positive** lorsque $\text{sgn}(S) = 1$, et que S est une suite de marches fermées **négative** lorsque $\text{sgn}(S) = -1$.

Le résultat fondamental de Mahajan et Vinay est que le déterminant d'une matrice carrée A peut être exprimé comme suit :

$$\det(A) = \sum_{\substack{S \text{ suite de marches fermées} \\ \text{dans } G(A)}} \text{sgn}(S) \omega(S).$$

Soit A une matrice avec n lignes et n colonnes. Pour implémenter le calcul du déterminant de A selon la formule ci-dessus, Mahajan et Vinay introduisent un nouveau graphe orienté et pondéré H_A , que nous allons maintenant décrire.

Le graphe H_A possède trois sommets distingués : s , t_0 , et t_1 . La construction de H_A permet d'assurer que les chemins de s à t_0 (respectivement t_1) dans H_A correspondent aux suites de marches fermées positives (respectivement négatives) dans $G(A)$. Les autres sommets sont des quadruplets $\langle p, h, u, i \rangle$ avec $p \in \{0, 1\}$, $0 \leq h \leq u \leq n-1$, et $i \in \{0, \dots, n-1\}$. Ceux-ci représentent des étapes de tentatives de construction de suites de marches fermées. Le sommet $\langle p, h, u, i \rangle$ représente le cas où des marches fermées $C_1, \dots, C_k$ ont été construites, et où on cherche à construire une marche fermée commençant par $C_{k+1} = v_0, v_1, \dots, v_m$, où

- le signe de la suite de marches fermées $C_1, \dots, C_k$ est $(-1)^p$;
- h est la tête de la marche fermée en cours de construction, c'est-à-dire $h = v_0$;
- u est le sommet courant, c'est-à-dire $u = v_m$;
- i est le nombre d'arcs parcourus jusque là, c'est-à-dire $i = m + \sum_{j=1}^k |C_j|$.

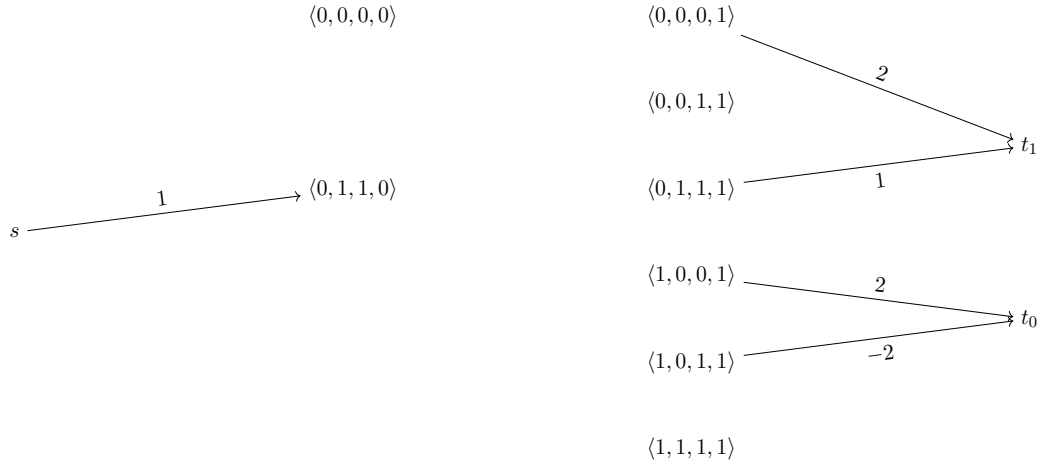
Le graphe H_A a les arcs suivants.

- (1) Un arc de poids 1 de s vers chaque sommet de la forme $\langle n \bmod 2, h, h, 0 \rangle$. Ces arcs permettent d'initialiser les suites de marches fermées.
- (2) Pour chaque coefficient $A_{u,v} \neq 0$, chaque $h < v$ et chaque $i < n-1$, un arc de poids $A_{u,v}$ de $\langle p, h, u, i \rangle$ vers $\langle p, h, v, i+1 \rangle$. Ces arcs correspondent à l'extension de la marche fermée en cours de construction (la marche C_{k+1} dans l'explication ci-dessus).
- (3) Pour chaque coefficient $A_{u,h} \neq 0$, chaque $h' \in \{h+1, \dots, n-1\}$, et chaque $i < n-1$, un arc de poids $A_{u,h}$ de $\langle p, h, u, i \rangle$ vers $\langle 1-p, h', h', i+1 \rangle$. Ces arcs correspondent à la fermeture de la marche fermée en cours de construction, et à l'initialisation d'une nouvelle marche fermée de tête h' .
- (4) Pour chaque coefficient $A_{u,h} \neq 0$ et chaque $p \in \{0, 1\}$, un arc de poids $A_{u,h}$ de $\langle p, h, u, n-1 \rangle$ vers t_{1-p} . Ces arcs correspondent à la fermeture de la dernière marche fermée de la suite.

Question 26. On considère la matrice A suivante :

$$\begin{pmatrix} 2 & 3 \\ -2 & 1 \end{pmatrix}$$

La figure ci-dessous représente certains sommets du graphe H_A correspondant, ainsi que certains arcs. Indiquer les arcs manquants entre les sommets représentés, ainsi que leurs poids.



Question 27. Soit A une matrice carrée avec n lignes et n colonnes. Montrer que si $C = h, u_1, \dots, u_m, h$ est une marche fermée dans $G(A)$, alors

$$\langle p, h, h, i \rangle \rightarrow \langle p, h, u_1, i+1 \rangle \rightarrow \dots \rightarrow \langle p, h, u_m, i+m \rangle$$

est un chemin dans H_A pour tout $p \in \{0, 1\}$ et tout $i \in \{0, \dots, n-1-m\}$.

En déduire qu'à chaque suite de marches fermées positive dans $G(A)$ correspond un chemin de s à t_0 dans H_A .

On peut démontrer que les chemins de s à t_0 dans H_A sont en fait en bijection avec les suites de marches fermées positives dans $G(A)$. Dans toute la suite, on admet ce résultat ainsi que le résultat analogue pour les chemins de s à t_1 dans H_A et les suites de marches fermées négatives dans $G(A)$.

Nous ne construirons pas le graphe H_A mais travaillerons de manière implicite avec celui-ci. Afin de calculer le déterminant de la matrice A en utilisant la formule de Mahajan-Vinay, nous allons calculer la somme des poids des chemins de s vers t_0 et t_1 dans H_A . Soit F_A la fonction qui à chaque sommet x de H_A associe la somme des poids des chemins de s à x dans H_A .

Nous allons commencer par calculer la valeur de $F_A(x)$ pour l'ensemble des sommets de la forme $x = \langle p, h, v, i \rangle$. On remarque que les sommets du graphe H_A peuvent être organisés en couches selon la valeur du dernier entier. Formellement, la **couche** i de H_A est l'ensemble des sommets de H_A de la forme $\langle p, h, v, i \rangle$.

Question 28. Exprimer la valeur de $F_A(\langle p, h, v, i \rangle)$ en fonction des valeurs de la couche $i-1$, c'est-à-dire des $F_A(\langle p', h', v', i-1 \rangle)$.

Implémentation

On veut représenter $F_A(x)$ pour chaque x de la forme $\langle p, h, v, i \rangle$. Pour cela, on suppose donnés les types et fonctions C suivants.

- Un type `Fourtable`.
- Une fonction `Fourtable *create (int n)`. Un appel `create(n)` alloue dans le tas une valeur de type `Fourtable` et renvoie un pointeur `t` vers celle-ci. La valeur `*t` ainsi créée a la taille nécessaire pour enregistrer les valeurs de F_A lorsque A est une matrice $n \times n$. Les valeurs contenues dans `*t` sont toutes initialisées à 0. On supposera que `free(t)` a une complexité en $O(1)$.
- Une fonction `void write(Fourtable *t, int val, int p, int h, int v, int i)` qui enregistre dans `*t` l'entier `val` comme valeur correspondant au sommet $\langle p, h, v, i \rangle$.
- Une fonction `int read(Fourtable *t, int p, int h, int v, int i)` qui renvoie la valeur correspondant au sommet $\langle p, h, v, i \rangle$ dans `*t`.

On suppose de plus que les fonctions `read` et `write` ont une complexité en $O(1)$, et que la fonction `create` a une complexité en $O(n^3)$, où n est la valeur donnée en argument.

Notons que $F_A(\langle p, h, u, 0 \rangle)$ ne dépend que de p, h, u et de la dimension de la matrice A .

Question 29. *Écrire une fonction `Fourtable *initialise (int n)` telle que `initialise(n)` renvoie un pointeur vers un `Fourtable` contenant les valeurs de F_A pour tous les sommets de la couche 0 de H_A , où A est une matrice $n \times n$.*

On attend une complexité en $O(n^3)$ pour cette fonction, sans la justifier.

Question 30. *Écrire une fonction `Fourtable *remplit (int n, int *A)` qui prend en entrée la représentation linéarisée A d'une matrice A de dimension $n \times n$, et renvoie un pointeur vers un `Fourtable` contenant les valeurs de F_A pour tous les sommets de la forme $\langle p, h, v, i \rangle$ avec $p \in \{0, 1\}$, $0 \leq h \leq v \leq n - 1$, et $0 \leq i \leq n - 1$.*

On attend une complexité $O(n^4)$ pour cette fonction, qu'il faudra justifier.

Question 31. *Écrire une fonction `int determinant (int n, int *A)` qui prend en entrée la représentation linéarisée A d'une matrice A de dimension $n \times n$ et renvoie son déterminant. On veillera à libérer correctement la mémoire allouée dans le tas.*

On attend une complexité en $O(n^4)$ pour cette fonction, qu'il faudra justifier.

Partie III : Méthode algébrique et probabiliste pour les couplages parfaits

Cette partie concerne un algorithme qui teste si un graphe possède un couplage parfait, c'est-à-dire un couplage qui couvre tous ses sommets. Elle est à composer dans le langage C. On pourra réutiliser le matériel de la partie précédente.

On ne considère que des graphes **non orientés** (voir page 3). Un tel graphe G d'ensemble de sommets $\{0, \dots, n-1\}$ est représenté par sa **matrice d'adjacence** A , de dimension $n \times n$, et dont les coefficients sont donnés par

$$A_{i,j} = \begin{cases} 1 & \text{si } \{i, j\} \text{ est une arête de } G, \\ 0 & \text{si } \{i, j\} \text{ n'est pas une arête de } G. \end{cases}$$

Nous allons utiliser des méthodes algébriques, basées sur le déterminant de la matrice de Tutte. La matrice de Tutte d'un graphe est une matrice symbolique, et son déterminant est un polynôme réel à plusieurs variables. Un polynôme réel à m variables $X_1, \dots, X_m$ est une somme finie de monômes, c'est-à-dire de termes de la forme $aX_1^{d_1} \dots X_m^{d_m}$ avec $a \in \mathbb{R}$ et $d_1, \dots, d_m \in \mathbb{N}$. Lorsque $d_i = 0$, la variable X_i est omise dans $aX_1^{d_1} \dots X_m^{d_m}$. Par exemple, le monôme $aX_1^0 \dots X_m^0$ est noté a .

Considérons un graphe non orienté G dont l'ensemble de sommets est $\{0, \dots, n-1\}$. La **matrice de Tutte** $T(G)$ de G est une matrice $n \times n$ dont les éléments sont des monômes sur l'ensemble de variables $\{X_{i,j} \mid 0 \leq i < j \leq n-1\}$. Les coefficients de $T(G)$ sont donnés par

$$T(G)_{i,j} = \begin{cases} 0 & \text{si } i = j \text{ ou si } \{i, j\} \text{ n'est pas une arête de } G, \\ X_{i,j} & \text{si } i < j \text{ et } \{i, j\} \text{ est une arête de } G, \\ -X_{j,i} & \text{si } i > j \text{ et } \{i, j\} \text{ est une arête de } G. \end{cases}$$

Un théorème de Tutte énonce que G possède un couplage parfait si, et seulement si, le déterminant de $T(G)$ est différent du polynôme nul (c'est-à-dire du polynôme dont toutes les valeurs sont 0).

Une partie du problème consiste donc à décider si un polynôme est nul ou non. Nous allons implémenter un algorithme **probabiliste** qui teste si un polynôme s'annule en un certain nombre de points choisis aléatoirement.

Dans la suite, on suppose donnée une fonction `int rand_range(int n)` qui prend en entrée un entier `n` et renvoie un entier tiré aléatoirement entre 0 et `n` (inclus) selon la distribution uniforme. On suppose que cette fonction s'exécute en temps constant.

On suppose que l'instruction suivante, qui crée un tableau d'entiers `t` de longueur `m`, s'exécute en temps $O(m)$:

```
int t [m];
```

Question 32. Écrire une fonction `int tirage_tutte(int n, int *A)` qui prend en entrée la représentation linéarisée `A` d'une matrice d'adjacence A de dimension $n \times n$, et renvoie le

déterminant d'une instantiation de la matrice de Tutte du graphe représenté par A obtenue en tirant aléatoirement les valeurs des variables $X_{i,j}$ dans l'ensemble $\{0, 1, 2, \dots, \mathbf{n}*\mathbf{n} - 1, \mathbf{n}*\mathbf{n}\}$. On attend une complexité en $O(\mathbf{n}^4)$ pour cette fonction, sans la justifier.

On va utiliser la fonction définie ci-dessus pour décider si un polynôme réel à plusieurs variables est nul ou non. On se propose d'appliquer le lemme de Schwartz-Zippel. Ce résultat permet, pour un polynôme non nul P , de borner le nombre de zéros de P (dans un ensemble fini donné) en fonction du degré total de P (et de la taille de cet ensemble). Le degré total d'un monôme $aX_1^{d_1} \dots X_n^{d_n}$ est la somme $d_1 + \dots + d_n$. Le **degré total** d'un polynôme est le degré total maximal des monômes qui le composent.

Soit P un polynôme réel non nul à m variables et de degré total $d \geq 0$, et soit S un sous-ensemble fini non vide de $\mathbb{R}$. On note $\Pr[P(r_1, r_2, \dots, r_m) = 0]$ la probabilité pour que P s'annule en $r_1, r_2, \dots, r_m$, où $r_1, r_2, \dots, r_m$ sont des réels tirés aléatoirement dans S , de manière uniforme et indépendante. Le lemme de Schwartz-Zippel énonce que

$$\Pr[P(r_1, r_2, \dots, r_m) = 0] \leq \frac{d}{|S|}.$$

On souhaite obtenir un algorithme probabiliste décidant si un graphe possède un couplage parfait. En s'aidant du lemme de Schwartz-Zippel, on pourra assurer la correction de cet algorithme et en borner l'erreur. On pourra utiliser la fonction `log2` définie lorsque l'entête `<math.h>` est incluse.

Question 33. *Écrire une fonction*

`bool parfait (int n, int *A, int p)`

*qui prend en entrée la matrice d'adjacence linéarisée A d'un graphe G ayant $\mathbf{n}$ sommets, et décide si G possède un couplage parfait avec une probabilité d'erreur inférieure à 2^{-p} , **qu'il faudra justifier**.*

*On attend une complexité en $O\left(p \frac{\mathbf{n}^4}{\log_2(\mathbf{n})}\right)$, **qu'il faudra justifier**.*

Fin du sujet.

A2025 – INFO I MPI

**ÉCOLE NATIONALE DES PONTS et CHAUSSÉES,
ISAE-SUPAERO, ENSTA PARIS,
TÉLÉCOM PARIS, MINES PARIS,
MINES SAINT-ÉTIENNE, MINES NANCY,
IMT ATLANTIQUE, ENSAE PARIS,
CHIMIE PARISTECH - PSL.**

**Concours Mines-Télécom,
Concours Centrale-Supélec (Cycle International).**

CONCOURS 2025**PREMIÈRE ÉPREUVE D'INFORMATIQUE**

Durée de l'épreuve : 3 heures

L'usage de la calculatrice ou de tout dispositif électronique est interdit.

*Les candidats sont priés de mentionner de façon apparente
sur la première page de la copie :*

INFORMATIQUE I - MPI

Cette épreuve concerne uniquement les candidats de la filière MPI.

L'énoncé de cette épreuve comporte 13 pages de texte.

Si, au cours de l'épreuve, un candidat repère ce qui lui semble être une erreur d'énoncé, il le signale sur sa copie et poursuit sa composition en expliquant les raisons des initiatives qu'il est amené à prendre.

Les sujets sont la propriété du GIP CCMP. Ils sont publiés sous les termes de la licence
Creative Commons Attribution - Pas d'Utilisation Commerciale - Pas de Modification 3.0 France.

Tout autre usage est soumis à une autorisation préalable du Concours commun Mines-Ponts.



Étude des tableaux associatifs

Préliminaires

L'épreuve est composée d'un problème unique, comportant 34 questions. Le problème consiste en l'étude et l'implémentation d'un tableau associatif en C, suivant différentes méthodes.

Le problème est divisé en 4 sections distinctes :

- l'implémentation du tableau associatif en utilisant une structure d'arbre (section 1),
- l'ajout d'un « ramasse miettes » pour gérer la mémoire (section 2),
- l'implémentation du tableau associatif en utilisant un algorithme probabiliste (section 3),
- la conception d'un analyseur syntaxique pour créer un tableau associatif à partir d'une chaîne de caractères (section 4).

Les quatre sections sont indépendantes.

Travail attendu

Les résultats attendus pour certaines questions pourront être admis afin de faciliter la résolution des questions suivantes.

Le langage C constitue l'unique langage autorisé pour le traitement des questions de programmation.

Une attention particulière sera portée quant à la correction et la lisibilité du code proposé. Si une tolérance raisonnable peut être envisagée sur certains aspects syntaxiques dans le cadre d'une épreuve réalisée sur support papier (par exemple, l'oubli d'un « ; »), un code difficilement lisible ou l'emploi de fonctions ou structures de contrôle non conformes aux standards du langage C, notamment celles empruntées à d'autres langages, sera sanctionné.

Un soin particulier sera également apporté à la gestion de la mémoire, à la bonne initialisation des structures ainsi qu'au contrôle systématique des allocations mémoires.

La gestion rigoureuse des erreurs sera valorisée (l'utilisation d'assert est recommandée), de même que la qualité, la simplicité et la lisibilité du code.

En conformité avec le programme officiel, il sera admis que certains en-têtes standards soient pré-supposés comme inclus dans le code : `<assert.h>`, `<stdbool.h>`, `<stddef.h>`, `<stdint.h>`, `<stdio.h>`, `<stdlib.h>`, et `<string.h>`.

Les tableaux associatifs

Les tableaux associatifs sont des structures de données permettant un accès rapide à une *valeur* à partir d'une *clef*.

Afin de simplifier l'écriture du code, nous considérerons que les clefs sont toujours de type `char*` et les valeurs de type `int`.

1. Implémentation par un arbre binaire de recherche

Le but de cette section est d'étudier une implémentation d'un tableau associatif reposant sur un arbre binaire de recherche.

Nous adopterons une approche fonctionnelle pour l'implémentation des structures de données, en les considérant comme immuables. Ainsi, chaque fonction de manipulation renverra systématiquement une nouvelle version modifiée de la structure, sans altérer l'originale.

Cette méthode présente l'avantage d'éliminer les effets de bord lors des appels de fonction. Toutefois, elle implique une augmentation du nombre d'allocations mémoire, puisque chaque modification nécessite la création d'une copie de la structure de données.

Dans cette section nous ne nous occuperons pas de la récupération de la mémoire, car nous introduirons en section 2 un « ramasse miettes » pour gérer la mémoire automatiquement.

Arbre binaire de recherche

Définitions : Soit A un arbre binaire de recherche. On notera avec une lettre minuscule (par exemple x ou y) un nœud de A , et avec une lettre majuscule (par exemple T_1 , T_2 ou T_3) un sous-arbre de A . Soit x un nœud, on notera x_{gauche} et x_{droit} , respectivement le sous-arbre gauche et le sous-arbre droit de x .

On rappelle que la taille d'un arbre est égale au nombre de nœuds qu'il contient.

Pour un arbre de racine x , on notera $h(x)$ sa hauteur, et on définit récursivement la hauteur d'un arbre par :

- la hauteur d'un arbre vide est -1 ,
- $h(x) = \max(h(x_{\text{gauche}}), h(x_{\text{droit}})) + 1$.

Pour un arbre de racine x , on définit la valeur d'équilibre par :

$$e(x) = h(x_{\text{gauche}}) - h(x_{\text{droit}})$$

On qualifiera d'*équilibré* un arbre binaire de recherche A si pour tout nœud x de A ,

$$-1 \leq e(x) \leq 1$$

Définition : On définit un nœud d'un arbre binaire de recherche par la structure suivante :

```

1. struct abrNoeud_s {
2.     char* clef;
3.     int valeur;
4.
5.     struct abrNoeud_s* fils_gauche;
6.     struct abrNoeud_s* fils_droit;
7. };
8. typedef struct abrNoeud_s abrNoeud;
```

On utilisera la valeur du champ `clef` pour ordonner les nœuds de l'arbre, en utilisant l'ordre alphabétique.

□ 1 – Nous nous intéressons aux valeurs du champ `clef` de la structure ci-dessus. Expliquer comment une chaîne de caractères est représentée en mémoire en C ? Combien d'octets sont nécessaires pour représenter la chaîne "arbre" par exemple ?

Première épreuve d'informatique MPI 2025

□ 2 – Sachant que le code ASCII du caractère ‘a’ est 97, quelle est sa représentation en binaire? Quelle est sa représentation en hexadécimal?

□ 3 – Écrire une fonction C `copie_chaine` qui renvoie une copie de la chaîne de caractères passée en paramètre. La signature de la fonction est :

```
char* copie_chaine(char* chaine)
```

Indication C : On considèrera que la chaîne de caractères passée en paramètre (`chaine`) n’est jamais NULL.

□ 4 – Écrire une fonction C `abr_creer_noeud` qui crée un nœud à partir d’une clef, d’une valeur et de 2 sous-arbres donnés, et qui renvoie ce nouveau nœud créé. La signature de la fonction est :

```
abrNoeud* abr_creer_noeud(char* clef, int valeur,  
                           abrNoeud* fils_gauche,  
                           abrNoeud* fils_droit)
```

Indication C : le champ `clef` du nœud créé doit contenir une copie de la chaîne de caractères passée en paramètre. Par contre, il n’est pas nécessaire de dupliquer les sous-arbres passés en paramètre.

□ 5 – Écrire une fonction C `abr_equilibre` qui renvoie la valeur d’équilibre de l’arbre dont le nœud racine est passé en paramètre. La signature de la fonction est :

```
int abr_equilibre(abrNoeud* noeud)
```

□ 6 – Pour un arbre de taille N , quelle est la complexité de la fonction `abr_equilibre`? Sans proposer de code, expliquer quelles seraient les modifications à apporter à la structure de donnée et/ou à l’algorithme pour améliorer cette complexité?

□ 7 – Soit A un arbre binaire de recherche équilibré et de hauteur h , quelle approximation peut-on faire du nombre minimal N de nœuds que peut contenir cet arbre?

□ 8 – En déduire une borne supérieure de la hauteur h d’un arbre binaire de recherche équilibré de taille N .

□ 9 – Écrire une fonction C récursive `abr_rechercher` qui, étant donné un arbre dont le nœud racine est passé en paramètre ainsi qu’une clef, renvoie la valeur associée à cette dernière. La signature de la fonction est :

```
int abr_rechercher(abrNoeud* noeud, char* clef)
```

Indication : On considèrera que la clef donnée est toujours dans l’arbre binaire de recherche.

Indication C : l’opérateur `==` permet de comparer des entiers ou des pointeurs mais pas de comparer le contenu de deux chaînes de caractères.

Première épreuve d'informatique MPI 2025

- ☐ 10 – En application de la question 8, quelle est la complexité de la fonction `abr_rechercher` pour un arbre équilibré de taille N ?
- ☐ 11 – Quel est l'intérêt d'avoir un arbre binaire de recherche qui soit tout le temps équilibré ?

2. Ajout d'un ramasse miettes

Un « ramasse miettes » est un ensemble de fonctions qui permet de gérer la mémoire de manière automatique. L'idée est de libérer la mémoire qui avait été allouée dynamiquement pour des structures ou des tableaux lorsque ces derniers ne sont plus utilisés.

Dans un contexte fonctionnel où les données sont immuables, il est parfois compliqué de déterminer le moment opportun pour libérer la mémoire allouée aux nœuds. En effet, l'immuabilité des structures permet de partager le même sous-arbre à plusieurs endroits d'un arbre, sans indication explicite de ce partage, rendant ainsi difficile de savoir si un nœud doit être libéré ou non. L'utilisation d'un ramasse miettes dans un tel cas est donc tout indiquée. C'est ce que nous allons étudier dans cette section.

Pour simplifier le problème, nous considérons qu'une structure ou un tableau devient inutilisé dès lors qu'il n'est plus atteignable à partir des variables globales du programme. Autrement dit, en partant des variables globales, il est impossible d'y accéder en suivant des pointeurs, même de manière transitive.

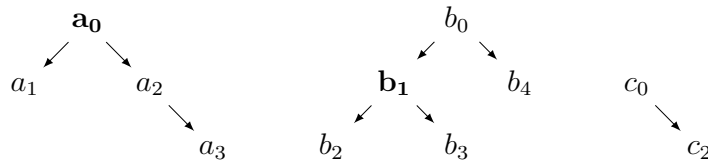
Pour pouvoir décrire l'algorithme de ramasse miettes qui nous intéresse, nous avons besoin de considérer qu'un nœud peut être « marqué ». Cela consiste à mémoriser une information dans la structure elle-même. Dans notre situation, nous utiliserons la structure suivante :

```
1. struct abrNoeud_s {  
2.     char* clef;  
3.     int valeur;  
4.  
5.     struct abrNoeud_s* fils_gauche;  
6.     struct abrNoeud_s* fils_droit;  
7.  
8.     // partie propre au ramasse-miettes  
9.     bool marque;  
10. };  
11. typedef struct abrNoeud_s abrNoeud;
```

Un nœud est alors dit « marqué » si et seulement si le champ `marque` a pour valeur `true`. Nous considérons qu'initialement aucun nœud n'est marqué. L'algorithme de ramasse miettes fonctionne en deux temps :

1. Marquer tous les nœuds atteignables par clôture transitive depuis les variables globales. C'est-à-dire tous les nœuds atteignables directement ou bien atteignables à partir d'un nœud lui-même atteignable.
2. Libérer tous les nœuds alloués mais non marqués et effacer la marque des nœuds marqués.

Première épreuve d'informatique MPI 2025

FIGURE 1 – Exemples d’arbres binaires de recherche, où les a_i , b_i , c_i sont des nœuds.

□ 12 – On considère que *seuls* a_0 et b_1 sont directement référencés par des variables globales (Figure 1). Quels sont les nœuds marqués par l’étape 1 de l’algorithme ? Quels nœuds seront libérés par le ramasse miettes ?

En déduire (sans démonstration) la complexité en temps de l’algorithme de marquage et de l’algorithme de libération.

Pour pouvoir libérer tous les nœuds non marqués il est nécessaire de maintenir une liste des nœuds de l’arbre alloués en mémoire. Nous proposons d’utiliser la structure de données suivante :

```

1. struct rmListeMemoire_s {
2.     abrNoeud* element;
3.     struct rmListeMemoire_s* suivant;
4. };
5. typedef struct rmListeMemoire_s rmListeMemoire;
  
```

□ 13 – Écrire une fonction C permettant d’insérer, en temps constant, un nœud d’arbre binaire de recherche dans une liste. La signature de la fonction est :

```

rmListeMemoire* rm_inserer_tete(abrNoeud* element,
                                rmListeMemoire* liste)
  
```

Indication : la donnée référencée par *liste* n’a pas besoin d’être modifiée.

Nous considérons maintenant la structure de donnée **RM** permettant de mémoriser dans le champ *arbres* les nœuds racines des différents arbres binaires de recherche accessibles (dans notre cas il peut y avoir jusqu’à 16 nœuds racines) ainsi que la liste des nœuds alloués :

```

1. #define MAX_NB_ARBRES 16
2. struct RM_s {
3.     abrNoeud* arbres[MAX_NB_ARBRES]; /* arbres accessibles */
4.     int nb_elements; /* nombre total de noeuds alloués */
5.     rmListeMemoire* premierElement;
6. };
7. typedef struct RM_s RM;
8.
9. RM* rm; /* variable globale pour l'ensemble du programme */
  
```

Le champ *premierElement* est une référence vers le premier élément d’une liste. Cette dernière est gérée de la façon suivante :

— lorsqu’un nœud d’un arbre binaire de recherche est alloué, il est ajouté à la liste,

Première épreuve d'informatique MPI 2025

— lorsqu'un nœud d'un arbre binaire de recherche est libéré, il est retiré de la liste.

□ 14 – La fonction `main` commence par l'instruction `rm = rm_creer_rm()` ; Écrire le code de la fonction `rm_creer_rm` permettant d'allouer et d'initialiser la structure de données qui est affectée à la variable `rm`. La signature de la fonction est :

RM* `rm_creer_rm(void)`

Le code suivant correspond aux deux étapes de l'algorithme de ramasse miettes :

```
1.  void rm_ramasse_miettes(RM* rm) {  
2.      assert(rm != NULL);  
3.      int nb_noeuds = rm->nb_elements;  
4.      for (int i = 0; i < MAX_NB_ARBRES; i++) {  
5.          if (rm->arbres[i] != NULL) {  
6.              rm_marquer_elements_accessible(rm->arbres[i]);  
7.          }  
8.      }  
9.      rm_recuperer_elements_inaccessibles(rm);  
10.     printf("noeuds collectes : %d", nb_noeuds - rm->nb_elements);  
11.     printf("noeuds encore accessibles : %d\n", rm->nb_elements);  
12. }
```

□ 15 – Écrire une fonction C correspondant à la première étape de l'algorithme : marquer un nœud donné ainsi que tous les nœuds atteignables à partir de ce dernier. La signature de la fonction est :

void `rm_marquer_elements_accessible(abrNoeud* element)`

Première épreuve d'informatique MPI 2025

Pour libérer tous les nœuds non marqués on considère la fonction suivante :

```

1.  void rm_recuperer_elements_inaccessibles(RM* rm) {
2.      assert(rm != NULL);
3.      rmListeMemoire* objet = rm->premierElement;
4.      rmListeMemoire* objets_conservees = NULL;
5.
6.      while (objet != NULL) {
7.          if (!objet->element->marque) {
8.              /* Ce nœud n'a pas été marqué,
9.                 on le retire de la liste et on le libère */
10.             rmListeMemoire* a_detruire = objet;
11.             objet = a_detruire->suivant;
12.             // PARTIE A
13.             // ici il faut libérer la mémoire
14.             // FIN PARTIE A
15.             rm->nb_elements--;
16.         } else {
17.             // PARTIE B
18.             /* Ce nœud a été marqué,
19.                on enlève la marque et on passe au nœud suivant */
20.             objet->element->marque = false;
21.             rmListeMemoire* a_garder = objet;
22.             objet = a_garder->suivant;
23.             // FIN PARTIE B
24.         }
25.     }
26.     rm->premierElement = objets_conservees;
27. }
```

L'algorithme est le suivant :

- on parcourt tous les éléments de la liste premierElement,
- si le nœud n'est pas marqué, il faut libérer la mémoire et enlever le nœud de la liste,
- si le nœud est marqué il faut enlever la marque et le laisser dans la liste.

☐ 16 – Compléter le code de la partie A de la fonction ci-dessus pour libérer la mémoire lorsque le nœud est marqué.

Indication : Seul le code entre *// PARTIE A* et *// FIN PARTIE A* est à compléter. Il n'est pas nécessaire de recopier le code autour.

☐ 17 – Quelle spécificité ont les nœuds de la liste référencée par le champ premierElement à la fin de l'exécution de la fonction ?

☐ 18 – Expliquer pourquoi le code proposé dans la partie B de la fonction ci-dessus n'est pas correct, et proposez une correction.

Indication : Seul le code entre *// PARTIE B* et *// FIN PARTIE B* est à compléter. Il n'est pas nécessaire de recopier le code autour.

3. Implémentation par l'utilisation d'un algorithme probabiliste

Nous allons nous intéresser dans cette section à une autre manière d'implémenter un tableau associatif, en utilisant un algorithme probabiliste : les « *skip lists* ».

3.1. Principe des skip lists

Une skip list est une liste chaînée optimisée pour accélérer la recherche d'un élément. Pour ce faire, certains nœuds stockent dans un tableau des pointeurs vers d'autres nœuds situés plus loin dans la liste. Chaque case du tableau correspond à un « niveau » et un pointeur de niveau i mène vers un nœud qui possède lui aussi un pointeur de niveau i .

Le niveau 0 contient l'ensemble des éléments, classés par ordre alphabétique en fonction de leur clef. Les niveaux supérieurs regroupent un sous-ensemble des éléments, de façon à espacer progressivement les nœuds. Par exemple, idéalement, le niveau 1 contiendrait un nœud sur deux, le niveau 2 un nœud sur quatre, et ainsi de suite (voir Figure 2).

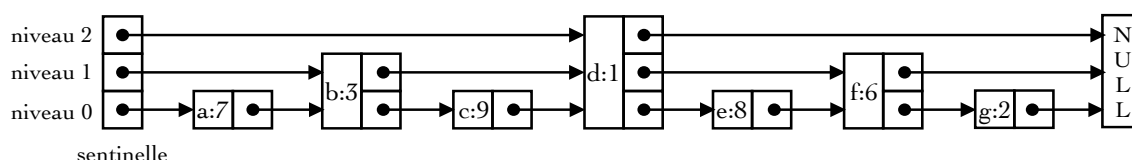


FIGURE 2 – Exemple d'une skip list idéale.

Le premier nœud de la skip list est une sentinelle : un élément spécial qui ne contient pas de donnée utile mais qui possède des pointeurs pour tous les niveaux de la liste.

Pour chercher un élément, on démarre à partir de la sentinelle au niveau le plus élevé. À ce niveau, on suit les pointeurs vers les nœuds suivants tant que la clef du nœud actuel est inférieure à celle recherchée. Dès qu'on rencontre un nœud dont la clef est supérieure, on descend d'un niveau et on répète la même opération. Lorsque la recherche atteint le niveau 0, elle est terminée et, si l'élément recherché y est présent, il est renvoyé (voir Figure 3).

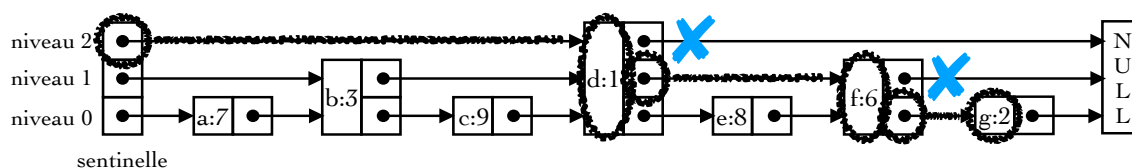


FIGURE 3 – Parcours lors de la recherche de la clef "g".

En pratique, chaque nœud a une probabilité de $\frac{1}{2}$ d'être présent dans le niveau supérieur.

3.2. Implémentation

Définition : On définit une skip list et un nœud d'une skip list par les structures suivantes :

```

1.  struct slNoeud_s {
2.      char* clef;
3.      int valeur;
4.
5.      struct slNoeud_s* suivant_par_niveau[];
6.  };
7.  typedef struct slNoeud_s slNoeud;
8.
9.  struct slListe_s {
10.     int niveau_actuel;
11.     slNoeud* sentinelle;
12.  };
13. typedef struct slListe_s slListe;

```

On définit une constante, `MAX_NIVEAU`, qui correspond au nombre maximal de niveaux qu'un nœud peut avoir. La sentinelle de la skip list est créée comme un nœud particulier, avec une clef vide, une valeur nulle, et un tableau de pointeurs de taille `MAX_NIVEAU+1` (de 0 à `MAX_NIVEAU` inclus) tous initialisés à `NULL`.

□ 19 – Écrire, en suivant l'algorithme décrit section 3.1, une fonction C `sl_rechercher` qui recherche, dans la skip list passée en paramètre, la clef donnée et qui renvoie :

- soit la valeur associée à la clef si elle est présente dans la skip list,
- soit `-1` si la clef n'est pas présente dans la skip list.

La signature de la fonction est :

```
int sl_rechercher(slListe* liste, char* clef)
```

□ 20 – On rappelle que la fonction `rand()` renvoie un entier aléatoire (de type `int`) compris entre 0 et un certain entier supérieur à 32767.

Quelles sont les propriétés de la valeur renvoyée par la fonction suivante ?

```

1.  int sl_mystere() {
2.      int niveau = 0;
3.      while ((rand() % 2) == 1 && niveau < MAX_NIVEAU) {
4.          niveau++;
5.      }
6.      return niveau;
7.  }

```

On suppose que la fonction suivante est disponible :

```
slNoeud* sl_creer_noeud(int niveau, char* clef, int valeur)
```

Cette fonction crée un nouveau nœud de niveau `niveau` avec la clef et la valeur passées en paramètre. Les pointeurs du nouveau nœud sont tous initialisés à `NULL`.

 Première épreuve d'informatique MPI 2025

La fonction `sl_ajoute_valeur`, qui ajoute une clef et une valeur dans la skip list passée en paramètre, a la structure suivante :

```

1.  void sl_ajoute_valeur(slListe* liste, char* clef, int valeur) {
2.      int niveau = sl_mystere();
3.      slNoeud* nouveau_noeud = sl_creer_noeud(niveau, clef, valeur);
4.
5.      slNoeud* noeuds_a_mettre_a_jour[MAX_NIVEAU + 1];
6.      slNoeud* courant = liste->sentinelle;
7.
8.      // PARTIE A
9.      /* parcourt l'ensemble des niveaux de la liste pour trouver
10.         dans chaque niveau le noeud après lequel insérer
11.         le nouveau noeud */
12.      // FIN PARTIE A
13.
14.      if (niveau > liste->niveau_actuel) {
15.          // PARTIE B
16.          /* si le niveau du nouveau noeud est plus grand
17.             que le niveau actuel de la liste
18.             préparer la sentinelle */
19.          // FIN PARTIE B
20.      }
21.
22.      // PARTIE C
23.      /* insérer le nouveau noeud */
24.      // FIN PARTIE C
25.  }
```

☐ 21 – En vous basant sur les instructions du commentaire, écrire le code correspondant à la partie A de la fonction `sl_ajoute_valeur`.

Indication C : Les nœuds devant être mis à jour sont stockés dans le tableau `noeuds_a_mettre_a_jour`.

Indication : Seul le code entre `// PARTIE A` et `// FIN PARTIE A` est à compléter. Il n'est pas nécessaire de recopier le code autour.

☐ 22 – En vous basant sur les instructions du commentaire, écrire le code correspondant à la partie B de la fonction `sl_ajoute_valeur`.

Indication C : On réutilisera la variable `noeuds_a_mettre_a_jour`.

Indication : Seul le code entre `// PARTIE B` et `// FIN PARTIE B` est à compléter. Il n'est pas nécessaire de recopier le code autour.

☐ 23 – En vous basant sur les instructions du commentaire, écrire le code correspondant à la partie C de la fonction `sl_ajoute_valeur`.

Indication : Seul le code entre `// PARTIE C` et `// FIN PARTIE C` est à compléter. Il n'est pas nécessaire de recopier le code autour.

Première épreuve d'informatique MPI 2025

□ 24 – L'algorithme des skip lists est un algorithme probabiliste. Est-ce un algorithme de type « Monte-Carlo » ou de type « Las Vegas » ? Justifier.

□ 25 – En supposant une skip list idéale comme dans la Figure 2, où chaque niveau possède exactement la moitié des éléments du niveau inférieur idéalement répartis, quelle est la complexité en temps de la recherche d'un élément dans une skip list de taille n ?

On considère désormais une skip list non idéale, où les éléments sont répartis de manière aléatoire avec une probabilité $p = \frac{1}{2}$ d'être présent dans le niveau supérieur.

□ 26 – Quelle est l'espérance du nombre d'éléments au niveau k ?

□ 27 – Quelle est l'espérance du niveau le plus haut d'une skip list de taille n ?

On acceptera dans la suite le résultat suivant : cette espérance est proportionnelle à $\log_2(n)$.

□ 28 – Quel est en moyenne le nombre de nœuds parcourus horizontalement dans un niveau k lors d'une recherche dans une skip list de taille n ?

□ 29 – Des questions 27 et 28, déduisez la complexité en moyenne en temps de la recherche d'un élément dans une skip list de taille n .

4. Analyse syntaxique

On souhaite construire une fonction analyse qui prend en paramètre une chaîne de caractères décrivant un tableau associatif sous la forme de couples `clef:valeur`.

Un exemple d'utilisation pourrait être : `analyse("{id:5,valeur:12,ok:0}")`

Le format de la chaîne de caractères passée en paramètre est spécifié par les règles suivantes :

- la chaîne doit commencer par une accolade ouvrante, être suivie d'une liste de couples `clef:valeur`, et terminée par une accolade fermante,
- chaque couple est composé d'une clef et d'une valeur séparées par le caractère `:`,
- ni la clef ni la valeur ne peuvent être absentes,
- la clef est une suite finie non vide de lettres minuscules (de `'a'` à `'z'`),
- la valeur est un entier non signé, sans 0 non significatif,
- les couples sont séparés par le caractère `,`.

Le but de cette partie est d'étudier la grammaire et l'automate de ce langage de description.

Première épreuve d'informatique MPI 2025

Définition : On utilise la notation $x \mid \dots \mid y$ pour indiquer la disjonction de symboles allant de x à y avec x et y des chiffres ou x et y des lettres.

$$N \rightarrow 0 \mid \dots \mid 9$$

On considère connues les règles de production suivantes :

$$M \rightarrow 1 \mid \dots \mid 9$$

$$L \rightarrow a \mid \dots \mid z$$

☐ 30 – Écrire la règle de production C correspondant à la description d'une clef.

☐ 31 – Écrire la règle de production V correspondant à la description d'une valeur.

On considère la grammaire suivante :

$$T \rightarrow \{S\}$$

$$S \rightarrow K \mid K, S$$

$$K \rightarrow C : V$$

☐ 32 – Le mot $\{\text{id:5,valeur:12,ok:0}\}$ appartient-il au langage engendré par cette grammaire? Le cas échéant, dessinez un arbre de dérivation.

Même question pour le mot $\{\text{james:007}\}$.

☐ 33 – Décrire un automate (avec un schéma ou une table de transitions) déterministe et sans transition vide, correspondant au langage reconnu par la règle T . On s'autorisera à étiqueter les transitions avec des disjonctions de lettres ou de chiffres.

☐ 34 – Vérifiez en précisant les étapes de l'automate que les séquences $\{\text{code:102}\}$, $\{\text{v:0}\}$, et $\{\text{a:1,b:2}\}$, sont bien reconnues par l'automate proposé. Dans quel état est l'automate lorsqu'on lui demande de reconnaître la séquence $\{\text{james:007}\}$?

A2025 – INFO II MPI

**ÉCOLE NATIONALE DES PONTS et CHAUSSÉES,
ISAE-SUPAERO, ENSTA PARIS,
TÉLÉCOM PARIS, MINES PARIS,
MINES SAINT-ÉTIENNE, MINES NANCY,
IMT ATLANTIQUE, ENSAE PARIS,
CHIMIE PARISTECH - PSL.**

**Concours Mines-Télécom,
Concours Centrale-Supélec (Cycle International).**

CONCOURS 2025**DEUXIÈME ÉPREUVE D'INFORMATIQUE**

Durée de l'épreuve : 4 heures

L'usage de la calculatrice ou de tout dispositif électronique est interdit.

*Les candidats sont priés de mentionner de façon apparente
sur la première page de la copie :*

INFORMATIQUE II - MPI

Cette épreuve concerne uniquement les candidats de la filière MPI.

L'énoncé de cette épreuve comporte 9 pages de texte.

Si, au cours de l'épreuve, un candidat repère ce qui lui semble être une erreur d'énoncé, il le signale sur sa copie et poursuit sa composition en expliquant les raisons des initiatives qu'il est amené à prendre.

Les sujets sont la propriété du GIP CCMP. Ils sont publiés sous les termes de la licence
Creative Commons Attribution - Pas d'Utilisation Commerciale - Pas de Modification 3.0 France.

Tout autre usage est soumis à une autorisation préalable du Concours commun Mines-Ponts.



Épreuve d'Informatique 2 MPI 2025

Le jeu de Shannon

Préliminaires

L'épreuve est composée d'un problème unique, comportant 33 questions.

Dans ce problème, nous nous intéressons au *jeu de Shannon*, qui est un jeu de stratégie dans lequel l'un des joueurs doit parvenir à établir une certaine connexion en posant des pièces de jeu, tandis que son opposant doit l'en empêcher en essayant de le bloquer. Le problème est divisé en trois sections. Dans la première section (page 1), nous introduisons les règles du jeu et une étude de tournois. Dans la deuxième section (page 5), nous analysons les stratégies de jeu conduisant l'un des joueurs à la victoire. Dans la troisième section (page 7), nous étudions la construction de deux arbres couvrants disjoints dans un graphe, ce qui est un prérequis de l'une des stratégies d'un joueur, vue dans la deuxième section.

Dans tout l'énoncé, un même identificateur écrit dans deux polices de caractères différentes désigne la même entité, mais du point de vue mathématique pour la police en italique (par exemple n) et du point de vue informatique pour celle en romain avec espacement fixe (par exemple `n`).

Travail attendu

Pour répondre à une question, il est permis de réutiliser le résultat d'une question précédente, même sans avoir réussi à établir ce résultat.

Selon les consignes, il faudra coder des fonctions à l'aide du langage de programmation C exclusivement, en reprenant le prototype de fonction fourni par le sujet, ou en pseudo-code (c.-à-d. dans une syntaxe souple mais conforme aux possibilités offertes par le langage C). Inclure les entêtes tels que `<assert.h>`, `<stdbool.h>`, etc., n'est pas demandé.

Quand l'énoncé demande de coder une fonction, sauf indication explicite, il n'est pas nécessaire de justifier que celle-ci est correcte ou de tester que des préconditions sont satisfaites.

Le barème tient compte de la clarté et de la concision des programmes : nous recommandons de choisir des noms de variables intelligibles ou encore de structurer de longs codes par des blocs ou par des fonctions auxiliaires dont on décrira le rôle.

1 Le jeu de Shannon

1.1 Implémentation du jeu en langage C

Un graphe non orienté et connexe $G = (V, E)$ est fixé, ainsi que deux sommets particuliers : le sommet s et le sommet t . La lettre n désigne le nombre de sommets.

Définition : Le *jeu de Shannon* sur le graphe G est un jeu qui se joue entre deux joueurs, *Positus* et *Minus*, qui agissent à tour de rôle. Chaque coup consiste à marquer une arête qui n'a pas été marquée précédemment. Dans ce jeu, *Positus* remporte la victoire s'il parvient à marquer un ensemble d'arêtes dans lequel les sommets s et t sont reliés par un chemin (appelé aussi une *chaîne* d'arêtes). *Minus* gagne le jeu si toutes les arêtes ont été marquées sans que *Positus* ne soit parvenu à relier les sommets s et t .

Épreuve d'Informatique 2 MPI 2025

En pratique, *Positus* « verrouille » des arêtes tandis que *Minus* « supprime » des arêtes encore jouables.

Indication C : Dans l'ensemble du sujet, les sommets du graphe $G = (V, E)$ sont numérotés entre 0 et $n - 1$, $S = \{0, 1, 2, \dots, n - 1\}$. Par convention, les sommets s et t correspondent aux sommets de numéros 0 et 1. Les arêtes des graphes sont de la forme $\{u, v\}$ et donc E est un sous-ensemble de $P_2(S) = \{X \subset S \mid |X| = 2\}$. Nous notons $M \subseteq E$ l'ensemble des arêtes marquées. Nous introduisons les déclarations suivantes.

```

1.  #define s 0
2.  #define t 1
3.
4.  struct link {
5.      int node;
6.      struct link * next;
7.  };
8.  typedef struct link link_t;
9.
10. struct game {
11.     int n;           // Nombre de sommets
12.     int nm;          // Nombre d'arêtes non marquées
13.     link_t ** adj_arr; // Voisins non marques
14.     int * cmps;      // Composantes connexes creees par Positus
15. };
16. typedef struct game game_t;
17.
18. game_t gm;
```

Le type `link_t` permet de représenter des maillons de listes simplement chaînées. Le graphe $(V, E \setminus M)$ est représenté par un tableau de listes d'adjacences. Le type `game_t` permet de représenter un jeu de Shannon en cours de déroulement. Dans les enregistrements de ce type,

- le champ `n` désigne le nombre de sommets,
- le champ `nm` désigne le nombre d'arêtes non marquées,
- le champ `adj_arr` renvoie à un tableau de pointeurs où, pour tout entier u compris entre 0 et $n - 1$, le pointeur `adj_array[u]` désigne le premier maillon d'une liste simplement chaînée contenant tous les voisins du sommet u dans le graphe $(V, E \setminus M)$,
- le champ `cmps` renvoie à un tableau permettant de représenter les sommets reliés entre eux par des arêtes marquées par *Positus* selon une convention détaillée aux questions 6 et 7.

Une variable globale `gm` est utilisée dans tout le sujet pour désigner le jeu en cours. On notera que les arêtes marquées au cours du jeu sont perdues et ne figurent plus dans la variable `gm`.

□ 1 – Écrire une fonction C `void gm_init(int n)` qui initialise la variable globale `gm` afin qu'elle contienne un jeu sur le graphe à n sommets et zéro arêtes. Il est attendu que le tableau `gm.cmps` contienne initialement les entiers $0, 1, \dots, n - 1$, dans cet ordre.

Épreuve d'Informatique 2 MPI 2025

□ 2 – Écrire une fonction C `void gm_addedge(int u, int v)` dont l'effet est d'ajouter une arête entre les sommets u et v au graphe stocké dans la variable globale `gm`. On veillera à représenter une arête par deux arcs orientés (u, v) et (v, u) .

□ 3 – Écrire une fonction C `bool gm_remove(int u, int v)` dont l'effet est de retirer une arête entre les sommets u et v au graphe `gm` et dont la valeur de retour est `true` si la suppression s'est bien déroulée, et `false` sinon.

Nous rappelons que la structure « unir & trouver » (ou, en anglais, *union find*) permet de représenter les classes d'équivalence d'une relation d'équivalence pour un ensemble E . On choisit d'implémenter cette structure à l'aide d'un tableau associatif A , de taille n pour un ensemble E de cardinal n dont les éléments sont numérotés de 0 à $n - 1$. Le tableau rend compte d'une forêt où chaque arbre est orienté des feuilles vers la racine et représente une classe d'équivalence. Pour un élément de numéro u , $A[u]$ vaut u s'il s'agit de la racine, que l'on appellera **identifiant** de la classe, ou $A[u]$ vaut le numéro de son parent dans l'arbre dans tout autre cas.

□ 4 – Écrire une fonction C `int uf_find(int n, int * A, int u)` dont la valeur de retour est l'identifiant de la classe à laquelle appartient l'élément u dans la relation représentée par A . Le tableau A ne doit pas être modifié.

□ 5 – Écrire une fonction C `bool uf_unite(int n, int * A, int u, int v)` dont l'effet est de réunir les classes des éléments u et v dans la relation d'équivalence représentée par A . Il n'y a pas d'effet si les éléments u et v appartiennent déjà à la même classe. La valeur de retour est `true` si A a été modifié et `false` sinon.

Nous appelons $M^+ \subseteq E$ l'ensemble des arêtes marquées par le joueur *Positus*. Nous notons $\leftrightarrow_P$ la relation d'équivalence dans laquelle deux sommets u et v sont en relation si et seulement s'il existe une chaîne entre u et v formée d'arêtes de M^+ . Les classes d'équivalence sont ainsi les composantes connexes du graphe (V, M^+) . Nous utilisons le champ `cmps` de la variable `gm` afin de stocker cette relation d'équivalence selon une structure « unir & trouver ».

□ 6 – Écrire une fonction C `int gm_find(int u)` dont la valeur de retour est l'identifiant de la composante connexe à laquelle appartient le sommet u dans le graphe (V, M^+) .

□ 7 – Écrire une fonction C `void gm_unite(int u, int v)` dont l'effet est de réunir les composantes connexes des sommets u et v .

□ 8 – Écrire une fonction C `bool has_P_won(void)` dont la valeur de retour indique si le jeu est dans un état gagnant pour le joueur *Positus*.

□ 9 – Écrire une fonction C `bool has_P_lost(void)` dont la valeur de retour indique si le jeu est dans un état perdant pour le joueur *Positus*.

Épreuve d'Informatique 2 MPI 2025

□ 10 – Écrire une fonction C `void play_P(int u, int v)` dont l'effet est de transformer la variable `gm` en jouant l'arête entre les sommets u et v par le joueur *Positus*. On se défendra, à l'aide d'une assertion, du cas où le coup joué par *Positus* est illicite.

□ 11 – Écrire une fonction C `void play_M(int u, int v)` dont l'effet est de transformer la variable `gm` en jouant l'arête entre les sommets u et v par le joueur *Minus*. On se défendra, à l'aide d'une assertion, du cas où le coup joué par *Minus* est illicite.

1.2 Étude de tournois en langage SQL

Un *tournoi* de jeux de Shannon est une compétition dans laquelle s'affrontent exactement deux joueurs (A et B) au cours de plusieurs matches. Le vainqueur d'un tournoi est le joueur qui a gagné le plus grand nombre de matches. Le rôle de chaque joueur (*Positus* ou *Minus*) peut changer entre chaque match.

On dispose d'une base de donnée SQL composée de trois tables (les clés primaires sont soulignées)

- Joueurs(JoueurID, NomJ)
- Tournois(TournoiID, NomT, JoueurAID, JoueurBID)
- Matches(MatchID, TournoiID, RoleJoueurA, VainqueurID)

dont voici certains extraits :

JoueurID	NomJ	TournoiID	NomT	JoueurAID	JoueurBID
2	Alice	82	TournoiDu12Dec	2	8
8	Bob	59	TournoiDu17Fev	7	8
7	Charlie	61	TournoiDu29Mai	8	2

MatchID	TournoiID	RoleJoueurA	VainqueurID
119	82	Positus	2
934	82	Minus	8
925	82	Minus	2
384	61	Positus	2

Par exemple, le tournoi du 12 décembre a opposé Alice et Bob, il s'est joué en 3 matches. Alice, qui a gagné deux des trois matches, a été victorieuse.

□ 12 – Écrire une requête SQL renvoyant le nombre de matches joués dans chaque tournoi.

□ 13 – Écrire une requête SQL renvoyant le nombre de tournois gagnés par chaque joueur.

□ 14 – Écrire une requête SQL renvoyant le nombre de fois où chaque joueur a joué le rôle de *Positus*.

Épreuve d'Informatique 2 MPI 2025

2 Stratégies de jeu

2.1 Stratégies gagnantes

☐ 15 – Démontrer, à l'aide d'un variant, que le jeu se termine toujours et justifier qu'il ne peut pas se terminer par un match nul.

☐ 16 – Rappeler la définition du terme *stratégie gagnante*.

☐ 17 – Nous supposons que *Positus* dispose d'une stratégie gagnante s'il joue en second. Démontrer qu'il dispose aussi d'une stratégie gagnante s'il joue en premier.

☐ 18 – Démontrer qu'il existe trois types d'instances du jeu de Shannon :

- un type d'instance dans laquelle *Positus* dispose d'une stratégie gagnante qu'il soit premier ou second joueur,
- un type dans laquelle *Minus* dispose d'une stratégie gagnante qu'il soit premier ou second joueur,
- et enfin un type dans laquelle le joueur qui commence la partie dispose d'une stratégie gagnante.

On donnera un exemple de graphe G illustrant chacun des trois types.

2.2 Une stratégie pour *Positus*

Définition : Selon la terminologie usuelle, nous appelons *arbre couvrant* tout sous-ensemble de $n - 1$ arêtes $T \subseteq E$ ne contenant pas de cycle. Il est rappelé les faits suivants :

- le graphe (V, T) est connexe ;
- tout ajout d'une arête $e \in E \setminus T$, à un arbre couvrant T , crée un unique cycle noté $C(T, e)$;
- si T est un arbre couvrant, alors, pour tout couple de sommets u et v de V , il existe un unique chemin simple reliant u et v avec des arêtes de T .

☐ 19 – Soient deux arbres couvrants $T \subseteq E$ et $T' \subseteq E$ et une arête α appartenant à T . Montrer qu'il existe une arête β appartenant à T' telle que l'ensemble $T'' = T \cup \{\beta\} \setminus \{\alpha\}$ est encore un arbre couvrant.

Indication C : Nous représentons tout arbre couvrant par un tableau T de taille n tel que $T[0]$ vaut 0 et, pour tout autre sommet u , $T[u]$ vaut le parent du sommet u dans l'arbre enraciné en 0.

Pour représenter un arc, nous déclarons :

Épreuve d'Informatique 2 MPI 2025

```

19. struct pair {
20.     int first;
21.     int second;
22. };
23. typedef struct pair pair_t;

```

□ 20 – Écrire une fonction C `pair_t edge_find(int n, int * T, int * Tprime, pair_t alpha` ainsi spécifiée :

Précondition : Les variables T et $Tprime$ contiennent des arbres couvrants T et T' sur les mêmes n sommets. La variable α est un arc de l'arbre T orienté des feuilles vers la racine.

Valeur de retour : Une arête β , appartenant à l'arbre T' , telle que $T'' = T \cup \{\beta\} \setminus \{\alpha\}$ est encore un arbre couvrant.

□ 21 – Écrire une fonction C `void edge_swap(int n, int * T, pair_t alpha, pair_t beta)` ainsi spécifiée :

Précondition : La variable T correspond à un arbre couvrant T à n sommets. La variable α est un arc de l'arbre T orienté des feuilles vers la racine. La variable β est une arête qui n'appartient pas à l'arbre T telle que $T'' = T \cup \{\beta\} \setminus \{\alpha\}$ est un arbre couvrant.

Effet : Le tableau T est transformé afin de contenir l'arbre T'' .

Dans ce qui suit, nous supposons que *Minus* joue en premier.

Définition : Pour tout ensemble de sommets $U \subseteq V$ qui contient les sommets s et t , nous notons G_U le *graphe induit par U* , c'est-à-dire le sous-graphe (U, E_U) où,

$$E_U = \{\{u, v\} \in E \mid u, v \in U\}.$$

Pour tout entier naturel k , nous notons $M_k^+ \subseteq E$ l'ensemble des k arêtes marquées par *Positus*, après que k coups ont été joués par *Positus*. De même, nous notons $M_k^- \subseteq E$ l'ensemble des k arêtes marquées par *Minus* après que k coups ont été joués par *Minus*. Enfin, nous appelons $(\boxtimes_k)$ l'assertion suivante :

Après que $2k$ coups ont été joués par *Minus* et *Positus*, il existe un ensemble de sommets U contenant les sommets s et t ainsi que deux arbres couvrant le graphe induit G_U , que l'on note $T_k \subseteq E_U$ et $T'_k \subseteq E_U$, tels que l'on a $T_k \cap T'_k = M_k^+$ et $(T_k \cup T'_k) \cap M_k^- = \emptyset$. ($\boxtimes_k$)

□ 22 – Soit k un entier naturel. Dans cette question, nous supposons que $2k$ coups ont d'ores et déjà été joués et que l'assertion $(\boxtimes_k)$ est vérifiée ; pour son $(k+1)^{\text{e}}$ coup, le joueur *Minus* choisit de marquer une arête appartenant à l'arbre T_k . Montrer que le joueur *Positus* peut alors trouver une arête à marquer telle que l'assertion $(\boxtimes_{k+1})$ est vérifiée.

□ 23 – Soit k un entier naturel. Dans cette question, nous supposons que $2k$ coups ont été joués et que l'assertion $(\boxtimes_k)$ est vérifiée ; le joueur *Minus* marque une arête n'appartenant ni à l'arbre T_k , ni à l'arbre T'_k . Montrer que le joueur *Positus* peut marquer une arête telle que l'assertion $(\boxtimes_{k+1})$ est vérifiée.

Épreuve d'Informatique 2 MPI 2025

□ 24 – Démontrer que, si au cours de la partie, il existe un entier naturel k_0 tel que l'assertion $(\mathfrak{X}_{k_0})$ est satisfaite, alors *Positus* possède une stratégie gagnante.

□ 25 – Dans cette question, nous supposons que la paire $\{s, t\}$ n'appartient pas à E et nous notons $\hat{G}$ le graphe obtenu à partir de G en ajoutant une arête supplémentaire entre les sommets s et t . Nous supposons que l'assertion $(\mathfrak{X}_0)$ est vraie pour le nouveau graphe $\hat{G}$. Démontrer que le joueur *Positus* dispose d'une stratégie gagnante lorsqu'il joue en premier.

3 Arbres couvrants disjoints

Dans toute cette section, nous supposons que la paire $\{s, t\}$ n'appartient pas à l'ensemble des arêtes E . Nous notons $\hat{G}$ le graphe obtenu à partir de G en ajoutant une arête supplémentaire entre les sommets s et t . Nous appelons $\hat{E}$ l'ensemble d'arêtes $E \cup \{s, t\}$.

3.1 Une condition suffisante pour $(\mathfrak{X}_0)$

Soient $T \subseteq \hat{E}$ et $T' \subseteq \hat{E}$ deux arbres couvrants de $\hat{G}$ et α_0 une arête n'appartenant pas à $T \cup T'$. Notons $L_0 = \{\alpha_0\}$. Notons $L_1 \subseteq \hat{E}$ l'unique cycle $C(T, \alpha_0)$ présent dans l'ensemble $T \cup \{\alpha_0\}$.

Définition : Pour tout couple d'arbres couvrants, $T \subseteq \hat{E}$ et $T' \subseteq \hat{E}$, notons $\delta(T, T')$ le cardinal de l'intersection $T \cap T'$. Deux arbres couvrants T_0 et T'_0 sont *éloignés* si le couple (T_0, T'_0) atteint le minimum global de la fonction δ parmi tous les couples d'arbres couvrants de $\hat{G}$.

Définition : Une arête $\alpha \in \hat{E}$ est *principale* s'il existe un couple d'arbres couvrants T et T' éloignés, tel que l'arête α n'appartient pas à $T \cup T'$.

□ 26 – En supposant qu'il existe une arête $\alpha_1 \in L_1$ appartenant à $T \cap T'$, construire deux arbres couvrants $\tilde{T}$ et $\tilde{T}'$ du graphe $\hat{G}$ tels que

$$\delta(\tilde{T}, \tilde{T}') < \delta(T, T').$$

Si L_1 ne contient aucune arête de $T \cap T'$, nous appelons $L_2 \subseteq \hat{E}$ la réunion des cycles $\bigcup_{\alpha \in L_1} C(T', \alpha)$ qui se créent lorsque l'on ajoute à T' l'une des arêtes de L_1 .

□ 27 – En supposant qu'il existe une arête $\alpha_2 \in L_2$ appartenant à $T \cap T'$, construire deux arbres couvrants $\tilde{T}$ et $\tilde{T}'$ du graphe $\hat{G}$ tels que

$$\delta(\tilde{T}, \tilde{T}') < \delta(T, T').$$

Plus généralement, nous construisons par récurrence la suite croissante $(L_k)_{k \in \mathbb{N}} \subseteq \hat{E}$ d'ensembles d'arêtes où

Épreuve d'Informatique 2 MPI 2025

- pour tout entier $k \geq 1$, si l'ensemble L_{2k} ne contient aucune arête de $T \cap T'$, alors on pose

$$L_{2k+1} = \bigcup_{\alpha \in L_{2k}} C(T, \alpha),$$

- pour tout entier $k \geq 0$, si l'ensemble L_{2k+1} ne contient aucune arête de $T \cap T'$, alors on pose

$$L_{2k+2} = \bigcup_{\alpha \in L_{2k+1}} C(T', \alpha).$$

□ 28 – Démontrer que l'un ou l'autre des cas suivants se produit :

- la suite $(L_k)_{k \in \mathbb{N}} \subseteq E^{\mathbb{N}}$ n'est définie que pour un nombre fini de termes et dans ce cas montrer qu'il existe deux arbres couvrants $\tilde{T}$ et $\tilde{T}'$ du graphe $\hat{G}$ tels que

$$\delta(\tilde{T}, \tilde{T}') < \delta(T, T') ;$$

- ou bien la suite $(L_k)_{k \in \mathbb{N}} \subseteq E$ est définie pour tout rang k et donc qu'il existe un rang à partir duquel la suite est égale à une constante $\Lambda(\alpha_0) \subseteq \hat{E}$.

□ 29 – Démontrer que, si l'arête $\{s, t\}$ est une arête principale du graphe $\hat{G}$, alors l'assertion $(\mathfrak{X}_0)$ est vérifiée.

3.2 Recherche des arêtes principales

□ 30 – Nommer un algorithme qui construit un arbre couvrant pour le graphe G et qui soit cohérent avec les choix opérés par le type `game_t` de représenter un graphe par des listes d'adjacence.

Soit T et T' un couple d'arbres couvrants éloignés. On pose

$$P = \bigcup_{\alpha_0 \in E \setminus (T \cup T')} \Lambda(\alpha_0).$$

On appelle U l'ensemble des arêtes incidentes à P . On appelle n_U le cardinal de U , m_P le cardinal de P , et $c = |E \setminus (T \cup T')|$ le nombre d'arêtes hors de T et T' . On note κ le nombre de composantes connexes de (U, P) .

Soit $\tilde{T}$ et $\tilde{T}'$ un autre couple d'arbres couvrants éloignés.

Notons $\tilde{c}_1$ le cardinal de $(E \setminus (\tilde{T} \cup \tilde{T}')) \cap P$.

□ 31 – Montrer que

$$c \geq \tilde{c}_1 \geq m_P - 2(n_U - \kappa).$$

En déduire que le cas 2 de la question 28 se produit pour toute arête α_0 uniquement quand T et T' sont éloignés.

□ 32 – D'après la question 28, pour toute arête α_0 n'appartenant pas à $T \cup T'$, l'ensemble $\Lambda(\alpha_0) \subseteq \hat{E}$ est bien défini. Démontrer que P est l'ensemble des arêtes principales.

Épreuve d'Informatique 2 MPI 2025

□ 33 – Expliquer comment on peut construire l'ensemble des arêtes principales et en déduire une stratégie gagnante pour un joueur. Une réponse en pseudo-code est permise, mais seule la logique de programmation sera évaluée.

FIN DE L'ÉPREUVE



Informatique

MPI

2025

4 heures

Calculatrice autorisée

Rush Hour

L'épreuve est composée de deux problèmes indépendants. L'objectif du premier problème est d'écrire un programme permettant de résoudre le casse-tête de déplacements nommé *Rush Hour* dont le but est de faire sortir une voiture d'un parking. Dans le second problème, on souhaite connaître le maximum de véhicules qu'il est possible de placer sur le terrain d'une *casse automobile*.

Les candidats devront répondre aux questions en utilisant le langage C dans le premier problème et le langage OCaml dans le second problème. Ils sont invités, lorsque c'est pertinent, à décrire succinctement le fonctionnement de leurs programmes, un dessin étant souvent plus parlant qu'un long discours. Le barème tient compte de la clarté et de la concision des programmes. Nous recommandons de choisir des noms de variables intelligibles et de structurer, si cela s'avère nécessaire, de longs codes par des fonctions auxiliaires dont on décrira le rôle.

On identifiera une même grandeur écrite dans deux polices différentes, en italique lorsque nous la considérerons d'un point de vue mathématique (par exemple n), et en police à largeur fixe lorsque nous la considérerons d'un point de vue informatique (par exemple `n`). Si $a, b \in \mathbb{Z}$, on note $\llbracket a, b \rrbracket$ l'ensemble $\{k \in \mathbb{Z} \mid a \leq k < b\} = \{a, a+1, \dots, b-1\}$. Un graphe non orienté est un couple $G = (S, A)$ où S est un ensemble fini non vide d'éléments appelés sommets et A un ensemble de parties de S à deux éléments, appelées arêtes. Si $x, y \in S$ sont deux sommets distincts de S , l'arête $\{x, y\}$ sera notée $x - y$. Un graphe orienté est un couple $G = (S, A)$ où S est un ensemble fini non vide d'éléments appelés sommets et A un ensemble d'éléments (x, y) de $S \times S$ tels que $x \neq y$, appelés arcs. Si $x, y \in S$ sont deux sommets distincts de S , l'arc (x, y) sera noté $x \rightarrow y$.

Par complexité en temps d'un algorithme, on entend nombre d'opérations élémentaires (comparaison, addition, soustraction, multiplication, division, affectation, test, etc.) nécessaires à l'exécution de l'algorithme dans le pire des cas.

Partie A – Rush Hour

Le jeu de plateau *Rush Hour* simule un embouteillage dans un parking à l'heure de pointe. Dans ce jeu à un joueur, on doit extraire le véhicule gris d'une grille dans laquelle plusieurs autres véhicules noirs bloquent la sortie. Il est possible de déplacer chaque véhicule, d'une ou plusieurs cases, vers l'avant ou l'arrière. Les véhicules sont soit des voitures, soit des camions. Les voitures occupent 2 cases tandis que les camions occupent 3 cases. La sortie est repérée par une flèche qui se situe sur le côté droit de la grille. La figure 1 illustre une position de départ du jeu.

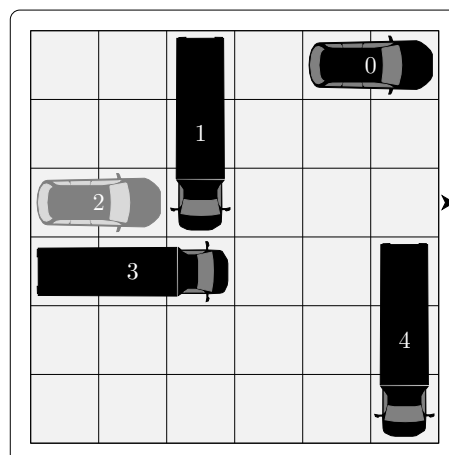


Figure 1.

Pour résoudre ce casse-tête, on commence par déplacer la voiture 0 d'une case en arrière. On se retrouve alors dans la configuration de la figure 2.a. Puis on déplace le camion 4 de 3 cases en arrière, le camion 3 de 3 cases en avant et le camion 1 de 3 cases en avant. On se retrouve alors dans la configuration de la figure 2.b qui nous permet de déplacer la voiture grise numérotée 2 de 3 cases vers l'avant et nous retrouver sans la configuration de la figure 2.c.

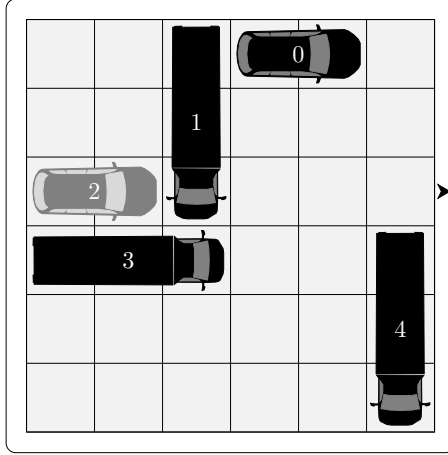


Figure 2.a

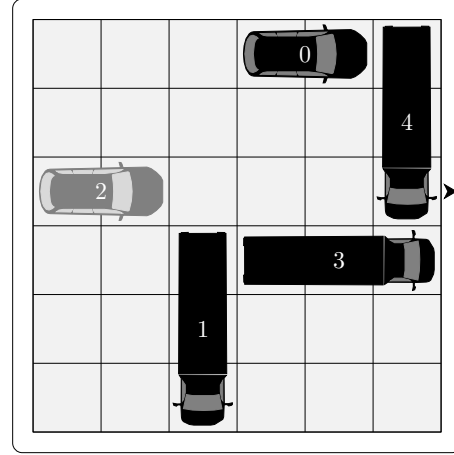


Figure 2.b

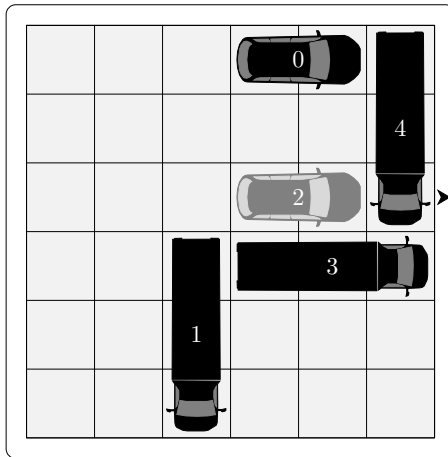


Figure 2.c

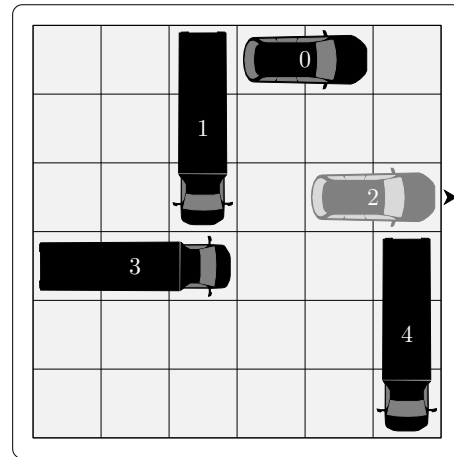


Figure 2.d

On déplace alors le camion 1 de 3 cases vers l'arrière, puis le camion 3 de 3 cases vers l'arrière et le camion 4 de 3 cases vers l'avant. On avance enfin la voiture grise numérotée 2 d'une case. Elle se retrouve devant la sortie indiquée par la flèche sur la droite de la grille. Cette configuration gagnante est indiquée à la figure 2.d.

Dans ce problème, nous supposons que la grille du jeu est carrée et comporte $M \times M$ cases. Comme tous les véhicules occupent au moins 2 cases, le nombre de véhicules présents sur la grille est majoré par $(M \times M)/2$. Dans notre programme, cet entier est noté MAX_V . Les cases sont repérées par leurs coordonnées $(i, j) \in \llbracket 0, M \rrbracket^2$, l'indice i indiquant le numéro de la ligne et l'indice j indiquant le numéro de la colonne. La ligne sur laquelle se trouve la sortie est indiquée par S . Dans l'exemple de la figure 3 disponible page suivante, on a $M = 6$ et $S = 2$. Dans la suite du sujet, nous supposons que la voiture grise est l'unique voiture placée horizontalement sur la ligne de la sortie. Comme c'est la seule voiture pouvant sortir de la grille, contrairement aux schémas présents dans ce sujet sur lesquels elle est en gris, elle ne fera l'objet d'aucun marquage particulier dans notre algorithme.

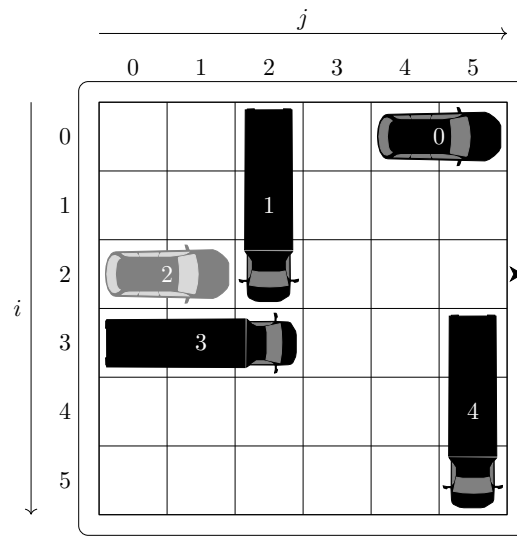


Figure 3.

Nous supposons que les entêtes suivants ont été inclus et que les constantes ci-dessous sont définies au début du programme.

```

1 #include <assert.h>
2 #include <stdbool.h>
3 #include <stdint.h>
4 #include <stdlib.h>
5
6 #define M 6
7 #define MAX_V (M * M) / 2
8 #define S 2

```

Les directives **#define** présentes ici ont pour but de définir des synonymes pour les constantes entières utilisées dans le sujet. Chaque fois que l'identifiant **M** sera présent dans la suite, il sera directement substitué par la constante 6 par le compilateur.

I – Introduction

La position de chaque véhicule sur la grille est représentée par la structure de donnée suivante :

```

1 struct vehicule_s {
2     int i;
3     int j;
4     int longueur;
5     char direction;
6 };
7 typedef struct vehicule_s vehicule;

```

Le couple (i, j) représente les coordonnées de la case occupée par le véhicule située la plus en haut à gauche sur la grille. Par exemple, sur la figure 3, les coordonnées du véhicule 0 sont $(0, 4)$ et celles du véhicule 1 sont $(0, 2)$. Le champ **longueur** indique la longueur, en nombre de cases, du véhicule : 2 pour une voiture et 3 pour un camion. Enfin, le champ **direction** indique la direction du véhicule : 'H' si le véhicule est placé de manière horizontale, comme le véhicule 0 de la figure 3, et 'V' si le véhicule est placé de manière verticale, comme le véhicule 1 de la figure 3.

La configuration des véhicules sur le plateau de jeu est représentée par la structure suivante :

```

1 struct etat_s {
2     vehicule v[MAX_V];
3     int n;
4     int distance;
5 };
6 typedef struct etat_s etat;

```

Le nombre de véhicules présents sur la grille est donné par le champ `n` et vérifie `n ≤ MAX_V`. Pour tout $k \in \llbracket 0, n \rrbracket$, le champ `v[k]` donne la position et les caractéristiques du véhicule d'indice k . Enfin, le champ `distance` n'est pas utilisé pour le moment et sera initialisé à 0.

- Q1.** Écrire une fonction `etat *e_nouveau(void)` renvoyant un état ne possédant aucune voiture sur son plateau, ainsi qu'une fonction `void e_libere(etat *e)` libérant la mémoire associée à cet état.
- Q2.** Écrire une fonction `bool e_egal(etat *e1, etat *e2)` déterminant si deux états sont égaux, c'est-à-dire possédant le même nombre de véhicules n et tels que pour tout $k \in \llbracket 0, n \rrbracket$ les véhicules d'indice k ont mêmes positions et mêmes caractéristiques. Deux états peuvent être égaux en ayant des valeurs différentes associées au champ `distance`.
- Q3.** Écrire une fonction `void e_ajoute_vehicule(int i, int j, int longueur, char direction, etat *e)` ajoutant à l'état e un véhicule à la position (i, j) dont la longueur et la direction sont données par les variables homonymes. On ne cherchera pas à vérifier que les cases sont libres, mais on utilisera une assertion pour vérifier que le nombre n de véhicules présents sur la grille reste inférieur ou égal à `MAX_V`.

Afin de faciliter la manipulation des états, nous allons définir une structure `plateau` représentant le plateau de jeu. Cette structure est définie comme suit :

```
1 const int vide = -1;
2
3 struct plateau_s {
4     int tab[M][M];
5 };
6 typedef struct plateau_s plateau;
```

- Q4.** Écrire une fonction `plateau *p_etat(etat *e)` renvoyant un plateau dont les cases sont initialisées à la valeur `vide`, excepté les cases sur lesquelles se trouvent les véhicules de l'état e qui sont chacune initialisée avec l'indice du véhicule la recouvrant.

Pour chaque état, seul un nombre fini de déplacements est valide. Pour manipuler cette collection, nous allons utiliser une liste chaînée, dont le maillon `deplacement` est défini comme ceci :

```
1 struct deplacement_s {
2     int k;
3     int pas;
4     struct deplacement_s *suivant;
5 };
6 typedef struct deplacement_s deplacement;
```

Un tel maillon permet de représenter un déplacement de la voiture d'indice k de `pas` cases dans le sens des i (ou des j) croissants. Par exemple, dans la solution du casse-tête de la figure 1, le premier mouvement de la voiture 0 d'une case vers la gauche aura une valeur de `k = 0` et de `pas = -1`. Le second mouvement du camion 4 de 3 cases vers le haut aura une valeur de `k = 4` et de `pas = -3`. Le champ `suivant` pointe vers le prochain maillon de la liste chaînée. La liste vide est représentée par le pointeur `NULL`.

- Q5.** Écrire une fonction `deplacement *d_cons(int k, int pas, deplacement *lst)` ajoutant en tête de la liste chaînée `lst`, un déplacement de la voiture d'indice k de `pas` cases.

Pour la suite du sujet, on suppose disposer d'une fonction `deplacement *e_deplacements(etat *e)` renvoyant la liste des déplacements valides dans l'état e .

- Q6.** Implémenter une fonction `void d_libere(deplacement *lst)` libérant la mémoire occupée par les différents éléments de la liste `lst`.

II – Implémentation d'une file

Dans cette partie, nous allons réaliser une file à l'aide d'un tableau circulaire. Sur l'exemple ci-dessous, nous avons une file d'entiers représentée par un tableau possédant 8 cases. On dit que la file a une capacité de 8 éléments.

8					17	25	9
0	1	2	3	4	5	6	7
prochain							

Ce tableau représente la file $17 \leftarrow 25 \leftarrow 9 \leftarrow 8$. Le prochain élément à sortir de la file est 17 et le dernier élément à avoir été ajouté est 8. L'indice **prochain**, qui est égal à 5 sur cet exemple, indique l'indice du prochain élément qui sortira de la file. Cette file possède 4 éléments. On dit que sa taille est égale à 4.

La file que nous allons implémenter est une file d'états. Elle est représentée par la structure suivante :

```

1 struct file_s {
2     etat **tab;
3     int prochain;
4     int taille;
5     int capacite;
6 };
7 typedef struct file_s file;
```

- Q7.** Écrire une fonction `file *f_nouveau(void)` qui renvoie une file de taille et de capacité nulle. Écrire d'autre part une fonction `void f_libere(file *f)` libérant la mémoire associée à la file f . Cette fonction ne devra pas libérer la mémoire associée aux états présents dans la file.
- Q8.** Écrire une fonction `void f_change_capacite(file *f, int c)` prenant en entrée une file de taille m ainsi qu'un entier $c \geq m$. Cette fonction devra modifier la capacité de la file f afin qu'elle devienne égale à c .
- Q9.** Écrire les fonctions `void f_enfile(file *f, etat *e)` et `etat *f_defile(file *f)` permettant respectivement d'ajouter un nouvel élément e à l'entrée de la file f et d'enlever et récupérer un élément à la sortie de la file. Lorsqu'on souhaite ajouter un nouvel élément dans une file dont la capacité est égale à la taille, on multiplie cette capacité par 2 (sauf pour le cas particulier où la capacité est nulle où on la change en 1) avant d'ajouter notre nouvel élément.

On souhaite montrer que la complexité amortie des fonctions `f_enfile` et `f_defile` est en $\mathcal{O}(1)$, c'est-à-dire que si l'on part d'une file vide, la complexité d'une succession de n appels, chaque appel se faisant à l'une de ces fonctions, a une complexité en $\mathcal{O}(n)$. Pour les deux questions suivantes, et seulement pour ces deux questions, la complexité d'une opération sera calculée comme le nombre d'accès aux éléments du tableau `tab`, que ces accès soient en lecture ou en écriture. Nous allons utiliser pour cela la méthode du potentiel. On définit le potentiel Φ d'une file f par

$$\Phi(f) = |4 \cdot \text{taille}(f) - 2 \cdot \text{capacite}(f)|.$$

On note $f_0, f_1, \dots, f_n$ les états successifs de la file. L'état f_0 est son état initial qui est une file de taille et de capacité nulle. Pour tout $k \in \llbracket 1, n \rrbracket$, f_k est l'état de la file après la k -ième opération.

$$f_0 \xrightarrow{\text{op}_1} f_1 \xrightarrow{\text{op}_2} f_2 \cdots f_{n-1} \xrightarrow{\text{op}_n} f_n.$$

Pour tout $k \in \llbracket 1, n \rrbracket$, on note C_k la complexité de l'opération op_k qui fait passer la file de l'état f_{k-1} à l'état f_k .

- Q10.** Montrer qu'il existe un entier $K \geq 0$ que l'on déterminera, tel que

$$\forall k \in \llbracket 1, n \rrbracket, \quad C_k + \Phi(f_k) - \Phi(f_{k-1}) \leq K.$$

- Q11.** En déduire que

$$\sum_{k=1}^n C_k \leq Kn$$

puis conclure.

III – Table de hachage

Dans cette partie, nous allons implémenter une structure d'ensemble à l'aide d'une table de hachage en adressage ouvert. Les éléments de cet ensemble sont de type `etat *`. Nous allons commencer par définir une fonction de hachage. Pour tout état e , on définit

$$\text{hash}(e) = \sum_{\substack{0 \leq i < M \\ 0 \leq j < M}} d_{i,j} 3^{iM+j}$$

où pour tout $i, j \in \llbracket 0, M \rrbracket$

$$d_{i,j} = \begin{cases} 0 & \text{si la case } (i, j) \text{ est inoccupée,} \\ 1 & \text{si la case } (i, j) \text{ est occupée par un véhicule horizontal,} \\ 2 & \text{si la case } (i, j) \text{ est occupée par un véhicule vertical.} \end{cases}$$

On suppose disposer d'une fonction `plateau *p_hash(etat *e)` qui étant donné un état e , renvoie un plateau p dont les éléments `tab[i][j]` sont égaux à $d_{i,j}$.

Q12. Écrire une fonction `int64_t e_hash(etat *e)` prenant en entrée un état e et renvoyant $\text{hash}(e)$.

Q13. Montrer que si e_1 et e_2 sont deux états accessibles à partir d'un même état initial e et que $\text{hash}(e_1) = \text{hash}(e_2)$, alors $e_1 = e_2$.

Pour stocker les états dans notre table de hachage, nous allons utiliser un tableau possédant un nombre de cases appelé capacité de la table de hachage. Cette capacité sera de la forme 2^p . La taille de la table de hachage est le nombre d'états qu'elle contient. Lorsqu'on souhaite ajouter un état e dans notre table de hachage, on commencera par calculer $h = \text{hash}(e) \bmod 2^p$. Si la case d'indice h est vide, on placera e dans cette case. Sinon, on cherchera la première case vide en partant de l'indice h et en avançant de un en un de manière circulaire dans le tableau, jusqu'à trouver une case vide. Par exemple, si l'on part d'une table de hachage vide de capacité 8, et qu'on ajoute successivement les états e_0, e_1, e_2, e_3 et e_4 ayant des hash respectifs de $h_0 = 1, h_1 = 7, h_2 = 3, h_3 = 3$ et $h_4 = 7$, on obtiendra la table de hachage suivante.

e_4	e_0		e_2	e_3			e_1
0	1	2	3	4	5	6	7

Lorsque la table de hachage est trop peuplée, on double sa capacité et on réinsère les éléments de l'ancienne table de hachage dans la nouvelle. Nous utiliserons la structure suivante pour représenter notre table de hachage.

```
1 struct tableh_s {
2     etat **tab;
3     int taille;
4     int capacite;
5 };
6 typedef struct tableh_s tableh;
```

Le tableau `tab` contient les différents états présents dans la table de hachage. Nous utiliserons le pointeur `NULL` pour représenter les cases vides de la table. On suppose que l'on dispose d'une fonction `tableh *t_nouveau(void)` créant une table de hachage vide ayant une capacité de 1 et d'une fonction `void t_libere(tableh *t)` libérant la mémoire associée à la table de hachage t , ainsi que la mémoire associée à tous les états qu'elle contient.

Q14. Écrire une fonction `bool t_contient(tableh *t, etat *e)` déterminant si l'état e est présent dans la table de hachage t .

On suppose disposer d'une fonction `void t_augmente_capacite(tableh *t)` qui prend en entrée une table de hachage t de capacité $c \geq 1$ et qui augmente sa capacité à $2c$.

Q15. Écrire une fonction `void t ajoute(tableh *t, etat *e)` qui ajoute un état e à la table de hachage t . Si m est la taille de la table et c sa capacité, on fera attention à ce que l'on ait toujours $m \leq 2c/3$.

Q16. Expliquer en quoi ce serait une mauvaise idée de remplacer le 3^{iM+j} par 4^{iM+j} dans la définition de $\text{hash}(e)$.

IV – Recherche d’une solution optimale

On supposera disposer dans cette partie d’une fonction `bool e_gagne(etat *e)` déterminant si la voiture grise est devant la sortie, ainsi que d’une fonction `etat *e_copie(etat *e)` effectuant une copie d’un état e .

Q17. Écrire une fonction `int resout(etat *e)` prenant en entrée un état e et renvoyant le nombre minimal de mouvements pour résoudre le jeu. La fonction devra renvoyer -1 dans le cas où il n’y a pas de solution. On souhaite de plus que la mémoire associée à l’état e soit libérée à la fin de l’appel à `resout`. On pourra utiliser le champ `distance` d’un état pour indiquer le nombre de coups minimal nécessaires pour atteindre cet état à partir de l’état initial.

Q18. Montrer que la fonction `resout` n’engendre aucun problème mémoire.

Partie B – La casse automobile

Dans cette partie, nous disposons d’un terrain sur lequel nous souhaitons entreposer le plus grand nombre de véhicules hors d’usage. Ce terrain est modélisé par un ensemble de cases du plan, extraites d’une grille régulière. La taille d’un terrain désigne le nombre de cases le constituant. Deux types de véhicules sont disponibles : les voitures, qui occupent 2 cases, et les camions, qui occupent 3 cases.

Le premier objectif de cette partie est d’implémenter un algorithme efficace, ayant une complexité polynomiale en la taille du terrain, permettant de déterminer le maximum de voitures qu’il est possible de disposer sur ce dernier. Cet algorithme nous permet notamment de décider s’il est possible de couvrir tout le terrain à l’aide de voitures. La figure 4 donne un exemple de couverture complète d’un terrain possédant 14 cases à l’aide de voitures.

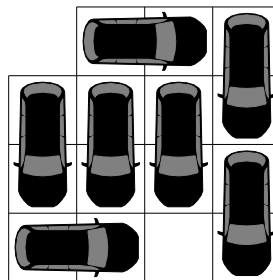


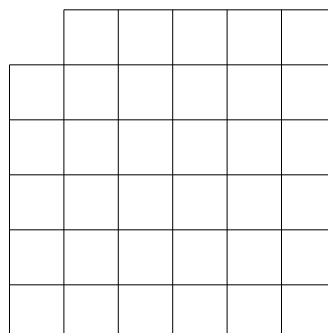
Figure 4.

Le second objectif de cette partie est de démontrer que, si l’on remplace les voitures, qui occupent 2 cases, par des camions, qui occupent 3 cases, le problème devient NP-complet.

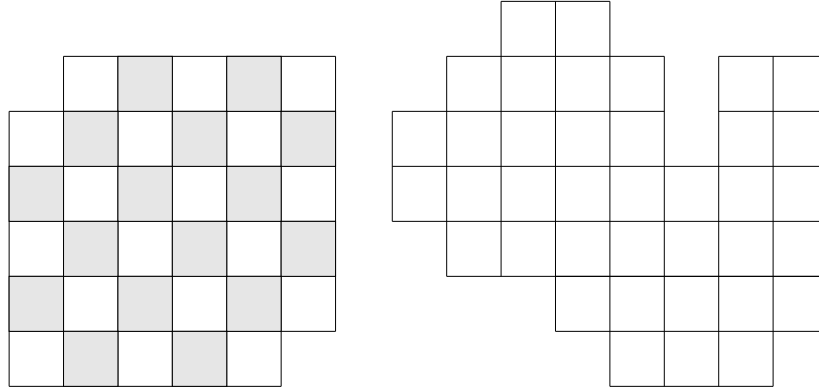
I – Remplir une casse avec le maximum de voitures

Dans cette partie B.I, tous les véhicules considérés sont des voitures et occupent 2 cases.

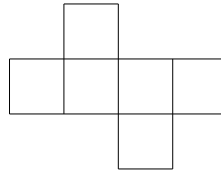
Q19. Prouver qu’il est impossible de couvrir tout le terrain suivant à l’aide de voitures. En déduire une condition nécessaire sur le terrain pour qu’il soit possible de le couvrir entièrement à l’aide de voitures.



- Q20.** Prouver qu'il est impossible de couvrir entièrement le terrain de gauche ci-dessous à l'aide de voitures. Ce terrain a une taille de 34 cases et la coloration de ses cases en blanc et en noir à la manière d'un échiquier n'a aucune incidence sur la disposition des voitures. Est-il possible de couvrir entièrement le terrain de droite ci-dessous ? On attend soit un exemple de couverture, soit une preuve de son impossibilité. En déduire une nouvelle condition nécessaire sur le terrain pour qu'il soit possible de le couvrir entièrement à l'aide de voitures.



- Q21.** En considérant le terrain ci-dessous, prouver que la condition nécessaire trouvée dans la question précédente n'est pas suffisante pour assurer l'existence d'une couverture complète à l'aide de voitures.



Dans la suite de cette partie, nous allons écrire un algorithme permettant de déterminer le plus grand nombre de voitures qu'il est possible de disposer sur un terrain, ainsi qu'une telle disposition. Pour cela, on définit le graphe non orienté $G = (S, A)$ dont l'ensemble des sommets S est constitué des cases du terrain, une arête $x - y$ appartenant à A si et seulement si x et y sont deux cases adjacentes, c'est-à-dire lorsque y est l'une des 4 cases au-dessus, en dessous, à gauche ou à droite de x . Nous choisissons arbitrairement de colorier une case du terrain en noir, puis de colorier le reste du terrain à la manière d'un échiquier en alternant les couleurs blanches et noires. On note N l'ensemble des sommets noirs et B l'ensemble des sommets blancs. Puisqu'une arête relie toujours deux sommets de couleurs différentes, $G = (N \sqcup B, A)$ est un graphe biparti. On numérote les r cases du terrain en commençant par les p cases noires, puis les q cases blanches. On a donc $r = p + q$, $N = \llbracket 0, p \rrbracket$ et $B = \llbracket p, r \rrbracket$. En OCaml, ce graphe sera représenté par un tableau de listes d'adjacence ($g : \text{int list array}$) ainsi que l'entier ($p : \text{int}$). Pour tout $x \in \llbracket 0, r \rrbracket$, $g.(x)$ est la liste des voisins du sommet x .

On rappelle qu'un couplage de $G = (S, A)$ est une partie C de A telle que chaque sommet n'est l'extrémité que d'au plus une arête de C . Un sommet $x \in S$ est dit libre lorsqu'il n'est l'extrémité d'aucune arête de C . Deux sommets $x, y \in S$ sont dits conjoints lorsqu'ils sont les extrémités d'une même arête de C . En OCaml, un couplage sera représenté par le tableau ($\text{conjoint} : \text{int option array}$) défini par

- $\text{conjoint}.(x) = \text{None}$ lorsque le sommet x est libre.
- $\text{conjoint}.(x) = \text{Some } y$ et $\text{conjoint}.(y) = \text{Some } x$ lorsque x et y sont conjoints.

Un chemin $x_0 - x_1 - \dots - x_k$ de longueur k sera représenté en OCaml par la liste $[x_0; \dots; x_k]$ et sera représenté par la variable ($w : \text{int list}$). Un chemin est dit élémentaire lorsque ses sommets sont deux à deux distincts. Un chemin est dit alternant pour C lorsqu'il est élémentaire et ses arêtes $x_i - x_{i+1}$ alternent entre des éléments de C et $A \setminus C$. On dit qu'il est augmentant lorsqu'il est alternant et que ses extrémités sont libres.

Si X et Y sont deux parties de A , on définit la différence symétrique $X \Delta Y = (X \cup Y) \setminus (X \cap Y)$. On admet que cette opération est associative et commutative. Si X est une partie de A et w est un chemin, $X \Delta w$ est défini en considérant le chemin w comme un ensemble d'arêtes.

Q22. Montrer que si C est un couplage et que w est un chemin augmentant pour C , alors $C \Delta w$ est un couplage.

Q23. Écrire une fonction `delta` (`conjoint : int option array`) (`w : int list`) : `unit` prenant en entrée un couplage C représentée par le tableau `conjoint` et un chemin w que l'on supposera augmentant et modifiant `conjoint` de manière à ce qu'il représente le couplage $C \Delta w$.

On définit le graphe orienté G_C possédant les mêmes sommets que G et possédant un arc orienté pour chaque arête $n - b$ de G où $n \in N$ et $b \in B$: si $n - b \in C$, cet arc est $b \rightarrow n$, sinon cet arc est $n \rightarrow b$. D'autre part, dans la suite de cette partie, on utilisera l'enregistrement

```
1 type couplage = {
2   graphe : int list array;
3   p : int;
4   conjoint : int option array;
5 }
```

pour représenter un couplage ainsi que le graphe biparti auquel il est associé. Si `c` est une variable de type `couplage`, `c.graphe` permet d'accéder au tableau de listes d'adjacence du graphe, `c.p` permet d'accéder à l'entier p et `c.conjoint` permet d'accéder au tableau `conjoint`. On construit un couplage à l'aide de la syntaxe

```
{ graphe = g; p = p; conjoint = c }
```

Enfin, on note respectivement N_ℓ et B_ℓ l'ensemble des sommets libres de N et de B .

Q24. Montrer qu'un chemin $x_0 - x_1 - \dots - x_k$ de G dont le premier élément x_0 est dans N_ℓ et le dernier élément x_k est dans B_ℓ est augmentant pour C si et seulement si $x_0 \rightarrow x_1 \rightarrow \dots \rightarrow x_k$ est un chemin de G_C .

Q25. Écrire une fonction `orienter` (`c : couplage`) : `int list array` prenant en entrée un couplage C et renvoyant le tableau de listes d'adjacence représentant le graphe orienté G_C .

Pour tout $x \in S$, on note $d(x)$ la plus petite longueur parmi les longueurs des chemins alternants reliant un élément $n \in N_\ell$ à x . Si un tel chemin n'existe pas, on pose par convention $d(x) = +\infty$. Afin de manipuler efficacement cette distance, nous utiliserons par la suite l'enregistrement

```
1 type distance = {
2   d : int array;
3   dmin : int;
4 }
```

où `d` est un tableau tel que `d.(x) = d(x)` pour tout $x \in \llbracket 0, r \rrbracket$, la valeur $+\infty$ étant représenté par l'entier `max_int`, et `dmin` est la longueur $d_{\min}$ du plus court chemin augmentant pour C , avec par convention `dmin = max_int` lorsqu'un tel chemin n'existe pas.

Q26. Écrire une fonction `distance` (`c : couplage`) : `distance` prenant en entrée un couplage C et renvoyant l'enregistrement (`dist : distance`) associé à la distance d . On pourra utiliser le module `Queue` de la bibliothèque standard OCaml offrant une implémentation d'une file impérative et dont la signature est rappelée ci-dessous.

- `Queue.create () : 'a Queue.t`, création d'une file vide.
- `Queue.push (x : 'a) (f : 'a Queue.t) : unit`, enfiler un élément x dans la file f .
- `Queue.pop (f : 'a Queue.t) : 'a`, défiler et renvoyer le prochain élément en attente de la file f .
- `Queue.is_empty (f : 'a Queue.t) : bool`, renvoie `true` lorsque la file f est vide, `false` sinon.

Nous souhaitons construire le plus grand ensemble de chemins augmentants pour C tel que ces chemins n'aient aucun sommet en commun et soient tous de longueur $d_{\min}$. Un tel ensemble est appelé ensemble de chemins de Hopcroft-Karp. Pour construire cet ensemble, on commence par définir le graphe G_{HK} possédant les mêmes sommets que G_C , en ne conservant que les arcs $x \rightarrow y$ tels que $d(y) = d(x) + 1$ et $d(y) \leq d_{\min}$. Ensuite, on initialise notre ensemble de chemins à l'ensemble vide. Puis, pour chaque sommet $x \in N_\ell$, on cherche un chemin reliant x à un sommet $y \in B_\ell$ dans G_{HK} à l'aide d'un parcours en profondeur. Tant qu'un tel chemin existe, on retire de G_{HK} les sommets empruntés par notre chemin, on ajoute ce chemin à notre ensemble de chemins et on réitère l'opération. Pour notre implémentation, on ne

cherchera pas à générer le graphe G_{HK} , mais on travaillera uniquement à partir du graphe G_C .

On souhaite donc écrire une fonction `ensemble_chemin` (`c : couplage`) (`dist : distance`) : `int list list` prenant en entrée un couplage C , ainsi que l'enregistrement `dist` calculé par la fonction `distance` et renvoyant un ensemble de chemins de Hopcroft-Karp sous forme de liste. Pour cela, on introduit l'exception

```
1 exception Chemin of int list
```

et on définit la fonction

```
1 let ensemble_hc (c : couplage) (dist : distance) : int list list =
2   let r = Array.length c.graphe in
3   let visite = Array.make r false in
4   let go = oriente c in
5   let ensemble = ref [] in
6   let rec chemin (x : int) (w : int list) : unit =
7     <code1>
8   in
9   <code2>
10  !ensemble
```

Q27. Compléter la partie `<code1>` de la fonction `ensemble_hc` afin de définir la fonction

`chemin (x : int) (w : int list) : unit.`

Cette fonction doit effectuer un parcours en profondeur dans G_{HK} à partir du sommet x en ayant déjà parcouru le chemin w codé à l'envers (le dernier sommet visité est en tête de liste), et renvoyer une exception `Chemin` w lorsqu'un chemin augmentant w a été trouvé.

Q28. Compléter la partie `<code2>` de la fonction `ensemble_hc`.

Dans la question suivante, on admet le lemme de Berge : un couplage dans un graphe est maximum pour l'inclusion si et seulement si il n'existe pas de chemin augmentant.

Q29. Définir la fonction `couplage_maximum` (`g : int list array`) (`p : int`) : `int option array` prenant en entrée un graphe biparti donné par son tableau de listes d'adjacence ainsi que par son nombre de sommets noirs et renvoyant un couplage maximum pour ce graphe.

II – Remplir une casse avec le maximum de camions

Dans cette partie, on souhaite montrer que le problème qui consiste, étant donné un terrain t ainsi qu'un entier n , de déterminer s'il est possible de placer au moins n camions sur le terrain t , est NP-complet. Dans la suite, on nomme ce problème 3CASSE.

Q30. Définir ce qu'est un problème de décision appartenant à la classe NP. Définir ce qu'est un problème NP-difficile, puis un problème NP-complet.

On définit les types

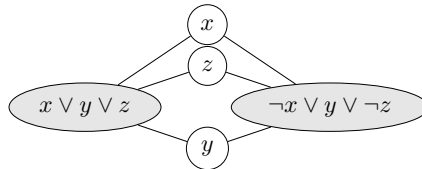
```
1 type terrain = bool array array
2 type direction = Horizontal | Vertical
3 type camion = {i : int; j : int; d : direction}
```

le type `terrain` représentant un terrain à l'aide d'un tableau bidimensionnel de booléens. Pour chaque case de cette grille, le booléen associé est égal à `true` si et seulement si elle fait partie du terrain. Le type `camion` représente quant à lui la position d'un camion sur le terrain, de manière similaire à ce qui a été décrit dans la partie A.I.

Q31. Écrire une fonction `est_valide` (`t : terrain`) (`n : int`) (`lst : camion list`) : `bool` prenant en entrée un terrain t , un entier n et une liste de positions de camions `lst` et renvoyant `true` si la liste possède n camions et qu'il est possible de les placer sur le terrain t comme spécifié. Si ce n'est pas le cas, la fonction renverra `false`.

Q32. En déduire que le problème 3CASSE est dans la classe NP.

On souhaite montrer que 3CASSE est NP-difficile. Pour cela, nous allons partir d'une variante du problème 3SAT, nommée Planar 3SAT et qui est également NP-complet. Tout comme pour 3SAT, les instances de ce problème de décision sont des formules logiques en forme normale conjonctive. Pour une formule φ on considère le graphe non orienté dont les sommets sont formés des variables et des clauses et dont les arêtes relient une variable à une clause si et seulement si la variable est présente dans la clause. Le problème Planar 3SAT est semblable à 3SAT mais restreint les formules à celles dont le graphe est planaire. Par exemple, la formule $\varphi = (x \vee y \vee z) \wedge (\neg x \vee y \vee \neg z)$ est planaire puisqu'on peut la représenter par le graphe suivant :



Étant donnée une formule φ de Planar 3SAT, nous allons construire une instance de 3CASSE. Pour cela, nous allons construire un terrain à l'aide de *gadgets*. L'essentiel de ces gadgets peut accueillir un nombre fixe de camions. Les gadgets représentant des clauses sont des exceptions et peuvent accueillir un camion en plus lorsque cette clause est vraie. Les gadgets sont reliés entre eux afin de créer des chemins, transportant des *signaux*. Chaque signal représente la valeur d'un littéral x ou de sa négation $\neg x$.

Concrètement, les gadgets sont reliés entre eux en superposant une de leur case, par laquelle sera transmis ce signal. Les gadgets sont agencés de manière à ce que tout camion placé sur le terrain se trouve entièrement sur un unique gadget. Par exemple, le terrain de la figure 5 est composé de 4 gadgets $\alpha, \beta, \gamma, \delta$ reliés par les cases a, b, c et d . Les gadgets α et γ sont reliés par la case a qui appartient aux deux gadgets. Sur ce terrain, nous avons placé deux camions. Le camion 1 appartient au gadget γ . Bien qu'il empiète sur le gadget α , on ne considère pas qu'il appartienne à ce gadget. Le camion 2 appartient quant à lui au gadget δ .

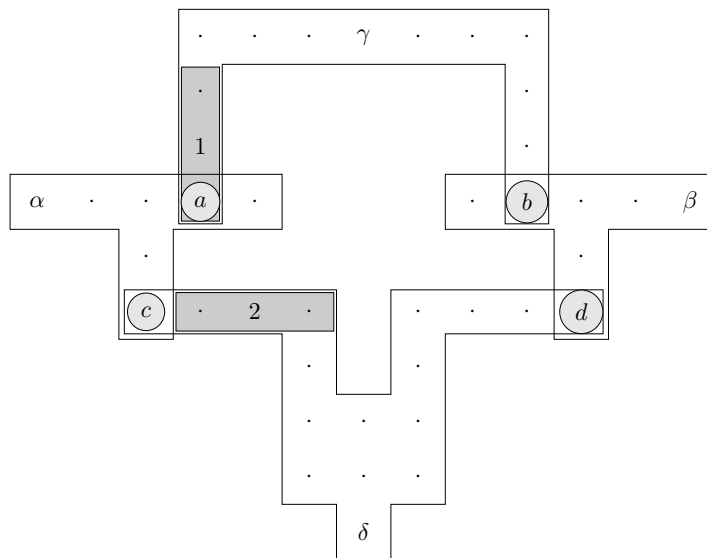
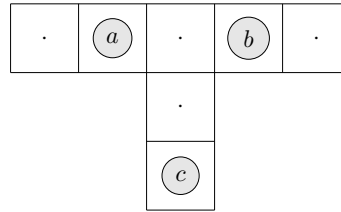


Figure 5.

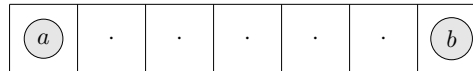
La disposition des camions sur le terrain va permettre de représenter les signaux. Si ε est un gadget et qu'un camion n'appartenant pas à ce gadget empiète sur la case e du gadget ε , on dit que le gadget ε possède une protrusion en e . On dit dans ce cas que, pour le gadget ε , le signal e est faux. Dans le cas contraire, on dit que ce signal est vrai. Par exemple, avec la disposition de camions ci-dessus, pour le gadget α , on dit que le signal a est faux et que le signal c est vrai.

Q33. Afin de représenter une clause $a \vee b \vee c$, nous allons utiliser le gadget suivant sur lequel il peut y avoir des protrusions sur les cases a, b, c .



Montrer que ce gadget permet seulement d'accueillir 0 ou 1 camion et que l'un des signaux a , b , c est vrai si et seulement si il est possible d'y accueillir un camion.

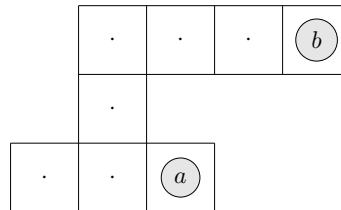
Q34. Afin de transmettre une valeur, nous allons utiliser le gadget suivant, sur lequel on souhaite placer deux camions.



Montrer que, si le signal a est faux, pour placer deux camions sur ce gadget, il est nécessaire de rendre le signal b faux pour le gadget suivant. Montrer que, si le signal a est vrai, il est possible de placer ces deux camions de manière à ce que le signal b soit vrai pour le gadget suivant.

L'idée du gadget précédent peut être généralisée afin d'obtenir un gadget permettant de transmettre un signal tout en se décalant d'un nombre de cases qui est un multiple de 3. Il est cependant nécessaire de transmettre des signaux tout en se décalant d'un nombre de cases quelconque.

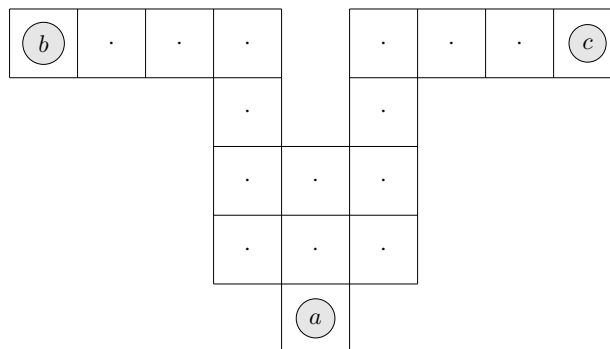
Q35. Afin de transmettre un signal tout en le décalant de deux cases, nous allons utiliser le gadget suivant, sur lequel on souhaite placer deux camions.



Montrer que, si le signal a est faux, pour placer deux camions sur ce gadget, il est nécessaire de rendre le signal b faux pour le gadget suivant. Montrer que, si le signal a est vrai, il est possible de placer ces deux camions de manière à ce que le signal b soit vrai pour le gadget suivant.

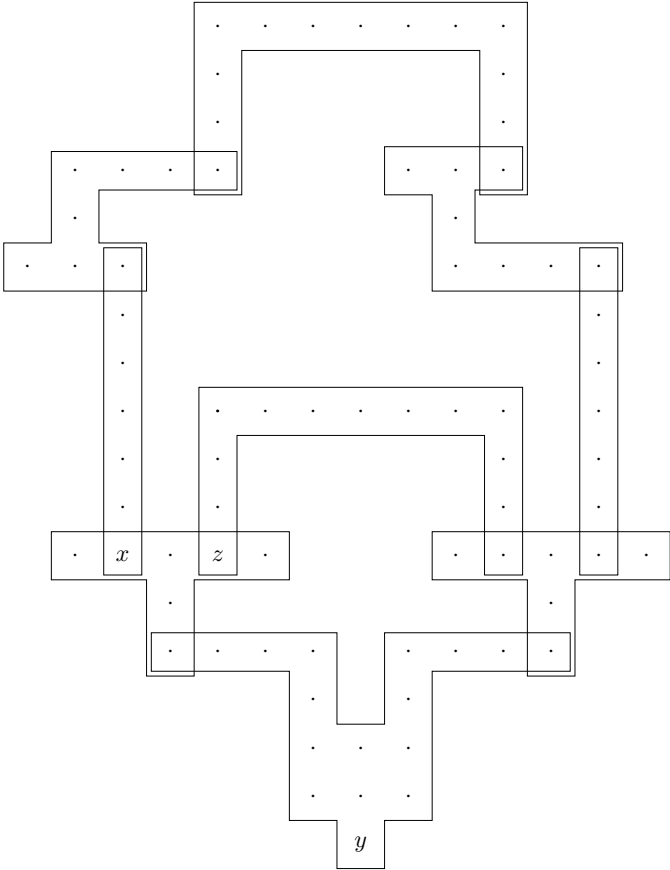
Un signal représentant un littéral x (ou sa négation $\neg x$) peut être utilisé dans plusieurs clauses. C'est pourquoi il est nécessaire de pouvoir le dupliquer.

Q36. Afin de diviser un signal en deux, nous allons utiliser le gadget suivant, sur lequel on souhaite placer cinq camions.



Montrer que, si le signal a est faux, pour placer cinq camions sur ce gadget, il est nécessaire de rendre les signaux b et c faux pour les gadgets suivants. Montrer que, si le signal a est vrai, il est possible de placer cinq camions de manière à ce que les signaux b et c soient vrais pour les gadgets suivants.

Q37. En utilisant les idées développées dans les questions 33 à 36 ainsi que le fait qu'il est possible de satisfaire la formule $\varphi = (x \vee y \vee z) \wedge (\neg x \vee y \vee \neg z)$, déterminer le plus grand nombre de camions qu'il est possible de placer sur le terrain suivant. On justifiera la réponse en donnant une telle disposition ainsi que les valeurs des variables x , y et z associées.



Q38. Montrer que le problème 3CASSE est NP-complet.

◇ Fin ◇

1020 - 6 mars 2025 - 10:48:00

SESSION 2025



MPI5IN

ÉPREUVE MUTUALISÉE AVEC E3A-POLYTECH**ÉPREUVE SPÉCIFIQUE - FILIÈRE MPI****INFORMATIQUE****Durée : 4 heures**

N.B. : le candidat attachera la plus grande importance à la clarté, à la précision et à la concision de la rédaction. Si un candidat est amené à repérer ce qui peut lui sembler être une erreur d'énoncé, il le signalera sur sa copie et devra poursuivre sa composition en expliquant les raisons des initiatives qu'il a été amené à prendre.

RAPPEL DES CONSIGNES

- Utiliser uniquement un stylo noir ou bleu foncé non effaçable pour la rédaction de votre composition ; d'autres couleurs, excepté le vert, bleu clair ou turquoise, peuvent être utilisées, mais exclusivement pour les schémas et la mise en évidence des résultats.
- Ne pas utiliser de correcteur.
- Écrire le mot FIN à la fin de votre composition.

Les calculatrices sont interdites.**Le sujet est composé de trois parties, toutes indépendantes.**

Partie I - Grammaire non contextuelle

Soit la grammaire non contextuelle $G = (\Sigma, V, P, S)$, l'ensemble des règles de production P étant défini par :

$$\begin{aligned} S &\rightarrow SaS \mid A \\ A &\rightarrow AbA \mid B \\ B &\rightarrow BcB \mid \varepsilon \end{aligned}$$

où Σ (respectivement V) est l'ensemble de symboles terminaux (respectivement non terminaux), $S \in V$ est le symbole initial de G et ε dénote le mot vide.

Soit u le mot abc .

Q1. Donner une dérivation à gauche de u . Que peut-on en déduire sur le mot u ?

Q2. Donner deux arbres de dérivation pour le mot aa . En déduire que G est ambiguë.

Une grammaire est dite *récursive gauche directe* si elle possède une règle de la forme $A \rightarrow A\alpha$, où α est une suite de symboles terminaux et non terminaux. A est dite *variable récursive gauche*. Une telle grammaire pose divers problèmes en analyse syntaxique descendante, on cherche alors à éliminer cette récursivité. Ceci est toujours possible pour les langages réguliers.

Q3. Identifier dans G les variables récursives gauches.

Pour éliminer la récursivité gauche, on utilise l'algorithme suivant :

Soit $A \rightarrow A\alpha \mid \beta$ une règle, où α est une suite de symboles terminaux et non terminaux et β est une suite de symboles terminaux et non terminaux ne commençant pas par A .

On remplace cette règle par :

- (i). une règle commençant par $A : A \rightarrow \beta A'$,
- (ii). une règle pour une nouvelle variable $A' : A' \rightarrow \alpha A' \mid \varepsilon$.

Q4. Construire la grammaire non récursive gauche directe G' équivalente à G .

Q5. Prouver que le langage reconnu par G' (ou G) est $\{a, b, c\}^*$.

Partie II - Problème de bin-packing

Cette partie comporte des questions nécessitant un code `OCaml`.

Soient $n, k \in \mathbb{N}^2$. On appelle *rangement* de n objets dans k boîtes une fonction $\mathcal{R} : \llbracket 1, n \rrbracket \rightarrow \llbracket 1, k \rrbracket$. L'ensemble $B_j = \mathcal{R}^{-1}(j) = \{i \in \llbracket 1, n \rrbracket, \mathcal{R}(i) = j\}$ est le contenu de la boîte $j \in \llbracket 1, k \rrbracket$.

Étant données des tailles $a = (a_1 \cdots a_n) \in (\mathbb{Q} \cap]0, 1])^n$, a_i étant la taille de l'objet i , on dit qu'un rangement $\mathcal{R}$ de n objets dans k boîtes est *valide* pour les tailles a si pour tout $j \in \llbracket 1, k \rrbracket$ on a $\sum_{i \in B_j} a_i \leq 1$.

On considère alors le problème de décision BIN-PACKING qui, étant donnés $n, k \in \mathbb{N}^2$ et $a \in (\mathbb{Q} \cap]0, 1])^n$, décide s'il existe un rangement $\mathcal{R}$ de n objets dans k boîtes valide pour les tailles a .

On peut décrire une solution de ce problème par un couple $(k, \mathcal{R})$.

Q6. Montrer que BIN-PACKING est dans NP.

Soit le problème PARTITION suivant :

Pour n entiers $c_1 \cdots c_n$, existe-t-il un sous-ensemble S de $\llbracket 1, n \rrbracket$, tel que $\sum_{i \in S} c_i = \sum_{i \notin S} c_i$?

On admet que le problème PARTITION est NP-complet.

Q7. Montrer par réduction depuis PARTITION que le problème BIN-PACKING est NP-complet.

On s'intéresse par la suite au problème MIN-BIN-PACKING qui demande de déterminer le plus petit k pour lequel il existe un rangement $\mathcal{R}$ de n objets de taille a dans k boîtes. On introduit pour résoudre ce problème une heuristique dite *Premier Casier Décroissant* (**algorithme 2**), se basant sur l'heuristique *Premier Casier* décrite dans l'**algorithme 1**. Dans cet algorithme, le rangement $\mathcal{R}$ est modélisé par une liste de boîtes $\mathcal{L}$.

Algorithme 1 - Algorithme Premier Casier

Entrées : $n \in \mathbb{N}, (a_1 \cdots a_n) \in (\mathbb{Q} \cap]0, 1])^n$

Sorties : Une solution $(k, \mathcal{R})$

début

$\mathcal{L}$: liste de boîtes ouvertes

$\mathcal{L} = \emptyset$

pour $i=1$ à n **faire**

 Rechercher la première boîte de $\mathcal{L}$ dans laquelle l'objet i peut être placé

si une telle boîte existe **alors**

 l'objet i est placé à l'intérieur

sinon

 une nouvelle boîte est ajoutée à $\mathcal{L}$ et l'objet i y est placé

$k := \text{longueur}(\mathcal{L})$

 Dédurre $\mathcal{R}$ de $\mathcal{L}$.

Algorithme 2 - Algorithme Premier Casier Décroissant

Entrées : $n \in \mathbb{N}, (a_1 \cdots a_n) \in (\mathbb{Q} \cap]0, 1])^n$

Sorties : Une solution $(k, \mathcal{R})$

début

σ = permutation telle que $a_{\sigma(1)} \geq a_{\sigma(2)} \cdots \geq a_{\sigma(n)}$

$k, \mathcal{R}_1 := \text{Premier Casier}(n, a_{\sigma(1)}, \cdots, a_{\sigma(n)})$

pour $i = 1$ à n **faire**

$\mathcal{R}(i) := \mathcal{R}_1(\sigma(i))$

Nous allons montrer que l'algorithme *Premier Casier Décroissant* est une $\frac{3}{2}$ -approximation pour le problème MIN-BIN-PACKING.

On note dans la suite k le nombre de boîtes retournées par l'**algorithme 2**, k^* le nombre de boîtes optimal (défini comme le nombre minimal de boîtes permettant de répondre au problème MIN-BIN-PACKING) et $k_1 = \lceil 2k/3 \rceil$, où $\lceil \cdot \rceil$ désigne la partie entière supérieure. B_{k_1} fait référence à la boîte déduite du rangement renvoyé par l'**algorithme 2**.

Q8. Montrer que si B_{k_1} contient un objet i de taille $a_i > \frac{1}{2}$, alors $k^* \geq k_1 \geq \frac{2}{3}k$.

Q9. Montrer que sinon, les boîtes $B_{k_1} \cdots B_k$ contiennent au moins $2(k - k_1) + 1$ objets, aucun d'eux ne pouvant rentrer dans les boîtes $B_1 \cdots B_{k_1-1}$.

Q10. Justifier que $\sum_{i=1}^n a_i > \min(k_1 - 1, 2(k - k_1) + 1)$. En admettant que pour $k \in \mathbb{N}$, $\frac{2}{3}k + \frac{2}{3} \geq \lceil 2k/3 \rceil$,

en déduire que $k^* \geq \left\lceil \frac{2}{3}k \right\rceil \geq \frac{2}{3}k$.

On définit des types record pour représenter les objets et les boîtes en OCaml :

```
type objet = {
  id : int; (*identifiant de la boîte*)
  taille : float; (*taille de la boîte*)
}

type boîte = {
  charge : float; (*somme des tailles des objets de la boîte*)
  objets : objet list; (*liste courante des tailles des objets dans la boîte*)
}
```

Q11. Écrire une constante `boite_vide` : `boite` représentant une boîte vide.

Q12. Écrire une fonction de signature

`ajoute_boite` : `objet` -> `boite` -> `boite`

qui renvoie la boîte obtenue en ajoutant un objet à une boîte donnée.

Q13. Écrire une fonction récursive de signature

`trouve_boite` : `boite list` -> `objet` -> `int`

telle que `trouve_boite bl o` retourne l'indice (à partir de 0) de la première boîte dans `bl` pouvant contenir `o` ou la taille de la liste `bl` si aucune boîte ne peut contenir `o`.

Q14. Écrire une fonction récursive de signature

`transforme` : `'a` -> (`'a` -> `'a`) -> `int` -> `'a list` -> `'a list`

telle que `transforme d f i l` retourne la liste `l` où l'élément `x` à l'indice `i` (à partir de 0) a été remplacé par `f x`. Si `i` est plus grand que la taille de la liste, `f d` est ajouté à la fin.

Q15. Écrire une fonction de signature

`premier_casier` : `objet list` -> `boite list`

qui applique l'**algorithme 1**. On retournera directement la liste de boîtes, on ne cherchera pas à calculer la fonction $\mathcal{R}$. On pourra utiliser la fonction `transforme` précédemment écrite.

Q16. Écrire une fonction de signature

`premier_casier_decroissant` : `objet list` -> `boite list`

qui applique l'**algorithme 2**.

On pourra utiliser `List.sort` : (`'a` -> `'a` -> `int`) -> `'a list` -> `'a list` qui trie la liste passée en argument dans l'ordre croissant suivant la fonction passée en argument (qui renvoie -1, 0 ou 1 pour inférieur, égal ou supérieur respectivement). On pourra utiliser la fonction de comparaison générique `compare` : `'a` -> `'a` -> `int`. Là encore, on retournera directement la liste des boîtes.

Partie III - Algorithmes de couplage

Cette partie comporte des questions nécessitant un code en C.

On s'intéresse ici à un problème d'appariement entre deux groupes d'individus U et V , que l'on suppose de même cardinalité n .

On cherche à appairer les éléments de V aux éléments de U , chaque élément devant être apparié à exactement un élément de l'autre ensemble. Chaque élément de V (respectivement de U) a, de plus, un ordre de préférence total entre tous les éléments de U (respectivement de V).

Définition 1 (Couplage, couplage parfait)

Un ensemble de paires $A \subseteq V \times U$ est un couplage si tout $x \in V$ et tout $y \in U$ apparaissent dans au plus un élément de A . Le couplage est dit parfait si tout $x \in V$ et tout $y \in U$ apparaissent dans un et un seul élément de A .

Notations

Dans toute la suite on notera :

- $|E|$ le cardinal de l'ensemble E ,
- $>^x$ l'ordre associé à l'élément $x \in V \cup U$: si $u_1 \in U$ préfère strictement $v_1 \in V$ à $v_2 \in V$ alors $v_1 >^{u_1} v_2$. Par extension, on définit de même la notion de préférence $\geq^x$.
- $m_A(x)$ le partenaire de $x \in V \cup U$ dans le couplage parfait A .

Définition 2 (Couplage parfait stable)

Un couplage parfait A est dit *stable* si, pour tous couples (v_1, u_1) et (v_2, u_2) de A , il est impossible que l'on ait $u_2 >^{v_1} u_1$ et $v_1 >^{u_2} v_2$.

Définition 3 (Pareto-optimalité)

Soient A et A' deux couplages parfaits stables et $E \subseteq V \cup U$. On dit que A Pareto-domine A' sur E si et seulement si pour tout $x \in E$, $m_A(x) \geq^x m_{A'}(x)$ et il existe $x \in E$ tel que $m_A(x) >^x m_{A'}(x)$. Un couplage est Pareto-optimal s'il est stable et n'est pas Pareto dominé sur $V \cup U$.

En économie, la Pareto-optimalité d'un système exprime le fait que l'on ne peut améliorer la satisfaction d'un individu du système sans diminuer la satisfaction d'un autre.

III.1 - Recherche de couplages stables

L'algorithme de Gale–Shapley (**algorithme 3**) permet de montrer qu'il existe toujours au moins un couplage parfait stable et d'en trouver un de façon efficace. L'idée de l'algorithme est tout d'abord de choisir un des deux ensembles (ici pour illustration V). Les éléments de V font des propositions aux éléments de U . Quand un élément de V fait une proposition à un élément de U , celui-ci peut soit la refuser définitivement, soit l'accepter provisoirement en attendant une éventuelle meilleure offre. Toute nouvelle acceptation vaut rejet définitif des propositions précédemment acceptées.

Algorithme 3 - Algorithme de Gale-Shapley

Entrées : $n = |V| = |U|$, les listes de préférences des éléments de V et U

Sorties : Un couplage parfait stable A

début

$A := \emptyset$

tant que il existe un élément de V non apparié **faire**

Choisir un tel v

$u :=$ élément de U que v préfère parmi ceux à qui il n'a pas encore proposé

si u n'est pas apparié dans A **alors**

$A := A \cup \{(v, u)\}$

sinon

si u est apparié dans A à v' ET $v >^u v'$ **alors**

$A := A \setminus \{(v', u)\}$

$A := A \cup \{(v, u)\}$

retourner A

Soit le jeu de données $\mathcal{D}$ suivant : $V = \{v_1, v_2, v_3\}$, $U = \{u_1, u_2, u_3\}$ et :

V	U
$u_2 >^{v_1} u_1 >^{v_1} u_3$	$v_1 >^{u_1} v_3 >^{u_1} v_2$
$u_1 >^{v_2} u_2 >^{v_2} u_3$	$v_2 >^{u_2} v_1 >^{u_2} v_3$
$u_1 >^{v_3} u_2 >^{v_3} u_3$	$v_2 >^{u_3} v_1 >^{u_3} v_3$

Q17. Donner la trace de l'**algorithme 3** sur ces données. Le choix de v dans l'algorithme se fera par ordre croissant des indices.

Q18. Montrer que l'**algorithme 3** termine.

Q19. Donner, sans le prouver, un invariant de boucle permettant de prouver que l'**algorithme 3** retourne un couplage parfait.

Q20. Montrer que le couplage parfait calculé est stable.

Q21. Montrer que cet algorithme effectue moins de n^2 itérations de boucles.

Pour des préférences données, il peut exister plus d'un couplage parfait stable. Soient A et A' deux tels couplages pour un problème donné. On dira qu'un élément de $V \cup U$ *préfère* A à A' s'il n'a pas le même partenaire dans les deux couplages et s'il préfère celui de A à celui de A' .

Q22. Montrer que si v préfère le couplage A , alors son partenaire u dans A préfère A' .

On peut montrer qu'en même temps le partenaire u' de v dans A' préfère A .

La correction de l'algorithme de Gale-Shapley ne dépend pas de l'élément $v \in V$ qui fait sa proposition à chaque tour. En fait, on montre dans la suite que, quel que soit l'élément $v \in V$ non encore apparié qui fait une proposition à chaque tour, l'algorithme calcule toujours le même couplage parfait stable. Dans la suite, on dira qu'un élément $u \in U$ est *réalisable* pour un élément $v \in V$ s'il existe un couplage parfait stable qui contient (v, u) .

Q23. Montrer qu'au cours de l'**algorithme 3**, chaque $v \in V$ n'est rejeté que par les éléments $u \in U$ qui ne sont pas réalisables pour v .

Soit $M(v) \in U$ l'élément réalisable le mieux classé sur la liste des préférences de v et $P(u) \in V$ l'élément réalisable le moins bien classé sur la liste des préférences de u . On note enfin $A^* = \{(v, M(v)), v \in V\}$.

Q24. Montrer que lors de l'exécution de l'**algorithme 3** :

- 1) Tout $v \in V$, s'il est apparié, l'est avec $u \in U$ supérieur ou égal à $M(v)$ dans sa liste de préférences.
- 2) Tout $v \in V$, s'il n'est pas apparié, n'a fait des propositions qu'à des $u \in U$ strictement supérieurs à $M(v)$ dans sa liste de préférences.

Q25. En déduire que quel que soit l'élément $v \in V$ choisi dans la boucle Tant que de l'**algorithme 3**, l'algorithme termine en produisant l'ensemble A^* .

Ainsi, l'algorithme calcule le meilleur couplage parfait stable possible du point de vue des éléments de V . Il s'avère également que l'algorithme de Gale-Shapley fait correspondre u avec $P(u)$, pour tout $u \in U$. L'algorithme avantage donc de manière claire le groupe ayant l'initiative (ici V) : le couplage stable produit est dit *Pareto-V-optimal* en ce sens qu'il n'est pas Pareto-dominé sur V .

III.2 - Couplages Pareto-optimaux

Dans cette sous-partie, on utilise également le jeu de données $\mathcal{D}$.

L'algorithme de Gale-Shapley ne retourne pas nécessairement un couplage Pareto-optimal. On s'intéresse ici à un algorithme susceptible de produire cette propriété.

Pour construire un couplage Pareto-optimal, on représente les données à l'aide d'un graphe orienté $\mathcal{G} = (S, E)$ où :

- les sommets S sont les éléments de $V \cup U$,
- les arcs E représentent les premiers choix : $(v, u) \in E$ si et seulement si u est le premier choix de v dans sa liste de préférences et $(u, v) \in E$ si et seulement si v est le premier choix de u .

Q26. Montrer que $\mathcal{G}$ contient au moins un circuit, c'est-à-dire un chemin dont les deux sommets extrémités sont identiques.

Q27. Justifier par l'absurde que tout $x \in V \cup U$ est dans au plus un circuit.

On propose alors l'**algorithme 4** pour calculer un couplage, dont on admettra qu'il est Pareto-optimal pour V .

Algorithme 4 - Algorithme de couplage par graphe orienté

Entrées : $n = |V| = |U|$, les listes de préférences des éléments de V et U

Sorties : Un couplage A

début

$A := \emptyset$

tant que *il existe un élément de V non apparié* **faire**

 Construire le graphe $\mathcal{G}$ sur les données courantes

pour chaque *arc (v, u) présent dans un circuit de $\mathcal{G}$* **faire**

$A = A \cup \{(v, u)\}$

 Supprimer du problème tous les $v \in V$ et les $u \in U$ appariés

 Mettre à jour les listes de préférences des individus restants

retourner A

Q28. Donner les graphes et la construction itérative de A produits par l'**algorithme 4** sur $\mathcal{D}$.

Q29. En utilisant les données $\mathcal{D}$, montrer que l'**algorithme 4** ne produit pas toujours un couplage stable.

On s'intéresse alors à une autre forme de stabilité pour assurer à la fois une propriété de Pareto et une stabilité dans les couplages.

Définition 4 (P-stabilité)

Un couplage A est P-stable s'il n'existe aucune paire $(v, v') \in V^2$ telle que v préfère $m_A(v')$ et v' préfère $m_A(v)$.

Q30. Montrer par l'absurde que l'**algorithme 4** produit un couplage P-stable.

Q31. Montrer que l'**algorithme 3** ne produit pas nécessairement un couplage P-stable.

III.3 - Implémentation

Dans la suite, les individus sont numérotés de 0 à $n - 1$. Ainsi $V = \{0, \dots, n - 1\}$ et $U = \{0, \dots, n - 1\}$.

On suppose disposer de deux matrices de taille $n \times n$ L_v et L_u donnant respectivement les listes de préférences des éléments de V et U : ainsi, $L_u[i][j]$ est le j -ème élément de V préféré par l'élément $i \in U$.

On définit ces tableaux en C par l'intermédiaire du code suivant :

```
/* La directive #define est utilisée pour définir une valeur pour la constante n qui
   servira à déclarer des tableaux de taille fixe. */
#define n 4

int main(void) {
    int Lv[n][n], Lu[n][n];
    (...)
}
```

III.3.1 - Algorithme de Gale-Shapley

Pour connaître rapidement le classement des éléments de V dans les listes de préférences des éléments de U , on définit un tableau $Rang$ de taille $n \times n$, tel que $Rang[i][j]$ donne le rang de $j \in V$ dans la liste de préférences de l'élément $i \in U$. Par convention, les rangs commencent à 0.

Q32. Écrire une fonction de prototype

void calcule_rang(int Lu[n][n], int Rang[n][n])

qui construit le tableau $Rang$.

Afin de pouvoir ajouter ou supprimer rapidement des couples (v, u) au couplage parfait stable A , on définit deux tableaux `CoupleV` et `CoupleU` tels que `CoupleV[i]` ou `CoupleU[i]` est le partenaire actuel de i . On définit enfin un dernier tableau `USuiv` de taille n stockant pour chaque élément de V le rang du prochain élément de U à qui il peut faire une proposition.

Q33. Proposer des valeurs d'initialisation pour ces trois tableaux.

Q34. Avec ces structures de données, écrire une fonction de prototype

```
void gale_shapley(int Lv[n][n], int Lu[n][n], int CoupleU[n], int CoupleV[n])
```

qui réalise l'**algorithme 3**. On ne retournera pas le couplage parfait stable, on remplira seulement les variables `CoupleU` et `CoupleV` passées en argument. Pour le choix de v , on prendra les éléments de V par ordre croissant de leur numéro en choisissant à chaque fois le premier libre.

III.3.2 - Algorithme de couplage par graphe orienté

Sans explicitement coder tout l'**algorithme 4** qui requiert de rechercher les circuits dans $\mathcal{G}$, on se propose de coder quelques fonctions utiles à la réalisation de l'algorithme.

Pour pouvoir numéroter les sommets du graphe de manière continue, les individus sont maintenant numérotés de 0 à $2n - 1$ de la manière suivante : $V = \{0, \dots, n - 1\}$ et $U = \{n, \dots, 2n - 1\}$.

On code le graphe orienté $\mathcal{G}$ par listes d'adjacence. On propose les types suivants :

```
struct noeud {
    int individu;           // Element de U union V
    struct noeud* suivant;
};
typedef struct noeud *liste;

struct graphe_s {
    int nb_sommets;
    liste *liste_adjacence;
};
typedef struct graphe_s graphe;
```

Q35. Écrire une fonction de prototype

```
void ajoute_arc(graphe *g, int i, int j)
```

qui ajoute l'arc (i, j) au graphe $\mathcal{G}$.

Q36. Écrire une fonction de prototype

```
void supprime_arc(graphe *g, int i, int j)
```

qui supprime l'arc (i, j) du graphe $\mathcal{G}$.

Q37. Écrire une fonction de prototype

```
void supprime_sommet(graphe *g, int s)
```

qui supprime le sommet s du graphe $\mathcal{G}$. Pour ce faire, on supprimera uniquement tous les arcs dont ce sommet est origine ou destination.

Q38. Écrire une fonction de prototype

```
graphe *construit_graphe(int Lv[n][n], int Lu[n][n])
```

qui construit le graphe $\mathcal{G}$ utilisé dans l'**algorithme 4** et dont la définition est donnée juste avant la **Q26**. On prendra bien garde à la numérotation des éléments de U .

FIN

SESSION 2025



MP7IN

ÉPREUVE MUTUALISÉE AVEC E3A-POLYTECH**ÉPREUVE SPÉCIFIQUE - FILIÈRE MP****INFORMATIQUE****Durée : 4 heures**

N.B. : le candidat attachera la plus grande importance à la clarté, à la précision et à la concision de la rédaction. Si un candidat est amené à repérer ce qui peut lui sembler être une erreur d'énoncé, il le signalera sur sa copie et devra poursuivre sa composition en expliquant les raisons des initiatives qu'il a été amené à prendre.

RAPPEL DES CONSIGNES

- Utiliser uniquement un stylo noir ou bleu foncé non effaçable pour la rédaction de votre composition ; d'autres couleurs, excepté le vert, bleu clair ou turquoise, peuvent être utilisées, mais exclusivement pour les schémas et la mise en évidence des résultats.
- Ne pas utiliser de correcteur.
- Écrire le mot FIN à la fin de votre composition.

Les calculatrices sont interdites.

Le sujet est constitué d'un unique problème et comporte cinq parties qui ne sont pas complètement indépendantes.

Répartition de la gestion d'un réseau minimisant la bande passante

Le Listenbourg (**figure 1**) est une république fictive de la péninsule ibérique, comportant environ 66 millions d'habitants, née sur les réseaux sociaux en octobre 2022. Au Listenbourg deux fournisseurs d'accès, MaxDébit et MinLatence, cherchent à se partager la gestion d'un tout nouveau réseau : l'Ultraneet.

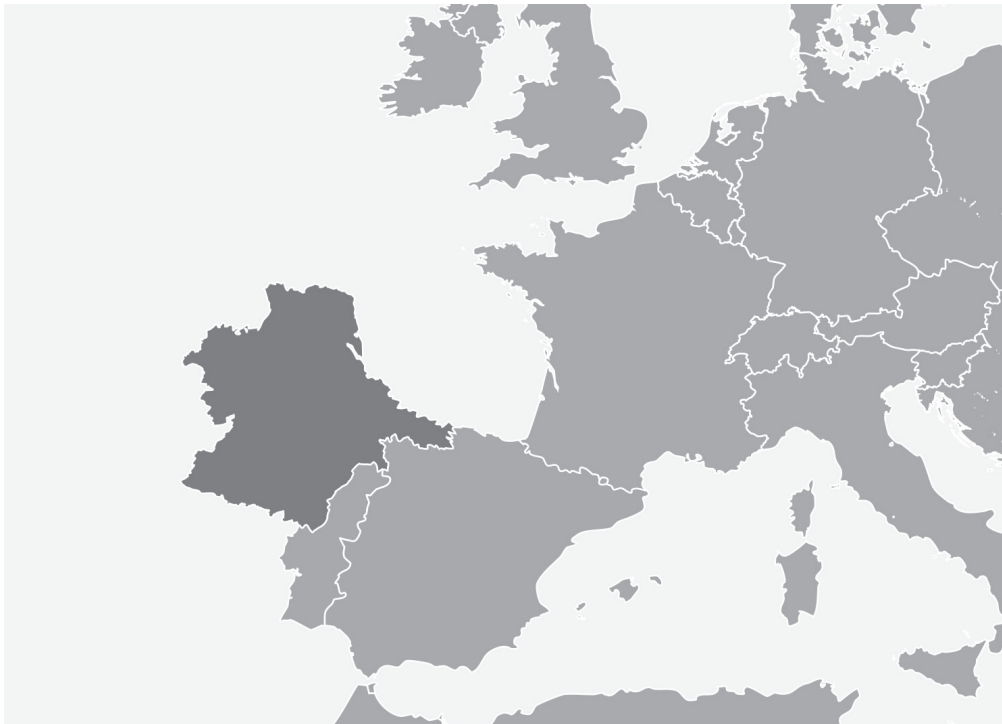


Figure 1 - La république imaginaire du Listenbourg

Le graphe simplifié du réseau Ultraneet de ce pays est représenté à la **figure 2**. Le nom des villes correspondant aux identifiants des sommets est enregistré dans la base de données (**figure 3**). Une arête relie deux villes lorsqu'un câblage physique direct est établi entre elles et la bande passante permise par ce câblage est indiquée comme poids. Il y a au plus une arête entre deux villes. La structure du réseau Ultraneet a été intégralement terminée lors du grand plan « Réseau pour Tous » mené par le premier ministre Petro Sank-Ærixh et il s'agit désormais d'en confier l'exploitation aux deux fournisseurs d'accès concurrents.

Dans ce sujet, on s'intéresse à une procédure de partage de la gestion du réseau entre les deux opérateurs. On suppose qu'une liaison entre deux villes ne peut être exploitée que par l'un ou par l'autre des deux fournisseurs d'accès qui en aura alors l'usage exclusif. Le gouvernement du Listenbourg ne souhaite pas s'embarrasser avec une procédure complexe d'appel d'offre et impose la procédure simple suivante pour répartir l'exploitation exclusive des liaisons physiques du réseau :

- tant qu'il reste des liaisons à attribuer, chaque opérateur, à tour de rôle et en commençant par MaxDébit, choisit parmi les liaisons restantes une liaison qu'il souhaite exploiter exclusivement.

Une fois la procédure de répartition terminée, c'est-à-dire une fois que toutes les liaisons ont été attribuées, chaque fournisseur d'accès dispose de son propre sous-réseau exclusif, dont la gestion lui revient entièrement. L'objectif d'un fournisseur d'accès est d'obtenir un sous-réseau permettant de proposer la meilleure bande passante possible entre deux villes quelconques du pays.

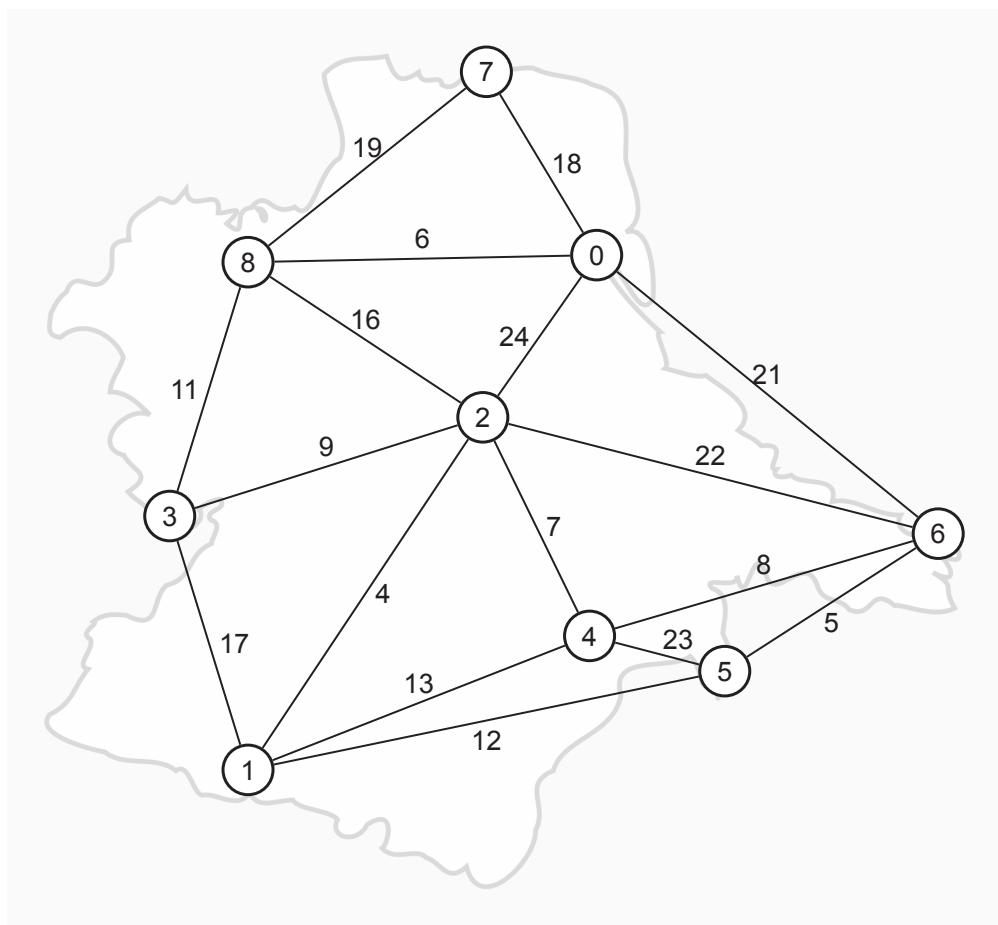


Figure 2 - Réseau simplifié du Listenbourg

La première partie du sujet s'intéresse à l'étude d'une base de données du réseau mise à disposition des fournisseurs d'accès (informatique de tronc commun, langage SQL). La deuxième partie introduit et étudie quelques propriétés de l'arbre couvrant de poids maximal (option informatique). La troisième partie s'intéresse à la recherche de l'arbre couvrant de poids maximal en utilisant l'algorithme de Borůvka (option informatique, langage OCaml). La quatrième partie se propose de montrer que la bande passante limite d'un réseau correspond au poids de l'arête de poids minimal de l'arbre couvrant de poids maximal du graphe (option informatique). La cinquième partie propose de modéliser la procédure de répartition de la gestion du réseau comme un jeu à deux joueurs (informatique de tronc commun, langage Python).

Les cinq parties sont essentiellement indépendantes dans leur traitement mais des notions utiles à la résolution d'une partie peuvent être introduites dans les parties précédentes. La **partie I** est indépendante des quatre autres. La **partie II** introduit des notions utiles à la résolution des **parties III** et **IV**. La **partie IV** introduit des concepts utilisés dans la **partie V**.

Dans tout le sujet, il est toujours admis d'utiliser un résultat ou un programme correspondant à une question précédente, même si cette question n'a pas été résolue.

Partie I - Base de données du réseau

Le ministère des affaires environnementales, des ressources, de l'agriculture et des forêts du Listenbourg met à disposition des deux fournisseurs d'accès une base de données relationnelle. Celle-ci comporte deux tables dont le schéma est le suivant :

- **villes**(id : *entier*, nom : *texte*, pop : *flottant*)
- **liaisons**(id1 : *entier*, id2 : *entier*, bp : *flottant*)

Un enregistrement de la table **villes** comporte l'identifiant unique d'une ville (id), le nom de cette ville (nom) et sa population en millions d'habitants (pop). *A priori*, plusieurs villes du Listenbourg peuvent porter le même nom. Un enregistrement de la table **liaisons** correspond à une liaison physique directe entre deux villes données par leur identifiant (id1, id2) ainsi que la bande passante en $Tb.s^{-1}$ correspondant à ce câblage (bp). On garantit que dans la représentation d'une liaison, l'identifiant de la première ville est toujours strictement inférieur à celui de la deuxième ville. Rappelons qu'il ne peut exister qu'au plus une liaison entre deux villes.

Un exemple simplifié du contenu de cette base de données, correspondant au graphe simplifié du réseau du Listenbourg de la **figure 2**, est donné **figure 3**.

villes		
id	nom	pop
0	Lurenberg	12.0
1	Veroja	10.0
2	Aschlöss	8.0
3	Stratord	5.2
4	Gossard	5.1
5	La Galinera	5.0
6	Sainte Marie	5.0
7	Atlanitkischer	4.7
8	Gasparländ	3.5

liaisons		
id1	id2	bp
0	2	24.0
0	6	21.0
0	7	18.0
0	8	6.0
1	2	4.0
1	3	17.0
1	4	13.0
1	5	12.0
2	3	9.0
2	4	7.0
2	6	22.0
2	8	16.0
3	8	11.0
4	5	23.0
4	6	8.0
5	6	5.0
7	8	19.0

Figure 3 - Contenu de la base de données correspondant au réseau simplifié de la **figure 2**

- Q1.** Au vu de la situation modélisée, proposer des clés primaires pour les tables **villes** et **liaisons**, en justifiant succinctement. Identifier les clés étrangères présentes dans ce schéma relationnel.
- Q2.** Pour chacune des questions suivantes on demande d'écrire une requête SQL portant sur le schéma relationnel proposé qui fonctionnerait quel que soit le contenu de la base de données et pas uniquement sur l'exemple de contenu proposé à la **figure 3**.
- (a) Écrire une requête SQL qui donne les noms des villes comportant strictement plus de cinq millions d'habitants ;
 - (b) Écrire une requête SQL qui donne la bande passante moyenne des liaisons incidentes à la ville d'identifiant 2 ;
 - (c) Écrire une requête SQL qui donne les noms des deux villes reliées par la liaison de bande passante maximale. *On suppose que toutes les liaisons ont une bande passante différente.*
- Q3.** Déterminer le résultat de la requête SQL suivante sur l'exemple de contenu de la base de données représentée à la **figure 3**.

```
SELECT nom, COUNT(*) AS d
FROM villes
  JOIN (SELECT id1 AS x, id2 FROM liaisons UNION SELECT id2 AS x, id1 FROM liaisons)
    ON id = x
GROUP BY id
HAVING d >= 4
ORDER BY id ASC
```

Partie II - Arbre couvrant de poids maximal

II.1 - Acyclicité, connexité et arbres

Un *graphe* (non orienté) est un couple $G = (S, A)$ formé d'un ensemble S fini non vide de *sommets* et d'un ensemble A d'*arêtes* constitué de parties de S de cardinal 2. On note $|S|$ le nombre de sommets de G et $|A|$ le nombre d'arêtes de G . Les voisins d'un sommet $x \in S$ sont notés $\mathcal{V}(x)$ et leur nombre $d(x) = |\mathcal{V}(x)|$ est le *degré* de x .

Un chemin dans un graphe est une suite $c = x_0 x_1 \dots x_n$ de sommets avec $\forall i \in \llbracket 0, n-1 \rrbracket, \{x_i, x_{i+1}\} \in A$. La longueur $|c|$ d'un tel chemin est $n \in \mathbb{N}$. Un chemin est *élémentaire* si tous les sommets empruntés par ce chemin sont deux à deux distincts. Un chemin est *simple* si toutes les arêtes empruntées par ce chemin sont deux à deux distinctes.

Un *cycle* est un chemin simple $x_0 x_1 \dots x_n$ avec $n \geq 3$ et $x_0 = x_n$. Un graphe est *acyclique* s'il ne comporte pas de cycle.

Un graphe est *connexe* si deux sommets quelconques sont toujours reliés par un chemin. Une *composante connexe* est un ensemble $C \subseteq S$ de sommets dont tous les couples de sommets sont reliés par au moins un chemin.

Un *arbre* est un graphe connexe acyclique.

Si $a = \{x, y\}$ est une partie de S de cardinal 2, on note $G + a$ le graphe $(S, A \cup \{a\})$, c'est-à-dire le graphe G auquel on a ajouté l'arête a et $G - a$ le graphe $(S, A \setminus \{a\})$, c'est-à-dire le graphe G auquel on a retiré l'arête a .

Q4. Montrer que si $G = (S, A)$ est un graphe connexe et que si $a = \{x, y\} \in A$ est une arête de G appartenant à un cycle de G , alors le graphe $G - a$ est encore connexe.

On admet la proposition suivante :

Proposition 1 : relation entre $|S|$ et $|A|$ pour les graphes connexes ou acycliques

Soit $G = (S, A)$ un graphe :

- (a) si G est connexe alors $|A| \geq |S| - 1$;
- (b) si G est acyclique alors $|A| \leq |S| - 1$.

Q5. Montrer que si $G = (S, A)$ est connexe et que $|A| = |S| - 1$ alors G est un arbre.

II.2 - Sous-graphe et arbre couvrant

Dans tout ce sujet un *sous-graphe* d'un graphe G est un graphe $G' = (S, A')$ avec $A' \subseteq A$. Notons qu'un sous-graphe de G comporte exactement les mêmes sommets que G . On identifie un sous-graphe $G' = (S, A')$ avec le sous-ensemble d'arêtes $A' \subseteq A$. Un *arbre couvrant* de G est un sous-graphe de G qui est un arbre.

Q6. Montrer qu'un graphe $G = (S, A)$ est connexe si et seulement s'il admet un arbre couvrant.

Q7. Soit $T = (S, A')$ un arbre couvrant de G avec $A' \subseteq A$ et $a \in A \setminus A'$.

- (a) Justifier que $T + a$ n'est pas acyclique.
- (b) Soit $a' \in A'$ une arête d'un cycle de $T + a$. Montrer que $T + a - a'$ est un arbre couvrant de G .

II.3 - Graphes pondérés et arbre couvrant de poids maximal

Un *graphe pondéré* est un triplet (S, A, p) où (S, A) est un graphe et $p : A \rightarrow \mathbb{R}$ une fonction de pondération des arêtes. Si $\{x, y\} \in A$ est une arête, $p(\{x, y\})$ est appelé le *poids* de cette arête. Le poids d'un graphe pondéré est la somme des poids de ses arêtes :

$$p(G) = \sum_{a \in A} p(a).$$

On étend la notion de sous-graphe aux graphes pondérés de la manière suivante : un sous-graphe d'un graphe pondéré $G = (S, A, p)$ est un graphe pondéré $G' = (S, A', p_{A'})$ où $A' \subseteq A$ et où $p_{A'}$ est la restriction de p à A' . On se permettra de ne pas systématiquement indiquer la fonction de pondération.

Le plan « Réseau pour Tous » du gouvernement a permis d'interconnecter toutes les villes du Listebourg. On considérera donc en général un graphe pondéré $G = (S, A, p)$ non orienté *connexe*, avec la fonction de pondération correspondant à la bande passante associée à chaque liaison. Chaque province ayant opté pour son propre câblage, deux villes ne sont jamais reliées par le même type de câblage et la bande passante de deux liaisons est *toujours* différente. Ceci se traduit donc par une fonction de pondération p injective : dans toute la suite du sujet, les poids des arêtes de G sont toujours deux à deux distincts.

On se propose de démontrer la **proposition 2** suivante :

Proposition 2 : existence et unicité de l'arbre couvrant de poids maximal

Soit $G = (S, A, p)$ un graphe *connexe* muni d'une fonction de pondération *injective*. Alors, il existe un unique arbre couvrant de poids maximal de G . On note $T^* = (S, A^*)$ cet arbre couvrant.

Q8. Montrer l'existence d'un arbre couvrant de poids maximal de G .

- Q9.** Pour montrer l'unicité, on se propose de procéder par l'absurde. Supposons l'existence de deux arbres couvrants de poids maximal $T_1 = (S, A_1)$ et $T_2 = (S, A_2)$ différents. Considérons une arête de poids maximal parmi les arêtes qui appartiennent à un seul des deux arbres, c'est-à-dire une arête de poids maximal dans $(A_1 \setminus A_2) \cup (A_2 \setminus A_1)$, qui est bien non vide puisque $A_1 \neq A_2$. Sans perte de généralité, on peut noter a_2 une telle arête et supposer que $a_2 \in A_2 \setminus A_1$.
- (a) Montrer qu'il existe une arête $a_1 \in A_1 \setminus A_2$ sur un cycle de $T_1 + a_2$.
 - (b) Conclure en considérant le sous-graphe $T_1 + a_2 - a_1$.

Partie III - Recherche d'un arbre couvrant de poids maximal

III.1 - Rappels et fonctions élémentaires en OCaml

On rappelle que, dans le langage OCaml :

- on peut représenter des tableaux par le type `'a array` ;
- si `e` est une expression de type `'a`, l'expression `Array.make n e` permet de créer un tableau de type `'a array` de $n \in \mathbb{N}$ cases dont toutes les cases contiennent la valeur de l'expression `e` ;

```
Array.make : int -> 'a -> 'a array
```

- si `a` est un tableau de type `'a array` de taille $n \in \mathbb{N}$, l'expression `Array.length a` permet d'obtenir la valeur `n` ;

```
Array.length : 'a array -> int
```

- si $0 \leq i < n$, l'expression `a.(i)` s'évalue en la valeur contenue à la case `i` du tableau `a` et si `e` est une expression de type `'a`, l'expression `a.(i) <- e` permet de remplacer la valeur contenue à la case `i` du tableau `a` par la valeur de l'expression `e` ;
- la fonction `List.iter` a le comportement suivant : si `f` est une fonction de type `'a -> unit`, alors l'expression `List.iter f [a1; a2; ...; an]` est équivalente à `f a1; f a2; ...; f an`

```
List.iter : ('a -> unit) -> 'a list -> unit
```

- une boucle `for i = a to b do ... done` permet de faire évoluer une variable `i` entre les valeurs entières `a` et `b` **incluses** ;
 - il existe deux fonctions `max` et `min` de type `'a -> 'a -> 'a`.
- Q10.** Écrire une fonction `max_tab : int array -> int` telle que, pour un tableau d'entiers `a` de taille $n \geq 1$, `max_tab a` renvoie la valeur du plus grand entier contenu dans `a`. On attend une complexité linéaire en n , ce que l'on justifiera en une phrase.

III.2 - Représentation des graphes en OCaml

On représente un graphe pondéré $G = (S, A, p)$, avec $S = \llbracket 0, n - 1 \rrbracket$ et $n \in \mathbb{N}^*$ par un tableau de listes d'adjacence pondérées.

```
type graphe = (int * float) list array
```

Si g représente un graphe et si $0 \leq i < n$ est un sommet, alors $g.(i)$ contient la liste des couples $\left\{ (j, p(\{i, j\})) \right\}_{j \in V(i)}$, c'est-à-dire la liste des couples de la forme (j, w) avec $w = p(\{i, j\})$ et j voisin de i .

Remarquons que puisque les graphes sont non orientés, si i est dans la liste d'adjacence de j , alors j est dans la liste d'adjacence de i avec le même poids.

Par exemple, si g est le graphe de la **figure 2**, on a $g.(0)$ qui vaut par exemple :

```
[(2, 24.0); (6, 21.0); (7, 18.0); (8, 6.0)]
```

Le nombre de sommets du graphe correspond donc à la taille du tableau de listes d'adjacences :

```
let nb_sommets g = Array.length g
```

Q11. Donner une valeur possible pour $g.(7)$.

On représente une arête par un triplet (w, i, j) où $w = p(\{i, j\})$ et où $i < j$. Remarquons que chaque arête est ainsi représentée exactement une fois et qu'une arête commence par son poids.

Q12. Écrire en OCaml une fonction `ajoute_arete : graphe -> (float * int * int) -> unit` telle que `ajoute_arete g (w, i, j)` permet d'ajouter au graphe g l'arête (w, i, j) . On suppose que cette arête n'était pas déjà présente et la complexité doit être en temps constant.

Q13. Écrire en OCaml une fonction `toutes_les_aretes : graphe -> (float * int * int) list` qui permet d'obtenir la liste des arêtes d'un graphe avec la convention ci-dessus. On attend une complexité linéaire en $|S| + |A|$, ce que l'on justifiera rapidement.

Si a_1 et a_2 sont deux arêtes de type `float * int * int`, l'expression $a_1 < a_2$ compare les arêtes pour l'ordre lexicographique des triplets donc pour la valeur du poids de l'arête en premier. Comme dans ce sujet on suppose toujours que tous les poids sont deux à deux distincts, on peut directement utiliser l'ordre OCaml `<` sur les arêtes pour comparer les poids des arêtes. Ainsi par exemple, `min a1 a2` s'évalue en l'arête de poids minimal entre a_1 et a_2 .

Q14. Écrire une fonction `min_arete : (float * int * int) list -> (float * int * int)` de manière récursive en OCaml qui renvoie l'arête de poids minimal d'une liste d'arêtes supposée non vide. Quelle est sa complexité ?

III.3 - Recherche des composantes connexes

Pour un graphe $G = (S, A, p)$ avec $S = \llbracket 0, n-1 \rrbracket$, on cherche à construire un tableau c de taille n représentant les composantes connexes du graphe. Les composantes connexes sont indexées entre 0 et $p-1$ où p est le nombre de composantes connexes du graphe. Pour $0 \leq i < n$ la case $c.(i)$ comporte l'indice k de la composante connexe du sommet i (ou la valeur -1 si cette composante n'a pas encore été déterminée).

On considère la fonction `explorer : graphe -> int array -> int -> int -> unit` suivante pour un graphe g , un tableau c de taille n , un identifiant de composante connexe $0 \leq k < p$ et un sommet de départ s , vérifiant la convention suivante : « un sommet $0 \leq i < n$ a été visité si et seulement si $c.(i) \geq 0$ ».

```

1 let rec explorer g c k s =
2   if c.(s) < 0 then begin
3     c.(s) <- k;
4     List.iter (fun (v, _) -> explorer g c k v) g.(s)
5   end

```

Q15. Décrire précisément ce que réalise un appel à `explorer g c k s`.

Q16. Écrire en OCaml une fonction `composantes_connexes : graphe -> int array` telle que, pour un graphe g , `composantes_connexes g` s'évalue en le tableau c des indices des composantes connexes. Cette fonction doit avoir une complexité en $O(|S| + |A|)$ que l'on ne demande pas de justifier.

Q17. En déduire une fonction `est_connexe : graphe -> bool` telle que `est_connexe g` s'évalue à `true` si le graphe g est connexe et à `false` sinon.

III.4 - Algorithme de Borůvka

Dans toute cette sous-partie, on considère un graphe $G = (S, A, p)$ connexe avec une fonction de pondération p injective. On note $T^* = (S, A^*)$ l'unique arbre couvrant de poids maximal de G .

Considérons $H = (S, B)$ un sous-graphe acyclique de G , avec donc $B \subseteq A$. On considère $C \subseteq S$ une composante connexe de H . Une arête **sûre** pour C est une arête de A ayant exactement une extrémité dans C dont le poids est maximal parmi les arêtes qui ont exactement une extrémité dans C .

Par exemple, pour le sous-graphe H de la **figure 4** et pour la composante connexe $\{1, 4, 5, 6\}$ l'arête de poids 22 est sûre (maximale parmi les arêtes de poids 4, 7, 17, 21 et 22 qui sont celles ayant exactement une extrémité dans cette composante). Sauf si H est connexe, chaque composante connexe de H comporte exactement une arête sûre. Une même arête peut être sûre pour deux composantes connexes C et C' . Dans l'exemple de la **figure 4**, l'arête de poids 22 est également l'arête sûre pour la composante connexe $\{0, 2\}$.

Q18. Déterminer l'arête sûre pour la composante $\{3\}$ ainsi que celle pour la composante $\{7, 8\}$.

Q19. On suppose dans cette question que $H = (S, B)$ est un sous-graphe acyclique de l'arbre couvrant $T^* = (S, A^*)$ de poids maximal de G , et donc que $B \subseteq A^*$. Montrer que A^* contient toutes les arêtes sûres des composantes connexes de H .

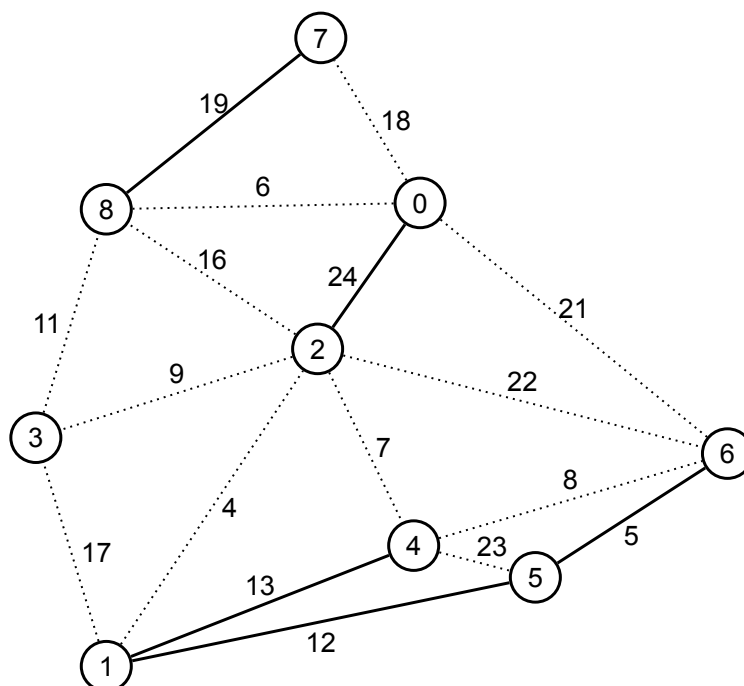


Figure 4 - Un sous-graphe H du graphe de la **figure 2**, comportant quatre composantes connexes. Les arêtes qui n'appartiennent pas au sous-graphe sont représentées en pointillés

L'idée de l'algorithme de Borůvka, dont le pseudocode est donné par l'**algorithme 1**, est de construire progressivement un sous-graphe acyclique $H = (S, B)$ de l'arbre couvrant de poids maximal $T^* = (S, A^*)$. Au départ $B = \emptyset$. À chaque étape, on ajoute au sous-graphe acyclique H en cours de construction toutes les arêtes sûres pour les composantes connexes de H .

Algorithme 1 : algorithme de Borůvka

Entrée : Graphe $G = (S, A, p)$ pondéré injectivement et connexe

Sortie : $H = (S, B)$ l'arbre couvrant de poids maximal de G

```

1  $H = (S, \emptyset)$ ;
2 tant que  $H$  n'est pas connexe faire
3   | ajouter à  $H$  toutes les arêtes sûres pour les composantes connexes de  $H$ ;
4 fin
```

Q20. Donner le déroulé de l'**algorithme 1** sur le graphe G de la **figure 2**. Indiquer à chaque étape et pour chaque composante connexe son arête sûre et indiquer l'ensemble des arêtes sûres ajoutées à chaque étape.

Q21. Montrer que l'algorithme de Borůvka termine.

Q22. Montrer que l'algorithme de Borůvka renvoie l'arbre couvrant de poids maximal $T^* = (S, A^*)$ de G .

On considère la fonction suivante qui prend en entrée un graphe g représentant le graphe $G = (S, A, p)$ connexe pondéré et un graphe h représentant un sous-graphe acyclique $H = (S, B)$ de G supposé non encore connexe.

```

1  let aretes_sures g h =
2    let c = composantes_connexes h in
3    let p = max_tab c + 1 in
4    let aretes = toutes_les_aretes g in
5    let sures = Array.make p (min_arete aretes) in
6    let traite a =
7      let (_, i, j) = a in
8      if c.(i) <> c.(j) then begin
9        sures.(c.(i)) <- max a sures.(c.(i));
10       sures.(c.(j)) <- max a sures.(c.(j));
11      end
12    in
13    List.iter traite aretes;
14    sures

```

Q23. Déterminer le type de cette fonction. Expliquer ce que réalise cette fonction et préciser la structure et le contenu de la valeur renvoyée. Déterminer sa complexité en fonction de $|A|$ en justifiant. *On remarquera que d'après la **proposition 1**, puisque le graphe G est connexe, on a $|S| - 1 \leq |A|$ et donc qu'un $O(|S|)$ est un $O(|A|)$.*

On suppose disposer d'une fonction :

```
ajoute_aretes_sures : graphe -> (float * int * int) array -> unit
```

telle que si h représente un sous-graphe acyclique de G et que $sures$ est la valeur renvoyée par `aretes_sures g h` alors `ajoute_aretes_sures h sures` ajoute à h toutes les arêtes contenues dans $sures$. On garantit que cette fonction est en complexité $O(|A|)$ et qu'elle n'ajoute qu'une seule fois une même arête à h même si cette arête est sûre pour deux composantes connexes différentes.

Q24. Écrire en OCaml une fonction `borukva : graphe -> graphe` telle que `borukva g` sur un graphe g s'évalue en un sous-graphe h de G correspondant à son arbre couvrant de poids maximal.

Q25. On s'intéresse maintenant à la complexité temporelle de cet algorithme.

- (a) Montrer qu'à chaque passage dans la boucle **tant que** à la ligne 2 de l'**algorithme 1**, le nombre de composantes connexes de H est divisé au moins par 2.
- (b) En déduire la complexité temporelle dans le pire cas de l'algorithme de Borůvka.
- (c) Quelle est la complexité temporelle de cet algorithme dans le meilleur cas ?

Partie IV - Chemin de bande passante maximale

Dans cette partie, on considère un graphe pondéré $G = (S, A, p)$, non nécessairement connexe, mais dont la fonction de pondération est injective.

On définit la *bande passante* d'un chemin $c = x_0x_1 \dots x_n$ comme la quantité :

$$\bar{b}(c) = \min_{0 \leq i \leq n-1} p(\{x_i, x_{i+1}\})$$

c'est-à-dire le poids minimal d'une arête de ce chemin. En effet, la bande passante d'un chemin est limitée par la plus petite bande passante des arêtes de ce chemin. La bande passante d'un chemin de longueur nulle est $+\infty$.

Pour deux sommets $x, y \in S$, un *chemin de bande passante maximale* entre x et y est un chemin c qui relie x à y dont la bande passante est maximale parmi tous les chemins entre x et y . Un chemin de bande passante maximale est ainsi un chemin permettant de maximiser la bande passante entre deux villes, ce que cherchent à réaliser les opérateurs MaxDébit et MinLatence. On note :

$$\bar{b}_{\max}(x, y) = \max \{ \bar{b}(c) \mid c \text{ chemin de } x \text{ à } y \}$$

la bande passante d'un chemin de bande passante maximale de x à y . On a $\bar{b}_{\max}(x, y) = -\infty$ si x et y ne sont pas reliés par au moins un chemin. Notons que cette quantité est bien définie, l'ensemble des arêtes étant fini, il y a un nombre fini de valeurs possibles pour la bande passante d'un chemin de x à y .

Q26. Déterminer un chemin de bande passante maximale entre Lurenberg (d'identifiant 0) et Veroja (d'identifiant 1) sur le graphe de la **figure 2**. Quelle est sa bande passante ?

On admet que dans un arbre il existe un unique chemin élémentaire entre tout couple de sommets.

Q27. Soient $x, y \in S$, montrer que l'unique chemin de x à y dans l'arbre couvrant de poids maximal T^* est un chemin de bande passante maximale entre x et y .

Un fournisseur d'accès cherche à pouvoir offrir un chemin avec la bande passante la plus grande possible entre tous les couples de villes possibles. Plus précisément, il s'intéresse à la quantité :

$$\bar{b}_{\lim}(G) = \min \{ \bar{b}_{\max}(x, y) \mid x, y \in S \}$$

appelée la *bande passante limite* d'un graphe G .

Q28. Justifier que

$$\bar{b}_{\lim}(G) = \begin{cases} -\infty & \text{si } G \text{ n'est pas connexe} \\ \min \{ p(a) \mid a \in A^* \} & \text{avec } T^* = (S, A^*) \text{ l'arbre couvrant de poids maximal de } G \text{ sinon.} \end{cases}$$

Partie V - Jeu sur un graphe

Dans cette partie, on suppose que $G = (S, A, p)$ est un graphe pondéré connexe muni d'une fonction de pondération injective.

V.1 - Procédure de partage vue comme un jeu

On peut voir la procédure de partage du réseau décrite dans l'introduction du sujet comme un jeu à deux joueurs. Les deux joueurs sont MaxDébit (joueur max associé à la valeur 1) et MinLatence (joueur min associé à la valeur -1).

Une *configuration* est la donnée de deux sous-graphes G_1 et G_{-1} de G , d'arêtes disjointes, correspondant aux choix des arêtes effectués jusque-là par les deux joueurs, ainsi que la donnée du joueur qui doit jouer (il s'agit du joueur qui *contrôle* cette configuration). Un *coup* possible consiste à choisir une nouvelle arête du graphe non encore attribuée.

Une configuration est finale lorsque toutes les arêtes ont été attribuées et donc lorsque les arêtes de G_1 et G_{-1} partitionnent celles de G . La configuration initiale est contrôlée par MaxDébit et est composée de deux sous-graphes sans arêtes.

Pour tester la viabilité de cette procédure, le gouvernement commence par effectuer un essai sur le réseau de l'île de Kor e rattach e au Listenbourg, repr esent e   la **figure 5**.

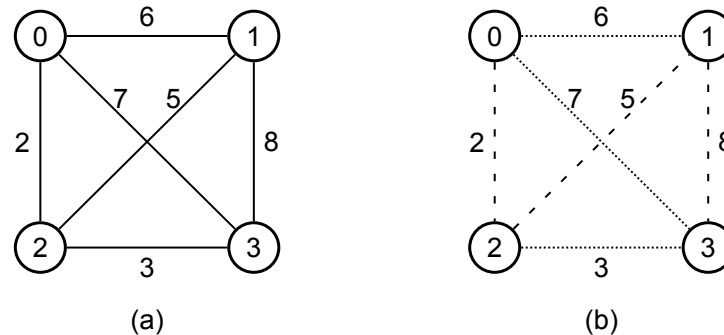


Figure 5 - (a) Le graphe correspondant au r seau de l' le de Kor e. (b) Exemple de configuration finale (contr ol e par MaxD bit) atteinte   l'issue d'une partie. Les ar tes choisies par le joueur MaxD bit sont repr esent es par un trait discontinu et celle par le joueur MinLatence par une ligne en pointill es

Un exemple possible de partie de ce jeu, dont l'issue est repr esent e   la **figure 5**, est la suivante :

- l'ar te $\{1, 3\}$ est choisie par MaxD bit ;
- l'ar te $\{0, 1\}$ est choisie par MinLatence ;
- l'ar te $\{1, 2\}$ est choisie par MaxD bit ;
- l'ar te $\{2, 3\}$ est choisie par MinLatence ;
- l'ar te $\{0, 2\}$ est choisie par MaxD bit ;
- l'ar te $\{0, 3\}$ est choisie par MinLatence.

Ici les deux op rateurs obtiennent la gestion d'un sous-r seau qui est exactement un arbre couvrant, mais ce n'est pas le cas en g n ral, le sous-graphe correspondant au sous-r seau obtenu par un joueur pouvant m me ne pas  tre connexe.

La **partie IV** montre que l'objectif d'un fournisseur d'acc s est de choisir un sous-graphe G' de mani re   maximiser la quantit  $\bar{b}_{\text{lim}}(G')$, c'est- -dire de maximiser la bande passante limite du sous-graphe. D'apr s la **partie IV**, pour cela, il s'agit tout d'abord d'obtenir un sous-graphe G' connexe (sans quoi $\bar{b}_{\text{lim}}(G') = -\infty$) puis, le cas  ch ant, de consid rer l'ar te de poids minimal de l'arbre couvrant de poids maximal de G' , $\bar{b}_{\text{lim}}(G')$  tant alors le poids de cette ar te. Un fournisseur d'acc s estime qu'il est gagnant si la bande passante limite de son sous-graphe $\bar{b}_{\text{lim}}(G')$ est strictement sup rieure   celle de son concurrent. Une configuration finale est donc gagnante pour MaxD bit si $\bar{b}_{\text{lim}}(G_1) > \bar{b}_{\text{lim}}(G_{-1})$ et gagnante pour MinLatence si $\bar{b}_{\text{lim}}(G_1) < \bar{b}_{\text{lim}}(G_{-1})$. Remarquons que si seul l'un des deux joueurs r ussit   obtenir un sous-graphe connexe, celui-ci est n cessairement gagnant. Il peut  galement y avoir match nul si aucun des deux joueurs n'obtient un graphe connexe   l'issue de la proc dure d'attribution.

En reprenant l'exemple propos    la **figure 5**, MaxD bit obtient un sous-graphe dont la plus petite ar te de son arbre couvrant est de poids 2 et MinLatence obtient un sous-graphe dont la plus petite ar te de son arbre couvrant est de poids 3. Le joueur MinLatence remporte donc la partie.

Q29. Justifier que ce jeu termine toujours, c'est- -dire que toute partie est n cessairement finie.

Q30. Justifier que la seule possibilit  de match nul est celle o  aucun des deux joueurs n'obtient un graphe connexe   l'issue de la partie.

Q31. Montrer, sur un exemple, que la strat gie consistant   toujours choisir l'ar te de poids maximal parmi les ar tes restantes n'est pas une strat gie gagnante pour le joueur MaxD bit.

V.2 - Implémentation en Python

On rappelle qu'en Python une boucle `for i in range(a, b)` permet de faire évoluer une variable `i` entre les valeurs `a` **include** et `b` **exclude**.

On modélise en Python le graphe pondéré $G = (S, A, p)$ par un objet `g` de type `Graphe` dont l'implémentation n'est pas précisée ici. Si `g` est un objet de type `Graphe`, `g.nb_sommets` est un entier correspondant à $n = |S|$ et `g.arettes` est une liste de longueur $m = |A|$ contenant les triplets (w, i, j) avec $0 \leq i < j < n$ et $w = p(\{i, j\})$ correspondant aux arêtes pondérées. On note $(a_k)_{k \in \llbracket 0, m-1 \rrbracket}$ les arêtes dans l'ordre de cette énumération.

Par exemple, si `g` représente le graphe de l'île de Korbe représenté à la **figure 5**, on aura par exemple :

```
>>> g.nb_sommets
4
>>> g.arettes
[(6.0, 0, 1), (2.0, 0, 2), (7.0, 0, 3), (5.0, 1, 2), (8.0, 1, 3), (3.0, 2, 3)]
```

On représente une configuration par un couple (t, etat) . La première composante `t` est un entier valant 1 si c'est au joueur MaxDébit de jouer et -1 si c'est au joueur MinLatence de jouer. La deuxième composante `etat` est une liste de longueur m avec, pour $k \in \llbracket 0, m-1 \rrbracket$, `etat[k] == 1` si l'arête a_k a déjà été choisie par le joueur MaxDébit, `etat[k] == -1` si l'arête a_k a déjà été choisie par le joueur MinLatence et `etat[k] == 0` si l'arête a_k n'a pas encore été choisie.

Par exemple, la configuration finale de la **figure 5** est représentée par le couple :

```
(1, [-1, 1, -1, 1, 1, -1])
```

En effet, l'état est contrôlé par le joueur MaxDébit, les arêtes d'indices 1, 3 et 4 ont été choisies par le joueur MaxDébit et les arêtes d'indices 0, 2, 5 par le joueur MinLatence.

La fonction suivante permet de renvoyer la configuration initiale correspondant à un graphe `g` :

```
def configuration_initiale(g):
    etat = [0] * len(g.arettes)
    return (1, etat)
```

La fonction suivante permet de savoir quel joueur contrôle une configuration. Elle renvoie 1 si c'est à MaxDébit de jouer et -1 si c'est à MinLatence de jouer.

```
def tour(c):
    t, etat = c
    return t
```

Q32. Écrire en Python une fonction `finale(c)` qui prend une configuration `c` et qui renvoie `True` si et seulement si la configuration est finale, c'est-à-dire si la partie est terminée.

On rappelle que si `etat` est une liste Python, l'instruction `suiv = etat.copy()` permet de créer une copie de la liste `etat` dans la variable `suiv`.

Q33. Écrire en Python une fonction `configurations_suivantes(c)` qui, étant donné une configuration `c` supposée non finale, renvoie la liste de toutes les configurations accessibles en un coup à partir de cette configuration, c'est-à-dire les configurations obtenues lorsque le joueur qui contrôle cette configuration choisit une nouvelle arête. La configuration `c` ne doit pas être modifiée : il est nécessaire d'en faire des copies.

V.3 - Calcul des attracteurs

On suppose disposer d'une fonction `gagnant(g, c)` qui, étant donné un graphe g et une configuration finale c , renvoie 1, 0 ou -1 suivant que la configuration finale est gagnante pour MaxDébit, un match nul ou gagnante pour MinLatence. Pour cela, cette fonction va calculer les sous-graphes G_1 et G_{-1} obtenus par les deux joueurs, vérifier si ceux-ci sont connexes, le cas échéant en calculer les arbres couvrants de poids maximal, puis le poids de l'arête de poids minimal de ces arbres couvrants, en déduire $\bar{b}_{\text{lim}}(G_1)$ et $\bar{b}_{\text{lim}}(G_{-1})$, les comparer et enfin renvoyer le gagnant (ou 0 pour match nul).

La fonction suivante permet d'attribuer un score dans $\{-1, 0, 1\}$ à une configuration non nécessairement finale.

```

1 def score(g, c):
2     if est_finale(c):
3         return gagnant(g, c)
4     t, etat = c
5     score_fils = [score(g, suiv) for suiv in configurations_suivantes(c)]
6     if t == 1:
7         return max(score_fils)
8     else:
9         return min(score_fils)

```

Q34. Indiquer à quoi correspondent les configurations (non nécessairement finales) de score -1 , de score 0 et de score 1.

Q35. Considérons une configuration non finale de score 1. Expliquer comment construire une stratégie gagnante pour MaxDébit à partir de cette configuration. On ne demande pas d'implémenter cette fonction en Python ni de montrer que cette stratégie est effectivement gagnante, mais uniquement d'expliquer comment l'obtenir.

Les trois sous-parties suivantes sont complètement indépendantes.

V.4 - Mémoïsation

L'implémentation précédente de la fonction `score` va conduire à recalculer un grand nombre de fois le score pour une même configuration. On se propose d'adopter une technique de mémoïsation. On se donne un dictionnaire `cache` qui sera, pour simplifier, une variable globale. Ce dictionnaire permet de mémoriser le score des configurations pour lesquelles celui-ci a déjà été calculé. Comme les listes ne peuvent pas être utilisées comme clés pour les dictionnaires, on utilisera la fonction `fige` ci-après pour convertir une configuration avec des listes en configuration avec des n -uplets avant de l'utiliser comme clé du dictionnaire `cache`.

```

def fige(c):
    t, etat = c
    return (t, tuple(etat))

```

Par exemple, en considérant la configuration obtenue après les trois premiers coups lors de la partie donnée en exemple dans la **sous-partie V.1**, on obtient :

```

>>> c_ex = (-1, [-1, 0, 0, 1, 1, 0])
>>> fige(c_ex)
(-1, (-1, 0, 0, 1, 1, 0))

```

- Q36.** Compléter le squelette de la fonction `score_memo` ci-dessous pour que celle-ci se comporte exactement comme la fonction `score` de la **sous-partie V.3**, mais en utilisant le dictionnaire `cache` pour ne pas recalculer plusieurs fois le score d'une même configuration.

```
cache = {}

def score_memo(g, c):
    # On peut utiliser dans cette fonction la variable globale `cache`
    ...
```

V.5 - Algorithme MinMax avec heuristique et exploration jusqu'à une profondeur p

Même en utilisant une fonction mémorisée, le coût du calcul du score de toutes les configurations avec l'approche précédente peut vite se révéler prohibitif. On se propose dans cette sous-partie d'utiliser l'algorithme MinMax avec une heuristique en limitant l'exploration jusqu'à une profondeur $p \in \mathbb{N}$. On suppose disposer d'une heuristique, c'est-à-dire une évaluation approximative de la qualité d'une configuration non finale sous la forme d'un score dans $[-1, 1]$, avec un score positif si la configuration semble favorable pour MaxDébit et un score négatif si la configuration semble favorable pour MinLatence. On suppose donc disposer d'une fonction heuristique telle que `heuristique(g, c)` pour un graphe g et une configuration non finale c renvoie un score dans $[-1, 1]$.

- Q37.** Écrire en Python une fonction `score_minmax(g, c, p)` sur le modèle de la fonction `score` de la **sous-partie V.3** (sans la mémorisation de la **sous-partie V.4**) telle que `score_minmax(g, c, p)` calcule un score pour une configuration c , mais en limitant l'exploration à une profondeur $p \in \mathbb{N}$, et en utilisant l'heuristique comme substitut lorsque l'on atteint la limite de profondeur.

V.6 - Vol de stratégie

- Q38.** Montrer que, quel que soit le graphe connexe à pondération injective, il n'existe pas de stratégie gagnante pour MinLatence. *On pourra remarquer qu'avoir un coup arbitraire en plus ne peut qu'être bénéfique et procéder par l'absurde en supposant que le deuxième joueur dispose d'une stratégie gagnante pour construire une stratégie gagnante pour le premier joueur.*

Ainsi, il existe pour MaxDébit une stratégie lui assurant de ne pas perdre : elle lui assure soit de gagner soit de forcer un match nul. Le gouvernement du Listenbourg semble avoir privilégié un des deux opérateurs.

FIN